# Namespace SixLabors.ImageSharp.Drawing

## Classes

ArcLineSegment
: Represents a line segment that contains radii and angles that will be rendered as an elliptical arc.

ClipPathExtensions
: Provides extension methods to IPath that allow the clipping of shapes.

ComplexPolygon
: Represents a complex polygon made up of one or more shapes overlayed on each other, where overlaps cause holes.

CubicBezierLineSegment
: Represents a line segment that contains a list of control points that will be rendered as a cubic bezier curve.

EllipsePolygon
: An elliptical shape made up of a single path made up of one or more ILineSegments.

EmptyPath
: A path that is always empty.

LinearGeometry
: Represents retained linearized geometry that can be consumed directly by drawing backends.

LinearLineSegment
: Represents a series of control points that will be joined by straight lines.

NormalizePathExtensions
: Provides extension methods to IPath that allow paths to be normalized.

OutlinePathExtensions
: Extensions to IPath that allow the generation of outlines.

Path
: An aggregate of ILineSegments making a single logical path.

PathBuilder
: Provides a fluent API for incrementally building complex shapes and paths from figures composed of lines, arcs, and bezier curves.

PathCollection
An aggregate of IPaths to apply common operations to them.

PathExtensions
Convenience methods that can be applied to shapes and paths.

PiePolygon
A pie sector polygon defined by a center point, radii, rotation, and arc sweep.

Polygon
A shape made up of a single closed path made up of one or more ILineSegments.

RectanglePolygon
A closed rectangular path defined by four straight edges.

Region
Represents an integer region composed from axis-aligned rectangles.

RegularPolygon
A regular polygon with a configurable number of equally spaced vertices placed on a circumscribing circle.

RoundedRectanglePolygon
A closed rectangular path with rounded corners.

SplitPathExtensions
Extensions to IPath for splitting paths into dash segments without performing stroke expansion.

StarPolygon
A star-shaped polygon defined by alternating inner and outer radii.

## Structs

LinearContour
Describes a single contour within a LinearGeometry.

LinearGeometryInfo
Describes geometry-wide metadata for a LinearGeometry instance.

LinearSegment
Represents one derived linear segment within a LinearGeometry.

PathPoint
Represents metadata about a point along a path.

SegmentEnumerator
: Enumerates the derived linear segments in a LinearGeometry.

## Interfaces

ILineSegment
: Represents a simple path segment.

IPath
: Represents a logic path that can be drawn.

IPathCollection
: Represents a collection of paths that can be enumerated and transformed as a single unit.

ISimplePath
: Represents a simple (non-composite) path defined by a series of points.

## Enums

BooleanOperation
: Specifies the type of boolean operation to perform on polygons.

ClipOperation
: Specifies how a clip shape is combined with the current clip region.

IntersectionRule
: Provides options for calculating intersection points.

LineCap
: Specifies the shape to be used at the ends of open lines or paths when stroking.

LineJoin
: Specifies how the connection between two consecutive line segments (a join) is rendered when stroking paths or polygons.

PathTypes
: Describes the different type of paths.

# Class ArcLineSegment

Namespace: SixLabors.ImageSharp.Drawing

Assembly: SixLabors.ImageSharp.Drawing.dll

Represents a line segment that contains radii and angles that will be rendered as an elliptical arc.

```
public class ArcLineSegment : ILineSegment
```

**Inheritance**

object ← ArcLineSegment

**Implements**

ILineSegment

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## ArcLineSegment(PointF, PointF, SizeF, float, bool, bool)

Initializes a new instance of the ArcLineSegment class.

```
public ArcLineSegment(PointF from, PointF to, SizeF radius, float rotation, bool largeArc,
bool sweep)
```

### Parameters

`from` PointF

The absolute coordinates of the current point on the path.

`to` PointF

The absolute coordinates of the final point of the arc.

`radius` SizeF

The radii of the ellipse (also known as its semi-major and semi-minor axes).

`rotation` float

The angle, in degrees, from the x-axis of the current coordinate system to the x-axis of the ellipse.

`largeArc` bool

The large arc flag, and is false if an arc spanning less than or equal to 180 degrees is chosen, or true if an arc spanning greater than 180 degrees is chosen.

`sweep` bool

The sweep flag, and is false if the line joining center to arc sweeps through decreasing angles, or true if it sweeps through increasing angles.

## ArcLineSegment(PointF, SizeF, float, float, float)

Initializes a new instance of the ArcLineSegment class.

```
public ArcLineSegment(PointF center, SizeF radius, float rotation, float startAngle,
float sweepAngle)
```

### Parameters

`center` PointF

The coordinates of the center of the ellipse.

`radius` SizeF

The radii of the ellipse (also known as its semi-major and semi-minor axes).

`rotation` float

The angle, in degrees, from the x-axis of the current coordinate system to the x-axis of the ellipse.

`startAngle` float

The start angle of the elliptical arc prior to the stretch and rotate operations. (0 is at the 3 o'clock position of the arc's circle).

`sweepAngle` float

The angle between `startAngle` and the end of the arc.

# Properties

## Bounds

Gets the bounds of the linearized segment output.

```
public RectangleF Bounds { get; }
```

## Property Value

RectangleF

## EndPoint

Gets the end point.

```
public PointF EndPoint { get; }
```

## Property Value

PointF

The end point.

## StartPoint

Gets the start point.

```
public PointF StartPoint { get; }
```

### Property Value

PointF

# Methods

## CopyTo(Span<PointF>, bool, Vector2)

Writes the segment's linearized points to `destination`, baked at the supplied device-space `scale`.

```
public void CopyTo(Span<PointF> destination, bool skipFirstPoint, Vector2 scale)
```

### Parameters

`destination` Span<PointF>

The destination point span.

`skipFirstPoint` bool

Whether to skip the first emitted point.

`scale` Vector2

The X/Y scale at which curves are flattened. Pass One for local-space output.

## LinearVertexCount(Vector2)

Returns the number of linear vertices emitted by this segment when flattened under the supplied device-space `scale`.

```
public int LinearVertexCount(Vector2 scale)
```

### Parameters

`scale` Vector2

The X/Y scale at which curves are flattened. Pass One for local-space counts.

## Returns

int

The number of linear vertices this segment emits.

# Transform(Matrix4×4)

Transforms the current ArcLineSegment using specified matrix.

```
public ILineSegment Transform(Matrix4x4 matrix)
```

## Parameters

`matrix` Matrix4×4

The transformation matrix.

## Returns

ILineSegment

An ArcLineSegment with the matrix applied to it.

# Enum BooleanOperation

Namespace: SixLabors.ImageSharp.Drawing

Assembly: SixLabors.ImageSharp.Drawing.dll

Specifies the type of boolean operation to perform on polygons.

```
public enum BooleanOperation
```

# Fields

`Difference = 2`

The difference operation, which subtracts the clipping polygon from the subject polygon.

`Intersection = 0`

The intersection operation, which results in the area common to both polygons.

`Union = 1`

The union operation, which results in the combined area of both polygons.

`Xor = 3`

The exclusive OR (XOR) operation, which results in the area covered by exactly one polygon, excluding the overlapping areas.

# Enum ClipOperation

Namespace: SixLabors.ImageSharp.Drawing

Assembly: SixLabors.ImageSharp.Drawing.dll

Specifies how a clip shape is combined with the current clip region.

```
public enum ClipOperation
```

# Fields

`Difference = 1`

Removes the clip shape from the current clip region.

`Intersection = 0`

Keeps only the area covered by both the current clip region and the clip shape.

# Class ClipPathExtensions

Namespace: SixLabors.ImageSharp.Drawing

Assembly: SixLabors.ImageSharp.Drawing.dll

Provides extension methods to IPath that allow the clipping of shapes.

```
public static class ClipPathExtensions
```

**Inheritance**

object ← ClipPathExtensions

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

# Methods

## Clip(IPath, BooleanOperation, params IPath[])

Clips the specified subject path with the provided clipping paths using NonZero.

```
public static IPath Clip(this IPath subjectPath, BooleanOperation operation, params
IPath[] clipPaths)
```

## Parameters

`subjectPath` IPath

The subject path.

`operation` BooleanOperation

The boolean operation to perform.

`clipPaths` IPath[]

The clipping paths.

## Returns

IPath

The clipped IPath.

# Clip(IPath, BooleanOperation, IntersectionRule, params IPath[])

Clips the specified subject path with the provided clipping paths.

```
public static IPath Clip(this IPath subjectPath, BooleanOperation operation,
IntersectionRule intersectionRule, params IPath[] clipPaths)
```

## Parameters

`subjectPath` IPath

The subject path.

`operation` BooleanOperation

The boolean operation to perform.

`intersectionRule` IntersectionRule

The fill rule used to interpret the subject and clipping paths.

`clipPaths` IPath[]

The clipping paths.

## Returns

IPath

The clipped IPath.

# Clip(IPath, BooleanOperation, IntersectionRule, IEnumerable<IPath>)

Clips the specified subject path with the provided clipping paths.

```
public static IPath Clip(this IPath subjectPath, BooleanOperation operation,
IntersectionRule intersectionRule, IEnumerable<IPath> clipPaths)
```

## Parameters

`subjectPath` IPath

The subject path.

`operation` BooleanOperation

The boolean operation to perform.

`intersectionRule` IntersectionRule

The fill rule used to interpret the subject and clipping paths.

`clipPaths` IEnumerable<IPath>

The clipping paths.

## Returns

IPath

The clipped IPath.

# Clip(IPath, BooleanOperation, IEnumerable<IPath>)

Clips the specified subject path with the provided clipping paths using NonZero.

```
public static IPath Clip(this IPath subjectPath, BooleanOperation operation,
IEnumerable<IPath> clipPaths)
```

## Parameters

`subjectPath` IPath

The subject path.

`operation` BooleanOperation

The boolean operation to perform.

`clipPaths` IEnumerable<IPath>

The clipping paths.

### Returns

IPath

The clipped IPath.

## Clip(IPath, params IPath[])

Clips the specified subject path with the provided clipping paths using Intersection and NonZero.

```
public static IPath Clip(this IPath subjectPath, params IPath[] clipPaths)
```

### Parameters

`subjectPath` IPath

The subject path.

`clipPaths` IPath[]

The clipping paths.

### Returns

IPath

The clipped IPath.

## Clip(IPath, IEnumerable<IPath>)

Clips the specified subject path with the provided clipping paths using Intersection and NonZero.

```
public static IPath Clip(this IPath subjectPath, IEnumerable<IPath> clipPaths)
```

## Parameters

`subjectPath` IPath

The subject path.

`clipPaths` IEnumerable<IPath>

The clipping paths.

## Returns

IPath

The clipped IPath.

# Class ComplexPolygon

Namespace: SixLabors.ImageSharp.Drawing

Assembly: SixLabors.ImageSharp.Drawing.dll

Represents a complex polygon made up of one or more shapes overlayed on each other, where overlaps cause holes.

```
public sealed class ComplexPolygon : IPath
```

### Inheritance

object ← ComplexPolygon

### Implements

IPath

### Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

### Extension Methods

ClipPathExtensions.Clip(IPath, BooleanOperation, params IPath[]) , ClipPathExtensions.Clip(IPath, BooleanOperation, IntersectionRule, params IPath[]) , ClipPathExtensions.Clip(IPath, BooleanOperation, IntersectionRule, IEnumerable<IPath>) , ClipPathExtensions.Clip(IPath, BooleanOperation, IEnumerable<IPath>) , ClipPathExtensions.Clip(IPath, params IPath[]) , ClipPathExtensions.Clip(IPath, IEnumerable<IPath>) , NormalizePathExtensions.Normalize(IPath) , OutlinePathExtensions.GenerateOutline(IPath, float) , OutlinePathExtensions.GenerateOutline(IPath, float, StrokeOptions) , OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>) , OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, StrokeOptions) , OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool) , OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool, StrokeOptions) , OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool, float) , OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool, float, StrokeOptions) , OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, float) , OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, float, StrokeOptions) , PathExtensions.ComputeArea(IPath) , PathExtensions.ComputeLength(IPath) ,

PathExtensions.Contains(IPath, PointF, IntersectionRule) , PathExtensions.Rotate(IPath, float) , PathExtensions.RotateDegree(IPath, float) , PathExtensions.Scale(IPath, float) , PathExtensions.Scale(IPath, float, float) , PathExtensions.Translate(IPath, PointF) , PathExtensions.Translate(IPath, float, float) , PathExtensions.TryGetPathPointAtDistance(IPath, float, out PathPoint) , PathExtensions.TryGetPathPointAtDistanceUnbounded(IPath, float, out PathPoint) , PathExtensions.TryGetSegment(IPath, float, float, bool, out IPath) , SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>) , SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>, bool) , SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>, bool, float) , SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>, float)

# Constructors

## ComplexPolygon(params IPath[])

Initializes a new instance of the ComplexPolygon class.

```
public ComplexPolygon(params IPath[] paths)
```

### Parameters

`paths` IPath[]

The paths.

## ComplexPolygon(PointF[], PointF[])

Initializes a new instance of the ComplexPolygon class.

```
public ComplexPolygon(PointF[] contour, PointF[] hole)
```

### Parameters

`contour` PointF[]

The points defining the outer contour.

`hole` PointF[]

The points defining the hole.

## ComplexPolygon(IEnumerable<IPath>)

Initializes a new instance of the ComplexPolygon class.

```
public ComplexPolygon(IEnumerable<IPath> paths)
```

### Parameters

`paths` IEnumerable<IPath>

The paths.

# Properties

## Bounds

Gets the bounds enclosing the path.

```
public RectangleF Bounds { get; }
```

### Property Value

RectangleF

## PathType

Gets a value indicating whether this instance is closed, open or a composite path with a mixture of open and closed figures.

```
public PathTypes PathType { get; }
```

### Property Value

PathTypes

## Paths

Gets the collection of paths that make up this shape.

```
public IEnumerable<IPath> Paths { get; }
```

### Property Value

IEnumerable<IPath>

# Methods

## AsClosedPath()

Returns this path with all figures closed.

```
public IPath AsClosedPath()
```

### Returns

IPath

A new closed IPath.

## ComputeArea(Vector2)

Calculates the non-negative path area after flattening curves at the supplied scale.

```
public float ComputeArea(Vector2 scale)
```

### Parameters

`scale` Vector2

The X/Y scale at which curves are flattened for area measurement.

### Returns

float

The path area.

## ComputeLength(Vector2)

Calculates the path length after flattening curves at the supplied scale.

```
public float ComputeLength(Vector2 scale)
```

### Parameters

`scale` Vector2

The X/Y scale at which curves are flattened for length measurement.

### Returns

float

The path length.

## Contains(PointF, IntersectionRule, Vector2)

Returns whether the supplied point is inside the path.

```
public bool Contains(PointF point, IntersectionRule intersectionRule, Vector2 scale)
```

### Parameters

`point` PointF

The point to test.

`intersectionRule` IntersectionRule

The fill rule used for containment.

`scale` Vector2

The X/Y scale at which curves are flattened for the containment test.

## Returns

bool

true when the point is inside or on the path boundary.

# Flatten()

Converts the IPath into a simple linear path.

```
public IEnumerable<ISimplePath> Flatten()
```

## Returns

IEnumerable<ISimplePath>

Returns the current IPath as simple linear path.

# ToLinearGeometry(Vector2)

Converts this path into a retained LinearGeometry, flattening curves at the precision of the supplied device-space `scale`.

```
public LinearGeometry ToLinearGeometry(Vector2 scale)
```

## Parameters

`scale` Vector2

The X/Y scale at which curves are flattened.

## Returns

LinearGeometry

The retained linear geometry.

# Transform(Matrix4×4)

Transforms the path using the specified matrix.

```
public IPath Transform(Matrix4x4 matrix)
```

## Parameters

`matrix` Matrix4×4

The matrix.

## Returns

IPath

A new path with the matrix applied to it.

# TryGetPathPointAtDistance(float, Vector2, out PathPoint)

Gets path information at the specified distance along the path.

```
public bool TryGetPathPointAtDistance(float distance, Vector2 scale, out
PathPoint pathPoint)
```

## Parameters

`distance` float

The distance along the path.

`scale` Vector2

The X/Y scale at which curves are flattened for distance measurement.

`pathPoint` PathPoint

When this method returns, contains the path information at `distance` if the distance resolves to a point on the path; otherwise, the default value.

## Returns

bool

true if `distance` resolves to a point on the path; otherwise, false.

# TryGetPathPointAtDistanceUnbounded(float, Vector2, out PathPoint)

Gets path information at the specified distance along the path, extrapolating along the boundary tangents when the distance falls before the start or beyond the end of an open path.

```
public bool TryGetPathPointAtDistanceUnbounded(float distance, Vector2 scale, out
PathPoint pathPoint)
```

## Parameters

`distance` float

The distance along the path.

`scale` Vector2

The X/Y scale at which curves are flattened for distance measurement.

`pathPoint` PathPoint

When this method returns, contains the path information at `distance` if the path is measurable; otherwise, the default value.

## Returns

bool

true if the point could be resolved; otherwise, false.

## Remarks

Unlike TryGetPathPointAtDistance(float, Vector2, out PathPoint), out-of-range distances resolve to a point on the virtual straight-line continuation of the path's first or last segment. Text layout uses this to place aligned glyphs that overflow their layout path. The method returns false only when the distance is not a finite number or the path has no measurable length.

# TryGetSegment(float, float, bool, Vector2, out IPath)

Creates a path segment between two distances along the path.

```
public bool TryGetSegment(float startDistance, float stopDistance, bool startOnBeginFigure,
Vector2 scale, out IPath path)
```

## Parameters

`startDistance` float

The segment start distance.

`stopDistance` float

The segment stop distance.

`startOnBeginFigure` bool

Whether the returned segment starts a new figure at the first segment point.

`scale` Vector2

The X/Y scale at which curves are flattened for segment extraction.

`path` IPath

When this method returns, contains the segment path if one was created; otherwise, an empty path.

## Returns

bool

true when a segment path was created.

# See Also

IPath

# Class CubicBezierLineSegment

Namespace: SixLabors.ImageSharp.Drawing

Assembly: SixLabors.ImageSharp.Drawing.dll

Represents a line segment that contains a list of control points that will be rendered as a cubic bezier curve.

```
public sealed class CubicBezierLineSegment : ILineSegment
```

**Inheritance**

object ← CubicBezierLineSegment

**Implements**

ILineSegment

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## CubicBezierLineSegment(PointF, PointF, PointF, PointF)

Initializes a new instance of the CubicBezierLineSegment class.

```
public CubicBezierLineSegment(PointF start, PointF controlPoint1, PointF controlPoint2,
PointF end)
```

### Parameters

`start` PointF

The start point of the curve.

`controlPoint1` PointF

The first control point.

`controlPoint2` PointF

The second control point.

`end` PointF

The end point of the curve.

## CubicBezierLineSegment(PointF, PointF, PointF, PointF, params PointF[])

Initializes a new instance of the CubicBezierLineSegment class.

```
public CubicBezierLineSegment(PointF start, PointF controlPoint1, PointF controlPoint2,
PointF end, params PointF[] additionalPoints)
```

### Parameters

`start` PointF

The start point of the curve.

`controlPoint1` PointF

The first control point.

`controlPoint2` PointF

The second control point.

`end` PointF

The end point of the curve.

`additionalPoints` PointF[]

Additional points appended after `end`; each group of three (two control points and an end point) defines a further cubic span.

## CubicBezierLineSegment(PointF[])

Initializes a new instance of the CubicBezierLineSegment class.

```
public CubicBezierLineSegment(PointF[] points)
```

### Parameters

`points` PointF[]

The control points. The length must be a multiple of 3 plus 1 (4, 7, 10...).

# Properties

## Bounds

Gets the bounds of the linearized segment output.

```
public RectangleF Bounds { get; }
```

### Property Value

RectangleF

## ControlPoints

Gets the control points.

```
public IReadOnlyList<PointF> ControlPoints { get; }
```

### Property Value

IReadOnlyList<PointF>

## EndPoint

Gets the end point.

```
public PointF EndPoint { get; }
```

### Property Value

PointF

The end point.

## StartPoint

Gets the start point.

```
public PointF StartPoint { get; }
```

### Property Value

PointF

# Methods

## CopyTo(Span<PointF>, bool, Vector2)

Writes the segment's linearized points to `destination`, baked at the supplied device-space `scale`.

```
public void CopyTo(Span<PointF> destination, bool skipFirstPoint, Vector2 scale)
```

### Parameters

`destination` Span<PointF>

The destination point span.

`skipFirstPoint` bool

Whether to skip the first emitted point.

`scale` Vector2

The X/Y scale at which curves are flattened. Pass One for local-space output.

## GetControlPoints()

Gets the control points of this curve.

```
public ReadOnlyMemory<PointF> GetControlPoints()
```

### Returns

ReadOnlyMemory<PointF>

The control points of this curve.

## LinearVertexCount(Vector2)

Returns the number of linear vertices emitted by this segment when flattened under the supplied device-space `scale`.

```
public int LinearVertexCount(Vector2 scale)
```

### Parameters

`scale` Vector2

The X/Y scale at which curves are flattened. Pass One for local-space counts.

### Returns

int

The number of linear vertices this segment emits.

## Transform(Matrix4×4)

Transforms this line segment using the specified matrix.

```
public CubicBezierLineSegment Transform(Matrix4x4 matrix)
```

## Parameters

`matrix` Matrix4×4

The matrix.

## Returns

CubicBezierLineSegment

A line segment with the matrix applied to it.

# See Also

ILineSegment

# Class EllipsePolygon

Namespace: SixLabors.ImageSharp.Drawing

Assembly: SixLabors.ImageSharp.Drawing.dll

An elliptical shape made up of a single path made up of one or more ILineSegments.

```
public sealed class EllipsePolygon : Polygon, IPath, ISimplePath
```

### Inheritance

object ← Path ← Polygon ← EllipsePolygon

### Implements

IPath, ISimplePath

### Inherited Members

Polygon.IsClosed , Path.Empty , Path.Points , Path.Bounds , Path.PathType , Path.LineSegments , Path.AsClosedPath() , Path.Flatten() , Path.ToLinearGeometry(Vector2) , Path.ComputeLength(Vector2) , Path.ComputeArea(Vector2) , Path.Contains(PointF, IntersectionRule, Vector2) , Path.TryGetPathPointAtDistance(float, Vector2, out PathPoint) , Path.TryGetPathPointAtDistanceUnbounded(float, Vector2, out PathPoint) , Path.TryGetSegment(float, float, bool, Vector2, out IPath) , Path.TryParseSvgPath(string, out IPath) , Path.TryParseSvgPath(ReadOnlySpan<char>, out IPath) , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

### Extension Methods

ClipPathExtensions.Clip(IPath, BooleanOperation, params IPath[]) , ClipPathExtensions.Clip(IPath, BooleanOperation, IntersectionRule, params IPath[]) , ClipPathExtensions.Clip(IPath, BooleanOperation, IntersectionRule, IEnumerable<IPath>) , ClipPathExtensions.Clip(IPath, BooleanOperation, IEnumerable<IPath>) , ClipPathExtensions.Clip(IPath, params IPath[]) , ClipPathExtensions.Clip(IPath, IEnumerable<IPath>) , NormalizePathExtensions.Normalize(IPath) , OutlinePathExtensions.GenerateOutline(IPath, float) , OutlinePathExtensions.GenerateOutline(IPath, float, StrokeOptions) , OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>) , OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, StrokeOptions) ,

OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool, StrokeOptions) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool, float) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool, float, StrokeOptions) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, float) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, float, StrokeOptions) ,
PathExtensions.ComputeArea(IPath) , PathExtensions.ComputeLength(IPath) ,
PathExtensions.Contains(IPath, PointF, IntersectionRule) , PathExtensions.Rotate(IPath, float) ,
PathExtensions.RotateDegree(IPath, float) , PathExtensions.Scale(IPath, float) ,
PathExtensions.Scale(IPath, float, float) , PathExtensions.Translate(IPath, PointF) ,
PathExtensions.Translate(IPath, float, float) ,
PathExtensions.TryGetPathPointAtDistance(IPath, float, out PathPoint) ,
PathExtensions.TryGetPathPointAtDistanceUnbounded(IPath, float, out PathPoint) ,
PathExtensions.TryGetSegment(IPath, float, float, bool, out IPath) ,
SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>) ,
SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>, bool) ,
SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>, bool, float) ,
SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>, float)

# Constructors

## EllipsePolygon(PointF, SizeF)

Initializes a new instance of the EllipsePolygon class.

```
public EllipsePolygon(PointF location, SizeF size)
```

## Parameters

`location` PointF

The location the center of the ellipse will be placed.

`size` SizeF

The width/height of the final ellipse.

## EllipsePolygon(PointF, float)

Initializes a new instance of the EllipsePolygon class.

```
public EllipsePolygon(PointF location, float radius)
```

### Parameters

`location` PointF

The location the center of the circle will be placed.

`radius` float

The radius of the final circle.

## EllipsePolygon(float, float, float)

Initializes a new instance of the EllipsePolygon class.

```
public EllipsePolygon(float x, float y, float radius)
```

### Parameters

`x` float

The x-coordinate of the center of the circle.

`y` float

The y-coordinate of the center of the circle.

`radius` float

The radius of the final circle.

## EllipsePolygon(float, float, float, float)

Initializes a new instance of the EllipsePolygon class.

```
public EllipsePolygon(float x, float y, float width, float height)
```

### Parameters

`x` float

The x-coordinate of the center of the ellipse.

`y` float

The y-coordinate of the center of the ellipse.

`width` float

The width the ellipse should have.

`height` float

The height the ellipse should have.

# Methods

## Transform(Matrix4×4)

Transforms the path using the specified matrix.

```
public override IPath Transform(Matrix4x4 matrix)
```

### Parameters

`matrix` Matrix4×4

The matrix.

### Returns

IPath

A new path with the matrix applied to it.

# Class EmptyPath

Namespace: SixLabors.ImageSharp.Drawing

Assembly: SixLabors.ImageSharp.Drawing.dll

A path that is always empty.

```
public sealed class EmptyPath : IPath
```

### Inheritance

object ← EmptyPath

### Implements

IPath

### Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() ,
object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

### Extension Methods

ClipPathExtensions.Clip(IPath, BooleanOperation, params IPath[]) ,
ClipPathExtensions.Clip(IPath, BooleanOperation, IntersectionRule, params IPath[]) ,
ClipPathExtensions.Clip(IPath, BooleanOperation, IntersectionRule, IEnumerable<IPath>) ,
ClipPathExtensions.Clip(IPath, BooleanOperation, IEnumerable<IPath>) ,
ClipPathExtensions.Clip(IPath, params IPath[]) ,
ClipPathExtensions.Clip(IPath, IEnumerable<IPath>) , NormalizePathExtensions.Normalize(IPath) ,
OutlinePathExtensions.GenerateOutline(IPath, float) ,
OutlinePathExtensions.GenerateOutline(IPath, float, StrokeOptions) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, StrokeOptions) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool, StrokeOptions) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool, float) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool, float,
StrokeOptions) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, float) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, float, StrokeOptions) ,
PathExtensions.ComputeArea(IPath) , PathExtensions.ComputeLength(IPath) ,
PathExtensions.Contains(IPath, PointF, IntersectionRule) , PathExtensions.Rotate(IPath, float) ,

PathExtensions.RotateDegree(IPath, float) , PathExtensions.Scale(IPath, float) , PathExtensions.Scale(IPath, float, float) , PathExtensions.Translate(IPath, PointF) , PathExtensions.Translate(IPath, float, float) , PathExtensions.TryGetPathPointAtDistance(IPath, float, out PathPoint) , PathExtensions.TryGetPathPointAtDistanceUnbounded(IPath, float, out PathPoint) , PathExtensions.TryGetSegment(IPath, float, float, bool, out IPath) , SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>) , SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>, bool) , SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>, bool, float) , SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>, float)

# Properties

## Bounds

Gets the bounds enclosing the path.

```
public RectangleF Bounds { get; }
```

### Property Value

RectangleF

## ClosedPath

Gets the closed path instance of the empty path.

```
public static EmptyPath ClosedPath { get; }
```

### Property Value

EmptyPath

## OpenPath

Gets the open path instance of the empty path.

```
public static EmptyPath OpenPath { get; }
```

### Property Value

EmptyPath

## PathType

Gets a value indicating whether this instance is closed, open or a composite path with a mixture of open and closed figures.

```
public PathTypes PathType { get; }
```

### Property Value

PathTypes

# Methods

## AsClosedPath()

Returns this path with all figures closed.

```
public IPath AsClosedPath()
```

### Returns

IPath

A new closed IPath.

## ComputeArea(Vector2)

Calculates the non-negative path area after flattening curves at the supplied scale.

```
public float ComputeArea(Vector2 scale)
```

## Parameters

`scale` Vector2

The X/Y scale at which curves are flattened for area measurement.

## Returns

float

The path area.

# ComputeLength(Vector2)

Calculates the path length after flattening curves at the supplied scale.

```
public float ComputeLength(Vector2 scale)
```

## Parameters

`scale` Vector2

The X/Y scale at which curves are flattened for length measurement.

## Returns

float

The path length.

# Contains(PointF, IntersectionRule, Vector2)

Returns whether the supplied point is inside the path.

```
public bool Contains(PointF point, IntersectionRule intersectionRule, Vector2 scale)
```

## Parameters

`point` PointF

The point to test.

`intersectionRule` IntersectionRule

The fill rule used for containment.

`scale` Vector2

The X/Y scale at which curves are flattened for the containment test.

## Returns

bool

true when the point is inside or on the path boundary.

# Flatten()

Converts the IPath into a simple linear path.

```
public IEnumerable<ISimplePath> Flatten()
```

## Returns

IEnumerable<ISimplePath>

Returns the current IPath as simple linear path.

# ToLinearGeometry(Vector2)

Converts this path into a retained LinearGeometry, flattening curves at the precision of the supplied device-space `scale`.

```
public LinearGeometry ToLinearGeometry(Vector2 scale)
```

### Parameters

`scale` Vector2

The X/Y scale at which curves are flattened.

### Returns

LinearGeometry

The retained linear geometry.

## Transform(Matrix4×4)

Transforms the path using the specified matrix.

```
public IPath Transform(Matrix4x4 matrix)
```

### Parameters

`matrix` Matrix4×4

The matrix.

### Returns

IPath

A new path with the matrix applied to it.

## TryGetPathPointAtDistance(float, Vector2, out PathPoint)

Gets path information at the specified distance along the path.

```
public bool TryGetPathPointAtDistance(float distance, Vector2 scale, out
```

```
PathPoint pathPoint)
```

## Parameters

`distance` float

The distance along the path.

`scale` Vector2

The X/Y scale at which curves are flattened for distance measurement.

`pathPoint` PathPoint

When this method returns, contains the path information at `distance` if the distance resolves to a point on the path; otherwise, the default value.

## Returns

bool

true if `distance` resolves to a point on the path; otherwise, false.

# TryGetPathPointAtDistanceUnbounded(float, Vector2, out PathPoint)

Gets path information at the specified distance along the path, extrapolating along the boundary tangents when the distance falls before the start or beyond the end of an open path.

```
public bool TryGetPathPointAtDistanceUnbounded(float distance, Vector2 scale, out
PathPoint pathPoint)
```

## Parameters

`distance` float

The distance along the path.

`scale` Vector2

The X/Y scale at which curves are flattened for distance measurement.

`pathPoint` PathPoint

When this method returns, contains the path information at `distance` if the path is measurable; otherwise, the default value.

## Returns

bool

true if the point could be resolved; otherwise, false.

## Remarks

Unlike TryGetPathPointAtDistance(float, Vector2, out PathPoint), out-of-range distances resolve to a point on the virtual straight-line continuation of the path's first or last segment. Text layout uses this to place aligned glyphs that overflow their layout path. The method returns false only when the distance is not a finite number or the path has no measurable length.

# TryGetSegment(float, float, bool, Vector2, out IPath)

Creates a path segment between two distances along the path.

```
public bool TryGetSegment(float startDistance, float stopDistance, bool startOnBeginFigure,
Vector2 scale, out IPath path)
```

## Parameters

`startDistance` float

The segment start distance.

`stopDistance` float

The segment stop distance.

`startOnBeginFigure` bool

Whether the returned segment starts a new figure at the first segment point.

`scale` Vector2

The X/Y scale at which curves are flattened for segment extraction.

`path` IPath

When this method returns, contains the segment path if one was created; otherwise, an empty path.

## Returns

bool

true when a segment path was created.

# Interface ILineSegment

Namespace: SixLabors.ImageSharp.Drawing

Assembly: SixLabors.ImageSharp.Drawing.dll

Represents a simple path segment.

```
public interface ILineSegment
```

# Properties

## Bounds

Gets the bounds of the linearized segment output.

```
RectangleF Bounds { get; }
```

## Property Value

RectangleF

## EndPoint

Gets the end point.

```
PointF EndPoint { get; }
```

## Property Value

PointF

The end point.

## StartPoint

Gets the start point.

```
PointF StartPoint { get; }
```

### Property Value

PointF

# Methods

## CopyTo(Span<PointF>, bool, Vector2)

Writes the segment's linearized points to `destination`, baked at the supplied device-space `scale`.

```
void CopyTo(Span<PointF> destination, bool skipFirstPoint, Vector2 scale)
```

### Parameters

`destination` Span<PointF>

The destination point span.

`skipFirstPoint` bool

Whether to skip the first emitted point.

`scale` Vector2

The X/Y scale at which curves are flattened. Pass One for local-space output.

## LinearVertexCount(Vector2)

Returns the number of linear vertices emitted by this segment when flattened under the supplied device-space `scale`.

```
int LinearVertexCount(Vector2 scale)
```

## Parameters

`scale` Vector2

The X/Y scale at which curves are flattened. Pass One for local-space counts.

## Returns

int

The number of linear vertices this segment emits.

# Transform(Matrix4×4)

Transforms the current line segment using the specified matrix.

```
ILineSegment Transform(Matrix4x4 matrix)
```

## Parameters

`matrix` Matrix4×4

The transformation matrix.

## Returns

ILineSegment

A line segment with the matrix applied to it.

# Interface IPath

Namespace: SixLabors.ImageSharp.Drawing

Assembly: SixLabors.ImageSharp.Drawing.dll

Represents a logic path that can be drawn.

```
public interface IPath
```

**Extension Methods**

ClipPathExtensions.Clip(IPath, BooleanOperation, params IPath[]) ,
ClipPathExtensions.Clip(IPath, BooleanOperation, IntersectionRule, params IPath[]) ,
ClipPathExtensions.Clip(IPath, BooleanOperation, IntersectionRule, IEnumerable<IPath>) ,
ClipPathExtensions.Clip(IPath, BooleanOperation, IEnumerable<IPath>) ,
ClipPathExtensions.Clip(IPath, params IPath[]) ,
ClipPathExtensions.Clip(IPath, IEnumerable<IPath>) , NormalizePathExtensions.Normalize(IPath) ,
OutlinePathExtensions.GenerateOutline(IPath, float) ,
OutlinePathExtensions.GenerateOutline(IPath, float, StrokeOptions) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, StrokeOptions) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool, StrokeOptions) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool, float) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool, float,
StrokeOptions) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, float) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, float, StrokeOptions) ,
PathExtensions.ComputeArea(IPath) , PathExtensions.ComputeLength(IPath) ,
PathExtensions.Contains(IPath, PointF, IntersectionRule) , PathExtensions.Rotate(IPath, float) ,
PathExtensions.RotateDegree(IPath, float) , PathExtensions.Scale(IPath, float) ,
PathExtensions.Scale(IPath, float, float) , PathExtensions.Translate(IPath, PointF) ,
PathExtensions.Translate(IPath, float, float) ,
PathExtensions.TryGetPathPointAtDistance(IPath, float, out PathPoint) ,
PathExtensions.TryGetPathPointAtDistanceUnbounded(IPath, float, out PathPoint) ,
PathExtensions.TryGetSegment(IPath, float, float, bool, out IPath) ,
SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>) ,
SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>, bool) ,

SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>, bool, float) ,
SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>, float)

# Properties

## Bounds

Gets the bounds enclosing the path.

```
RectangleF Bounds { get; }
```

## Property Value

RectangleF

## PathType

Gets a value indicating whether this instance is closed, open or a composite path with a mixture of open and closed figures.

```
PathTypes PathType { get; }
```

## Property Value

PathTypes

# Methods

## AsClosedPath()

Returns this path with all figures closed.

```
IPath AsClosedPath()
```

## Returns

IPath

A new closed IPath.

# ComputeArea(Vector2)

Calculates the non-negative path area after flattening curves at the supplied scale.

```
float ComputeArea(Vector2 scale)
```

## Parameters

`scale` Vector2

The X/Y scale at which curves are flattened for area measurement.

## Returns

float

The path area.

# ComputeLength(Vector2)

Calculates the path length after flattening curves at the supplied scale.

```
float ComputeLength(Vector2 scale)
```

## Parameters

`scale` Vector2

The X/Y scale at which curves are flattened for length measurement.

## Returns

float

The path length.

## Contains(PointF, IntersectionRule, Vector2)

Returns whether the supplied point is inside the path.

```
bool Contains(PointF point, IntersectionRule intersectionRule, Vector2 scale)
```

### Parameters

`point` PointF

The point to test.

`intersectionRule` IntersectionRule

The fill rule used for containment.

`scale` Vector2

The X/Y scale at which curves are flattened for the containment test.

### Returns

bool

true when the point is inside or on the path boundary.

## Flatten()

Converts the IPath into a simple linear path.

```
IEnumerable<ISimplePath> Flatten()
```

### Returns

IEnumerable<ISimplePath>

Returns the current IPath as simple linear path.

## ToLinearGeometry(Vector2)

Converts this path into a retained LinearGeometry, flattening curves at the precision of the supplied device-space `scale`.

```
LinearGeometry ToLinearGeometry(Vector2 scale)
```

### Parameters

`scale` Vector2

The X/Y scale at which curves are flattened.

### Returns

LinearGeometry

The retained linear geometry.

## Transform(Matrix4×4)

Transforms the path using the specified matrix.

```
IPath Transform(Matrix4x4 matrix)
```

### Parameters

`matrix` Matrix4×4

The matrix.

### Returns

IPath

A new path with the matrix applied to it.

## TryGetPathPointAtDistance(float, Vector2, out PathPoint)

Gets path information at the specified distance along the path.

```
bool TryGetPathPointAtDistance(float distance, Vector2 scale, out PathPoint pathPoint)
```

## Parameters

`distance` float

The distance along the path.

`scale` Vector2

The X/Y scale at which curves are flattened for distance measurement.

`pathPoint` PathPoint

When this method returns, contains the path information at `distance` if the distance resolves to a point on the path; otherwise, the default value.

## Returns

bool

true if `distance` resolves to a point on the path; otherwise, false.

# TryGetPathPointAtDistanceUnbounded(float, Vector2, out PathPoint)

Gets path information at the specified distance along the path, extrapolating along the boundary tangents when the distance falls before the start or beyond the end of an open path.

```
bool TryGetPathPointAtDistanceUnbounded(float distance, Vector2 scale, out
PathPoint pathPoint)
```

## Parameters

`distance` float

The distance along the path.

`scale` Vector2

The X/Y scale at which curves are flattened for distance measurement.

`pathPoint` PathPoint

When this method returns, contains the path information at `distance` if the path is measurable; otherwise, the default value.

## Returns

bool

true if the point could be resolved; otherwise, false.

## Remarks

Unlike TryGetPathPointAtDistance(float, Vector2, out PathPoint), out-of-range distances resolve to a point on the virtual straight-line continuation of the path's first or last segment. Text layout uses this to place aligned glyphs that overflow their layout path. The method returns false only when the distance is not a finite number or the path has no measurable length.

# TryGetSegment(float, float, bool, Vector2, out IPath)

Creates a path segment between two distances along the path.

```
bool TryGetSegment(float startDistance, float stopDistance, bool startOnBeginFigure, Vector2
scale, out IPath path)
```

## Parameters

`startDistance` float

The segment start distance.

`stopDistance` float

The segment stop distance.

`startOnBeginFigure` bool

Whether the returned segment starts a new figure at the first segment point.

`scale` Vector2

The X/Y scale at which curves are flattened for segment extraction.

`path` IPath

When this method returns, contains the segment path if one was created; otherwise, an empty path.

## Returns

bool

true when a segment path was created.

# Interface IPathCollection

Namespace: SixLabors.ImageSharp.Drawing

Assembly: SixLabors.ImageSharp.Drawing.dll

Represents a collection of paths that can be enumerated and transformed as a single unit.

```
public interface IPathCollection : IEnumerable<IPath>, IEnumerable
```

**Inherited Members**
IEnumerable<IPath>.GetEnumerator()

**Extension Methods**
PathExtensions.ComputeArea(IPathCollection) , PathExtensions.Rotate(IPathCollection, float) , PathExtensions.RotateDegree(IPathCollection, float) , PathExtensions.Scale(IPathCollection, float) , PathExtensions.Scale(IPathCollection, float, float) , PathExtensions.Translate(IPathCollection, PointF) , PathExtensions.Translate(IPathCollection, float, float)

# Properties

## Bounds

Gets the bounds enclosing all paths in the collection.

```
RectangleF Bounds { get; }
```

## Property Value

RectangleF

# Methods

## Transform(Matrix4×4)

Transforms all paths in the collection using the specified matrix.

```
IPathCollection Transform(Matrix4x4 matrix)
```

## Parameters

`matrix` Matrix4×4

The transformation matrix.

## Returns

IPathCollection

A new path collection with the matrix applied to it.

# Interface ISimplePath

Namespace: SixLabors.ImageSharp.Drawing

Assembly: SixLabors.ImageSharp.Drawing.dll

Represents a simple (non-composite) path defined by a series of points.

```
public interface ISimplePath
```

# Properties

## IsClosed

Gets a value indicating whether this instance is a closed path.

```
bool IsClosed { get; }
```

### Property Value

bool

## Points

Gets the points that make this up as a simple linear path.

```
ReadOnlyMemory<PointF> Points { get; }
```

### Property Value

ReadOnlyMemory<PointF>

# Enum IntersectionRule

Namespace: SixLabors.ImageSharp.Drawing

Assembly: SixLabors.ImageSharp.Drawing.dll

Provides options for calculating intersection points.

```
public enum IntersectionRule
```

# Fields

`EvenOdd = 0`

Only odd numbered sub-regions are filled.

`NonZero = 1`

Only non-zero sub-regions are filled.

# Enum LineCap

Namespace: SixLabors.ImageSharp.Drawing

Assembly: SixLabors.ImageSharp.Drawing.dll

Specifies the shape to be used at the ends of open lines or paths when stroking.

```
public enum LineCap
```

# Fields

`Butt = 0`

The stroke ends exactly at the endpoint. No extension is added beyond the path's end coordinates.

`Round = 2`

The stroke ends with a semicircular cap, extending beyond the endpoint by half the line width.

`Square = 1`

The stroke extends beyond the endpoint by half the line width, producing a square edge.

# Enum LineJoin

Namespace: SixLabors.ImageSharp.Drawing

Assembly: SixLabors.ImageSharp.Drawing.dll

Specifies how the connection between two consecutive line segments (a join) is rendered when stroking paths or polygons.

```
public enum LineJoin
```

# Fields

`Bevel = 3`

Joins lines by connecting the outer corners directly with a straight line, forming a flat edge at the join point.

`Miter = 0`

Joins lines by extending their outer edges until they meet at a sharp corner. If the miter limit is exceeded, the join is truncated at the limit distance.

`MiterRevert = 1`

Joins lines by extending their outer edges to form a miter. If the miter limit is exceeded, the join falls back to a bevel.

`MiterRound = 4`

Joins lines by forming a miter, but if the miter limit is exceeded, the join falls back to a round join instead of a bevel.

`Round = 2`

Joins lines by connecting them with a circular arc centered at the join point, producing a smooth, rounded corner.

# Struct LinearContour

Namespace: SixLabors.ImageSharp.Drawing

Assembly: SixLabors.ImageSharp.Drawing.dll

Describes a single contour within a LinearGeometry.

```
public readonly struct LinearContour
```

**Inherited Members**
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

# Remarks

A contour identifies a contiguous point run in Points and the corresponding range in the derived segment stream exposed by GetSegments().

# Properties

## Bounds

Gets the bounds of the stored contour points.

```
public required RectangleF Bounds { get; init; }
```

### Property Value

RectangleF

## IsClosed

Gets a value indicating whether the contour is closed.

```
public required bool IsClosed { get; init; }
```

### Property Value

bool

### Remarks

When true, the final derived segment for the contour joins the last stored point back to the first stored point. Closed contours do not duplicate the first point at the end of their stored point run.

## PointCount

Gets the number of stored points belonging to this contour.

```
public required int PointCount { get; init; }
```

### Property Value

int

## PointStart

Gets the zero-based index of the first point belonging to this contour in Points.

```
public required int PointStart { get; init; }
```

### Property Value

int

## SegmentCount

Gets the number of derived segments belonging to this contour.

```
public required int SegmentCount { get; init; }
```

## Property Value

int

# SegmentStart

Gets the zero-based index of the first derived segment belonging to this contour.

```
public required int SegmentStart { get; init; }
```

## Property Value

int

# Class LinearGeometry

Namespace: SixLabors.ImageSharp.Drawing

Assembly: SixLabors.ImageSharp.Drawing.dll

Represents retained linearized geometry that can be consumed directly by drawing backends.

```
public sealed class LinearGeometry
```

**Inheritance**

object ← LinearGeometry

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Remarks

A LinearGeometry instance stores contour-local point data plus the metadata required to interpret those points as a sequence of final linear segments.

Closed contours do not duplicate their first point at the end of the stored point run. Closure is represented by IsClosed, and the closing segment is derived by GetSegments().

The retained storage model is:

- Points stores the concatenated point data for every contour.
- Contours maps each contour to its point run and derived segment range.
- Info exposes geometry-wide metadata such as bounds and total segment count.

# Constructors

## LinearGeometry(LinearGeometryInfo, IReadOnlyList<LinearContour>, IReadOnlyList<PointF>)

Initializes a new instance of the LinearGeometry class.

```
public LinearGeometry(LinearGeometryInfo info, IReadOnlyList<LinearContour> contours,
IReadOnlyList<PointF> points)
```

### Parameters

`info` LinearGeometryInfo

The geometry metadata.

`contours` IReadOnlyList<LinearContour>

The contour metadata.

`points` IReadOnlyList<PointF>

The point storage.

# Properties

## Contours

Gets the contour metadata describing how Points is partitioned.

```
public IReadOnlyList<LinearContour> Contours { get; }
```

### Property Value

IReadOnlyList<LinearContour>

### Remarks

Each entry defines one contour's point run and the corresponding segment range in the derived segment stream.

## Info

Gets geometry-wide metadata for this retained result.

```
public LinearGeometryInfo Info { get; }
```

### Property Value

LinearGeometryInfo

## Points

Gets the retained point storage for all contours in this geometry.

```
public IReadOnlyList<PointF> Points { get; }
```

### Property Value

IReadOnlyList<PointF>

### Remarks

Points are stored per contour in contour order. A closed contour does not repeat its first point at the end of its stored point run.

# Methods

## ComputeArea()

Calculates the total absolute area of all contour point runs.

```
public float ComputeArea()
```

### Returns

float

The total contour area.

### Remarks

Each contour with at least three points contributes the absolute value of its shoelace area, regardless of whether it is marked closed. The result is memoized.

## ComputeLength()

Calculates the total length of all derived linear segments.

```
public float ComputeLength()
```

### Returns

float

The total segment length.

### Remarks

The result is memoized. Concurrent first calls may compute it more than once, which is benign because the computation is deterministic.

## Contains(PointF, IntersectionRule)

Returns whether the supplied point is inside the geometry.

```
public bool Contains(PointF point, IntersectionRule intersectionRule)
```

### Parameters

`point` PointF

The point to test.

`intersectionRule` IntersectionRule

The fill rule used for containment.

### Returns

bool

true when the point is inside or on the geometry boundary.

### Remarks

Only closed contours participate in the test; open contours never contain points. A point lying exactly on a closed contour edge is treated as contained under both fill rules.

## CreateOpenPolyline(PointF[])

Creates retained geometry for one open polyline.

```
public static LinearGeometry CreateOpenPolyline(PointF[] points)
```

### Parameters

`points` PointF[]

The polyline points.

### Returns

LinearGeometry

The retained open polyline geometry.

## CreateOpenPolyline(PointF[], Vector2)

Creates retained geometry for one open polyline, baked under the supplied device-space `scale`.

```
public static LinearGeometry CreateOpenPolyline(PointF[] points, Vector2 scale)
```

### Parameters

`points` PointF[]

The polyline points.

`scale` Vector2

The X/Y scale at which the polyline is baked.

### Returns

LinearGeometry

The retained open polyline geometry.

### Remarks

When `scale` is One the input array is retained directly without copying, so the caller must not mutate it afterwards.

## GetSegments()

Gets an enumerator for the derived linear segments represented by Points and Contours.

```
public SegmentEnumerator GetSegments()
```

### Returns

SegmentEnumerator

A zero-allocation enumerator that yields the final linear segments in contour order.

## TryGetPathPointAtDistance(float, out PathPoint)

Gets path information at the specified distance along the geometry.

```
public bool TryGetPathPointAtDistance(float distance, out PathPoint pathPoint)
```

### Parameters

`distance` float

The distance along the geometry.

`pathPoint` PathPoint

When this method returns, contains the path information at `distance` if the distance resolves to a point on the geometry; otherwise, the default value.

## Returns

bool

true if `distance` resolves to a point on the geometry; otherwise, false.

## Remarks

The distance must resolve to a point on the geometry itself: negative or non-finite distances fail, and when any contour is open a distance beyond the total length fails. When every contour is closed the distance wraps modulo the total length instead.

At a distance that lands exactly on a shared vertex the outgoing segment wins, so the reported tangent and angle are those of the segment leaving the vertex.

# TryGetPathPointAtDistanceUnbounded(float, out PathPoint)

Gets path information at the specified distance along the geometry, extrapolating along the boundary tangents when the distance falls before the start or beyond the end of an open geometry.

```
public bool TryGetPathPointAtDistanceUnbounded(float distance, out PathPoint pathPoint)
```

## Parameters

`distance` float

The distance along the geometry.

`pathPoint` PathPoint

When this method returns, contains the path information at `distance` if the geometry is measurable; otherwise, the default value.

## Returns

bool

true if the point could be resolved; otherwise, false.

## Remarks

Unlike TryGetPathPointAtDistance(float, out PathPoint), out-of-range distances resolve to a point on the virtual straight-line continuation of the geometry's first or last segment. The method returns false only when the distance is not a finite number or the geometry has no measurable length.

# TryGetSegment(float, float, bool, out IPath)

Creates a path segment between two distances along the geometry.

```
public bool TryGetSegment(float startDistance, float stopDistance, bool startOnBeginFigure,
out IPath path)
```

## Parameters

`startDistance` float

The segment start distance.

`stopDistance` float

The segment stop distance.

`startOnBeginFigure` bool

Whether the returned segment starts a new figure at the first segment point.

`path` IPath

When this method returns, contains the segment path if one was created; otherwise, an empty path.

## Returns

bool

true when a segment path was created.

# Struct LinearGeometryInfo

Namespace: SixLabors.ImageSharp.Drawing

Assembly: SixLabors.ImageSharp.Drawing.dll

Describes geometry-wide metadata for a LinearGeometry instance.

```
public readonly struct LinearGeometryInfo
```

**Inherited Members**
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

# Remarks

This metadata is computed eagerly during lowering so backends do not need to enumerate the geometry again to discover basic information such as total segment count or bounds.

# Properties

## Bounds

Gets the bounds of all points stored in the containing LinearGeometry.

```
public required RectangleF Bounds { get; init; }
```

### Property Value

RectangleF

## ContourCount

Gets the total number of contours in the containing LinearGeometry.

```
public required int ContourCount { get; init; }
```

## Property Value

int

# PointCount

Gets the total number of stored points across all contours.

```
public required int PointCount { get; init; }
```

## Property Value

int

# SegmentCount

Gets the total number of derived linear segments across all contours.

```
public required int SegmentCount { get; init; }
```

## Property Value

int

# Class LinearLineSegment

Namespace: SixLabors.ImageSharp.Drawing

Assembly: SixLabors.ImageSharp.Drawing.dll

Represents a series of control points that will be joined by straight lines.

```
public sealed class LinearLineSegment : ILineSegment
```

**Inheritance**

object ← LinearLineSegment

**Implements**

ILineSegment

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## LinearLineSegment(PointF, PointF)

Initializes a new instance of the LinearLineSegment class.

```
public LinearLineSegment(PointF start, PointF end)
```

## Parameters

`start` PointF

The start point.

`end` PointF

The end point.

## LinearLineSegment(PointF, PointF, params PointF[])

Initializes a new instance of the LinearLineSegment class.

```
public LinearLineSegment(PointF point1, PointF point2, params PointF[] additionalPoints)
```

### Parameters

`point1` PointF

The first point.

`point2` PointF

The second point.

`additionalPoints` PointF[]

Additional points appended after `point2`.

## LinearLineSegment(PointF[])

Initializes a new instance of the LinearLineSegment class.

```
public LinearLineSegment(PointF[] points)
```

### Parameters

`points` PointF[]

The points; at least two are required.

# Properties

## Bounds

Gets the bounds of the linearized segment output.

```
public RectangleF Bounds { get; }
```

### Property Value

RectangleF

## EndPoint

Gets the end point.

```
public PointF EndPoint { get; }
```

### Property Value

PointF

The end point.

## StartPoint

Gets the start point.

```
public PointF StartPoint { get; }
```

### Property Value

PointF

# Methods

## CopyTo(Span<PointF>, bool, Vector2)

Writes the segment's linearized points to `destination`, baked at the supplied device-space `scale`.

```
public void CopyTo(Span<PointF> destination, bool skipFirstPoint, Vector2 scale)
```

### Parameters

`destination` Span<PointF>

The destination point span.

`skipFirstPoint` bool

Whether to skip the first emitted point.

`scale` Vector2

The X/Y scale at which curves are flattened. Pass One for local-space output.

## LinearVertexCount(Vector2)

Returns the number of linear vertices emitted by this segment when flattened under the supplied device-space `scale`.

```
public int LinearVertexCount(Vector2 scale)
```

### Parameters

`scale` Vector2

The X/Y scale at which curves are flattened. Pass One for local-space counts.

### Returns

int

The number of linear vertices this segment emits.

## Transform(Matrix4×4)

Transforms the current LineSegment using specified matrix.

```
public LinearLineSegment Transform(Matrix4x4 matrix)
```

## Parameters

`matrix` Matrix4×4

The matrix.

## Returns

LinearLineSegment

A line segment with the matrix applied to it.

# See Also

ILineSegment

# Struct LinearSegment

Namespace: SixLabors.ImageSharp.Drawing

Assembly: SixLabors.ImageSharp.Drawing.dll

Represents one derived linear segment within a LinearGeometry.

```
public readonly struct LinearSegment
```

**Inherited Members**
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

# Remarks

Instances are produced by SegmentEnumerator and contain the per-segment values required by current backend scene-building code without forcing each backend to recompute them from the endpoints on every iteration.

# Properties

## ContourIndex

Gets the zero-based index of the contour that owns this segment.

```
public required int ContourIndex { get; init; }
```

### Property Value

int

## End

Gets the segment end point.

```
public required PointF End { get; init; }
```

### Property Value

PointF

## IsHorizontal

Gets a value indicating whether the segment is horizontal.

```
public required bool IsHorizontal { get; init; }
```

### Property Value

bool

### Remarks

A segment is horizontal when Start.Y equals End.Y.

## MaxY

Gets the larger of Start.Y and End.Y.

```
public required float MaxY { get; init; }
```

### Property Value

float

## MinY

Gets the smaller of Start.Y and End.Y.

```
public required float MinY { get; init; }
```

## Property Value

float

# Start

Gets the segment start point.

```
public required PointF Start { get; init; }
```

## Property Value

PointF

# Class NormalizePathExtensions

Namespace: SixLabors.ImageSharp.Drawing

Assembly: SixLabors.ImageSharp.Drawing.dll

Provides extension methods to IPath that allow paths to be normalized.

```
public static class NormalizePathExtensions
```

**Inheritance**

object ← NormalizePathExtensions

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

# Methods

## Normalize(IPath)

Creates a normalized positive-winding polygon representation of the specified path.

```
public static IPath Normalize(this IPath path)
```

## Parameters

`path` IPath

The path to normalize.

## Returns

IPath

A path whose contours represent the normalized filled area of `path` using positive winding.

# Class OutlinePathExtensions

Namespace: SixLabors.ImageSharp.Drawing

Assembly: SixLabors.ImageSharp.Drawing.dll

Extensions to IPath that allow the generation of outlines.

```
public static class OutlinePathExtensions
```

**Inheritance**

object ← OutlinePathExtensions

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

# Methods

## GenerateOutline(IPath, float)

Generates an outline of the path.

```
public static IPath GenerateOutline(this IPath path, float width)
```

## Parameters

`path` IPath

The path to outline.

`width` float

The outline width.

## Returns

IPath

A new IPath representing the outline.

## GenerateOutline(IPath, float, StrokeOptions)

Generates an outline of the path.

```
public static IPath GenerateOutline(this IPath path, float width,
StrokeOptions strokeOptions)
```

### Parameters

`path` IPath

The path to outline.

`width` float

The outline width.

`strokeOptions` StrokeOptions

The stroke geometry options.

### Returns

IPath

A new IPath representing the outline.

## GenerateOutline(IPath, float, ReadOnlySpan<float>)

Generates an outline of the path with alternating on and off segments based on the pattern.

```
public static IPath GenerateOutline(this IPath path, float width,
ReadOnlySpan<float> pattern)
```

### Parameters

`path` IPath

The path to outline.

`width` float

The outline width.

`pattern` ReadOnlySpan<float>

The pattern made of multiples of the width.

## Returns

IPath

A new IPath representing the outline.

# GenerateOutline(IPath, float, ReadOnlySpan<float>, StrokeOptions)

Generates an outline of the path with alternating on and off segments based on the pattern.

```
public static IPath GenerateOutline(this IPath path, float width, ReadOnlySpan<float>
pattern, StrokeOptions strokeOptions)
```

## Parameters

`path` IPath

The path to outline.

`width` float

The outline width.

`pattern` ReadOnlySpan<float>

The pattern made of multiples of the width.

`strokeOptions` StrokeOptions

The stroke geometry options.

## Returns

IPath

A new IPath representing the outline.

# GenerateOutline(IPath, float, ReadOnlySpan<float>, bool)

Generates an outline of the path with alternating on and off segments based on the pattern.

```
public static IPath GenerateOutline(this IPath path, float width, ReadOnlySpan<float>
pattern, bool startOff)
```

## Parameters

`path` IPath

The path to outline.

`width` float

The outline width.

`pattern` ReadOnlySpan<float>

The pattern made of multiples of the width.

`startOff` bool

Whether the first item in the pattern is on or off.

## Returns

IPath

A new IPath representing the outline.

# GenerateOutline(IPath, float, ReadOnlySpan<float>, bool, StrokeOptions)

Generates an outline of the path with alternating on and off segments based on the pattern.

```
public static IPath GenerateOutline(this IPath path, float width, ReadOnlySpan<float>
pattern, bool startOff, StrokeOptions strokeOptions)
```

## Parameters

`path` IPath

The path to outline.

`width` float

The outline width.

`pattern` ReadOnlySpan<float>

The pattern made of multiples of the width.

`startOff` bool

Whether the first item in the pattern is on or off.

`strokeOptions` StrokeOptions

The stroke geometry options.

## Returns

IPath

A new IPath representing the outline.

# GenerateOutline(IPath, float, ReadOnlySpan<float>, bool, float)

Generates an outline of the path with alternating on and off segments based on the pattern.

```
public static IPath GenerateOutline(this IPath path, float width, ReadOnlySpan<float>
pattern, bool startOff, float offset)
```

## Parameters

`path` IPath

The path to outline.

`width` float

The outline width.

`pattern` ReadOnlySpan<float>

The pattern made of multiples of the width.

`startOff` bool

Whether the first item in the pattern is on or off.

`offset` float

The distance into the dash pattern, expressed as a multiple of `width`.

## Returns

IPath

A new IPath representing the outline.

# GenerateOutline(IPath, float, ReadOnlySpan<float>, bool, float, StrokeOptions)

Generates an outline of the path with alternating on and off segments based on the pattern.

```
public static IPath GenerateOutline(this IPath path, float width, ReadOnlySpan<float>
pattern, bool startOff, float offset, StrokeOptions strokeOptions)
```

## Parameters

`path` IPath

The path to outline.

`width` float

The outline width.

`pattern` ReadOnlySpan<float>

The pattern made of multiples of the width.

`startOff` bool

Whether the first item in the pattern is on or off.

`offset` float

The distance into the dash pattern, expressed as a multiple of `width`.

`strokeOptions` StrokeOptions

The stroke geometry options.

## Returns

IPath

A new IPath representing the outline.

# GenerateOutline(IPath, float, ReadOnlySpan<float>, float)

Generates an outline of the path with alternating on and off segments based on the pattern.

```
public static IPath GenerateOutline(this IPath path, float width, ReadOnlySpan<float>
pattern, float offset)
```

## Parameters

`path` IPath

The path to outline.

`width` float

The outline width.

`pattern` ReadOnlySpan<float>

The pattern made of multiples of the width.

`offset` float

The distance into the dash pattern, expressed as a multiple of `width`.

## Returns

IPath

A new IPath representing the outline.

# GenerateOutline(IPath, float, ReadOnlySpan<float>, float, StrokeOptions)

Generates an outline of the path with alternating on and off segments based on the pattern.

```
public static IPath GenerateOutline(this IPath path, float width, ReadOnlySpan<float>
pattern, float offset, StrokeOptions strokeOptions)
```

## Parameters

`path` IPath

The path to outline.

`width` float

The outline width.

`pattern` ReadOnlySpan<float>

The pattern made of multiples of the width.

`offset` float

The distance into the dash pattern, expressed as a multiple of `width`.

`strokeOptions` StrokeOptions

The stroke geometry options.

## Returns

IPath

A new IPath representing the outline.

# Class Path

Namespace: SixLabors.ImageSharp.Drawing

Assembly: SixLabors.ImageSharp.Drawing.dll

An aggregate of ILineSegments making a single logical path.

```
public class Path : IPath, ISimplePath
```

### Inheritance

object ← Path

### Implements

IPath, ISimplePath

### Derived

Polygon

### Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

### Extension Methods

ClipPathExtensions.Clip(IPath, BooleanOperation, params IPath[]) , ClipPathExtensions.Clip(IPath, BooleanOperation, IntersectionRule, params IPath[]) , ClipPathExtensions.Clip(IPath, BooleanOperation, IntersectionRule, IEnumerable<IPath>) , ClipPathExtensions.Clip(IPath, BooleanOperation, IEnumerable<IPath>) , ClipPathExtensions.Clip(IPath, params IPath[]) , ClipPathExtensions.Clip(IPath, IEnumerable<IPath>) , NormalizePathExtensions.Normalize(IPath) , OutlinePathExtensions.GenerateOutline(IPath, float) , OutlinePathExtensions.GenerateOutline(IPath, float, StrokeOptions) , OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>) , OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, StrokeOptions) , OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool) , OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool, StrokeOptions) , OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool, float) , OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool, float, StrokeOptions) , OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, float) ,

OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, float, StrokeOptions) , PathExtensions.ComputeArea(IPath) , PathExtensions.ComputeLength(IPath) , PathExtensions.Contains(IPath, PointF, IntersectionRule) , PathExtensions.Rotate(IPath, float) , PathExtensions.RotateDegree(IPath, float) , PathExtensions.Scale(IPath, float) , PathExtensions.Scale(IPath, float, float) , PathExtensions.Translate(IPath, PointF) , PathExtensions.Translate(IPath, float, float) , PathExtensions.TryGetPathPointAtDistance(IPath, float, out PathPoint) , PathExtensions.TryGetPathPointAtDistanceUnbounded(IPath, float, out PathPoint) , PathExtensions.TryGetSegment(IPath, float, float, bool, out IPath) , SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>) , SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>, bool) , SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>, bool, float) , SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>, float)

# Constructors

## Path(params ILineSegment[])

Initializes a new instance of the Path class.

```
public Path(params ILineSegment[] segments)
```

### Parameters

`segments` ILineSegment[]

The segments.

## Path(Path)

Initializes a new instance of the Path class.

```
public Path(Path path)
```

### Parameters

`path` Path

The path.

## Path(PointF[])

Initializes a new instance of the Path class.

```
public Path(PointF[] points)
```

### Parameters

`points` PointF[]

The collection of points; processed as a series of linear line segments.

## Path(IEnumerable<ILineSegment>)

Initializes a new instance of the Path class.

```
public Path(IEnumerable<ILineSegment> segments)
```

### Parameters

`segments` IEnumerable<ILineSegment>

The segments.

# Properties

## Bounds

Gets the bounds enclosing the path.

```
public RectangleF Bounds { get; }
```

### Property Value

RectangleF

## Empty

Gets the default empty path.

```
public static IPath Empty { get; }
```

### Property Value

IPath

## IsClosed

Gets a value indicating whether this instance is a closed path.

```
public virtual bool IsClosed { get; }
```

### Property Value

bool

## LineSegments

Gets readonly collection of line segments.

```
public IReadOnlyList<ILineSegment> LineSegments { get; }
```

### Property Value

IReadOnlyList<ILineSegment>

## PathType

Gets a value indicating whether this instance is closed, open or a composite path with a mixture of open and closed figures.

```
public PathTypes PathType { get; }
```

### Property Value

PathTypes

## Points

Gets the points that make this up as a simple linear path.

```
public ReadOnlyMemory<PointF> Points { get; }
```

### Property Value

ReadOnlyMemory<PointF>

# Methods

## AsClosedPath()

Returns this path with all figures closed.

```
public IPath AsClosedPath()
```

### Returns

IPath

A new closed IPath.

## ComputeArea(Vector2)

Calculates the non-negative path area after flattening curves at the supplied scale.

```
public virtual float ComputeArea(Vector2 scale)
```

## Parameters

`scale` Vector2

The X/Y scale at which curves are flattened for area measurement.

## Returns

float

The path area.

# ComputeLength(Vector2)

Calculates the path length after flattening curves at the supplied scale.

```
public virtual float ComputeLength(Vector2 scale)
```

## Parameters

`scale` Vector2

The X/Y scale at which curves are flattened for length measurement.

## Returns

float

The path length.

# Contains(PointF, IntersectionRule, Vector2)

Returns whether the supplied point is inside the path.

```
public virtual bool Contains(PointF point, IntersectionRule intersectionRule, Vector2 scale)
```

## Parameters

`point` PointF

The point to test.

`intersectionRule` IntersectionRule

The fill rule used for containment.

`scale` Vector2

The X/Y scale at which curves are flattened for the containment test.

## Returns

bool

true when the point is inside or on the path boundary.

# Flatten()

Converts the IPath into a simple linear path.

```
public IEnumerable<ISimplePath> Flatten()
```

## Returns

IEnumerable<ISimplePath>

Returns the current IPath as simple linear path.

# ToLinearGeometry(Vector2)

Converts this path into a retained LinearGeometry, flattening curves at the precision of the supplied device-space `scale`.

```
public virtual LinearGeometry ToLinearGeometry(Vector2 scale)
```

### Parameters

`scale` Vector2

The X/Y scale at which curves are flattened.

### Returns

LinearGeometry

The retained linear geometry.

## Transform(Matrix4×4)

Transforms the path using the specified matrix.

```
public virtual IPath Transform(Matrix4x4 matrix)
```

### Parameters

`matrix` Matrix4×4

The matrix.

### Returns

IPath

A new path with the matrix applied to it.

## TryGetPathPointAtDistance(float, Vector2, out PathPoint)

Gets path information at the specified distance along the path.

```
public virtual bool TryGetPathPointAtDistance(float distance, Vector2 scale, out
```

```
PathPoint pathPoint)
```

## Parameters

`distance` float

The distance along the path.

`scale` Vector2

The X/Y scale at which curves are flattened for distance measurement.

`pathPoint` PathPoint

When this method returns, contains the path information at `distance` if the distance resolves to a point on the path; otherwise, the default value.

## Returns

bool

true if `distance` resolves to a point on the path; otherwise, false.

# TryGetPathPointAtDistanceUnbounded(float, Vector2, out PathPoint)

Gets path information at the specified distance along the path, extrapolating along the boundary tangents when the distance falls before the start or beyond the end of an open path.

```
public virtual bool TryGetPathPointAtDistanceUnbounded(float distance, Vector2 scale, out
PathPoint pathPoint)
```

## Parameters

`distance` float

The distance along the path.

`scale` Vector2

The X/Y scale at which curves are flattened for distance measurement.

`pathPoint` PathPoint

When this method returns, contains the path information at `distance` if the path is measurable; otherwise, the default value.

## Returns

bool

true if the point could be resolved; otherwise, false.

## Remarks

Unlike TryGetPathPointAtDistance(float, Vector2, out PathPoint), out-of-range distances resolve to a point on the virtual straight-line continuation of the path's first or last segment. Text layout uses this to place aligned glyphs that overflow their layout path. The method returns false only when the distance is not a finite number or the path has no measurable length.

# TryGetSegment(float, float, bool, Vector2, out IPath)

Creates a path segment between two distances along the path.

```
public virtual bool TryGetSegment(float startDistance, float stopDistance, bool
startOnBeginFigure, Vector2 scale, out IPath path)
```

## Parameters

`startDistance` float

The segment start distance.

`stopDistance` float

The segment stop distance.

`startOnBeginFigure` bool

Whether the returned segment starts a new figure at the first segment point.

`scale` Vector2

The X/Y scale at which curves are flattened for segment extraction.

`path` IPath

When this method returns, contains the segment path if one was created; otherwise, an empty path.

### Returns

bool

true when a segment path was created.

## TryParseSvgPath(ReadOnlySpan<char>, out IPath?)

Converts an SVG path string into an IPath.

```
public static bool TryParseSvgPath(ReadOnlySpan<char> svgPath, out IPath? value)
```

### Parameters

`svgPath` ReadOnlySpan<char>

The string containing the SVG path data.

`value` IPath

When this method returns, contains the logic path converted from the given SVG path string; otherwise, null. This parameter is passed uninitialized.

### Returns

bool

true if the input value can be parsed and converted; otherwise, false.

## TryParseSvgPath(string, out IPath?)

Converts an SVG path string into an IPath.

```
public static bool TryParseSvgPath(string svgPath, out IPath? value)
```

## Parameters

`svgPath` string

The string containing the SVG path data.

`value` IPath

When this method returns, contains the logic path converted from the given SVG path string; otherwise, null. This parameter is passed uninitialized.

## Returns

bool

true if the input value can be parsed and converted; otherwise, false.

# See Also

IPath

# Class PathBuilder

Namespace: SixLabors.ImageSharp.Drawing

Assembly: SixLabors.ImageSharp.Drawing.dll

Provides a fluent API for incrementally building complex shapes and paths from figures composed of lines, arcs, and bezier curves.

```
public class PathBuilder
```

**Inheritance**

object ← PathBuilder

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## PathBuilder()

Initializes a new instance of the PathBuilder class.

```
public PathBuilder()
```

## PathBuilder(Matrix4×4)

Initializes a new instance of the PathBuilder class.

```
public PathBuilder(Matrix4x4 defaultTransform)
```

### Parameters

`defaultTransform` Matrix4×4

The default transform. This is always composed with any transform set via SetTransform(Matrix4×4) or SetOrigin(PointF).

# Properties

## Transform

Gets the current transformation matrix.

```
public Matrix4x4 Transform { get; }
```

### Property Value

Matrix4×4

The current transformation matrix.

### Remarks

Returns a copy of the matrix. Because Matrix4×4 is a value type, modifications to the returned value do not affect the internal state. To change the transform, call SetTransform(Matrix4×4).

# Methods

## AddArc(Point, int, int, int, int, int)

Adds an elliptical arc to the current figure.

```
public PathBuilder AddArc(Point center, int radiusX, int radiusY, int rotation, int
startAngle, int sweepAngle)
```

### Parameters

`center` Point

The center Point of the ellipse from which the arc is taken.

`radiusX` int

The x-radius of the ellipse.

`radiusY` int

The y-radius of the ellipse.

`rotation` int

The angle, in degrees, from the x-axis of the current coordinate system to the x-axis of the ellipse.

`startAngle` int

The start angle of the elliptical arc prior to the stretch and rotate operations. (0 is at the 3 o'clock position of the arc's circle).

`sweepAngle` int

The angle between `startAngle` and the end of the arc.

## Returns

PathBuilder

The PathBuilder.

# AddArc(PointF, float, float, float, bool, bool, PointF)

Adds an elliptical arc to the current figure. The arc curves from the `startPoint` to `endPoint`, choosing one of four possible routes: clockwise or counterclockwise, and smaller or larger.

The arc sweep is always less than 360 degrees. The method appends a line to the last point if either radii are zero, or if last point is equal to `endPoint`. In addition the method scales the radii to fit last point and `endPoint` if both are greater than zero but too small to describe an arc.

```
public PathBuilder AddArc(PointF startPoint, float radiusX, float radiusY, float rotation,
bool largeArc, bool sweep, PointF endPoint)
```

## Parameters

`startPoint` PointF

The start point of the arc.

`radiusX` float

The x-radius of the ellipse.

`radiusY` float

The y-radius of the ellipse.

`rotation` float

The rotation along the X-axis; measured in degrees clockwise.

`largeArc` bool

The large arc flag, and is false if an arc spanning less than or equal to 180 degrees is chosen, or true if an arc spanning greater than 180 degrees is chosen.

`sweep` bool

The sweep flag, and is false if the line joining center to arc sweeps through decreasing angles, or true if it sweeps through increasing angles.

`endPoint` PointF

The end point of the arc.

## Returns

PathBuilder

The PathBuilder.

# AddArc(PointF, float, float, float, float, float)

Adds an elliptical arc to the current figure.

```
public PathBuilder AddArc(PointF center, float radiusX, float radiusY, float rotation, float
```

```
startAngle, float sweepAngle)
```

## Parameters

`center` PointF

The center PointF of the ellipse from which the arc is taken.

`radiusX` float

The x-radius of the ellipse.

`radiusY` float

The y-radius of the ellipse.

`rotation` float

The angle, in degrees, from the x-axis of the current coordinate system to the x-axis of the ellipse.

`startAngle` float

The start angle of the elliptical arc prior to the stretch and rotate operations. (0 is at the 3 o'clock position of the arc's circle).

`sweepAngle` float

The angle between `startAngle` and the end of the arc.

## Returns

PathBuilder

The PathBuilder.

# AddArc(Rectangle, int, int, int)

Adds an elliptical arc to the current figure.

```
public PathBuilder AddArc(Rectangle rectangle, int rotation, int startAngle, int sweepAngle)
```

## Parameters

`rectangle` Rectangle

A Rectangle that represents the rectangular bounds of the ellipse from which the arc is taken.

`rotation` int

The angle, in degrees, from the x-axis of the current coordinate system to the x-axis of the ellipse.

`startAngle` int

The start angle of the elliptical arc prior to the stretch and rotate operations. (0 is at the 3 o'clock position of the arc's circle).

`sweepAngle` int

The angle between `startAngle` and the end of the arc.

## Returns

PathBuilder

The PathBuilder.

# AddArc(RectangleF, float, float, float)

Adds an elliptical arc to the current figure.

```
public PathBuilder AddArc(RectangleF rectangle, float rotation, float startAngle,
float sweepAngle)
```

## Parameters

`rectangle` RectangleF

A RectangleF that represents the rectangular bounds of the ellipse from which the arc is taken.

`rotation` float

The angle, in degrees, from the x-axis of the current coordinate system to the x-axis of the ellipse.

`startAngle` float

The start angle of the elliptical arc prior to the stretch and rotate operations. (0 is at the 3 o'clock position of the arc's circle).

`sweepAngle` float

The angle between `startAngle` and the end of the arc.

### Returns

PathBuilder

The PathBuilder.

## AddArc(int, int, int, int, int, int, int)

Adds an elliptical arc to the current figure.

```
public PathBuilder AddArc(int x, int y, int radiusX, int radiusY, int rotation, int
startAngle, int sweepAngle)
```

### Parameters

`x` int

The x-coordinate of the center point of the ellipse from which the arc is taken.

`y` int

The y-coordinate of the center point of the ellipse from which the arc is taken.

`radiusX` int

The x-radius of the ellipse.

`radiusY` int

The y-radius of the ellipse.

`rotation` int

The angle, in degrees, from the x-axis of the current coordinate system to the x-axis of the ellipse.

`startAngle` int

The start angle of the elliptical arc prior to the stretch and rotate operations. (0 is at the 3 o'clock position of the arc's circle).

`sweepAngle` int

The angle between `startAngle` and the end of the arc.

## Returns

PathBuilder

The PathBuilder.

# AddArc(float, float, float, float, float, float, float)

Adds an elliptical arc to the current figure.

```
public PathBuilder AddArc(float x, float y, float radiusX, float radiusY, float rotation,
float startAngle, float sweepAngle)
```

## Parameters

`x` float

The x-coordinate of the center point of the ellipse from which the arc is taken.

`y` float

The y-coordinate of the center point of the ellipse from which the arc is taken.

`radiusX` float

The x-radius of the ellipse.

`radiusY` float

The y-radius of the ellipse.

`rotation` float

The angle, in degrees, from the x-axis of the current coordinate system to the x-axis of the ellipse.

`startAngle` float

The start angle of the elliptical arc prior to the stretch and rotate operations. (0 is at the 3 o'clock position of the arc's circle).

`sweepAngle` float

The angle between `startAngle` and the end of the arc.

## Returns

PathBuilder

The PathBuilder.

# AddCubicBezier(PointF, PointF, PointF, PointF)

Adds a cubic bezier curve to the current figure joining the `startPoint` point to the `endPoint`.

```
public PathBuilder AddCubicBezier(PointF startPoint, PointF controlPoint1, PointF
controlPoint2, PointF endPoint)
```

## Parameters

`startPoint` PointF

The start point.

`controlPoint1` PointF

The first control point.

`controlPoint2` PointF

The second control point.

`endPoint` PointF

The end point.

### Returns

PathBuilder

The PathBuilder.

## AddLine(PointF, PointF)

Adds a line segment connecting the two points to the current figure.

```
public PathBuilder AddLine(PointF start, PointF end)
```

### Parameters

`start` PointF

The start point of the line.

`end` PointF

The end point of the line.

### Returns

PathBuilder

The PathBuilder.

## AddLine(float, float, float, float)

Adds a line segment connecting the two points to the current figure.

```
public PathBuilder AddLine(float x1, float y1, float x2, float y2)
```

### Parameters

`x1` float

The x-coordinate of the start point.

`y1` float

The y-coordinate of the start point.

`x2` float

The x-coordinate of the end point.

`y2` float

The y-coordinate of the end point.

## Returns

PathBuilder

The PathBuilder.

# AddLines(params PointF[])

Adds a series of line segments connecting the supplied points to the current figure.

```
public PathBuilder AddLines(params PointF[] points)
```

## Parameters

`points` PointF[]

The points to connect.

## Returns

PathBuilder

The PathBuilder.

# AddLines(IEnumerable<PointF>)

Adds a series of line segments connecting the supplied points to the current figure.

```
public PathBuilder AddLines(IEnumerable<PointF> points)
```

## Parameters

`points` IEnumerable<PointF>

The points to connect.

## Returns

PathBuilder

The PathBuilder.

# AddPie(PointF, SizeF, float, float)

Adds a pie sector to the current path as a closed figure.

```
public PathBuilder AddPie(PointF center, SizeF radius, float startAngle, float sweepAngle)
```

## Parameters

`center` PointF

The center point of the pie sector.

`radius` SizeF

The x and y radii of the pie ellipse.

`startAngle` float

The pie start angle in degrees.

`sweepAngle` float

The pie sweep angle in degrees.

## Returns

PathBuilder

The PathBuilder.

## AddPie(PointF, SizeF, float, float, float)

Adds a pie sector to the current path as a closed figure.

```
public PathBuilder AddPie(PointF center, SizeF radius, float rotation, float startAngle,
float sweepAngle)
```

### Parameters

`center` PointF

The center point of the pie sector.

`radius` SizeF

The x and y radii of the pie ellipse.

`rotation` float

The ellipse rotation in degrees.

`startAngle` float

The pie start angle in degrees.

`sweepAngle` float

The pie sweep angle in degrees.

### Returns

PathBuilder

The PathBuilder.

## AddPie(float, float, float, float, float, float)

Adds a pie sector to the current path as a closed figure.

```
public PathBuilder AddPie(float x, float y, float radiusX, float radiusY, float startAngle,
float sweepAngle)
```

## Parameters

`x` float

The x-coordinate of the pie center.

`y` float

The y-coordinate of the pie center.

`radiusX` float

The x-radius of the pie ellipse.

`radiusY` float

The y-radius of the pie ellipse.

`startAngle` float

The pie start angle in degrees.

`sweepAngle` float

The pie sweep angle in degrees.

## Returns

PathBuilder

The PathBuilder.

# AddPie(float, float, float, float, float, float, float)

Adds a pie sector to the current path as a closed figure.

```
public PathBuilder AddPie(float x, float y, float radiusX, float radiusY, float rotation,
```

```
float startAngle, float sweepAngle)
```

## Parameters

`x` float

The x-coordinate of the pie center.

`y` float

The y-coordinate of the pie center.

`radiusX` float

The x-radius of the pie ellipse.

`radiusY` float

The y-radius of the pie ellipse.

`rotation` float

The ellipse rotation in degrees.

`startAngle` float

The pie start angle in degrees.

`sweepAngle` float

The pie sweep angle in degrees.

## Returns

PathBuilder

The PathBuilder.

# AddPolygon(params PointF[])

Adds a polygon to the current path as a closed figure.

```
public PathBuilder AddPolygon(params PointF[] points)
```

## Parameters

`points` PointF[]

The polygon vertices.

## Returns

PathBuilder

The PathBuilder.

# AddPolygon(IEnumerable<PointF>)

Adds a polygon to the current path as a closed figure.

```
public PathBuilder AddPolygon(IEnumerable<PointF> points)
```

## Parameters

`points` IEnumerable<PointF>

The polygon vertices.

## Returns

PathBuilder

The PathBuilder.

# AddQuadraticBezier(PointF, PointF, PointF)

Adds a quadratic bezier curve to the current figure joining the `startPoint` point to the `endPoint`.

```
public PathBuilder AddQuadraticBezier(PointF startPoint, PointF controlPoint,
PointF endPoint)
```

## Parameters

`startPoint` PointF

The start point.

`controlPoint` PointF

The control point.

`endPoint` PointF

The end point.

### Returns

PathBuilder

The PathBuilder.

## AddRectangle(Rectangle)

Adds a rectangle to the current path as a closed figure.

```
public PathBuilder AddRectangle(Rectangle rectangle)
```

### Parameters

`rectangle` Rectangle

The rectangle bounds.

### Returns

PathBuilder

The PathBuilder.

## AddRectangle(RectangleF)

Adds a rectangle to the current path as a closed figure.

```
public PathBuilder AddRectangle(RectangleF rectangle)
```

## Parameters

`rectangle` RectangleF

The rectangle bounds.

## Returns

PathBuilder

The PathBuilder.

# AddRectangle(float, float, float, float)

Adds a rectangle to the current path as a closed figure.

```
public PathBuilder AddRectangle(float x, float y, float width, float height)
```

## Parameters

`x` float

The x-coordinate of the rectangle.

`y` float

The y-coordinate of the rectangle.

`width` float

The rectangle width.

`height` float

The rectangle height.

## Returns

PathBuilder

The PathBuilder.

## AddRegularPolygon(PointF, int, float)

Adds a regular polygon to the current path as a closed figure.

```
public PathBuilder AddRegularPolygon(PointF center, int vertices, float radius)
```

### Parameters

`center` PointF

The center point of the polygon.

`vertices` int

The number of polygon vertices.

`radius` float

The polygon radius.

### Returns

PathBuilder

The PathBuilder.

## AddRegularPolygon(PointF, int, float, float)

Adds a regular polygon to the current path as a closed figure.

```
public PathBuilder AddRegularPolygon(PointF center, int vertices, float radius, float angle)
```

### Parameters

`center` PointF

The center point of the polygon.

`vertices` int

The number of polygon vertices.

`radius` float

The polygon radius.

`angle` float

The polygon rotation angle in degrees.

## Returns

PathBuilder

The PathBuilder.

# AddRegularPolygon(float, float, int, float)

Adds a regular polygon to the current path as a closed figure.

```
public PathBuilder AddRegularPolygon(float x, float y, int vertices, float radius)
```

## Parameters

`x` float

The x-coordinate of the polygon center.

`y` float

The y-coordinate of the polygon center.

`vertices` int

The number of polygon vertices.

`radius` float

The polygon radius.

## Returns

PathBuilder

The PathBuilder.

## AddRegularPolygon(float, float, int, float, float)

Adds a regular polygon to the current path as a closed figure.

```
public PathBuilder AddRegularPolygon(float x, float y, int vertices, float radius,
float angle)
```

### Parameters

x float

The x-coordinate of the polygon center.

y float

The y-coordinate of the polygon center.

vertices int

The number of polygon vertices.

radius float

The polygon radius.

angle float

The polygon rotation angle in degrees.

### Returns

PathBuilder

The PathBuilder.

## AddRoundedRectangle(Rectangle, SizeF)

Adds a rounded rectangle to the current path as a closed figure.

```
public PathBuilder AddRoundedRectangle(Rectangle rectangle, SizeF radius)
```

## Parameters

`rectangle` Rectangle

The rectangle bounds.

`radius` SizeF

The x and y radii of each corner.

## Returns

PathBuilder

The PathBuilder.

# AddRoundedRectangle(Rectangle, float)

Adds a rounded rectangle to the current path as a closed figure.

```
public PathBuilder AddRoundedRectangle(Rectangle rectangle, float radius)
```

## Parameters

`rectangle` Rectangle

The rectangle bounds.

`radius` float

The x and y radius of each corner.

## Returns

PathBuilder

The PathBuilder.

## AddRoundedRectangle(RectangleF, SizeF)

Adds a rounded rectangle to the current path as a closed figure.

```
public PathBuilder AddRoundedRectangle(RectangleF rectangle, SizeF radius)
```

### Parameters

`rectangle` RectangleF

The rectangle bounds.

`radius` SizeF

The x and y radii of each corner.

### Returns

PathBuilder

The PathBuilder.

## AddRoundedRectangle(RectangleF, float)

Adds a rounded rectangle to the current path as a closed figure.

```
public PathBuilder AddRoundedRectangle(RectangleF rectangle, float radius)
```

### Parameters

`rectangle` RectangleF

The rectangle bounds.

`radius` float

The x and y radius of each corner.

### Returns

PathBuilder

The PathBuilder.

## AddRoundedRectangle(float, float, float, float, SizeF)

Adds a rounded rectangle to the current path as a closed figure.

```
public PathBuilder AddRoundedRectangle(float x, float y, float width, float height,
SizeF radius)
```

### Parameters

x float

The x-coordinate of the rectangle.

y float

The y-coordinate of the rectangle.

width float

The rectangle width.

height float

The rectangle height.

radius SizeF

The x and y radii of each corner.

### Returns

PathBuilder

The PathBuilder.

## AddRoundedRectangle(float, float, float, float, float)

Adds a rounded rectangle to the current path as a closed figure.

```
public PathBuilder AddRoundedRectangle(float x, float y, float width, float height,
float radius)
```

## Parameters

`x` float

The x-coordinate of the rectangle.

`y` float

The y-coordinate of the rectangle.

`width` float

The rectangle width.

`height` float

The rectangle height.

`radius` float

The x and y radius of each corner.

## Returns

PathBuilder

The PathBuilder.

# AddSegment(ILineSegment)

Adds a segment to the current figure. The segment is transformed by the current transform and the current point is moved to the segment end point.

```
public PathBuilder AddSegment(ILineSegment segment)
```

## Parameters

`segment` ILineSegment

The segment to add.

### Returns

PathBuilder

The PathBuilder.

## AddStar(PointF, int, float, float)

Adds a star to the current path as a closed figure.

```
public PathBuilder AddStar(PointF center, int prongs, float innerRadii, float outerRadii)
```

### Parameters

`center` PointF

The center point of the star.

`prongs` int

The number of star prongs.

`innerRadii` float

The inner star radius.

`outerRadii` float

The outer star radius.

### Returns

PathBuilder

The PathBuilder.

## AddStar(PointF, int, float, float, float)

Adds a star to the current path as a closed figure.

```
public PathBuilder AddStar(PointF center, int prongs, float innerRadii, float outerRadii,
float angle)
```

## Parameters

`center` PointF

The center point of the star.

`prongs` int

The number of star prongs.

`innerRadii` float

The inner star radius.

`outerRadii` float

The outer star radius.

`angle` float

The star rotation angle in degrees.

## Returns

PathBuilder

The PathBuilder.

# AddStar(float, float, int, float, float)

Adds a star to the current path as a closed figure.

```
public PathBuilder AddStar(float x, float y, int prongs, float innerRadii, float outerRadii)
```

## Parameters

`x` float

The x-coordinate of the star center.

`y` float

The y-coordinate of the star center.

`prongs` int

The number of star prongs.

`innerRadii` float

The inner star radius.

`outerRadii` float

The outer star radius.

## Returns

PathBuilder

The PathBuilder.

# AddStar(float, float, int, float, float, float)

Adds a star to the current path as a closed figure.

```
public PathBuilder AddStar(float x, float y, int prongs, float innerRadii, float outerRadii,
float angle)
```

## Parameters

`x` float

The x-coordinate of the star center.

`y` float

The y-coordinate of the star center.

`prongs` int

The number of star prongs.

`innerRadii` float

The inner star radius.

`outerRadii` float

The outer star radius.

`angle` float

The star rotation angle in degrees.

## Returns

PathBuilder

The PathBuilder.

# ArcTo(float, float, float, bool, bool, PointF)

Adds an elliptical arc to the current figure. The arc curves from the last point to `point`, choosing one of four possible routes: clockwise or counterclockwise, and smaller or larger.

The arc sweep is always less than 360 degrees. The method appends a line to the last point if either radii are zero, or if last point is equal to `point`. In addition the method scales the radii to fit last point and `point` if both are greater than zero but too small to describe an arc.

```
public PathBuilder ArcTo(float radiusX, float radiusY, float rotation, bool largeArc, bool
sweep, PointF point)
```

## Parameters

`radiusX` float

The x-radius of the ellipse.

`radiusY` float

The y-radius of the ellipse.

`rotation` float

The rotation along the X-axis; measured in degrees clockwise.

`largeArc` bool

The large arc flag, and is false if an arc spanning less than or equal to 180 degrees is chosen, or true if an arc spanning greater than 180 degrees is chosen.

`sweep` bool

The sweep flag, and is false if the line joining center to arc sweeps through decreasing angles, or true if it sweeps through increasing angles.

`point` PointF

The end point of the arc.

### Returns

PathBuilder

The PathBuilder.

## Build()

Builds a path from the current working set of figures.

```
public IPath Build()
```

### Returns

IPath

The built IPath. A single non-empty figure is returned directly; multiple figures are combined into a ComplexPolygon.

## Clear()

Clears all drawn paths, leaving any applied transforms.

```
[MemberNotNull("currentFigure")]
public void Clear()
```

## CloseAllFigures()

Closes all figures in the path, including the current figure.

```
public PathBuilder CloseAllFigures()
```

### Returns

PathBuilder

The PathBuilder.

## CloseFigure()

Closes the current figure.

```
public PathBuilder CloseFigure()
```

### Returns

PathBuilder

The PathBuilder.

## CubicBezierTo(Vector2, Vector2, Vector2)

Draws a cubic bezier from the current point to the `point`

```
public PathBuilder CubicBezierTo(Vector2 secondControlPoint, Vector2 thirdControlPoint,
Vector2 point)
```

### Parameters

`secondControlPoint` Vector2

The second control point. The current point acts as the first.

`thirdControlPoint` Vector2

The third control point.

`point` Vector2

The end point of the curve.

### Returns

PathBuilder

The PathBuilder.

## LineTo(PointF)

Draws a line connecting the current point to the new point.

```
public PathBuilder LineTo(PointF point)
```

### Parameters

`point` PointF

The end point of the line.

### Returns

PathBuilder

The PathBuilder.

## LineTo(float, float)

Draws a line connecting the current point to the new point.

```
public PathBuilder LineTo(float x, float y)
```

## Parameters

`x` float

The x-coordinate of the end point.

`y` float

The y-coordinate of the end point.

## Returns

PathBuilder

The PathBuilder

# MoveTo(PointF)

Moves the current point to the supplied point, starting a new figure.

```
public PathBuilder MoveTo(PointF point)
```

## Parameters

`point` PointF

The point to move to.

## Returns

PathBuilder

The PathBuilder.

# MoveTo(float, float)

Moves the current point to the supplied point, starting a new figure.

```
public PathBuilder MoveTo(float x, float y)
```

## Parameters

`x` float

The x-coordinate of the point to move to.

`y` float

The y-coordinate of the point to move to.

## Returns

PathBuilder

The PathBuilder

# QuadraticBezierTo(Vector2, Vector2)

Draws a quadratic bezier from the current point to the `point`

```
public PathBuilder QuadraticBezierTo(Vector2 secondControlPoint, Vector2 point)
```

## Parameters

`secondControlPoint` Vector2

The second control point. The current point acts as the first.

`point` Vector2

The end point of the curve.

## Returns

PathBuilder

The PathBuilder.

## Reset()

Resets this instance, clearing any drawn paths and resetting any transforms.

```
public PathBuilder Reset()
```

### Returns

PathBuilder

The PathBuilder.

## ResetOrigin()

Resets the origin to the default.

```
public PathBuilder ResetOrigin()
```

### Returns

PathBuilder

The PathBuilder.

## ResetTransform()

Resets the transform to the default.

```
public PathBuilder ResetTransform()
```

### Returns

PathBuilder

The PathBuilder.

## SetOrigin(PointF)

Sets the origin that all subsequent points are relative to.

```
public PathBuilder SetOrigin(PointF origin)
```

### Parameters

`origin` PointF

The origin.

### Returns

PathBuilder

The PathBuilder.

## SetTransform(Matrix4×4)

Sets the transform to apply to all subsequently added segments. The supplied transform is composed with the default transform provided at construction.

```
public PathBuilder SetTransform(Matrix4x4 transform)
```

### Parameters

`transform` Matrix4×4

The transform to apply.

### Returns

PathBuilder

The PathBuilder.

## StartFigure()

Starts a new figure but leaves the previous one open.

```
public PathBuilder StartFigure()
```

## Returns

PathBuilder

The PathBuilder.

# Class PathCollection

Namespace: SixLabors.ImageSharp.Drawing

Assembly: SixLabors.ImageSharp.Drawing.dll

An aggregate of IPaths to apply common operations to them.

```
public class PathCollection : IPathCollection, IEnumerable<IPath>, IEnumerable
```

### Inheritance

object ← PathCollection

### Implements

IPathCollection, IEnumerable<IPath>, IEnumerable

### Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

### Extension Methods

PathExtensions.ComputeArea(IPathCollection) , PathExtensions.Rotate(IPathCollection, float) , PathExtensions.RotateDegree(IPathCollection, float) , PathExtensions.Scale(IPathCollection, float) , PathExtensions.Scale(IPathCollection, float, float) , PathExtensions.Translate(IPathCollection, PointF) , PathExtensions.Translate(IPathCollection, float, float)

# Constructors

## PathCollection(params IPath[])

Initializes a new instance of the PathCollection class.

```
public PathCollection(params IPath[] paths)
```

## Parameters

`paths` IPath[]

The collection of paths.

### PathCollection(IEnumerable<IPath>)

Initializes a new instance of the PathCollection class.

```
public PathCollection(IEnumerable<IPath> paths)
```

#### Parameters

`paths` IEnumerable<IPath>

The collection of paths.

## Properties

### Bounds

Gets the bounds enclosing all paths in the collection.

```
public RectangleF Bounds { get; }
```

#### Property Value

RectangleF

## Methods

### GetEnumerator()

Returns an enumerator that iterates through the collection.

```
public IEnumerator<IPath> GetEnumerator()
```

## Returns

IEnumerator<IPath>

An enumerator that can be used to iterate through the collection.

# Transform(Matrix4×4)

Transforms all paths in the collection using the specified matrix.

```
public IPathCollection Transform(Matrix4x4 matrix)
```

## Parameters

`matrix` Matrix4×4

The transformation matrix.

## Returns

IPathCollection

A new path collection with the matrix applied to it.

# See Also

IPath

# Class PathExtensions

Namespace: SixLabors.ImageSharp.Drawing

Assembly: SixLabors.ImageSharp.Drawing.dll

Convenience methods that can be applied to shapes and paths.

```
public static class PathExtensions
```

**Inheritance**

object ← PathExtensions

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

# Methods

## ComputeArea(IPath)

Calculates the total area represented by the specified path.

```
public static float ComputeArea(this IPath path)
```

## Parameters

`path` IPath

The path for which to compute the area.

## Returns

float

The total path area, in square units.

## ComputeArea(IPathCollection)

Calculates the total area of all paths in the specified collection.

```
public static float ComputeArea(this IPathCollection paths)
```

### Parameters

`paths` IPathCollection

A collection of paths for which to compute the combined area. Cannot be null.

### Returns

float

The total area, in square units, represented by all paths in the collection.

## ComputeLength(IPath)

Calculates the path length after flattening curves at local-space precision.

```
public static float ComputeLength(this IPath path)
```

### Parameters

`path` IPath

The path to compute the length for.

### Returns

float

The path length.

## Contains(IPath, PointF, IntersectionRule)

Returns whether the supplied point is inside the path.

```
public static bool Contains(this IPath path, PointF point, IntersectionRule
intersectionRule)
```

## Parameters

`path` IPath

The path to test.

`point` PointF

The point to test.

`intersectionRule` IntersectionRule

The fill rule used for containment.

## Returns

bool

true when the point is inside or on the path boundary.

# Rotate(IPath, float)

Creates a path rotated by the specified radians around its center.

```
public static IPath Rotate(this IPath path, float radians)
```

## Parameters

`path` IPath

The path to rotate.

`radians` float

The radians to rotate the path.

## Returns

IPath

An IPath with a rotate transform applied.

# Rotate(IPathCollection, float)

Creates a path collection rotated by the specified radians around its center.

```
public static IPathCollection Rotate(this IPathCollection path, float radians)
```

## Parameters

`path` IPathCollection

The path collection to rotate.

`radians` float

The radians to rotate the paths.

## Returns

IPathCollection

An IPathCollection with a rotate transform applied.

# RotateDegree(IPath, float)

Creates a path rotated by the specified degrees around its center.

```
public static IPath RotateDegree(this IPath shape, float degree)
```

## Parameters

`shape` IPath

The path to rotate.

`degree` float

The degree to rotate the path.

### Returns

IPath

An IPath with a rotate transform applied.

## RotateDegree(IPathCollection, float)

Creates a path collection rotated by the specified degrees around its center.

```
public static IPathCollection RotateDegree(this IPathCollection shape, float degree)
```

### Parameters

`shape` IPathCollection

The path collection to rotate.

`degree` float

The degree to rotate the paths.

### Returns

IPathCollection

An IPathCollection with a rotate transform applied.

## Scale(IPath, float)

Creates a path scaled by the supplied amount around its center.

```
public static IPath Scale(this IPath path, float scale)
```

### Parameters

`path` IPath

The path to scale.

`scale` float

The amount to scale along both the x and y axis.

## Returns

IPath

An IPath with a scale transform applied.

# Scale(IPath, float, float)

Creates a path scaled by the supplied amounts around its center.

```
public static IPath Scale(this IPath path, float scaleX, float scaleY)
```

## Parameters

`path` IPath

The path to scale.

`scaleX` float

The amount to scale along the X axis.

`scaleY` float

The amount to scale along the Y axis.

## Returns

IPath

An IPath with a scale transform applied.

# Scale(IPathCollection, float)

Creates a path collection scaled by the supplied amount around its center.

```
public static IPathCollection Scale(this IPathCollection path, float scale)
```

## Parameters

`path` IPathCollection

The path collection to scale.

`scale` float

The amount to scale along both the x and y axis.

## Returns

IPathCollection

An IPathCollection with a scale transform applied.

# Scale(IPathCollection, float, float)

Creates a path collection scaled by the supplied amounts around its center.

```
public static IPathCollection Scale(this IPathCollection path, float scaleX, float scaleY)
```

## Parameters

`path` IPathCollection

The path collection to scale.

`scaleX` float

The amount to scale along the X axis.

`scaleY` float

The amount to scale along the Y axis.

## Returns

IPathCollection

An IPathCollection with a scale transform applied.

## Translate(IPath, PointF)

Creates a path translated by the supplied position.

```
public static IPath Translate(this IPath path, PointF position)
```

### Parameters

`path` IPath

The path to translate.

`position` PointF

The translation position.

### Returns

IPath

An IPath with a translate transform applied.

## Translate(IPath, float, float)

Creates a path translated by the supplied position.

```
public static IPath Translate(this IPath path, float x, float y)
```

### Parameters

`path` IPath

The path to translate.

`x` float

The amount to translate along the X axis.

`y` float

The amount to translate along the Y axis.

### Returns

IPath

An IPath with a translate transform applied.

## Translate(IPathCollection, PointF)

Creates a path collection translated by the supplied position.

```
public static IPathCollection Translate(this IPathCollection path, PointF position)
```

### Parameters

`path` IPathCollection

The path collection to translate.

`position` PointF

The translation position.

### Returns

IPathCollection

An IPathCollection with a translate transform applied.

## Translate(IPathCollection, float, float)

Creates a path collection translated by the supplied position.

```
public static IPathCollection Translate(this IPathCollection path, float x, float y)
```

## Parameters

`path` IPathCollection

The path collection to translate.

`x` float

The amount to translate along the X axis.

`y` float

The amount to translate along the Y axis.

## Returns

IPathCollection

An IPathCollection with a translate transform applied.

# TryGetPathPointAtDistance(IPath, float, out PathPoint)

Gets path information at the specified distance along the path.

```
public static bool TryGetPathPointAtDistance(this IPath path, float distance, out
PathPoint pathPoint)
```

## Parameters

`path` IPath

The path to measure.

`distance` float

The distance along the path.

`pathPoint` PathPoint

When this method returns, contains the path information at `distance` if the distance resolves to a point on the path; otherwise, the default value.

## Returns

bool

true if `distance` resolves to a point on the path; otherwise, false.

# TryGetPathPointAtDistanceUnbounded(IPath, float, out PathPoint)

Gets path information at the specified distance along the path, extrapolating along the boundary tangents when the distance falls before the start or beyond the end of an open path.

```
public static bool TryGetPathPointAtDistanceUnbounded(this IPath path, float distance, out PathPoint pathPoint)
```

## Parameters

`path` IPath

The path to measure.

`distance` float

The distance along the path.

`pathPoint` PathPoint

When this method returns, contains the path information at `distance` if the path is measurable; otherwise, the default value.

## Returns

bool

true if the point could be resolved; otherwise, false.

## Remarks

Unlike TryGetPathPointAtDistance(IPath, float, out PathPoint), out-of-range distances resolve to a point on the virtual straight-line continuation of the path's first or last segment. Text layout uses this to place aligned glyphs that overflow their layout path. The method returns false only when the distance is not a finite number or the path has no measurable length.

# TryGetSegment(IPath, float, float, bool, out IPath)

Creates a path segment between two distances along the path.

```
public static bool TryGetSegment(this IPath path, float startDistance, float stopDistance,
bool startOnBeginFigure, out IPath segment)
```

## Parameters

`path` IPath

The path to measure.

`startDistance` float

The segment start distance.

`stopDistance` float

The segment stop distance.

`startOnBeginFigure` bool

Whether the returned segment starts a new figure at the first segment point.

`segment` IPath

When this method returns, contains the segment path if one was created; otherwise, an empty path.

## Returns

bool

true when a segment path was created.

# Struct PathPoint

Namespace: SixLabors.ImageSharp.Drawing

Assembly: SixLabors.ImageSharp.Drawing.dll

Represents metadata about a point along a path.

```
public readonly struct PathPoint
```

**Inherited Members**
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

# Properties

## Angle

Gets the forward angle of the segment, in degrees.

```
public float Angle { get; init; }
```

## Property Value

float

## Point

Gets the point on the path.

```
public PointF Point { get; init; }
```

## Property Value

PointF

# Tangent

Gets the normalized forward tangent of the segment.

```
public PointF Tangent { get; init; }
```

## Property Value

PointF

# Enum PathTypes

Namespace: SixLabors.ImageSharp.Drawing

Assembly: SixLabors.ImageSharp.Drawing.dll

Describes the different type of paths.

```
public enum PathTypes
```

# Fields

`Closed = 1`

Denotes a path describing a single simple closed shape.

`Mixed = 2`

Denotes a path containing one or more child paths that could be open or closed.

`Open = 0`

Denotes a path containing a single simple open path.

# Class PiePolygon

Namespace: SixLabors.ImageSharp.Drawing

Assembly: SixLabors.ImageSharp.Drawing.dll

A pie sector polygon defined by a center point, radii, rotation, and arc sweep.

```
public sealed class PiePolygon : Polygon, IPath, ISimplePath
```

### Inheritance

object ← Path ← Polygon ← PiePolygon

### Implements

IPath, ISimplePath

### Inherited Members

Polygon.IsClosed , Path.Empty , Path.Points , Path.Bounds , Path.PathType , Path.LineSegments , Path.AsClosedPath() , Path.Flatten() , Path.ToLinearGeometry(Vector2) , Path.ComputeLength(Vector2) , Path.ComputeArea(Vector2) , Path.Contains(PointF, IntersectionRule, Vector2) , Path.TryGetPathPointAtDistance(float, Vector2, out PathPoint) , Path.TryGetPathPointAtDistanceUnbounded(float, Vector2, out PathPoint) , Path.TryGetSegment(float, float, bool, Vector2, out IPath) , Path.TryParseSvgPath(string, out IPath) , Path.TryParseSvgPath(ReadOnlySpan<char>, out IPath) , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

### Extension Methods

ClipPathExtensions.Clip(IPath, BooleanOperation, params IPath[]) , ClipPathExtensions.Clip(IPath, BooleanOperation, IntersectionRule, params IPath[]) , ClipPathExtensions.Clip(IPath, BooleanOperation, IntersectionRule, IEnumerable<IPath>) , ClipPathExtensions.Clip(IPath, BooleanOperation, IEnumerable<IPath>) , ClipPathExtensions.Clip(IPath, params IPath[]) , ClipPathExtensions.Clip(IPath, IEnumerable<IPath>) , NormalizePathExtensions.Normalize(IPath) , OutlinePathExtensions.GenerateOutline(IPath, float) , OutlinePathExtensions.GenerateOutline(IPath, float, StrokeOptions) , OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>) , OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, StrokeOptions) ,

OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool, StrokeOptions) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool, float) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool, float, StrokeOptions) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, float) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, float, StrokeOptions) ,
PathExtensions.ComputeArea(IPath) , PathExtensions.ComputeLength(IPath) ,
PathExtensions.Contains(IPath, PointF, IntersectionRule) , PathExtensions.Rotate(IPath, float) ,
PathExtensions.RotateDegree(IPath, float) , PathExtensions.Scale(IPath, float) ,
PathExtensions.Scale(IPath, float, float) , PathExtensions.Translate(IPath, PointF) ,
PathExtensions.Translate(IPath, float, float) ,
PathExtensions.TryGetPathPointAtDistance(IPath, float, out PathPoint) ,
PathExtensions.TryGetPathPointAtDistanceUnbounded(IPath, float, out PathPoint) ,
PathExtensions.TryGetSegment(IPath, float, float, bool, out IPath) ,
SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>) ,
SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>, bool) ,
SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>, bool, float) ,
SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>, float)

# Constructors

## PiePolygon(PointF, SizeF, float, float)

Initializes a new instance of the PiePolygon class.

```
public PiePolygon(PointF center, SizeF radius, float startAngle, float sweepAngle)
```

### Parameters

`center` PointF

The center point of the pie sector.

`radius` SizeF

The x and y radii of the pie ellipse.

`startAngle` float

The pie start angle in degrees.

`sweepAngle` float

The pie sweep angle in degrees.

## PiePolygon(PointF, SizeF, float, float, float)

Initializes a new instance of the PiePolygon class.

```
public PiePolygon(PointF center, SizeF radius, float rotation, float startAngle,
float sweepAngle)
```

### Parameters

`center` PointF

The center point of the pie sector.

`radius` SizeF

The x and y radii of the pie ellipse.

`rotation` float

The ellipse rotation in degrees.

`startAngle` float

The pie start angle in degrees.

`sweepAngle` float

The pie sweep angle in degrees.

## PiePolygon(float, float, float, float, float, float)

Initializes a new instance of the PiePolygon class.

```
public PiePolygon(float x, float y, float radiusX, float radiusY, float startAngle,
```

```
float sweepAngle)
```

## Parameters

`x` float

The x-coordinate of the pie center.

`y` float

The y-coordinate of the pie center.

`radiusX` float

The x-radius of the pie ellipse.

`radiusY` float

The y-radius of the pie ellipse.

`startAngle` float

The pie start angle in degrees.

`sweepAngle` float

The pie sweep angle in degrees.

# PiePolygon(float, float, float, float, float, float, float)

Initializes a new instance of the PiePolygon class.

```
public PiePolygon(float x, float y, float radiusX, float radiusY, float rotation, float
startAngle, float sweepAngle)
```

## Parameters

`x` float

The x-coordinate of the pie center.

`y` float

The y-coordinate of the pie center.

`radiusX` float

The x-radius of the pie ellipse.

`radiusY` float

The y-radius of the pie ellipse.

`rotation` float

The ellipse rotation in degrees.

`startAngle` float

The pie start angle in degrees.

`sweepAngle` float

The pie sweep angle in degrees.

# Methods

## Transform(Matrix4×4)

Transforms the path using the specified matrix.

```
public override IPath Transform(Matrix4x4 matrix)
```

### Parameters

`matrix` Matrix4×4

The matrix.

### Returns

IPath

A new path with the matrix applied to it.

# Class Polygon

Namespace: SixLabors.ImageSharp.Drawing

Assembly: SixLabors.ImageSharp.Drawing.dll

A shape made up of a single closed path made up of one or more ILineSegments.

```
public class Polygon : Path, IPath, ISimplePath
```

### Inheritance

object ← Path ← Polygon

### Implements

IPath, ISimplePath

### Derived

EllipsePolygon, PiePolygon, RegularPolygon, RoundedRectanglePolygon, StarPolygon

### Inherited Members

Path.Empty , Path.Points , Path.Bounds , Path.PathType , Path.LineSegments ,
Path.AsClosedPath() , Path.Flatten() , Path.ToLinearGeometry(Vector2) ,
Path.ComputeLength(Vector2) , Path.ComputeArea(Vector2) ,
Path.Contains(PointF, IntersectionRule, Vector2) ,
Path.TryGetPathPointAtDistance(float, Vector2, out PathPoint) ,
Path.TryGetPathPointAtDistanceUnbounded(float, Vector2, out PathPoint) ,
Path.TryGetSegment(float, float, bool, Vector2, out IPath) ,
Path.TryParseSvgPath(string, out IPath) ,
Path.TryParseSvgPath(ReadOnlySpan<char>, out IPath) , object.Equals(object) ,
object.Equals(object, object) , object.GetHashCode() , object.GetType() ,
object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

### Extension Methods

ClipPathExtensions.Clip(IPath, BooleanOperation, params IPath[]) ,
ClipPathExtensions.Clip(IPath, BooleanOperation, IntersectionRule, params IPath[]) ,
ClipPathExtensions.Clip(IPath, BooleanOperation, IntersectionRule, IEnumerable<IPath>) ,
ClipPathExtensions.Clip(IPath, BooleanOperation, IEnumerable<IPath>) ,
ClipPathExtensions.Clip(IPath, params IPath[]) ,
ClipPathExtensions.Clip(IPath, IEnumerable<IPath>) , NormalizePathExtensions.Normalize(IPath) ,
OutlinePathExtensions.GenerateOutline(IPath, float) ,
OutlinePathExtensions.GenerateOutline(IPath, float, StrokeOptions) ,

OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, StrokeOptions) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool, StrokeOptions) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool, float) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool, float, StrokeOptions) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, float) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, float, StrokeOptions) ,
PathExtensions.ComputeArea(IPath) , PathExtensions.ComputeLength(IPath) ,
PathExtensions.Contains(IPath, PointF, IntersectionRule) , PathExtensions.Rotate(IPath, float) ,
PathExtensions.RotateDegree(IPath, float) , PathExtensions.Scale(IPath, float) ,
PathExtensions.Scale(IPath, float, float) , PathExtensions.Translate(IPath, PointF) ,
PathExtensions.Translate(IPath, float, float) ,
PathExtensions.TryGetPathPointAtDistance(IPath, float, out PathPoint) ,
PathExtensions.TryGetPathPointAtDistanceUnbounded(IPath, float, out PathPoint) ,
PathExtensions.TryGetSegment(IPath, float, float, bool, out IPath) ,
SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>) ,
SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>, bool) ,
SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>, bool, float) ,
SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>, float)

# Constructors

## Polygon(ILineSegment)

Initializes a new instance of the Polygon class.

```
public Polygon(ILineSegment segment)
```

### Parameters

`segment` ILineSegment

The segment.

## Polygon(params ILineSegment[])

Initializes a new instance of the Polygon class.

```
public Polygon(params ILineSegment[] segments)
```

### Parameters

`segments` ILineSegment[]

The segments.

## Polygon(PointF[])

Initializes a new instance of the Polygon class.

```
public Polygon(PointF[] points)
```

### Parameters

`points` PointF[]

The collection of points; processed as a series of linear line segments.

## Polygon(IEnumerable<ILineSegment>)

Initializes a new instance of the Polygon class.

```
public Polygon(IEnumerable<ILineSegment> segments)
```

### Parameters

`segments` IEnumerable<ILineSegment>

The segments.

# Properties

## IsClosed

Gets a value indicating whether this instance is a closed path.

```
public override bool IsClosed { get; }
```

### Property Value

bool

# Methods

## Transform(Matrix4×4)

Transforms the path using the specified matrix.

```
public override IPath Transform(Matrix4x4 matrix)
```

### Parameters

`matrix` Matrix4×4

The matrix.

### Returns

IPath

A new path with the matrix applied to it.

# Class RectanglePolygon

Namespace: SixLabors.ImageSharp.Drawing

Assembly: SixLabors.ImageSharp.Drawing.dll

A closed rectangular path defined by four straight edges.

```
public sealed class RectanglePolygon : IPath, ISimplePath
```

### Inheritance

object ← RectanglePolygon

### Implements

IPath, ISimplePath

### Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

### Extension Methods

ClipPathExtensions.Clip(IPath, BooleanOperation, params IPath[]) , ClipPathExtensions.Clip(IPath, BooleanOperation, IntersectionRule, params IPath[]) , ClipPathExtensions.Clip(IPath, BooleanOperation, IntersectionRule, IEnumerable<IPath>) , ClipPathExtensions.Clip(IPath, BooleanOperation, IEnumerable<IPath>) , ClipPathExtensions.Clip(IPath, params IPath[]) , ClipPathExtensions.Clip(IPath, IEnumerable<IPath>) , NormalizePathExtensions.Normalize(IPath) , OutlinePathExtensions.GenerateOutline(IPath, float) , OutlinePathExtensions.GenerateOutline(IPath, float, StrokeOptions) , OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>) , OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, StrokeOptions) , OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool) , OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool, StrokeOptions) , OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool, float) , OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool, float, StrokeOptions) , OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, float) , OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, float, StrokeOptions) , PathExtensions.ComputeArea(IPath) , PathExtensions.ComputeLength(IPath) , PathExtensions.Contains(IPath, PointF, IntersectionRule) , PathExtensions.Rotate(IPath, float) ,

PathExtensions.RotateDegree(IPath, float) , PathExtensions.Scale(IPath, float) , PathExtensions.Scale(IPath, float, float) , PathExtensions.Translate(IPath, PointF) , PathExtensions.Translate(IPath, float, float) , PathExtensions.TryGetPathPointAtDistance(IPath, float, out PathPoint) , PathExtensions.TryGetPathPointAtDistanceUnbounded(IPath, float, out PathPoint) , PathExtensions.TryGetSegment(IPath, float, float, bool, out IPath) , SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>) , SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>, bool) , SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>, bool, float) , SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>, float)

# Constructors

## RectanglePolygon(PointF, PointF)

Initializes a new instance of the RectanglePolygon class.

```
public RectanglePolygon(PointF topLeft, PointF bottomRight)
```

### Parameters

`topLeft` PointF

The PointF which specifies the rectangle's top-left point in a two-dimensional plane.

`bottomRight` PointF

The PointF which specifies the rectangle's bottom-right point in a two-dimensional plane.

## RectanglePolygon(PointF, SizeF)

Initializes a new instance of the RectanglePolygon class.

```
public RectanglePolygon(PointF point, SizeF size)
```

### Parameters

`point` PointF

The PointF which specifies the rectangle's point in a two-dimensional plane.

`size` SizeF

The SizeF which specifies the rectangle's height and width.

## RectanglePolygon(RectangleF)

Initializes a new instance of the RectanglePolygon class.

```
public RectanglePolygon(RectangleF rectangle)
```

### Parameters

`rectangle` RectangleF

The rectangle.

## RectanglePolygon(float, float, float, float)

Initializes a new instance of the RectanglePolygon class.

```
public RectanglePolygon(float x, float y, float width, float height)
```

### Parameters

`x` float

The horizontal position of the rectangle.

`y` float

The vertical position of the rectangle.

`width` float

The width of the rectangle.

`height` float

The height of the rectangle.

# Properties

## Bottom

Gets the y-coordinate of the bottom edge.

```
public float Bottom { get; }
```

## Property Value

float

## Bounds

Gets the bounds enclosing the path.

```
public RectangleF Bounds { get; }
```

## Property Value

RectangleF

## Center

Gets the center point.

```
public PointF Center { get; }
```

## Property Value

PointF

## Height

Gets the height.

```
public float Height { get; }
```

### Property Value

float

## IsClosed

Gets a value indicating whether this instance is a closed path.

```
public bool IsClosed { get; }
```

### Property Value

bool

## Left

Gets the x-coordinate of the left edge.

```
public float Left { get; }
```

### Property Value

float

## Location

Gets the top-left location of the rectangle.

```
public PointF Location { get; }
```

### Property Value

PointF

## PathType

Gets a value indicating whether this instance is closed, open or a composite path with a mixture of open and closed figures.

```
public PathTypes PathType { get; }
```

### Property Value

PathTypes

## Points

Gets the points that make this up as a simple linear path.

```
public ReadOnlyMemory<PointF> Points { get; }
```

### Property Value

ReadOnlyMemory<PointF>

## Right

Gets the x-coordinate of the right edge.

```
public float Right { get; }
```

### Property Value

float

## Size

Gets the size.

```
public SizeF Size { get; }
```

### Property Value

SizeF

## Top

Gets the y-coordinate of the top edge.

```
public float Top { get; }
```

### Property Value

float

## Width

Gets the width.

```
public float Width { get; }
```

### Property Value

float

## X

Gets the x-coordinate of the top-left corner.

```
public float X { get; }
```

### Property Value

float

## Y

Gets the y-coordinate of the top-left corner.

```
public float Y { get; }
```

### Property Value

float

# Methods

## AsClosedPath()

Returns this path with all figures closed.

```
public IPath AsClosedPath()
```

### Returns

IPath

A new closed IPath.

## ComputeArea(Vector2)

Calculates the non-negative path area after flattening curves at the supplied scale.

```
public float ComputeArea(Vector2 scale)
```

### Parameters

`scale` Vector2

The X/Y scale at which curves are flattened for area measurement.

### Returns

float

The path area.

## ComputeLength(Vector2)

Calculates the path length after flattening curves at the supplied scale.

```
public float ComputeLength(Vector2 scale)
```

### Parameters

`scale` Vector2

The X/Y scale at which curves are flattened for length measurement.

### Returns

float

The path length.

## Contains(PointF, IntersectionRule, Vector2)

Returns whether the supplied point is inside the path.

```
public bool Contains(PointF point, IntersectionRule intersectionRule, Vector2 scale)
```

## Parameters

`point` PointF

The point to test.

`intersectionRule` IntersectionRule

The fill rule used for containment.

`scale` Vector2

The X/Y scale at which curves are flattened for the containment test.

## Returns

bool

true when the point is inside or on the path boundary.

# Flatten()

Converts the IPath into a simple linear path.

```
public IEnumerable<ISimplePath> Flatten()
```

## Returns

IEnumerable<ISimplePath>

Returns the current IPath as simple linear path.

# ToLinearGeometry(Vector2)

Converts this path into a retained LinearGeometry, flattening curves at the precision of the supplied device-space `scale`.

```
public LinearGeometry ToLinearGeometry(Vector2 scale)
```

## Parameters

`scale` Vector2

The X/Y scale at which curves are flattened.

## Returns

LinearGeometry

The retained linear geometry.

# Transform(Matrix4×4)

Transforms the path using the specified matrix.

```
public IPath Transform(Matrix4x4 matrix)
```

## Parameters

`matrix` Matrix4×4

The matrix.

## Returns

IPath

A new path with the matrix applied to it.

# TryGetPathPointAtDistance(float, Vector2, out PathPoint)

Gets path information at the specified distance along the path.

```
public bool TryGetPathPointAtDistance(float distance, Vector2 scale, out
PathPoint pathPoint)
```

## Parameters

`distance` float

The distance along the path.

`scale` Vector2

The X/Y scale at which curves are flattened for distance measurement.

`pathPoint` PathPoint

When this method returns, contains the path information at `distance` if the distance resolves to a point on the path; otherwise, the default value.

## Returns

bool

true if `distance` resolves to a point on the path; otherwise, false.

# TryGetPathPointAtDistanceUnbounded(float, Vector2, out PathPoint)

Gets path information at the specified distance along the path, extrapolating along the boundary tangents when the distance falls before the start or beyond the end of an open path.

```
public bool TryGetPathPointAtDistanceUnbounded(float distance, Vector2 scale, out
PathPoint pathPoint)
```

## Parameters

`distance` float

The distance along the path.

`scale` Vector2

The X/Y scale at which curves are flattened for distance measurement.

`pathPoint` PathPoint

When this method returns, contains the path information at `distance` if the path is measurable; otherwise, the default value.

## Returns

bool

true if the point could be resolved; otherwise, false.

## Remarks

Unlike TryGetPathPointAtDistance(float, Vector2, out PathPoint), out-of-range distances resolve to a point on the virtual straight-line continuation of the path's first or last segment. Text layout uses this to place aligned glyphs that overflow their layout path. The method returns false only when the distance is not a finite number or the path has no measurable length.

# TryGetSegment(float, float, bool, Vector2, out IPath)

Creates a path segment between two distances along the path.

```
public bool TryGetSegment(float startDistance, float stopDistance, bool startOnBeginFigure,
Vector2 scale, out IPath path)
```

## Parameters

`startDistance` float

The segment start distance.

`stopDistance` float

The segment stop distance.

`startOnBeginFigure` bool

Whether the returned segment starts a new figure at the first segment point.

`scale` Vector2

The X/Y scale at which curves are flattened for segment extraction.

`path` IPath

When this method returns, contains the segment path if one was created; otherwise, an empty path.

## Returns

bool

true when a segment path was created.

# Operators

## explicit operator RectanglePolygon(Polygon)

Converts a polygon to a rectangle polygon from its bounds.

```
public static explicit operator RectanglePolygon(Polygon polygon)
```

## Parameters

`polygon` Polygon

The polygon to convert.

## Returns

RectanglePolygon

The rectangle polygon covering the source polygon's bounds.

# Class Region

Namespace: SixLabors.ImageSharp.Drawing

Assembly: SixLabors.ImageSharp.Drawing.dll

Represents an integer region composed from axis-aligned rectangles.

```
public sealed class Region
```

**Inheritance**

object ← Region

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Remarks

The region stores exact integer coverage using a normalized rect-set model: horizontal Y bands contain sorted X intervals. Region operations preserve the covered area as a union of rectangles; ToPath() exports that area as boundary geometry.

# Constructors

## Region()

Initializes a new instance of the Region class.

```
public Region()
```

## Region(Region)

Initializes a new instance of the Region class containing the same area as the specified region.

```
public Region(Region region)
```

### Parameters

`region` Region

The region to copy.

## Region(Rectangle)

Initializes a new instance of the Region class containing the specified rectangle.

```
public Region(Rectangle rectangle)
```

### Parameters

`rectangle` Rectangle

The rectangle to add to the region.

# Properties

## Bounds

Gets the bounding rectangle of the region.

```
public Rectangle Bounds { get; }
```

### Property Value

Rectangle

### Remarks

Returns Empty when the region is empty.

## IsEmpty

Gets a value indicating whether the region contains no area.

```
public bool IsEmpty { get; }
```

## Property Value

bool

## Rectangles

Gets the non-overlapping rectangles that describe the region.

```
public IReadOnlyList<Rectangle> Rectangles { get; }
```

## Property Value

IReadOnlyList<Rectangle>

## Remarks

The rectangles are an exported view of the region area. They preserve the region shape as a rect-set and are not collapsed to Bounds.

# Methods

## Add(Rectangle)

Adds a rectangle to the region.

```
public void Add(Rectangle rectangle)
```

## Parameters

`rectangle` Rectangle

The rectangle to add.

### Remarks

Rectangles with non-positive width or height do not change the region. The rectangle is unioned with the existing rect-set.

## Clear()

Removes all rectangles from the region.

```
public void Clear()
```

## Contains(Point)

Returns a value indicating whether the region contains the specified point.

```
public bool Contains(Point point)
```

### Parameters

`point` Point

The point to test.

### Returns

bool

true if the region contains the point; otherwise, false.

## Contains(int, int)

Returns a value indicating whether the region contains the specified point.

```
public bool Contains(int x, int y)
```

## Parameters

`x` int

The x-coordinate to test.

`y` int

The y-coordinate to test.

## Returns

bool

true if the region contains the point; otherwise, false.

# Intersect(Region)

Intersects this region with the specified region.

```
public bool Intersect(Region region)
```

## Parameters

`region` Region

The region to intersect with this region.

## Returns

bool

true when the resulting region is not empty; otherwise, false.

## Remarks

The operation intersects every overlapping X interval pair in every overlapping Y band and unions the results. This is the rect-set form of region intersection.

# Intersect(Rectangle)

Intersects this region with the specified rectangle.

```
public bool Intersect(Rectangle rectangle)
```

## Parameters

`rectangle` Rectangle

The rectangle to intersect with this region.

## Returns

bool

true when the resulting region is not empty; otherwise, false.

## Remarks

This operation clips the existing rect-set to the rectangle. It does not replace the result with one bounding rectangle.

# Intersects(Rectangle)

Returns a value indicating whether the region intersects the specified rectangle.

```
public bool Intersects(Rectangle rectangle)
```

## Parameters

`rectangle` Rectangle

The rectangle to test.

## Returns

bool

true if the region intersects the rectangle; otherwise, false.

# ToPath()

Creates a path representing the region.

```
public IPath ToPath()
```

## Returns

IPath

The path representing the region.

## Remarks

The returned path describes the exact boundary of the region. Complex regions may produce multiple closed figures. Returns ClosedPath when the region is empty.

# Class RegularPolygon

Namespace: SixLabors.ImageSharp.Drawing

Assembly: SixLabors.ImageSharp.Drawing.dll

A regular polygon with a configurable number of equally spaced vertices placed on a circumscribing circle.

```
public class RegularPolygon : Polygon, IPath, ISimplePath
```

### Inheritance

object ← Path ← Polygon ← RegularPolygon

### Implements

IPath, ISimplePath

### Inherited Members

Polygon.IsClosed , Polygon.Transform(Matrix4×4) , Path.Empty , Path.Points , Path.Bounds , Path.PathType , Path.LineSegments , Path.AsClosedPath() , Path.Flatten() , Path.ToLinearGeometry(Vector2) , Path.ComputeLength(Vector2) , Path.ComputeArea(Vector2) , Path.Contains(PointF, IntersectionRule, Vector2) , Path.TryGetPathPointAtDistance(float, Vector2, out PathPoint) , Path.TryGetPathPointAtDistanceUnbounded(float, Vector2, out PathPoint) , Path.TryGetSegment(float, float, bool, Vector2, out IPath) , Path.TryParseSvgPath(string, out IPath) , Path.TryParseSvgPath(ReadOnlySpan<char>, out IPath) , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

### Extension Methods

ClipPathExtensions.Clip(IPath, BooleanOperation, params IPath[]) , ClipPathExtensions.Clip(IPath, BooleanOperation, IntersectionRule, params IPath[]) , ClipPathExtensions.Clip(IPath, BooleanOperation, IntersectionRule, IEnumerable<IPath>) , ClipPathExtensions.Clip(IPath, BooleanOperation, IEnumerable<IPath>) , ClipPathExtensions.Clip(IPath, params IPath[]) , ClipPathExtensions.Clip(IPath, IEnumerable<IPath>) , NormalizePathExtensions.Normalize(IPath) , OutlinePathExtensions.GenerateOutline(IPath, float) , OutlinePathExtensions.GenerateOutline(IPath, float, StrokeOptions) , OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>) ,

OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, StrokeOptions) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool, StrokeOptions) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool, float) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool, float, StrokeOptions) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, float) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, float, StrokeOptions) ,
PathExtensions.ComputeArea(IPath) , PathExtensions.ComputeLength(IPath) ,
PathExtensions.Contains(IPath, PointF, IntersectionRule) , PathExtensions.Rotate(IPath, float) ,
PathExtensions.RotateDegree(IPath, float) , PathExtensions.Scale(IPath, float) ,
PathExtensions.Scale(IPath, float, float) , PathExtensions.Translate(IPath, PointF) ,
PathExtensions.Translate(IPath, float, float) ,
PathExtensions.TryGetPathPointAtDistance(IPath, float, out PathPoint) ,
PathExtensions.TryGetPathPointAtDistanceUnbounded(IPath, float, out PathPoint) ,
PathExtensions.TryGetSegment(IPath, float, float, bool, out IPath) ,
SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>) ,
SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>, bool) ,
SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>, bool, float) ,
SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>, float)

# Constructors

## RegularPolygon(PointF, int, float)

Initializes a new instance of the RegularPolygon class.

```
public RegularPolygon(PointF location, int vertices, float radius)
```

### Parameters

`location` PointF

The location the center of the polygon will be placed.

`vertices` int

The number of vertices the RegularPolygon should have.

`radius` float

The radius of the circle that would touch all vertices.

## RegularPolygon(PointF, int, float, float)

Initializes a new instance of the RegularPolygon class.

```
public RegularPolygon(PointF location, int vertices, float radius, float angle)
```

### Parameters

`location` PointF

The location the center of the polygon will be placed.

`vertices` int

The number of vertices the RegularPolygon should have.

`radius` float

The radius of the circle that would touch all vertices.

`angle` float

The angle of rotation in degrees.

## RegularPolygon(float, float, int, float)

Initializes a new instance of the RegularPolygon class.

```
public RegularPolygon(float x, float y, int vertices, float radius)
```

### Parameters

`x` float

The x-coordinate of the center of the polygon.

`y` float

The y-coordinate of the center of the polygon.

`vertices` int

The number of vertices the RegularPolygon should have.

`radius` float

The radius of the circle that would touch all vertices.

## RegularPolygon(float, float, int, float, float)

Initializes a new instance of the RegularPolygon class.

```
public RegularPolygon(float x, float y, int vertices, float radius, float angle)
```

### Parameters

`x` float

The x-coordinate of the center of the polygon.

`y` float

The y-coordinate of the center of the polygon.

`vertices` int

The number of vertices the RegularPolygon should have.

`radius` float

The radius of the circle that would touch all vertices.

`angle` float

The angle of rotation in degrees.

# Class RoundedRectanglePolygon

Namespace: SixLabors.ImageSharp.Drawing

Assembly: SixLabors.ImageSharp.Drawing.dll

A closed rectangular path with rounded corners.

```
public sealed class RoundedRectanglePolygon : Polygon, IPath, ISimplePath
```

### Inheritance

object ← Path ← Polygon ← RoundedRectanglePolygon

### Implements

IPath, ISimplePath

### Inherited Members

Polygon.IsClosed , Path.Empty , Path.Points , Path.Bounds , Path.PathType , Path.LineSegments , Path.AsClosedPath() , Path.Flatten() , Path.ToLinearGeometry(Vector2) , Path.ComputeLength(Vector2) , Path.ComputeArea(Vector2) , Path.Contains(PointF, IntersectionRule, Vector2) , Path.TryGetPathPointAtDistance(float, Vector2, out PathPoint) , Path.TryGetPathPointAtDistanceUnbounded(float, Vector2, out PathPoint) , Path.TryGetSegment(float, float, bool, Vector2, out IPath) , Path.TryParseSvgPath(string, out IPath) , Path.TryParseSvgPath(ReadOnlySpan<char>, out IPath) , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

### Extension Methods

ClipPathExtensions.Clip(IPath, BooleanOperation, params IPath[]) , ClipPathExtensions.Clip(IPath, BooleanOperation, IntersectionRule, params IPath[]) , ClipPathExtensions.Clip(IPath, BooleanOperation, IntersectionRule, IEnumerable<IPath>) , ClipPathExtensions.Clip(IPath, BooleanOperation, IEnumerable<IPath>) , ClipPathExtensions.Clip(IPath, params IPath[]) , ClipPathExtensions.Clip(IPath, IEnumerable<IPath>) , NormalizePathExtensions.Normalize(IPath) , OutlinePathExtensions.GenerateOutline(IPath, float) , OutlinePathExtensions.GenerateOutline(IPath, float, StrokeOptions) , OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>) , OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, StrokeOptions) ,

OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool, StrokeOptions) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool, float) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool, float, StrokeOptions) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, float) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, float, StrokeOptions) ,
PathExtensions.ComputeArea(IPath) , PathExtensions.ComputeLength(IPath) ,
PathExtensions.Contains(IPath, PointF, IntersectionRule) , PathExtensions.Rotate(IPath, float) ,
PathExtensions.RotateDegree(IPath, float) , PathExtensions.Scale(IPath, float) ,
PathExtensions.Scale(IPath, float, float) , PathExtensions.Translate(IPath, PointF) ,
PathExtensions.Translate(IPath, float, float) ,
PathExtensions.TryGetPathPointAtDistance(IPath, float, out PathPoint) ,
PathExtensions.TryGetPathPointAtDistanceUnbounded(IPath, float, out PathPoint) ,
PathExtensions.TryGetSegment(IPath, float, float, bool, out IPath) ,
SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>) ,
SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>, bool) ,
SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>, bool, float) ,
SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>, float)

# Constructors

## RoundedRectanglePolygon(RectangleF, SizeF)

Initializes a new instance of the RoundedRectanglePolygon class.

```
public RoundedRectanglePolygon(RectangleF rectangle, SizeF radius)
```

## Parameters

`rectangle` RectangleF

The rectangle bounds.

`radius` SizeF

The x and y radii of each corner.

# RoundedRectanglePolygon(RectangleF, SizeF, SizeF, SizeF, SizeF)

Initializes a new instance of the RoundedRectanglePolygon class.

```
public RoundedRectanglePolygon(RectangleF rectangle, SizeF topLeftRadius, SizeF
topRightRadius, SizeF bottomRightRadius, SizeF bottomLeftRadius)
```

## Parameters

`rectangle` RectangleF

The rectangle bounds.

`topLeftRadius` SizeF

The x and y radii of the top-left corner.

`topRightRadius` SizeF

The x and y radii of the top-right corner.

`bottomRightRadius` SizeF

The x and y radii of the bottom-right corner.

`bottomLeftRadius` SizeF

The x and y radii of the bottom-left corner.

# RoundedRectanglePolygon(RectangleF, float)

Initializes a new instance of the RoundedRectanglePolygon class.

```
public RoundedRectanglePolygon(RectangleF rectangle, float radius)
```

## Parameters

`rectangle` RectangleF

The rectangle bounds.

`radius` float

The x and y radius of each corner.

## RoundedRectanglePolygon(float, float, float, float, SizeF)

Initializes a new instance of the RoundedRectanglePolygon class.

```
public RoundedRectanglePolygon(float x, float y, float width, float height, SizeF radius)
```

### Parameters

`x` float

The x-coordinate of the rectangle.

`y` float

The y-coordinate of the rectangle.

`width` float

The rectangle width.

`height` float

The rectangle height.

`radius` SizeF

The x and y radii of each corner.

## RoundedRectanglePolygon(float, float, float, float, SizeF, SizeF, SizeF, SizeF)

Initializes a new instance of the RoundedRectanglePolygon class.

```
public RoundedRectanglePolygon(float x, float y, float width, float height, SizeF
topLeftRadius, SizeF topRightRadius, SizeF bottomRightRadius, SizeF bottomLeftRadius)
```

## Parameters

`x` float

The x-coordinate of the rectangle.

`y` float

The y-coordinate of the rectangle.

`width` float

The rectangle width.

`height` float

The rectangle height.

`topLeftRadius` SizeF

The x and y radii of the top-left corner.

`topRightRadius` SizeF

The x and y radii of the top-right corner.

`bottomRightRadius` SizeF

The x and y radii of the bottom-right corner.

`bottomLeftRadius` SizeF

The x and y radii of the bottom-left corner.

# RoundedRectanglePolygon(float, float, float, float, float)

Initializes a new instance of the RoundedRectanglePolygon class.

```
public RoundedRectanglePolygon(float x, float y, float width, float height, float radius)
```

## Parameters

`x` float

The x-coordinate of the rectangle.

`y` float

The y-coordinate of the rectangle.

`width` float

The rectangle width.

`height` float

The rectangle height.

`radius` float

The x and y radius of each corner.

# Methods

## Transform(Matrix4×4)

Transforms the path using the specified matrix.

```
public override IPath Transform(Matrix4x4 matrix)
```

### Parameters

`matrix` Matrix4×4

The matrix.

### Returns

IPath

A new path with the matrix applied to it.

# Struct SegmentEnumerator

Namespace: SixLabors.ImageSharp.Drawing

Assembly: SixLabors.ImageSharp.Drawing.dll

Enumerates the derived linear segments in a LinearGeometry.

```
public ref struct SegmentEnumerator
```

**Inherited Members**
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

# Remarks

The enumerator derives segments from Points and Contours. Segments are yielded in contour order. Within each contour, adjacent stored points form segments in point order, and a closed contour contributes one additional closing segment from its last stored point back to its first.

# Properties

## Current

Gets the current derived linear segment.

```
public readonly LinearSegment Current { get; }
```

### Property Value

LinearSegment

# Methods

## MoveNext()

Advances to the next derived segment.

```
public bool MoveNext()
```

## Returns

bool

true if a segment was produced; otherwise false.

# Class SplitPathExtensions

Namespace: SixLabors.ImageSharp.Drawing

Assembly: SixLabors.ImageSharp.Drawing.dll

Extensions to IPath for splitting paths into dash segments without performing stroke expansion.

```
public static class SplitPathExtensions
```

**Inheritance**

object ← SplitPathExtensions

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

# Methods

## GenerateDashes(IPath, float, ReadOnlySpan<float>)

Splits the given path into dash segments based on the provided pattern. Returns a composite path containing only the "on" segments as open sub-paths.

```
public static IPath GenerateDashes(this IPath path, float strokeWidth,
ReadOnlySpan<float> pattern)
```

## Parameters

`path` IPath

The centerline path to split.

`strokeWidth` float

The stroke width (pattern elements are multiples of this).

`pattern` ReadOnlySpan<float>

The dash pattern. Each element is a multiple of `strokeWidth`.

### Returns

IPath

A path containing the "on" dash segments.

## GenerateDashes(IPath, float, ReadOnlySpan<float>, bool)

Splits the given path into dash segments based on the provided pattern. Returns a composite path containing only the "on" segments as open sub-paths.

```
public static IPath GenerateDashes(this IPath path, float strokeWidth, ReadOnlySpan<float>
pattern, bool startOff)
```

### Parameters

`path` IPath

The centerline path to split.

`strokeWidth` float

The stroke width (pattern elements are multiples of this).

`pattern` ReadOnlySpan<float>

The dash pattern. Each element is a multiple of `strokeWidth`.

`startOff` bool

Whether the first item in the pattern is off rather than on.

### Returns

IPath

A path containing the "on" dash segments.

# GenerateDashes(IPath, float, ReadOnlySpan<float>, bool, float)

Splits the given path into dash segments based on the provided pattern. Returns a composite path containing only the "on" segments as open sub-paths.

```
public static IPath GenerateDashes(this IPath path, float strokeWidth, ReadOnlySpan<float>
pattern, bool startOff, float offset)
```

## Parameters

`path` IPath

The centerline path to split.

`strokeWidth` float

The stroke width (pattern elements are multiples of this).

`pattern` ReadOnlySpan<float>

The dash pattern. Each element is a multiple of `strokeWidth`.

`startOff` bool

Whether the first item in the pattern is off rather than on.

`offset` float

The distance into the dash pattern, expressed as a multiple of `strokeWidth`.

## Returns

IPath

A path containing the "on" dash segments.

# GenerateDashes(IPath, float, ReadOnlySpan<float>, float)

Splits the given path into dash segments based on the provided pattern. Returns a composite path containing only the "on" segments as open sub-paths.

```
public static IPath GenerateDashes(this IPath path, float strokeWidth, ReadOnlySpan<float>
pattern, float offset)
```

## Parameters

`path` IPath

The centerline path to split.

`strokeWidth` float

The stroke width (pattern elements are multiples of this).

`pattern` ReadOnlySpan<float>

The dash pattern. Each element is a multiple of `strokeWidth`.

`offset` float

The distance into the dash pattern, expressed as a multiple of `strokeWidth`.

## Returns

IPath

A path containing the "on" dash segments.

# Class StarPolygon

Namespace: SixLabors.ImageSharp.Drawing

Assembly: SixLabors.ImageSharp.Drawing.dll

A star-shaped polygon defined by alternating inner and outer radii.

```
public sealed class StarPolygon : Polygon, IPath, ISimplePath
```

### Inheritance

object ← Path ← Polygon ← StarPolygon

### Implements

IPath, ISimplePath

### Inherited Members

Polygon.IsClosed , Polygon.Transform(Matrix4×4) , Path.Empty , Path.Points , Path.Bounds , Path.PathType , Path.LineSegments , Path.AsClosedPath() , Path.Flatten() , Path.ToLinearGeometry(Vector2) , Path.ComputeLength(Vector2) , Path.ComputeArea(Vector2) , Path.Contains(PointF, IntersectionRule, Vector2) , Path.TryGetPathPointAtDistance(float, Vector2, out PathPoint) , Path.TryGetPathPointAtDistanceUnbounded(float, Vector2, out PathPoint) , Path.TryGetSegment(float, float, bool, Vector2, out IPath) , Path.TryParseSvgPath(string, out IPath) , Path.TryParseSvgPath(ReadOnlySpan<char>, out IPath) , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

### Extension Methods

ClipPathExtensions.Clip(IPath, BooleanOperation, params IPath[]) , ClipPathExtensions.Clip(IPath, BooleanOperation, IntersectionRule, params IPath[]) , ClipPathExtensions.Clip(IPath, BooleanOperation, IntersectionRule, IEnumerable<IPath>) , ClipPathExtensions.Clip(IPath, BooleanOperation, IEnumerable<IPath>) , ClipPathExtensions.Clip(IPath, params IPath[]) , ClipPathExtensions.Clip(IPath, IEnumerable<IPath>) , NormalizePathExtensions.Normalize(IPath) , OutlinePathExtensions.GenerateOutline(IPath, float) , OutlinePathExtensions.GenerateOutline(IPath, float, StrokeOptions) , OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>) , OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, StrokeOptions) ,

OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool, StrokeOptions) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool, float) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, bool, float, StrokeOptions) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, float) ,
OutlinePathExtensions.GenerateOutline(IPath, float, ReadOnlySpan<float>, float, StrokeOptions) ,
PathExtensions.ComputeArea(IPath) , PathExtensions.ComputeLength(IPath) ,
PathExtensions.Contains(IPath, PointF, IntersectionRule) , PathExtensions.Rotate(IPath, float) ,
PathExtensions.RotateDegree(IPath, float) , PathExtensions.Scale(IPath, float) ,
PathExtensions.Scale(IPath, float, float) , PathExtensions.Translate(IPath, PointF) ,
PathExtensions.Translate(IPath, float, float) ,
PathExtensions.TryGetPathPointAtDistance(IPath, float, out PathPoint) ,
PathExtensions.TryGetPathPointAtDistanceUnbounded(IPath, float, out PathPoint) ,
PathExtensions.TryGetSegment(IPath, float, float, bool, out IPath) ,
SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>) ,
SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>, bool) ,
SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>, bool, float) ,
SplitPathExtensions.GenerateDashes(IPath, float, ReadOnlySpan<float>, float)

# Constructors

## StarPolygon(PointF, int, float, float)

Initializes a new instance of the StarPolygon class.

```
public StarPolygon(PointF location, int prongs, float innerRadii, float outerRadii)
```

## Parameters

`location` PointF

The center point of the star.

`prongs` int

The number of star prongs.

`innerRadii` float

The inner star radius.

`outerRadii` float

The outer star radius.

## StarPolygon(PointF, int, float, float, float)

Initializes a new instance of the StarPolygon class.

```
public StarPolygon(PointF location, int prongs, float innerRadii, float outerRadii,
float angle)
```

### Parameters

`location` PointF

The center point of the star.

`prongs` int

The number of star prongs.

`innerRadii` float

The inner star radius.

`outerRadii` float

The outer star radius.

`angle` float

The angle of rotation in degrees.

## StarPolygon(float, float, int, float, float)

Initializes a new instance of the StarPolygon class.

```
public StarPolygon(float x, float y, int prongs, float innerRadii, float outerRadii)
```

## Parameters

`x` float

The x-coordinate of the star center.

`y` float

The y-coordinate of the star center.

`prongs` int

The number of star prongs.

`innerRadii` float

The inner star radius.

`outerRadii` float

The outer star radius.

# StarPolygon(float, float, int, float, float, float)

Initializes a new instance of the StarPolygon class.

```
public StarPolygon(float x, float y, int prongs, float innerRadii, float outerRadii,
float angle)
```

## Parameters

`x` float

The x-coordinate of the star center.

`y` float

The y-coordinate of the star center.

`prongs` int

The number of star prongs.

`innerRadii` float

The inner star radius.

`outerRadii` float

The outer star radius.

`angle` float

The angle of rotation in degrees.

# Namespace SixLabors.ImageSharp.Drawing.Helpers

## Classes

MatrixUtilities

Provides helper methods for extracting scale properties from transformation matrices.

# Class MatrixUtilities

Namespace: SixLabors.ImageSharp.Drawing.Helpers

Assembly: SixLabors.ImageSharp.Drawing.dll

Provides helper methods for extracting scale properties from transformation matrices.

```
public static class MatrixUtilities
```

**Inheritance**

object ← MatrixUtilities

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

# Remarks

Scale extraction operates on the 2D linear part of the matrix (M11, M12, M21, M22). Scale magnitudes are the lengths of the transformed X and Y basis vectors, so they are rotation-invariant and always non-negative. Translation does not affect any result.

# Methods

## GetAverageScale(in Matrix4×4)

Extracts the average 2D scale factor from a Matrix4×4. This is the mean of the X and Y axis scale magnitudes, suitable for uniformly scaling radii under non-uniform or projective transforms.

```
public static float GetAverageScale(in Matrix4x4 matrix)
```

### Parameters

`matrix` Matrix4×4

The transformation matrix.

### Returns

float

The average scale factor.

## GetResidual(Vector2, Matrix4×4)

Computes the transform remaining after the X and Y scale magnitudes have been baked into geometry.

```
public static Matrix4x4 GetResidual(Vector2 scale, Matrix4x4 matrix)
```

### Parameters

`scale` Vector2

The scale magnitudes baked into geometry. Components must be non-zero.

`matrix` Matrix4×4

The original transformation matrix.

### Returns

Matrix4×4

The residual transform.

### Remarks

The invariant is `Matrix4x4.CreateScale(scale.X, scale.Y, 1) * residual == matrix`: applying the residual to geometry that has already been scaled reproduces the original transform. Rotation, reflection, skew, and translation all remain in the residual.

## GetScale(in Matrix4×4)

Extracts the X and Y scale magnitudes from a 2D transform matrix.

```
public static Vector2 GetScale(in Matrix4x4 matrix)
```

## Parameters

`matrix` Matrix4×4

The transformation matrix.

## Returns

Vector2

The X and Y scale magnitudes.

## Remarks

The magnitudes are the lengths of the transformed X and Y basis vectors. They are always non-negative; reflection and rotation are not represented in the result.

# IsAffine(in Matrix4×4)

Returns a value indicating whether the matrix is affine in 2D: no perspective terms, so every position is transformed by the same linear part and shapes are mapped identically wherever they sit. Projective matrices fail this test because their perspective divisor varies with the input position.

```
public static bool IsAffine(in Matrix4x4 matrix)
```

## Parameters

`matrix` Matrix4×4

The transformation matrix.

## Returns

bool

true when the matrix contains no perspective terms; otherwise, false.

# IsTranslationOnly(in Matrix4×4)

Returns a value indicating whether the matrix is a pure 2D translation: an identity linear part with no perspective terms, so geometry moves by the M41 and M42 offsets without any scale, rotation, or skew.

```
public static bool IsTranslationOnly(in Matrix4x4 matrix)
```

## Parameters

`matrix` Matrix4×4

The transformation matrix.

## Returns

bool

true when the matrix only translates; otherwise, false.

# PreservesAxisAlignedRectangles(in Matrix4×4)

Returns a value indicating whether the matrix maps axis-aligned rectangles to axis-aligned rectangles.

```
public static bool PreservesAxisAlignedRectangles(in Matrix4x4 matrix)
```

## Parameters

`matrix` Matrix4×4

The transformation matrix.

## Returns

bool

true when axis-aligned rectangles remain axis-aligned; otherwise, false.

## Remarks

Translation, scaling, reflection, and 90 degree rotations (axis swaps) preserve axis alignment. Skew and free rotation do not.

# Namespace SixLabors.ImageSharp.Drawing.Processing

## Classes

BackdropAcrylicLayerEffect
: Blurs and tints the backdrop beneath a compositing layer, producing a frosted-glass acrylic material. This is the tinted extension of BackdropBlurLayerEffect and has no CSS equivalent; CSS composes the tint from the element's own translucent background instead.

BackdropBlurLayerEffect
: Blurs the backdrop beneath a compositing layer. The CSS equivalent is `backdrop-filter: blur()`.

BackdropBrightnessLayerEffect
: Adjusts the brightness of the backdrop beneath a compositing layer. The CSS equivalent is `backdrop-filter: brightness()`.

BackdropColorMatrixLayerEffect
: Transforms the colours of the backdrop beneath a compositing layer through a colour matrix. The named backdrop colour effects all specialize this; use it directly for matrices they do not cover.

BackdropContrastLayerEffect
: Adjusts the contrast of the backdrop beneath a compositing layer. The CSS equivalent is `backdrop-filter: contrast()`.

BackdropDropShadowLayerEffect
: Composites a drop shadow of the backdrop's silhouette beneath it. The CSS equivalent is `backdrop-filter: drop-shadow()`.

BackdropGrayscaleLayerEffect
: Desaturates the backdrop beneath a compositing layer towards grayscale using ITU-R BT.709 luminance coefficients, matching the CSS definition. The CSS equivalent is `backdrop-filter: grayscale()`.

BackdropHueRotateLayerEffect
: Rotates the hue of the backdrop beneath a compositing layer. The CSS equivalent is `backdrop-filter: hue-rotate()`.

BackdropInvertLayerEffect

Inverts the colours of the backdrop beneath a compositing layer. The CSS equivalent is `backdrop-filter: invert()`.

BackdropLayerEffect
: Describes an effect applied to the backdrop of a compositing layer: the pixels already on the canvas beneath the layer's region are filtered when the layer is opened, and the layer's content then renders above the filtered backdrop. This is the CSS `backdrop-filter` model; the filtered result is clipped to the layer's region.

BackdropOpacityLayerEffect
: Scales the opacity of the backdrop beneath a compositing layer. The CSS equivalent is `backdrop-filter: opacity()`.

BackdropSaturateLayerEffect
: Adjusts the colour saturation of the backdrop beneath a compositing layer. The CSS equivalent is `backdrop-filter: saturate()`.

BackdropSepiaLayerEffect
: Shifts the colours of the backdrop beneath a compositing layer towards sepia. The CSS equivalent is `backdrop-filter: sepia()`.

BlurLayerEffect
: Blurs the contents of a compositing layer when it is restored.

Brush
: Represents a logical configuration of a brush which can be used to source pixel colors.

BrushRenderer<TPixel>
: Renders a Brush against individual coverage scanlines.

BrushWorkspace<TPixel>
: Worker-local scratch workspace used by prepared brushes during row composition.

Brushes
: A collection of methods for creating generic brushes.

ColorMatrixLayerEffect
: Transforms the colours of a compositing layer's contents through a colour matrix when the layer is restored. Covers the CSS filter family: grayscale, sepia, saturation, hue rotation, brightness, contrast, and invert are all colour matrices.

DrawingCanvas
: Represents a stateful, retained-mode drawing canvas over a frame target.

DrawingCanvasFactoryExtensions
: Extension methods for creating drawing canvas instances over ImageSharp image frames.

DrawingCanvas<TPixel>
: A drawing canvas over a frame target.

DrawingClipDescriptor
: Represents one clip primitive carried by a drawing command.

DrawingOptions
: Provides options for influencing drawing operations, combining graphics rendering settings, the fill-rule intersection mode, and an optional coordinate transform.

DrawingOptionsDefaultsExtensions
: Adds extensions that help working with DrawingOptions.

DrawingTextCache
: Stores reusable text drawing data shared by one or more drawing canvases.

DropShadowLayerEffect
: Composites a drop shadow beneath the contents of a compositing layer when it is restored. The shadow is the content's own silhouette tinted with the shadow colour, blurred, and offset.

EllipticGradientBrush
: Provides an implementation of a brush for painting elliptical gradients. The ellipse is defined by a center point, a point on the longest axis, and the ratio between the longest and shortest axes.

GlowLayerEffect
: Composites a glow beneath the contents of a compositing layer when it is restored: the content's own silhouette tinted with the glow colour and blurred, spreading evenly in all directions.

GradientBrush
: Base class for gradient brushes. Derived brushes define a parameterization that maps each point to a scalar gradient position; this base class maps that position to a color via the sorted color stops and the GradientRepetitionMode.

ImageBrush
: The untyped base class for image brushes, used to support non-generic brush references in drawing contexts.

ImageBrush<TPixel>
: Provides an implementation of an image brush for painting images within areas.

InnerShadowLayerEffect
: Composites a shadow inside the contents of a compositing layer when it is restored: the darkness outside the content's silhouette is tinted with the shadow colour, blurred, offset, and clipped onto the content itself, so the shadow hugs the content's inside edges.

LayerEffect
: Describes an image effect applied to the contents of a compositing layer when it is restored. The layer isolates the content the effect operates on, so the effect sees only what was drawn into the layer, against transparency, before the result is composited onto the canvas.

LinearGradientBrush
: Provides a brush that paints linear gradients within an area. Supports both classic two-point gradients and three-point (rotated) gradients.

PaintExtensions
: Adds image-processing extensions that paint each frame through DrawingCanvas.

PaintProcessor
: Defines the image processor used by Paint(IImageProcessingContext, DrawingOptions, Canvas Action) to execute a canvas callback for each image frame.

PathGradientBrush
: Provides an implementation of a brush for painting gradients between multiple color positions in 2D coordinates.

PatternBrush
: Provides an implementation of a pattern brush for painting patterns.

PatternPen
: Defines a pen that can apply a pattern to a line with a set brush and thickness.

Pen
: The base class for pens that can apply a pattern to a line with a set brush and thickness

Pens
: Contains a collection of common pen styles.

RadialGradientBrush
: A radial gradient brush defined by either one circle or two circles. When one circle is provided, the gradient parameter is the distance from the center divided by the radius. When two circles are provided, the gradient parameter is computed along the family of circles interpolating between the start and end circles.

RasterizerDefaultsExtensions

Adds extensions that allow configuring the drawing backend implementation.

RecolorBrush
: Provides an implementation of a brush that can recolor an image. Pixels within Threshold of SourceColor are blended towards TargetColor, with the blend strength falling off as the color distance approaches the threshold.

RichGlyphOptions
: Provides GlyphOptions for rendering a glyph by id with per-glyph Brush and Pen support. The paint is carried to the renderer via a RichTextRun, so it can vary per glyph across a run.

RichTextOptions
: Provides configuration options for rendering and shaping of rich text.

RichTextRun
: Represents a run of drawable text spanning a series of graphemes within a string.

SolidBrush
: Provides an implementation of a solid brush for painting solid color areas.

SolidPen
: Defines a pen that draws a continuous, unpatterned stroke with a set brush and thickness.

StrokeOptions
: Provides configuration options for geometric stroke generation.

SweepGradientBrush
: Provides an implementation of a brush for painting sweep (conic) gradients within areas. Angles increase counter-clockwise from +X on the design grid.

## Structs

ColorStop
: A struct that defines a single color stop: a color pinned to a position along a gradient.

DrawingClipState
: Represents an ordered set of exact clip primitives carried by a drawing command.

PenOptions
: Provides a set of configurations options for pens.

## Enums

DrawingClipEdgeMode

Identifies how rectangle clip edges contribute coverage.

DrawingClipKind
: Identifies the clip primitive represented by a drawing clip descriptor.

GradientRepetitionMode
: Modes to repeat a gradient.

WrapMode
: Defines how an ImageBrush samples beyond its source region along a single axis.

## Delegates

CanvasAction
: Represents the per-frame painting callback executed by Paint(IImageProcessingContext, CanvasAction).

# Class BackdropAcrylicLayerEffect

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Blurs and tints the backdrop beneath a compositing layer, producing a frosted-glass acrylic material. This is the tinted extension of BackdropBlurLayerEffect and has no CSS equivalent; CSS composes the tint from the element's own translucent background instead.

```
public sealed class BackdropAcrylicLayerEffect : BackdropLayerEffect
```

**Inheritance**

object ← LayerEffect ← BackdropLayerEffect ← BackdropAcrylicLayerEffect

**Inherited Members**

BackdropLayerEffect.Reach , LayerEffect.IsPassThrough , LayerEffect.WriteBackOptions , LayerEffect.WriteBackOffset , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## BackdropAcrylicLayerEffect(float, Color)

Initializes a new instance of the BackdropAcrylicLayerEffect class.

```
public BackdropAcrylicLayerEffect(float sigma, Color tint)
```

## Parameters

`sigma` float

The Gaussian blur sigma, in pixels.

`tint` Color

The tint blended over the blurred backdrop. Its alpha controls the tint strength, so an opaque tint hides the backdrop entirely.

# Properties

## Sigma

Gets the Gaussian blur sigma, in pixels.

```
public float Sigma { get; }
```

## Property Value

float

## Tint

Gets the tint blended over the blurred backdrop.

```
public Color Tint { get; }
```

## Property Value

Color

# Class BackdropBlurLayerEffect

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Blurs the backdrop beneath a compositing layer. The CSS equivalent is `backdrop-filter: blur()`.

```
public sealed class BackdropBlurLayerEffect : BackdropLayerEffect
```

**Inheritance**

object ← LayerEffect ← BackdropLayerEffect ← BackdropBlurLayerEffect

**Inherited Members**

BackdropLayerEffect.Reach , LayerEffect.WriteBackOptions , LayerEffect.WriteBackOffset , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## BackdropBlurLayerEffect(float)

Initializes a new instance of the BackdropBlurLayerEffect class.

```
public BackdropBlurLayerEffect(float sigma)
```

### Parameters

`sigma` float

The Gaussian blur sigma, in pixels; zero leaves the backdrop unchanged.

# Properties

## IsPassThrough

Gets a value indicating whether the effect leaves the layer content unchanged, in which case its application is skipped entirely.

```
public override bool IsPassThrough { get; }
```

## Property Value

bool

# Sigma

Gets the Gaussian blur sigma, in pixels.

```
public float Sigma { get; }
```

## Property Value

float

# Class BackdropBrightnessLayerEffect

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Adjusts the brightness of the backdrop beneath a compositing layer. The CSS equivalent is `backdrop-filter: brightness()`.

```
public sealed class BackdropBrightnessLayerEffect : BackdropColorMatrixLayerEffect
```

**Inheritance**

object ← LayerEffect ← BackdropLayerEffect ← BackdropColorMatrixLayerEffect ← BackdropBrightnessLayerEffect

**Inherited Members**

BackdropColorMatrixLayerEffect.Matrix , BackdropColorMatrixLayerEffect.IsPassThrough , BackdropLayerEffect.Reach , LayerEffect.WriteBackOptions , LayerEffect.WriteBackOffset , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## BackdropBrightnessLayerEffect(float)

Initializes a new instance of the BackdropBrightnessLayerEffect class.

```
public BackdropBrightnessLayerEffect(float amount)
```

## Parameters

`amount` float

The brightness amount: 0 is black, 1 leaves the backdrop unchanged, and values above 1 brighten.

# Properties

## Amount

Gets the brightness amount.

```
public float Amount { get; }
```

## Property Value

float

# Class BackdropColorMatrixLayerEffect

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Transforms the colours of the backdrop beneath a compositing layer through a colour matrix. The named backdrop colour effects all specialize this; use it directly for matrices they do not cover.

```
public class BackdropColorMatrixLayerEffect : BackdropLayerEffect
```

**Inheritance**

object ← LayerEffect ← BackdropLayerEffect ← BackdropColorMatrixLayerEffect

**Derived**

BackdropBrightnessLayerEffect, BackdropContrastLayerEffect, BackdropGrayscaleLayerEffect, BackdropHueRotateLayerEffect, BackdropInvertLayerEffect, BackdropOpacityLayerEffect, BackdropSaturateLayerEffect, BackdropSepiaLayerEffect

**Inherited Members**

BackdropLayerEffect.Reach , LayerEffect.WriteBackOptions , LayerEffect.WriteBackOffset , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## BackdropColorMatrixLayerEffect(ColorMatrix)

Initializes a new instance of the BackdropColorMatrixLayerEffect class.

```
public BackdropColorMatrixLayerEffect(ColorMatrix matrix)
```

### Parameters

`matrix` ColorMatrix

The colour matrix applied to the backdrop.

# Properties

## IsPassThrough

Gets a value indicating whether the effect leaves the layer content unchanged, in which case its application is skipped entirely.

```
public override bool IsPassThrough { get; }
```

### Property Value

bool

## Matrix

Gets the colour matrix applied to the backdrop.

```
public ColorMatrix Matrix { get; }
```

### Property Value

ColorMatrix

# Class BackdropContrastLayerEffect

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Adjusts the contrast of the backdrop beneath a compositing layer. The CSS equivalent is `backdrop-filter: contrast()`.

```
public sealed class BackdropContrastLayerEffect : BackdropColorMatrixLayerEffect
```

**Inheritance**

object ← LayerEffect ← BackdropLayerEffect ← BackdropColorMatrixLayerEffect ← BackdropContrastLayerEffect

**Inherited Members**

BackdropColorMatrixLayerEffect.Matrix , BackdropColorMatrixLayerEffect.IsPassThrough , BackdropLayerEffect.Reach , LayerEffect.WriteBackOptions , LayerEffect.WriteBackOffset , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## BackdropContrastLayerEffect(float)

Initializes a new instance of the BackdropContrastLayerEffect class.

```
public BackdropContrastLayerEffect(float amount)
```

## Parameters

`amount` float

The contrast amount: 0 is fully grey, 1 leaves the backdrop unchanged, and values above 1 increase contrast.

# Properties

## Amount

Gets the contrast amount.

```
public float Amount { get; }
```

## Property Value

float

# Class BackdropDropShadowLayerEffect

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Composites a drop shadow of the backdrop's silhouette beneath it. The CSS equivalent is `backdrop-filter: drop-shadow()`.

```
public sealed class BackdropDropShadowLayerEffect : BackdropLayerEffect
```

**Inheritance**

object ← LayerEffect ← BackdropLayerEffect ← BackdropDropShadowLayerEffect

**Inherited Members**

BackdropLayerEffect.Reach , LayerEffect.IsPassThrough , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## BackdropDropShadowLayerEffect(Point, float, Color)

Initializes a new instance of the BackdropDropShadowLayerEffect class.

```
public BackdropDropShadowLayerEffect(Point offset, float sigma, Color color)
```

### Parameters

`offset` Point

The shadow offset, in pixels.

`sigma` float

The Gaussian blur sigma, in pixels; zero draws a hard shadow.

`color` Color

The shadow colour. Its alpha scales the backdrop's alpha.

# Properties

## Color

Gets the shadow colour.

```
public Color Color { get; }
```

### Property Value

Color

## Offset

Gets the shadow offset, in pixels.

```
public Point Offset { get; }
```

### Property Value

Point

## Sigma

Gets the Gaussian blur sigma, in pixels.

```
public float Sigma { get; }
```

### Property Value

float

## WriteBackOffset

Gets the offset, in pixels, at which the processed pixels are written back relative to the region they were read from.

```
public override Point WriteBackOffset { get; }
```

### Property Value

Point

## WriteBackOptions

Gets the graphics options used to composite the processed pixels back onto the layer, or null to replace the processed region outright.

```
public override GraphicsOptions? WriteBackOptions { get; }
```

### Property Value

GraphicsOptions

# Class BackdropGrayscaleLayerEffect

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Desaturates the backdrop beneath a compositing layer towards grayscale using ITU-R BT.709 luminance coefficients, matching the CSS definition. The CSS equivalent is `backdrop-filter: grayscale()`.

```
public sealed class BackdropGrayscaleLayerEffect : BackdropColorMatrixLayerEffect
```

**Inheritance**

object ← LayerEffect ← BackdropLayerEffect ← BackdropColorMatrixLayerEffect ← BackdropGrayscaleLayerEffect

**Inherited Members**

BackdropColorMatrixLayerEffect.Matrix , BackdropColorMatrixLayerEffect.IsPassThrough , BackdropLayerEffect.Reach , LayerEffect.WriteBackOptions , LayerEffect.WriteBackOffset , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## BackdropGrayscaleLayerEffect(float)

Initializes a new instance of the BackdropGrayscaleLayerEffect class.

```
public BackdropGrayscaleLayerEffect(float amount)
```

## Parameters

`amount` float

The grayscale amount between 0 and 1: 0 leaves the backdrop unchanged and 1 is fully grayscale.

# Properties

## Amount

Gets the grayscale amount.

```
public float Amount { get; }
```

## Property Value

float

# Class BackdropHueRotateLayerEffect

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Rotates the hue of the backdrop beneath a compositing layer. The CSS equivalent is `backdrop-filter: hue-rotate()`.

```
public sealed class BackdropHueRotateLayerEffect : BackdropColorMatrixLayerEffect
```

**Inheritance**

object ← LayerEffect ← BackdropLayerEffect ← BackdropColorMatrixLayerEffect ← BackdropHueRotateLayerEffect

**Inherited Members**

BackdropColorMatrixLayerEffect.Matrix , BackdropColorMatrixLayerEffect.IsPassThrough , BackdropLayerEffect.Reach , LayerEffect.WriteBackOptions , LayerEffect.WriteBackOffset , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## BackdropHueRotateLayerEffect(float)

Initializes a new instance of the BackdropHueRotateLayerEffect class.

```
public BackdropHueRotateLayerEffect(float degrees)
```

### Parameters

`degrees` float

The hue rotation in degrees; 0 leaves the backdrop unchanged.

# Properties

# Degrees

Gets the hue rotation in degrees.

```
public float Degrees { get; }
```

## Property Value

float

# Class BackdropInvertLayerEffect

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Inverts the colours of the backdrop beneath a compositing layer. The CSS equivalent is `backdrop-filter: invert()`.

```
public sealed class BackdropInvertLayerEffect : BackdropColorMatrixLayerEffect
```

**Inheritance**

object ← LayerEffect ← BackdropLayerEffect ← BackdropColorMatrixLayerEffect ← BackdropInvertLayerEffect

**Inherited Members**

BackdropColorMatrixLayerEffect.Matrix , BackdropColorMatrixLayerEffect.IsPassThrough , BackdropLayerEffect.Reach , LayerEffect.WriteBackOptions , LayerEffect.WriteBackOffset , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## BackdropInvertLayerEffect(float)

Initializes a new instance of the BackdropInvertLayerEffect class.

```
public BackdropInvertLayerEffect(float amount)
```

### Parameters

`amount` float

The inversion amount between 0 and 1: 0 leaves the backdrop unchanged and 1 fully inverts.

# Properties

# Amount

Gets the inversion amount.

```
public float Amount { get; }
```

## Property Value

float

# Class BackdropLayerEffect

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Describes an effect applied to the backdrop of a compositing layer: the pixels already on the canvas beneath the layer's region are filtered when the layer is opened, and the layer's content then renders above the filtered backdrop. This is the CSS `backdrop-filter` model; the filtered result is clipped to the layer's region.

```
public abstract class BackdropLayerEffect : LayerEffect
```

**Inheritance**

object ← LayerEffect ← BackdropLayerEffect

**Derived**

BackdropAcrylicLayerEffect, BackdropBlurLayerEffect, BackdropColorMatrixLayerEffect, BackdropDropShadowLayerEffect, WebGPUBackdropShaderLayerEffect

**Inherited Members**

LayerEffect.IsPassThrough , LayerEffect.WriteBackOptions , LayerEffect.WriteBackOffset , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## BackdropLayerEffect(BackdropLayerEffect)

Initializes a new instance of the BackdropLayerEffect class with the observable behavior of another backdrop effect when this effect is not executed natively.

```
protected BackdropLayerEffect(BackdropLayerEffect fallbackEffect)
```

### Parameters

`fallbackEffect` BackdropLayerEffect

The backdrop effect whose behavior is used as the fallback.

## BackdropLayerEffect(Action<IImageProcessingContext>)

Initializes a new instance of the BackdropLayerEffect class.

```
protected BackdropLayerEffect(Action<IImageProcessingContext> operation)
```

### Parameters

`operation` Action<IImageProcessingContext>

The operation that implements the effect.

# Properties

## Reach

Gets the output reach. Backdrop effects filter the pixels already beneath the layer clipped to the region, so their output never extends past it.

```
public override sealed int Reach { get; }
```

### Property Value

int

# Class BackdropOpacityLayerEffect

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Scales the opacity of the backdrop beneath a compositing layer. The CSS equivalent is `backdrop-filter: opacity()`.

```
public sealed class BackdropOpacityLayerEffect : BackdropColorMatrixLayerEffect
```

**Inheritance**

object ← LayerEffect ← BackdropLayerEffect ← BackdropColorMatrixLayerEffect ← BackdropOpacityLayerEffect

**Inherited Members**

BackdropColorMatrixLayerEffect.Matrix , BackdropColorMatrixLayerEffect.IsPassThrough , BackdropLayerEffect.Reach , LayerEffect.WriteBackOptions , LayerEffect.WriteBackOffset , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## BackdropOpacityLayerEffect(float)

Initializes a new instance of the BackdropOpacityLayerEffect class.

```
public BackdropOpacityLayerEffect(float amount)
```

## Parameters

`amount` float

The opacity amount between 0 and 1: 0 is fully transparent and 1 leaves the backdrop unchanged.

# Properties

## Amount

Gets the opacity amount.

```
public float Amount { get; }
```

## Property Value

float

# Class BackdropSaturateLayerEffect

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Adjusts the colour saturation of the backdrop beneath a compositing layer. The CSS equivalent is `backdrop-filter: saturate()`.

```
public sealed class BackdropSaturateLayerEffect : BackdropColorMatrixLayerEffect
```

**Inheritance**

object ← LayerEffect ← BackdropLayerEffect ← BackdropColorMatrixLayerEffect ← BackdropSaturateLayerEffect

**Inherited Members**

BackdropColorMatrixLayerEffect.Matrix , BackdropColorMatrixLayerEffect.IsPassThrough , BackdropLayerEffect.Reach , LayerEffect.WriteBackOptions , LayerEffect.WriteBackOffset , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## BackdropSaturateLayerEffect(float)

Initializes a new instance of the BackdropSaturateLayerEffect class.

```
public BackdropSaturateLayerEffect(float amount)
```

## Parameters

`amount` float

The saturation amount: 0 is fully desaturated, 1 leaves the backdrop unchanged, and values above 1 over-saturate.

# Properties

## Amount

Gets the saturation amount.

```
public float Amount { get; }
```

## Property Value

float

# Class BackdropSepiaLayerEffect

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Shifts the colours of the backdrop beneath a compositing layer towards sepia. The CSS equivalent is `backdrop-filter: sepia()`.

```
public sealed class BackdropSepiaLayerEffect : BackdropColorMatrixLayerEffect
```

**Inheritance**

object ← LayerEffect ← BackdropLayerEffect ← BackdropColorMatrixLayerEffect ← BackdropSepiaLayerEffect

**Inherited Members**

BackdropColorMatrixLayerEffect.Matrix , BackdropColorMatrixLayerEffect.IsPassThrough , BackdropLayerEffect.Reach , LayerEffect.WriteBackOptions , LayerEffect.WriteBackOffset , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## BackdropSepiaLayerEffect(float)

Initializes a new instance of the BackdropSepiaLayerEffect class.

```
public BackdropSepiaLayerEffect(float amount)
```

### Parameters

`amount` float

The sepia amount between 0 and 1: 0 leaves the backdrop unchanged and 1 is fully sepia.

# Properties

# Amount

Gets the sepia amount.

```
public float Amount { get; }
```

## Property Value

float

# Class BlurLayerEffect

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Blurs the contents of a compositing layer when it is restored.

```
public sealed class BlurLayerEffect : LayerEffect
```

**Inheritance**

object ← LayerEffect ← BlurLayerEffect

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## BlurLayerEffect(float)

Initializes a new instance of the BlurLayerEffect class.

```
public BlurLayerEffect(float sigma)
```

### Parameters

`sigma` float

The Gaussian blur sigma, in pixels; zero leaves the content unchanged.

# Properties

## IsPassThrough

Gets a value indicating whether the effect leaves the layer content unchanged, in which case its application is skipped entirely.

```
public override bool IsPassThrough { get; }
```

### Property Value

bool

## Reach

Gets the distance, in pixels, the effect can push content beyond its source region. Layer and processing bounds are expanded by this reach so the effect output is not cut off. Effects constructed over a fallback inherit the fallback's reach.

```
public override int Reach { get; }
```

### Property Value

int

## Sigma

Gets the Gaussian blur sigma, in pixels.

```
public float Sigma { get; }
```

### Property Value

float

## WriteBackOffset

Gets the offset, in pixels, at which the processed pixels are written back relative to the region they were read from.

```
public override Point WriteBackOffset { get; }
```

## Property Value

Point

# WriteBackOptions

Gets the graphics options used to composite the processed pixels back onto the layer, or null to replace the processed region outright.

```
public override GraphicsOptions? WriteBackOptions { get; }
```

## Property Value

GraphicsOptions

# Class Brush

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Represents a logical configuration of a brush which can be used to source pixel colors.

```
public abstract class Brush : IEquatable<Brush>
```

**Inheritance**

object ← Brush

**Implements**

IEquatable<Brush>

**Derived**

GradientBrush, ImageBrush, PathGradientBrush, PatternBrush, RecolorBrush, SolidBrush

**Inherited Members**

object.Equals(object, object) , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

# Remarks

A brush creates a BrushRenderer<TPixel> that performs the logic for retrieving pixel values for specific locations.

# Methods

## CreateRenderer<TPixel>(Configuration, GraphicsOptions, int, RectangleF)

Creates the prepared execution object for this brush.

```
public abstract BrushRenderer<TPixel> CreateRenderer<TPixel>(Configuration configuration,
GraphicsOptions options, int canvasWidth, RectangleF region) where TPixel :
unmanaged, IPixel<TPixel>
```

## Parameters

`configuration` Configuration

The configuration instance to use when performing operations.

`options` GraphicsOptions

The graphic options.

`canvasWidth` int

The canvas width for the current render pass.

`region` RectangleF

The region the brush will be applied to.

## Returns

BrushRenderer<TPixel>

The BrushRenderer<TPixel> for this brush.

## Type Parameters

`TPixel`

The pixel type.

## Remarks

The `region` when being applied to things like shapes would usually be the bounding box of the shape not necessarily the bounds of the whole image.

# Equals(Brush?)

Indicates whether the current object is equal to another object of the same type.

```
public abstract bool Equals(Brush? other)
```

## Parameters

`other` Brush

An object to compare with this object.

### Returns

bool

true if the current object is equal to the `other` parameter; otherwise, false.

## Equals(object?)

Determines whether the specified object is equal to the current object.

```
public override bool Equals(object? obj)
```

### Parameters

`obj` object

The object to compare with the current object.

### Returns

bool

true if the specified object is equal to the current object; otherwise, false.

## GetHashCode()

Serves as the default hash function.

```
public override abstract int GetHashCode()
```

### Returns

int

A hash code for the current object.

# Transform(Matrix4×4, Rectangle, Rectangle)

Returns a new brush with its defining geometry transformed by the given matrix.

```
public virtual Brush Transform(Matrix4x4 matrix, Rectangle sourceInterest,
Rectangle preparedInterest)
```

## Parameters

`matrix` Matrix4×4

The transformation matrix to apply.

`sourceInterest` Rectangle

The command interest before preparation.

`preparedInterest` Rectangle

The command interest after preparation.

## Returns

Brush

A transformed brush, or `this` if the brush has no spatial parameters.

# Class BrushRenderer<TPixel>

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Renders a Brush against individual coverage scanlines.

```
public abstract class BrushRenderer<TPixel> where TPixel : unmanaged, IPixel<TPixel>
```

## Type Parameters

`TPixel`

The pixel format.

**Inheritance**

object ← BrushRenderer<TPixel>

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## BrushRenderer(Configuration, GraphicsOptions, int)

Initializes a new instance of the BrushRenderer<TPixel> class.

```
protected BrushRenderer(Configuration configuration, GraphicsOptions options,
int canvasWidth)
```

## Parameters

`configuration` Configuration

The configuration instance to use when performing operations.

`options` GraphicsOptions

The graphics options.

`canvasWidth` int

The canvas width for the current render pass.

# Properties

## CanvasWidth

Gets the canvas width for the current render pass.

```
protected int CanvasWidth { get; }
```

### Property Value

int

## Configuration

Gets the configuration instance to use when performing operations.

```
protected Configuration Configuration { get; }
```

### Property Value

Configuration

## Options

Gets the graphics options.

```
protected GraphicsOptions Options { get; }
```

## Property Value

GraphicsOptions

# Methods

## Apply(Span<TPixel>, ReadOnlySpan<float>, int, int, BrushWorkspace<TPixel>)

Applies the opacity weighting for each pixel in a scanline to the target based on the pattern contained in the brush.

```
public abstract void Apply(Span<TPixel> destinationRow, ReadOnlySpan<float> scanline, int x,
int y, BrushWorkspace<TPixel> workspace)
```

### Parameters

`destinationRow` Span<TPixel>

The destination row slice to shade.

`scanline` ReadOnlySpan<float>

The coverage values for the current destination scanline.

`x` int

The x-position in the target pixel space that the start of the scanline data corresponds to.

`y` int

The y-position in the target pixel space that the scanline corresponds to.

`workspace` BrushWorkspace<TPixel>

The worker-local scratch workspace for temporary blending buffers.

# Class BrushWorkspace<TPixel>

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Worker-local scratch workspace used by prepared brushes during row composition.

```
public sealed class BrushWorkspace<TPixel> : IDisposable where TPixel :
unmanaged, IPixel<TPixel>
```

## Type Parameters

`TPixel`

The target pixel format.

**Inheritance**

object ← BrushWorkspace<TPixel>

**Implements**

IDisposable

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Methods

## Dispose()

Performs application-defined tasks associated with freeing, releasing, or resetting unmanaged resources.

```
public void Dispose()
```

## GetAmounts(int)

Gets the shared amount buffer for the requested length.

```
public Span<float> GetAmounts(int length)
```

### Parameters

`length` int

The number of elements required.

### Returns

Span<float>

A slice of the worker-local pooled amount buffer.

## GetBlendScratch(int, int)

Gets the shared vector scratch for the requested row length and vector row count.

```
public Span<Vector4> GetBlendScratch(int length, int vectorRows)
```

### Parameters

`length` int

The number of pixels in the row.

`vectorRows` int

The number of temporary vector rows required.

### Returns

Span<Vector4>

A slice of the worker-local pooled vector scratch buffer.

## GetOverlays(int)

Gets the shared overlay buffer for the requested length.

```
public Span<TPixel> GetOverlays(int length)
```

## Parameters

`length` int

The number of elements required.

## Returns

Span<TPixel>

A slice of the worker-local pooled overlay buffer.

# Class Brushes

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

A collection of methods for creating generic brushes.

```
public static class Brushes
```

**Inheritance**

object ← Brushes

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

# Methods

## BackwardDiagonal(Color)

Creates a brush that paints diagonal line hatching from upper right to lower left using the foreground color on a transparent background.

```
public static PatternBrush BackwardDiagonal(Color foreColor)
```

## Parameters

`foreColor` Color

The foreground color.

## Returns

PatternBrush

A new PatternBrush.

## BackwardDiagonal(Color, Color)

Creates a brush that paints diagonal line hatching from upper right to lower left using the specified foreground and background colors.

```
public static PatternBrush BackwardDiagonal(Color foreColor, Color backColor)
```

### Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

### Returns

PatternBrush

A new PatternBrush.

## Cross(Color)

Creates a brush that paints intersecting horizontal and vertical line hatching using the foreground color on a transparent background.

```
public static PatternBrush Cross(Color foreColor)
```

### Parameters

`foreColor` Color

The foreground color.

### Returns

PatternBrush

A new PatternBrush.

## Cross(Color, Color)

Creates a brush that paints intersecting horizontal and vertical line hatching using the specified foreground and background colors.

```
public static PatternBrush Cross(Color foreColor, Color backColor)
```

### Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

### Returns

PatternBrush

A new PatternBrush.

## DarkDownwardDiagonal(Color)

Creates a brush that paints thicker downward diagonal lines spaced more closely than ForwardDiagonal using the foreground color on a transparent background.

```
public static PatternBrush DarkDownwardDiagonal(Color foreColor)
```

### Parameters

`foreColor` Color

The foreground color.

## Returns

PatternBrush

A new PatternBrush.

# DarkDownwardDiagonal(Color, Color)

Creates a brush that paints thicker downward diagonal lines spaced more closely than ForwardDiagonal using the specified foreground and background colors.

```
public static PatternBrush DarkDownwardDiagonal(Color foreColor, Color backColor)
```

## Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

## Returns

PatternBrush

A new PatternBrush.

# DarkHorizontal(Color)

Creates a brush that paints thicker horizontal lines spaced more closely than Horizontal using the foreground color on a transparent background.

```
public static PatternBrush DarkHorizontal(Color foreColor)
```

## Parameters

`foreColor` Color

The foreground color.

### Returns

PatternBrush

A new PatternBrush.

## DarkHorizontal(Color, Color)

Creates a brush that paints thicker horizontal lines spaced more closely than Horizontal using the specified foreground and background colors.

```
public static PatternBrush DarkHorizontal(Color foreColor, Color backColor)
```

### Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

### Returns

PatternBrush

A new PatternBrush.

## DarkUpwardDiagonal(Color)

Creates a brush that paints thicker upward diagonal lines spaced more closely than BackwardDiagonal using the foreground color on a transparent background.

```
public static PatternBrush DarkUpwardDiagonal(Color foreColor)
```

## Parameters

`foreColor` Color

The foreground color.

## Returns

PatternBrush

A new PatternBrush.

# DarkUpwardDiagonal(Color, Color)

Creates a brush that paints thicker upward diagonal lines spaced more closely than BackwardDiagonal using the specified foreground and background colors.

```
public static PatternBrush DarkUpwardDiagonal(Color foreColor, Color backColor)
```

## Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

## Returns

PatternBrush

A new PatternBrush.

# DarkVertical(Color)

Creates a brush that paints thicker vertical lines spaced more closely than Vertical using the foreground color on a transparent background.

```
public static PatternBrush DarkVertical(Color foreColor)
```

### Parameters

`foreColor` Color

The foreground color.

### Returns

PatternBrush

A new PatternBrush.

## DarkVertical(Color, Color)

Creates a brush that paints thicker vertical lines spaced more closely than Vertical using the specified foreground and background colors.

```
public static PatternBrush DarkVertical(Color foreColor, Color backColor)
```

### Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

### Returns

PatternBrush

A new PatternBrush.

## DashedDownwardDiagonal(Color)

Creates a brush that paints dashed diagonal lines from upper left to lower right using the foreground color on a transparent background.

```
public static PatternBrush DashedDownwardDiagonal(Color foreColor)
```

## Parameters

`foreColor` Color

The foreground color.

## Returns

PatternBrush

A new PatternBrush.

# DashedDownwardDiagonal(Color, Color)

Creates a brush that paints dashed diagonal lines from upper left to lower right using the specified foreground and background colors.

```
public static PatternBrush DashedDownwardDiagonal(Color foreColor, Color backColor)
```

## Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

## Returns

PatternBrush

A new PatternBrush.

## DashedHorizontal(Color)

Creates a brush that paints dashed horizontal lines using the foreground color on a transparent background.

```
public static PatternBrush DashedHorizontal(Color foreColor)
```

### Parameters

`foreColor` Color

The foreground color.

### Returns

PatternBrush

A new PatternBrush.

## DashedHorizontal(Color, Color)

Creates a brush that paints dashed horizontal lines using the specified foreground and background colors.

```
public static PatternBrush DashedHorizontal(Color foreColor, Color backColor)
```

### Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

### Returns

PatternBrush

A new PatternBrush.

## DashedUpwardDiagonal(Color)

Creates a brush that paints dashed diagonal lines from upper right to lower left using the foreground color on a transparent background.

```
public static PatternBrush DashedUpwardDiagonal(Color foreColor)
```

### Parameters

`foreColor` Color

The foreground color.

### Returns

PatternBrush

A new PatternBrush.

## DashedUpwardDiagonal(Color, Color)

Creates a brush that paints dashed diagonal lines from upper right to lower left using the specified foreground and background colors.

```
public static PatternBrush DashedUpwardDiagonal(Color foreColor, Color backColor)
```

### Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

## Returns

PatternBrush

A new PatternBrush.

# DashedVertical(Color)

Creates a brush that paints dashed vertical lines using the foreground color on a transparent background.

```
public static PatternBrush DashedVertical(Color foreColor)
```

## Parameters

`foreColor` Color

The foreground color.

## Returns

PatternBrush

A new PatternBrush.

# DashedVertical(Color, Color)

Creates a brush that paints dashed vertical lines using the specified foreground and background colors.

```
public static PatternBrush DashedVertical(Color foreColor, Color backColor)
```

## Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

### Returns

PatternBrush

A new PatternBrush.

## DiagonalBrick(Color)

Creates a brush that paints staggered brick shapes running diagonally upward using the foreground color on a transparent background.

```
public static PatternBrush DiagonalBrick(Color foreColor)
```

### Parameters

`foreColor` Color

The foreground color.

### Returns

PatternBrush

A new PatternBrush.

## DiagonalBrick(Color, Color)

Creates a brush that paints staggered brick shapes running diagonally upward using the specified foreground and background colors.

```
public static PatternBrush DiagonalBrick(Color foreColor, Color backColor)
```

### Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

## Returns

PatternBrush

A new PatternBrush.

# DiagonalCross(Color)

Creates a brush that paints intersecting forward and backward diagonal line hatching using the foreground color on a transparent background.

```
public static PatternBrush DiagonalCross(Color foreColor)
```

## Parameters

`foreColor` Color

The foreground color.

## Returns

PatternBrush

A new PatternBrush.

# DiagonalCross(Color, Color)

Creates a brush that paints intersecting forward and backward diagonal line hatching using the specified foreground and background colors.

```
public static PatternBrush DiagonalCross(Color foreColor, Color backColor)
```

## Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

## Returns

PatternBrush

A new PatternBrush.

# Divot(Color)

Creates a brush that paints a divot-style hatch using the foreground color on a transparent background.

```
public static PatternBrush Divot(Color foreColor)
```

## Parameters

`foreColor` Color

The foreground color.

## Returns

PatternBrush

A new PatternBrush.

# Divot(Color, Color)

Creates a brush that paints a divot-style hatch using the specified foreground and background colors.

```
public static PatternBrush Divot(Color foreColor, Color backColor)
```

### Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

### Returns

PatternBrush

A new PatternBrush.

## DottedDiamond(Color)

Creates a brush that paints intersecting forward and backward diagonal dotted lines using the foreground color on a transparent background.

```
public static PatternBrush DottedDiamond(Color foreColor)
```

### Parameters

`foreColor` Color

The foreground color.

### Returns

PatternBrush

A new PatternBrush.

## DottedDiamond(Color, Color)

Creates a brush that paints intersecting forward and backward diagonal dotted lines using the specified foreground and background colors.

```
public static PatternBrush DottedDiamond(Color foreColor, Color backColor)
```

## Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

## Returns

PatternBrush

A new PatternBrush.

# DottedGrid(Color)

Creates a brush that paints intersecting horizontal and vertical dotted lines using the foreground color on a transparent background.

```
public static PatternBrush DottedGrid(Color foreColor)
```

## Parameters

`foreColor` Color

The foreground color.

## Returns

PatternBrush

A new PatternBrush.

## DottedGrid(Color, Color)

Creates a brush that paints intersecting horizontal and vertical dotted lines using the specified foreground and background colors.

```
public static PatternBrush DottedGrid(Color foreColor, Color backColor)
```

### Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

### Returns

PatternBrush

A new PatternBrush.

## ForwardDiagonal(Color)

Creates a brush that paints diagonal line hatching from upper left to lower right using the foreground color on a transparent background.

```
public static PatternBrush ForwardDiagonal(Color foreColor)
```

### Parameters

`foreColor` Color

The foreground color.

### Returns

PatternBrush

A new PatternBrush.

## ForwardDiagonal(Color, Color)

Creates a brush that paints diagonal line hatching from upper left to lower right using the specified foreground and background colors.

```
public static PatternBrush ForwardDiagonal(Color foreColor, Color backColor)
```

### Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

### Returns

PatternBrush

A new PatternBrush.

## Horizontal(Color)

Creates a brush that paints horizontal line hatching using the foreground color on a transparent background.

```
public static PatternBrush Horizontal(Color foreColor)
```

### Parameters

`foreColor` Color

The foreground color.

## Returns

PatternBrush

A new PatternBrush.

# Horizontal(Color, Color)

Creates a brush that paints horizontal line hatching using the specified foreground and background colors.

```
public static PatternBrush Horizontal(Color foreColor, Color backColor)
```

## Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

## Returns

PatternBrush

A new PatternBrush.

# HorizontalBrick(Color)

Creates a brush that paints staggered brick shapes arranged horizontally using the foreground color on a transparent background.

```
public static PatternBrush HorizontalBrick(Color foreColor)
```

## Parameters

`foreColor` Color

The foreground color.

### Returns

PatternBrush

A new PatternBrush.

## HorizontalBrick(Color, Color)

Creates a brush that paints staggered brick shapes arranged horizontally using the specified foreground and background colors.

```
public static PatternBrush HorizontalBrick(Color foreColor, Color backColor)
```

### Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

### Returns

PatternBrush

A new PatternBrush.

## LargeCheckerBoard(Color)

Creates a brush that paints a checkerboard hatch with larger squares than SmallCheckerBoard using the foreground color on a transparent background.

```
public static PatternBrush LargeCheckerBoard(Color foreColor)
```

## Parameters

`foreColor` Color

The foreground color.

## Returns

PatternBrush

A new PatternBrush.

# LargeCheckerBoard(Color, Color)

Creates a brush that paints a checkerboard hatch with larger squares than SmallCheckerBoard using the specified foreground and background colors.

```
public static PatternBrush LargeCheckerBoard(Color foreColor, Color backColor)
```

## Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

## Returns

PatternBrush

A new PatternBrush.

# LargeConfetti(Color)

Creates a brush that paints a confetti-style hatch with larger pieces than SmallConfetti using the foreground color on a transparent background.

```
public static PatternBrush LargeConfetti(Color foreColor)
```

## Parameters

`foreColor` Color

The foreground color.

## Returns

PatternBrush

A new PatternBrush.

# LargeConfetti(Color, Color)

Creates a brush that paints a confetti-style hatch with larger pieces than SmallConfetti using the specified foreground and background colors.

```
public static PatternBrush LargeConfetti(Color foreColor, Color backColor)
```

## Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

## Returns

PatternBrush

A new PatternBrush.

# LightDownwardDiagonal(Color)

Creates a brush that paints downward diagonal lines spaced more closely than ForwardDiagonal using the foreground color on a transparent background.

```
public static PatternBrush LightDownwardDiagonal(Color foreColor)
```

## Parameters

`foreColor` Color

The foreground color.

## Returns

PatternBrush

A new PatternBrush.

# LightDownwardDiagonal(Color, Color)

Creates a brush that paints downward diagonal lines spaced more closely than ForwardDiagonal using the specified foreground and background colors.

```
public static PatternBrush LightDownwardDiagonal(Color foreColor, Color backColor)
```

## Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

## Returns

PatternBrush

A new PatternBrush.

# LightHorizontal(Color)

Creates a brush that paints horizontal lines spaced more closely than Horizontal using the foreground color on a transparent background.

```
public static PatternBrush LightHorizontal(Color foreColor)
```

## Parameters

`foreColor` Color

The foreground color.

## Returns

PatternBrush

A new PatternBrush.

# LightHorizontal(Color, Color)

Creates a brush that paints horizontal lines spaced more closely than Horizontal using the specified foreground and background colors.

```
public static PatternBrush LightHorizontal(Color foreColor, Color backColor)
```

## Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

## Returns

PatternBrush

A new PatternBrush.

## LightUpwardDiagonal(Color)

Creates a brush that paints upward diagonal lines spaced more closely than BackwardDiagonal using the foreground color on a transparent background.

```
public static PatternBrush LightUpwardDiagonal(Color foreColor)
```

### Parameters

`foreColor` Color

The foreground color.

### Returns

PatternBrush

A new PatternBrush.

## LightUpwardDiagonal(Color, Color)

Creates a brush that paints upward diagonal lines spaced more closely than BackwardDiagonal using the specified foreground and background colors.

```
public static PatternBrush LightUpwardDiagonal(Color foreColor, Color backColor)
```

### Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

## Returns

PatternBrush

A new PatternBrush.

# LightVertical(Color)

Creates a brush that paints vertical lines spaced more closely than Vertical using the foreground color on a transparent background.

```
public static PatternBrush LightVertical(Color foreColor)
```

## Parameters

`foreColor` Color

The foreground color.

## Returns

PatternBrush

A new PatternBrush.

# LightVertical(Color, Color)

Creates a brush that paints vertical lines spaced more closely than Vertical using the specified foreground and background colors.

```
public static PatternBrush LightVertical(Color foreColor, Color backColor)
```

## Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

### Returns

PatternBrush

A new PatternBrush.

## Min(Color)

Creates a brush that paints horizontal line hatching for the minimum hatch style using the foreground color on a transparent background.

```
public static PatternBrush Min(Color foreColor)
```

### Parameters

`foreColor` Color

The foreground color.

### Returns

PatternBrush

A new PatternBrush.

## Min(Color, Color)

Creates a brush that paints horizontal line hatching for the minimum hatch style using the specified foreground and background colors.

```
public static PatternBrush Min(Color foreColor, Color backColor)
```

### Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

## Returns

PatternBrush

A new PatternBrush.

# NarrowHorizontal(Color)

Creates a brush that paints narrow horizontal lines spaced more closely than LightHorizontal using the foreground color on a transparent background.

```
public static PatternBrush NarrowHorizontal(Color foreColor)
```

## Parameters

`foreColor` Color

The foreground color.

## Returns

PatternBrush

A new PatternBrush.

# NarrowHorizontal(Color, Color)

Creates a brush that paints narrow horizontal lines spaced more closely than LightHorizontal using the specified foreground and background colors.

```
public static PatternBrush NarrowHorizontal(Color foreColor, Color backColor)
```

## Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

## Returns

PatternBrush

A new PatternBrush.

# NarrowVertical(Color)

Creates a brush that paints narrow vertical lines spaced more closely than LightVertical using the foreground color on a transparent background.

```
public static PatternBrush NarrowVertical(Color foreColor)
```

## Parameters

`foreColor` Color

The foreground color.

## Returns

PatternBrush

A new PatternBrush.

# NarrowVertical(Color, Color)

Creates a brush that paints narrow vertical lines spaced more closely than LightVertical using the specified foreground and background colors.

```
public static PatternBrush NarrowVertical(Color foreColor, Color backColor)
```

## Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

## Returns

PatternBrush

A new PatternBrush.

# OutlinedDiamond(Color)

Creates a brush that paints outlined diamond shapes formed by crossing diagonal lines using the foreground color on a transparent background.

```
public static PatternBrush OutlinedDiamond(Color foreColor)
```

## Parameters

`foreColor` Color

The foreground color.

## Returns

PatternBrush

A new PatternBrush.

# OutlinedDiamond(Color, Color)

Creates a brush that paints outlined diamond shapes formed by crossing diagonal lines using the specified foreground and background colors.

```
public static PatternBrush OutlinedDiamond(Color foreColor, Color backColor)
```

## Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

## Returns

PatternBrush

A new PatternBrush.

# Percent05(Color)

Creates a brush that paints a 5-percent hatch using the foreground color on a transparent background; the foreground-to-background color ratio is 5:95.

```
public static PatternBrush Percent05(Color foreColor)
```

## Parameters

`foreColor` Color

The foreground color.

## Returns

PatternBrush

A new PatternBrush.

# Percent05(Color, Color)

Creates a brush that paints a 5-percent hatch using the specified foreground and background colors; the foreground-to-background color ratio is 5:95.

```
public static PatternBrush Percent05(Color foreColor, Color backColor)
```

## Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

## Returns

PatternBrush

A new PatternBrush.

# Percent10(Color)

Creates a brush that paints a 10-percent hatch using the foreground color on a transparent background; the foreground-to-background color ratio is 10:90.

```
public static PatternBrush Percent10(Color foreColor)
```

## Parameters

`foreColor` Color

The foreground color.

## Returns

PatternBrush

A new PatternBrush.

## Percent10(Color, Color)

Creates a brush that paints a 10-percent hatch using the specified foreground and background colors; the foreground-to-background color ratio is 10:90.

```
public static PatternBrush Percent10(Color foreColor, Color backColor)
```

### Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

### Returns

PatternBrush

A new PatternBrush.

## Percent20(Color)

Creates a brush that paints a 20-percent hatch using the foreground color on a transparent background; the foreground-to-background color ratio is 20:80.

```
public static PatternBrush Percent20(Color foreColor)
```

### Parameters

`foreColor` Color

The foreground color.

## Returns

PatternBrush

A new PatternBrush.

# Percent20(Color, Color)

Creates a brush that paints a 20-percent hatch using the specified foreground and background colors; the foreground-to-background color ratio is 20:80.

```
public static PatternBrush Percent20(Color foreColor, Color backColor)
```

## Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

## Returns

PatternBrush

A new PatternBrush.

# Percent25(Color)

Creates a brush that paints a 25-percent hatch using the foreground color on a transparent background; the foreground-to-background color ratio is 25:75.

```
public static PatternBrush Percent25(Color foreColor)
```

## Parameters

`foreColor` Color

The foreground color.

### Returns

PatternBrush

A new PatternBrush.

## Percent25(Color, Color)

Creates a brush that paints a 25-percent hatch using the specified foreground and background colors; the foreground-to-background color ratio is 25:75.

```
public static PatternBrush Percent25(Color foreColor, Color backColor)
```

### Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

### Returns

PatternBrush

A new PatternBrush.

## Percent30(Color)

Creates a brush that paints a 30-percent hatch using the foreground color on a transparent background; the foreground-to-background color ratio is 30:70.

```
public static PatternBrush Percent30(Color foreColor)
```

## Parameters

`foreColor` Color

The foreground color.

## Returns

PatternBrush

A new PatternBrush.

# Percent30(Color, Color)

Creates a brush that paints a 30-percent hatch using the specified foreground and background colors; the foreground-to-background color ratio is 30:70.

```
public static PatternBrush Percent30(Color foreColor, Color backColor)
```

## Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

## Returns

PatternBrush

A new PatternBrush.

# Percent40(Color)

Creates a brush that paints a 40-percent hatch using the foreground color on a transparent background; the foreground-to-background color ratio is 40:60.

```
public static PatternBrush Percent40(Color foreColor)
```

## Parameters

`foreColor` Color

The foreground color.

## Returns

PatternBrush

A new PatternBrush.

# Percent40(Color, Color)

Creates a brush that paints a 40-percent hatch using the specified foreground and background colors; the foreground-to-background color ratio is 40:60.

```
public static PatternBrush Percent40(Color foreColor, Color backColor)
```

## Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

## Returns

PatternBrush

A new PatternBrush.

# Percent50(Color)

Creates a brush that paints a 50-percent hatch using the foreground color on a transparent background; the foreground-to-background color ratio is 50:50.

```
public static PatternBrush Percent50(Color foreColor)
```

## Parameters

`foreColor` Color

The foreground color.

## Returns

PatternBrush

A new PatternBrush.

# Percent50(Color, Color)

Creates a brush that paints a 50-percent hatch using the specified foreground and background colors; the foreground-to-background color ratio is 50:50.

```
public static PatternBrush Percent50(Color foreColor, Color backColor)
```

## Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

## Returns

PatternBrush

A new PatternBrush.

# Percent60(Color)

Creates a brush that paints a 60-percent hatch using the foreground color on a transparent background; the foreground-to-background color ratio is 60:40.

```
public static PatternBrush Percent60(Color foreColor)
```

## Parameters

`foreColor` Color

The foreground color.

## Returns

PatternBrush

A new PatternBrush.

# Percent60(Color, Color)

Creates a brush that paints a 60-percent hatch using the specified foreground and background colors; the foreground-to-background color ratio is 60:40.

```
public static PatternBrush Percent60(Color foreColor, Color backColor)
```

## Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

## Returns

PatternBrush

A new PatternBrush.

## Percent70(Color)

Creates a brush that paints a 70-percent hatch using the foreground color on a transparent background; the foreground-to-background color ratio is 70:30.

```
public static PatternBrush Percent70(Color foreColor)
```

### Parameters

`foreColor` Color

The foreground color.

### Returns

PatternBrush

A new PatternBrush.

## Percent70(Color, Color)

Creates a brush that paints a 70-percent hatch using the specified foreground and background colors; the foreground-to-background color ratio is 70:30.

```
public static PatternBrush Percent70(Color foreColor, Color backColor)
```

### Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

## Returns

PatternBrush

A new PatternBrush.

# Percent75(Color)

Creates a brush that paints a 75-percent hatch using the foreground color on a transparent background; the foreground-to-background color ratio is 75:25.

```
public static PatternBrush Percent75(Color foreColor)
```

## Parameters

`foreColor` Color

The foreground color.

## Returns

PatternBrush

A new PatternBrush.

# Percent75(Color, Color)

Creates a brush that paints a 75-percent hatch using the specified foreground and background colors; the foreground-to-background color ratio is 75:25.

```
public static PatternBrush Percent75(Color foreColor, Color backColor)
```

## Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

### Returns

PatternBrush

A new PatternBrush.

## Percent80(Color)

Creates a brush that paints an 80-percent hatch using the foreground color on a transparent background; the foreground-to-background color ratio is 80:20.

```
public static PatternBrush Percent80(Color foreColor)
```

### Parameters

`foreColor` Color

The foreground color.

### Returns

PatternBrush

A new PatternBrush.

## Percent80(Color, Color)

Creates a brush that paints an 80-percent hatch using the specified foreground and background colors; the foreground-to-background color ratio is 80:20.

```
public static PatternBrush Percent80(Color foreColor, Color backColor)
```

### Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

### Returns

PatternBrush

A new PatternBrush.

## Percent90(Color)

Creates a brush that paints a 90-percent hatch using the foreground color on a transparent background; the foreground-to-background color ratio is 90:10.

```
public static PatternBrush Percent90(Color foreColor)
```

### Parameters

`foreColor` Color

The foreground color.

### Returns

PatternBrush

A new PatternBrush.

## Percent90(Color, Color)

Creates a brush that paints a 90-percent hatch using the specified foreground and background colors; the foreground-to-background color ratio is 90:10.

```
public static PatternBrush Percent90(Color foreColor, Color backColor)
```

## Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

## Returns

PatternBrush

A new PatternBrush.

# Plaid(Color)

Creates a brush that paints a plaid-material hatch using the foreground color on a transparent background.

```
public static PatternBrush Plaid(Color foreColor)
```

## Parameters

`foreColor` Color

The foreground color.

## Returns

PatternBrush

A new PatternBrush.

# Plaid(Color, Color)

Creates a brush that paints a plaid-material hatch using the specified foreground and background colors.

```
public static PatternBrush Plaid(Color foreColor, Color backColor)
```

## Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

## Returns

PatternBrush

A new PatternBrush.

# Shingle(Color)

Creates a brush that paints layered shingle shapes running diagonally downward using the foreground color on a transparent background.

```
public static PatternBrush Shingle(Color foreColor)
```

## Parameters

`foreColor` Color

The foreground color.

## Returns

PatternBrush

A new PatternBrush.

# Shingle(Color, Color)

Creates a brush that paints layered shingle shapes running diagonally downward using the specified foreground and background colors.

```
public static PatternBrush Shingle(Color foreColor, Color backColor)
```

## Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

## Returns

PatternBrush

A new PatternBrush.

# SmallCheckerBoard(Color)

Creates a brush that paints a small checkerboard hatch using the foreground color on a transparent background.

```
public static PatternBrush SmallCheckerBoard(Color foreColor)
```

## Parameters

`foreColor` Color

The foreground color.

## Returns

PatternBrush

A new PatternBrush.

## SmallCheckerBoard(Color, Color)

Creates a brush that paints a small checkerboard hatch using the specified foreground and background colors.

```
public static PatternBrush SmallCheckerBoard(Color foreColor, Color backColor)
```

### Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

### Returns

PatternBrush

A new PatternBrush.

## SmallConfetti(Color)

Creates a brush that paints a small confetti-style hatch using the foreground color on a transparent background.

```
public static PatternBrush SmallConfetti(Color foreColor)
```

### Parameters

`foreColor` Color

The foreground color.

### Returns

PatternBrush

A new PatternBrush.

## SmallConfetti(Color, Color)

Creates a brush that paints a small confetti-style hatch using the specified foreground and background colors.

```
public static PatternBrush SmallConfetti(Color foreColor, Color backColor)
```

### Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

### Returns

PatternBrush

A new PatternBrush.

## SmallGrid(Color)

Creates a brush that paints intersecting horizontal and vertical lines spaced more closely than Cross using the foreground color on a transparent background.

```
public static PatternBrush SmallGrid(Color foreColor)
```

### Parameters

`foreColor` Color

The foreground color.

## Returns

PatternBrush

A new PatternBrush.

# SmallGrid(Color, Color)

Creates a brush that paints intersecting horizontal and vertical lines spaced more closely than Cross using the specified foreground and background colors.

```
public static PatternBrush SmallGrid(Color foreColor, Color backColor)
```

## Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

## Returns

PatternBrush

A new PatternBrush.

# Solid(Color)

Creates a brush that paints a solid color.

```
public static SolidBrush Solid(Color color)
```

## Parameters

`color` Color

The brush color.

### Returns

SolidBrush

A new SolidBrush.

## SolidDiamond(Color)

Creates a brush that paints a filled diamond checkerboard hatch using the foreground color on a transparent background.

```
public static PatternBrush SolidDiamond(Color foreColor)
```

### Parameters

`foreColor` Color

The foreground color.

### Returns

PatternBrush

A new PatternBrush.

## SolidDiamond(Color, Color)

Creates a brush that paints a filled diamond checkerboard hatch using the specified foreground and background colors.

```
public static PatternBrush SolidDiamond(Color foreColor, Color backColor)
```

### Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

## Returns

PatternBrush

A new PatternBrush.

# Sphere(Color)

Creates a brush that paints adjacent sphere-like shapes using the foreground color on a transparent background.

```
public static PatternBrush Sphere(Color foreColor)
```

## Parameters

`foreColor` Color

The foreground color.

## Returns

PatternBrush

A new PatternBrush.

# Sphere(Color, Color)

Creates a brush that paints adjacent sphere-like shapes using the specified foreground and background colors.

```
public static PatternBrush Sphere(Color foreColor, Color backColor)
```

## Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

## Returns

PatternBrush

A new PatternBrush.

# Trellis(Color)

Creates a brush that paints a trellis-style hatch using the foreground color on a transparent background.

```
public static PatternBrush Trellis(Color foreColor)
```

## Parameters

`foreColor` Color

The foreground color.

## Returns

PatternBrush

A new PatternBrush.

# Trellis(Color, Color)

Creates a brush that paints a trellis-style hatch using the specified foreground and background colors.

```
public static PatternBrush Trellis(Color foreColor, Color backColor)
```

### Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

### Returns

PatternBrush

A new PatternBrush.

## Vertical(Color)

Creates a brush that paints vertical line hatching using the foreground color on a transparent background.

```
public static PatternBrush Vertical(Color foreColor)
```

### Parameters

`foreColor` Color

The foreground color.

### Returns

PatternBrush

A new PatternBrush.

## Vertical(Color, Color)

Creates a brush that paints vertical line hatching using the specified foreground and background colors.

```
public static PatternBrush Vertical(Color foreColor, Color backColor)
```

## Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

## Returns

PatternBrush

A new PatternBrush.

# Wave(Color)

Creates a brush that paints horizontal lines formed from wave shapes using the foreground color on a transparent background.

```
public static PatternBrush Wave(Color foreColor)
```

## Parameters

`foreColor` Color

The foreground color.

## Returns

PatternBrush

A new PatternBrush.

## Wave(Color, Color)

Creates a brush that paints horizontal lines formed from wave shapes using the specified foreground and background colors.

```
public static PatternBrush Wave(Color foreColor, Color backColor)
```

### Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

### Returns

PatternBrush

A new PatternBrush.

## Weave(Color)

Creates a brush that paints a woven-material hatch using the foreground color on a transparent background.

```
public static PatternBrush Weave(Color foreColor)
```

### Parameters

`foreColor` Color

The foreground color.

### Returns

PatternBrush

A new PatternBrush.

## Weave(Color, Color)

Creates a brush that paints a woven-material hatch using the specified foreground and background colors.

```
public static PatternBrush Weave(Color foreColor, Color backColor)
```

### Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

### Returns

PatternBrush

A new PatternBrush.

## WideDownwardDiagonal(Color)

Creates a brush that paints wide downward diagonal lines with ForwardDiagonal spacing using the foreground color on a transparent background.

```
public static PatternBrush WideDownwardDiagonal(Color foreColor)
```

### Parameters

`foreColor` Color

The foreground color.

## Returns

PatternBrush

A new PatternBrush.

# WideDownwardDiagonal(Color, Color)

Creates a brush that paints wide downward diagonal lines with ForwardDiagonal spacing using the specified foreground and background colors.

```
public static PatternBrush WideDownwardDiagonal(Color foreColor, Color backColor)
```

## Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

## Returns

PatternBrush

A new PatternBrush.

# WideUpwardDiagonal(Color)

Creates a brush that paints wide upward diagonal lines with BackwardDiagonal spacing using the foreground color on a transparent background.

```
public static PatternBrush WideUpwardDiagonal(Color foreColor)
```

## Parameters

`foreColor` Color

The foreground color.

### Returns

PatternBrush

A new PatternBrush.

## WideUpwardDiagonal(Color, Color)

Creates a brush that paints wide upward diagonal lines with BackwardDiagonal spacing using the specified foreground and background colors.

```
public static PatternBrush WideUpwardDiagonal(Color foreColor, Color backColor)
```

### Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

### Returns

PatternBrush

A new PatternBrush.

## ZigZag(Color)

Creates a brush that paints horizontal lines formed from zigzags using the foreground color on a transparent background.

```
public static PatternBrush ZigZag(Color foreColor)
```

## Parameters

`foreColor` Color

The foreground color.

## Returns

PatternBrush

A new PatternBrush.

# ZigZag(Color, Color)

Creates a brush that paints horizontal lines formed from zigzags using the specified foreground and background colors.

```
public static PatternBrush ZigZag(Color foreColor, Color backColor)
```

## Parameters

`foreColor` Color

The foreground color.

`backColor` Color

The background color.

## Returns

PatternBrush

A new PatternBrush.

# Delegate CanvasAction

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Represents the per-frame painting callback executed by Paint(IImageProcessingContext, CanvasAction).

```
public delegate void CanvasAction(DrawingCanvas canvas)
```

## Parameters

`canvas` DrawingCanvas

The drawing canvas for the current image frame.

# Class ColorMatrixLayerEffect

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Transforms the colours of a compositing layer's contents through a colour matrix when the layer is restored. Covers the CSS filter family: grayscale, sepia, saturation, hue rotation, brightness, contrast, and invert are all colour matrices.

```
public sealed class ColorMatrixLayerEffect : LayerEffect
```

**Inheritance**

object ← LayerEffect ← ColorMatrixLayerEffect

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## ColorMatrixLayerEffect(ColorMatrix)

Initializes a new instance of the ColorMatrixLayerEffect class.

```
public ColorMatrixLayerEffect(ColorMatrix matrix)
```

### Parameters

`matrix` ColorMatrix

The colour matrix applied to the layer content.

# Properties

## IsPassThrough

Gets a value indicating whether the effect leaves the layer content unchanged, in which case its application is skipped entirely.

```
public override bool IsPassThrough { get; }
```

### Property Value

bool

## Matrix

Gets the colour matrix applied to the layer content.

```
public ColorMatrix Matrix { get; }
```

### Property Value

ColorMatrix

## Reach

Gets the distance, in pixels, the effect can push content beyond its source region. Layer and processing bounds are expanded by this reach so the effect output is not cut off. Effects constructed over a fallback inherit the fallback's reach.

```
public override int Reach { get; }
```

### Property Value

int

## WriteBackOffset

Gets the offset, in pixels, at which the processed pixels are written back relative to the region they were read from.

```
public override Point WriteBackOffset { get; }
```

## Property Value

Point

# WriteBackOptions

Gets the graphics options used to composite the processed pixels back onto the layer, or null to replace the processed region outright.

```
public override GraphicsOptions? WriteBackOptions { get; }
```

## Property Value

GraphicsOptions

# Struct ColorStop

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

A struct that defines a single color stop: a color pinned to a position along a gradient.

```
public readonly struct ColorStop
```

**Inherited Members**
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

# Constructors

## ColorStop(float, in Color)

Initializes a new instance of the ColorStop struct.

```
public ColorStop(float ratio, in Color color)
```

### Parameters

`ratio` float

The position of the stop along the gradient, where 0 is the start and 1 is the end of the gradient.

`color` Color

The color of the gradient at the stop position.

# Properties

## Color

Gets the color of the gradient at the stop position.

```
public Color Color { get; }
```

## Property Value

Color

# Ratio

Gets the position of the stop along the gradient, where 0 is the start and 1 is the end.

```
public float Ratio { get; }
```

## Property Value

float

# Class DrawingCanvas

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Represents a stateful, retained-mode drawing canvas over a frame target.

```
public abstract class DrawingCanvas : IDisposable
```

**Inheritance**

object ← DrawingCanvas

**Implements**

IDisposable

**Derived**

DrawingCanvas<TPixel>

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

# Remarks

Draw calls are recorded into an ordered command stream rather than rasterized immediately; the root canvas replays the recorded timeline when it is disposed. Drawing state (options, transform, clip and layer scope) is managed through the Save()/Restore() state stack.

# Properties

## Bounds

Gets the local bounds of this canvas.

```
public abstract Rectangle Bounds { get; }
```

## Property Value

Rectangle

## SaveCount

Gets the number of saved states currently on the canvas stack.

```
public abstract int SaveCount { get; }
```

## Property Value

int

# Methods

## Apply(IPath, Action<IImageProcessingContext>)

Applies an image-processing operation to a path region.

```
public abstract void Apply(IPath path, Action<IImageProcessingContext> operation)
```

## Parameters

`path` IPath

The path region to process.

`operation` Action<IImageProcessingContext>

The image-processing operation to apply to the region.

## Remarks

The operation affects only pixels covered by the supplied path.

# Apply(IPath, Action<IImageProcessingContext>, GraphicsOptions, Point)

Applies an image-processing operation to a path region and composites the processed pixels back with explicit options, optionally offset from where they were read.

```
public abstract void Apply(IPath path, Action<IImageProcessingContext> operation,
GraphicsOptions writeBackOptions, Point writeBackOffset)
```

## Parameters

`path` IPath

The path region to process.

`operation` Action<IImageProcessingContext>

The image-processing operation to apply to the region.

`writeBackOptions` GraphicsOptions

The graphics options used to composite the processed pixels back.

`writeBackOffset` Point

The offset at which the processed pixels are written back.

## Remarks

The write-back affects only pixels covered by the supplied path translated by the offset.

# Apply(PathBuilder, Action<IImageProcessingContext>)

Applies an image-processing operation to a region described by a path builder.

```
public abstract void Apply(PathBuilder pathBuilder, Action<IImageProcessingContext>
operation)
```

## Parameters

`pathBuilder` PathBuilder

The path builder describing the region to process.

`operation` Action<IImageProcessingContext>

The image-processing operation to apply to the region.

## Apply(Rectangle, Action<IImageProcessingContext>)

Applies an image-processing operation to a local region.

```
public abstract void Apply(Rectangle region, Action<IImageProcessingContext> operation)
```

### Parameters

`region` Rectangle

The local region to process.

`operation` Action<IImageProcessingContext>

The image-processing operation to apply to the region.

## Apply(Rectangle, Action<IImageProcessingContext>, GraphicsOptions, Point)

Applies an image-processing operation to a local region and composites the processed pixels back with explicit options, optionally offset from where they were read. The target region is untouched while the operation runs, so the processed pixels can be blended against the original content; for example a drop shadow composites the tinted, blurred region back beneath the content with DestOver at the shadow offset.

```
public abstract void Apply(Rectangle region, Action<IImageProcessingContext> operation,
GraphicsOptions writeBackOptions, Point writeBackOffset)
```

### Parameters

`region` Rectangle

The local region to process.

`operation` Action<IImageProcessingContext>

The image-processing operation to apply to the region.

`writeBackOptions` GraphicsOptions

The graphics options used to composite the processed pixels back.

`writeBackOffset` Point

The offset at which the processed pixels are written back.

## Clear(Brush)

Clears the whole canvas with the given brush using replacement composition: the brush output overwrites the covered pixels outright, including their alpha, rather than blending over them.

```
public void Clear(Brush brush)
```

### Parameters

`brush` Brush

Brush used to shade the destination pixels.

## Clear(Brush, IPath)

Fills a path region with the given brush using replacement composition: the brush output overwrites the covered pixels outright, including their alpha, rather than blending over them.

```
public abstract void Clear(Brush brush, IPath path)
```

### Parameters

`brush` Brush

Brush used to shade destination pixels during the clear.

`path` IPath

The path region to clear.

## Clear(Brush, Rectangle)

Clears a local region with the given brush using replacement composition: the brush output overwrites the covered pixels outright, including their alpha, rather than blending over them.

```
public void Clear(Brush brush, Rectangle region)
```

### Parameters

`brush` Brush

Brush used to shade the destination pixels.

`region` Rectangle

Region to clear in local coordinates.

## Clip(ClipOperation, params IPath[])

Narrows the current clip region by applying the specified clipping operation with the supplied clip paths.

```
public abstract void Clip(ClipOperation operation, params IPath[] clipPaths)
```

### Parameters

`operation` ClipOperation

The operation to apply to the current clip.

`clipPaths` IPath[]

The clip paths to combine with the current clip, in local coordinates.

### Remarks

The clip paths are transformed by the active transform at the point this is called.

# Clip(params IPath[])

Narrows the current clip region by intersecting it with the supplied clip paths.

```
public abstract void Clip(params IPath[] clipPaths)
```

## Parameters

`clipPaths` IPath[]

The clip paths to intersect with the current clip, in local coordinates.

## Remarks

The clip paths are transformed by the active transform at the point this is called, then intersected with the existing clip; clipping only ever narrows. The resulting clip is part of the current saved state and is restored by Restore(). Multiple paths combine as a union before intersecting (e.g. a region built from several rectangles).

# CopyPixelsFrom(DrawingCanvas, Rectangle, Point)

Copies pixels from another canvas into this canvas.

```
public abstract void CopyPixelsFrom(DrawingCanvas source, Rectangle sourceRectangle,
Point targetPoint)
```

## Parameters

`source` DrawingCanvas

The source canvas.

`sourceRectangle` Rectangle

The source rectangle in source-local coordinates.

`targetPoint` Point

The target point in this canvas' local coordinates.

## Remarks

Any queued commands on both canvases are applied before the copy.

# CreateRegion(Rectangle)

Creates a child canvas over a subregion in local coordinates.

```
public abstract DrawingCanvas CreateRegion(Rectangle region)
```

## Parameters

`region` Rectangle

The child region in local coordinates.

## Returns

DrawingCanvas

A child canvas with local origin at (0,0).

# CreateScene()

Seals the commands recorded so far into a retained backend scene and clears them from the canvas queue.

```
public abstract DrawingBackendScene CreateScene()
```

## Returns

DrawingBackendScene

The created backend scene.

## Remarks

The active clip range is closed before the scene is built and reopened afterwards, so this acts as an ordering boundary in the recorded timeline. Image resources retained for the sealed

commands are transferred to the returned scene, which owns and disposes them.

## Dispose()

Releases the canvas. Disposing the root canvas closes any open layers and clips and replays the recorded command timeline into the target frame; child region canvases share the root's command stream and do not flush on disposal.

```
public abstract void Dispose()
```

### Remarks

The root canvas also releases image resources retained for deferred draws and, when it owns the text drawing cache, clears that cache. Disposal is idempotent.

## Draw(Pen, IPath)

Draws a path outline in local coordinates using the given pen.

```
public abstract void Draw(Pen pen, IPath path)
```

### Parameters

`pen` Pen

Pen used to generate the outline fill path.

`path` IPath

The path to stroke.

## Draw(Pen, IPathCollection)

Draws all paths in a collection using the provided pen.

```
public void Draw(Pen pen, IPathCollection paths)
```

## Parameters

`pen` Pen

Pen used to generate outlines.

`paths` IPathCollection

Path collection to stroke.

# Draw(Pen, PathBuilder)

Draws a path outline built by the provided builder using the given pen.

```
public void Draw(Pen pen, PathBuilder pathBuilder)
```

## Parameters

`pen` Pen

Pen used to generate the outline fill path.

`pathBuilder` PathBuilder

The path builder describing the path to stroke.

# Draw(Pen, Rectangle)

Draws a rectangular outline using the provided pen.

```
public void Draw(Pen pen, Rectangle region)
```

## Parameters

`pen` Pen

Pen used to generate the rectangle outline.

`region` Rectangle

Rectangle region to stroke.

## DrawArc(Pen, PointF, SizeF, float, float, float)

Draws an arc outline using the provided pen.

```
public void DrawArc(Pen pen, PointF center, SizeF radius, float rotation, float startAngle,
float sweepAngle)
```

### Parameters

`pen` Pen

Pen used to generate the arc outline.

`center` PointF

Arc center point in local coordinates.

`radius` SizeF

Arc radii in local coordinates.

`rotation` float

Ellipse rotation in degrees.

`startAngle` float

Arc start angle in degrees.

`sweepAngle` float

Arc sweep angle in degrees.

## DrawBezier(Pen, params PointF[])

Draws a cubic bezier outline using the provided pen.

```
public void DrawBezier(Pen pen, params PointF[] points)
```

## Parameters

`pen` Pen

Pen used to generate the bezier outline.

`points` PointF[]

Bezier control points.

# DrawEllipse(Pen, PointF, SizeF)

Draws an ellipse outline using the provided pen.

```
public void DrawEllipse(Pen pen, PointF center, SizeF size)
```

## Parameters

`pen` Pen

Pen used to generate the ellipse outline.

`center` PointF

Ellipse center point in local coordinates.

`size` SizeF

Ellipse width and height in local coordinates.

# DrawGlyphs(Brush, Pen, IEnumerable<GlyphPathCollection>)

Draws layered glyph geometry.

```
public abstract void DrawGlyphs(Brush brush, Pen pen, IEnumerable<GlyphPathCollection>
glyphs)
```

## Parameters

`brush` Brush

Brush used to fill glyph layers.

`pen` Pen

Pen used to outline dominant painted layers.

`glyphs` IEnumerable<GlyphPathCollection>

Layered glyph geometry to draw.

## DrawImage(Image, Rectangle, RectangleF, WrapMode, WrapMode, IResampler?)

Draws an image source region into a destination rectangle, tiling the painted area by repeating the destination rectangle outwards per the supplied WrapModes.

```
public abstract void DrawImage(Image image, Rectangle sourceRect, RectangleF
destinationRect, WrapMode wrapX, WrapMode wrapY, IResampler? sampler = null)
```

### Parameters

`image` Image

The source image.

`sourceRect` Rectangle

The source rectangle within `image`.

`destinationRect` RectangleF

The destination rectangle in local canvas coordinates (defines a single tile cell).

`wrapX` WrapMode

The horizontal wrap mode applied when sampling beyond `destinationRect`.

`wrapY` WrapMode

The vertical wrap mode applied when sampling beyond `destinationRect`.

`sampler` IResampler

Optional resampler used when scaling or transforming the image. Defaults to Bicubic.

## DrawImage(Image, Rectangle, RectangleF, IResampler?)

Draws an image source region into a destination rectangle.

```
public abstract void DrawImage(Image image, Rectangle sourceRect, RectangleF
destinationRect, IResampler? sampler = null)
```

### Parameters

`image` Image

The source image.

`sourceRect` Rectangle

The source rectangle within `image`.

`destinationRect` RectangleF

The destination rectangle in local canvas coordinates.

`sampler` IResampler

Optional resampler used when scaling or transforming the image. Defaults to Bicubic.

## DrawLine(Pen, params PointF[])

Draws a polyline outline using the provided pen and drawing options.

```
public abstract void DrawLine(Pen pen, params PointF[] points)
```

### Parameters

`pen` Pen

Pen used to generate the line outline.

`points` PointF[]

Polyline points.

## DrawPie(Pen, PointF, SizeF, float, float)

Draws a pie sector outline using the provided pen.

```
public void DrawPie(Pen pen, PointF center, SizeF radius, float startAngle,
float sweepAngle)
```

### Parameters

`pen` Pen

Pen used to generate the pie outline.

`center` PointF

The center point of the pie sector in local coordinates.

`radius` SizeF

The x and y radii of the pie sector in local coordinates.

`startAngle` float

The start angle of the pie sector in degrees.

`sweepAngle` float

The sweep angle of the pie sector in degrees.

## DrawPie(Pen, PointF, SizeF, float, float, float)

Draws a pie sector outline using the provided pen.

```
public void DrawPie(Pen pen, PointF center, SizeF radius, float rotation, float startAngle,
float sweepAngle)
```

## Parameters

`pen` Pen

Pen used to generate the pie outline.

`center` PointF

The center point of the pie sector in local coordinates.

`radius` SizeF

The x and y radii of the pie sector in local coordinates.

`rotation` float

Ellipse rotation in degrees.

`startAngle` float

The start angle of the pie sector in degrees.

`sweepAngle` float

The sweep angle of the pie sector in degrees.

# DrawText(LineLayout, IPath, Brush?, Pen?)

Draws one prepared line layout along a path baseline onto this canvas.

```
public abstract void DrawText(LineLayout lineLayout, IPath path, Brush? brush, Pen? pen)
```

## Parameters

`lineLayout` LineLayout

The prepared line layout to draw.

`path` IPath

The path used as the text baseline in local canvas coordinates.

`brush` Brush

Optional brush used to fill glyphs.

`pen` Pen

Optional pen used to outline glyphs.

## DrawText(LineLayout, PointF, Brush?, Pen?)

Draws one prepared line layout onto this canvas.

```
public abstract void DrawText(LineLayout lineLayout, PointF location, Brush? brush,
Pen? pen)
```

### Parameters

`lineLayout` LineLayout

The prepared line layout to draw.

`location` PointF

The drawing location in local canvas coordinates.

`brush` Brush

Optional brush used to fill glyphs.

`pen` Pen

Optional pen used to outline glyphs.

## DrawText(TextBlock, IPath, float, Brush?, Pen?)

Draws a prepared text block along a path baseline onto this canvas.

```
public abstract void DrawText(TextBlock textBlock, IPath path, float wrappingLength, Brush?
brush, Pen? pen)
```

### Parameters

`textBlock` TextBlock

The prepared text block to draw.

`path` IPath

The path used as the text baseline in local canvas coordinates.

`wrappingLength` float

The wrapping length in pixels. Use `-1` to disable wrapping.

`brush` Brush

Optional brush used to fill glyphs.

`pen` Pen

Optional pen used to outline glyphs.

## DrawText(TextBlock, PointF, float, Brush?, Pen?)

Draws a prepared text block onto this canvas.

```
public abstract void DrawText(TextBlock textBlock, PointF location, float wrappingLength,
Brush? brush, Pen? pen)
```

### Parameters

`textBlock` TextBlock

The prepared text block to draw.

`location` PointF

The drawing location in local canvas coordinates.

`wrappingLength` float

The wrapping length in pixels. Use `-1` to disable wrapping.

`brush` Brush

Optional brush used to fill glyphs.

`pen` Pen

Optional pen used to outline glyphs.

## DrawText(RichTextOptions, ReadOnlySpan<char>, IPath, Brush?, Pen?)

Draws text along a path baseline onto this canvas.

```
public abstract void DrawText(RichTextOptions textOptions, ReadOnlySpan<char> text, IPath path, Brush? brush, Pen? pen)
```

### Parameters

`textOptions` RichTextOptions

The text rendering options.

`text` ReadOnlySpan<char>

The text to draw.

`path` IPath

The path used as the text baseline in local canvas coordinates.

`brush` Brush

Optional brush used to fill glyphs.

`pen` Pen

Optional pen used to outline glyphs.

## DrawText(RichTextOptions, ReadOnlySpan<char>, Brush?, Pen?)

Draws text onto this canvas.

```
public abstract void DrawText(RichTextOptions textOptions, ReadOnlySpan<char> text, Brush?
brush, Pen? pen)
```

## Parameters

`textOptions` RichTextOptions

The text rendering options.

`text` ReadOnlySpan<char>

The text to draw.

`brush` Brush

Optional brush used to fill glyphs.

`pen` Pen

Optional pen used to outline glyphs.

# DrawText(ReadOnlySpan<ushort>, ReadOnlySpan<Vector2>, RichGlyphOptions, Brush?, Pen?)

Draws positioned glyphs onto this canvas.

```
public abstract void DrawText(ReadOnlySpan<ushort> glyphIds, ReadOnlySpan<Vector2> points,
RichGlyphOptions options, Brush? brush, Pen? pen)
```

## Parameters

`glyphIds` ReadOnlySpan<ushort>

The glyph identifiers.

`points` ReadOnlySpan<Vector2>

The absolute glyph origins in pixel units.

`options` RichGlyphOptions

The glyph rendering options, including the font and optional glyph paint.

`brush` Brush

Default brush used to fill glyphs when Brush is not set.

`pen` Pen

Default pen used to outline glyphs when Pen is not set.

## DrawText(ushort, RichGlyphOptions, Brush?, Pen?)

Draws a single glyph, identified by its glyph id, onto this canvas.

```
public abstract void DrawText(ushort glyphId, RichGlyphOptions options, Brush? brush,
Pen? pen)
```

### Parameters

`glyphId` ushort

The id of the glyph within the font face referenced by `options`.

`options` RichGlyphOptions

The glyph rendering options, including the font, origin, grapheme index and optional per-glyph paint.

`brush` Brush

Default brush used to fill the glyph when Brush is not set.

`pen` Pen

Default pen used to outline the glyph when Pen is not set.

## Fill(Brush)

Fills the whole canvas using the given brush.

```
public void Fill(Brush brush)
```

### Parameters

`brush` Brush

Brush used to shade destination pixels.

## Fill(Brush, IPath)

Fills a path in local coordinates using the given brush.

```
public abstract void Fill(Brush brush, IPath path)
```

### Parameters

`brush` Brush

Brush used to shade covered pixels.

`path` IPath

The path to fill.

## Fill(Brush, IPathCollection)

Fills all paths in a collection using the given brush.

```
public void Fill(Brush brush, IPathCollection paths)
```

### Parameters

`brush` Brush

Brush used to shade covered pixels.

`paths` IPathCollection

Path collection to fill.

## Fill(Brush, PathBuilder)

Fills a path built by the provided builder using the given brush.

```
public void Fill(Brush brush, PathBuilder pathBuilder)
```

### Parameters

`brush` Brush

Brush used to shade covered pixels.

`pathBuilder` PathBuilder

The path builder describing the fill region.

## Fill(Brush, Rectangle)

Fills a local region using the given brush.

```
public void Fill(Brush brush, Rectangle region)
```

### Parameters

`brush` Brush

Brush used to shade destination pixels.

`region` Rectangle

Region to fill in local coordinates.

## FillArc(Brush, PointF, SizeF, float, float, float)

Fills the closed arc shape produced by joining the arc endpoints with a straight line.

```
public void FillArc(Brush brush, PointF center, SizeF radius, float rotation, float
startAngle, float sweepAngle)
```

## Parameters

`brush` Brush

Brush used to shade covered pixels.

`center` PointF

Arc center point in local coordinates.

`radius` SizeF

Arc radii in local coordinates.

`rotation` float

Ellipse rotation in degrees.

`startAngle` float

Arc start angle in degrees.

`sweepAngle` float

Arc sweep angle in degrees.

# FillEllipse(Brush, PointF, SizeF)

Fills an ellipse using the provided brush.

```
public void FillEllipse(Brush brush, PointF center, SizeF size)
```

## Parameters

`brush` Brush

Brush used to shade covered pixels.

`center` PointF

Ellipse center point in local coordinates.

`size` SizeF

Ellipse width and height in local coordinates.

## FillPie(Brush, PointF, SizeF, float, float)

Fills a pie sector using the provided brush.

```
public void FillPie(Brush brush, PointF center, SizeF radius, float startAngle,
float sweepAngle)
```

### Parameters

`brush` Brush

Brush used to shade covered pixels.

`center` PointF

The center point of the pie sector in local coordinates.

`radius` SizeF

The x and y radii of the pie sector in local coordinates.

`startAngle` float

The start angle of the pie sector in degrees.

`sweepAngle` float

The sweep angle of the pie sector in degrees.

## FillPie(Brush, PointF, SizeF, float, float, float)

Fills a pie sector using the provided brush.

```
public void FillPie(Brush brush, PointF center, SizeF radius, float rotation, float
```

```
startAngle, float sweepAngle)
```

## Parameters

`brush` Brush

Brush used to shade covered pixels.

`center` PointF

The center point of the pie sector in local coordinates.

`radius` SizeF

The x and y radii of the pie sector in local coordinates.

`rotation` float

Ellipse rotation in degrees.

`startAngle` float

The start angle of the pie sector in degrees.

`sweepAngle` float

The sweep angle of the pie sector in degrees.

# Flush()

Seals the drawing commands recorded so far into an ordered range so that subsequent commands begin a new range.

```
public abstract void Flush()
```

## Remarks

This does not rasterize or composite anything into the target frame. Recorded commands are replayed into the target only when the root canvas is disposed, or when they are lowered explicitly through CreateScene(), RenderScene(DrawingBackendScene) or CopyPixels From(DrawingCanvas, Rectangle, Point). Use this to establish an ordering boundary in the recorded timeline without ending the canvas.

# MeasureText(RichTextOptions, ReadOnlySpan<char>)

Measures the full set of layout metrics for the supplied text.

```
public abstract TextMetrics MeasureText(RichTextOptions textOptions,
ReadOnlySpan<char> text)
```

## Parameters

`textOptions` RichTextOptions

The text shaping and layout options.

`text` ReadOnlySpan<char>

The text to measure.

## Returns

TextMetrics

A TextMetrics value containing the metrics for the laid-out text.

# RenderScene(DrawingBackendScene)

Records a previously created backend scene into this canvas' command timeline.

```
public abstract void RenderScene(DrawingBackendScene scene)
```

## Parameters

`scene` DrawingBackendScene

The backend scene to render.

## Remarks

The active command range is sealed before the scene is inserted so the scene renders in submission order relative to the commands recorded around it. Like other recorded work, the

scene is replayed into the target frame when the root canvas is disposed or otherwise flushed, not at the point of this call.

## Restore()

Restores the most recently saved state.

```
public abstract void Restore()
```

### Remarks

If the most recently saved state was created by a `SaveLayer` overload, the layer is closed and becomes part of subsequent rendering.

## RestoreTo(int)

Restores to a specific save count.

```
public abstract void RestoreTo(int saveCount)
```

### Parameters

`saveCount` int

The save count to restore to.

### Remarks

State frames above `saveCount` are discarded, and the last discarded frame becomes the current state. If any discarded state was created by a `SaveLayer` overload, those layers are closed and become part of subsequent rendering.

## Save()

Saves the current drawing state on the state stack.

```
public abstract int Save()
```

## Returns

int

The save count after the state has been pushed.

## Remarks

This operation stores the current canvas state by reference. If the same DrawingOptions instance is mutated after Save(), those mutations are visible when restoring.

# Save(DrawingOptions)

Saves the current drawing state and replaces the active state with the provided options.

```
public abstract int Save(DrawingOptions options)
```

## Parameters

`options` DrawingOptions

Drawing options for the new active state.

## Returns

int

The save count after the previous state has been pushed.

## Remarks

The provided `options` instance is stored by reference. Mutating it after this call mutates the active/restored state behavior.

# SaveLayer()

Saves the current drawing state and begins an isolated compositing layer over the whole canvas.

```
public int SaveLayer()
```

## Returns

int

The save count after the layer state has been pushed.

# SaveLayer(GraphicsOptions)

Saves the current drawing state and begins an isolated compositing layer over the whole canvas.

```
public int SaveLayer(GraphicsOptions layerOptions)
```

## Parameters

`layerOptions` GraphicsOptions

Graphics options controlling how the layer is composited on restore.

## Returns

int

The save count after the layer state has been pushed.

# SaveLayer(GraphicsOptions, IPath, LayerEffect)

Saves the current drawing state and begins an isolated compositing layer whose content is transformed by an effect confined to a path region when the layer is restored.

```
public abstract int SaveLayer(GraphicsOptions layerOptions, IPath region,
LayerEffect effect)
```

## Parameters

`layerOptions` GraphicsOptions

Graphics options controlling how the closed layer is composited against the parent canvas.

`region` IPath

The path region the effect processes, in local coordinates.

`effect` LayerEffect

The effect applied to the layer content on restore.

## Returns

int

The save count after the layer state has been pushed.

## Remarks

The effect processes only pixels covered by the supplied path; its output lands on the path translated by the effect's offset. The layer bounds are derived from the path and expanded by the effect's reach.

# SaveLayer(GraphicsOptions, IPath, LayerEffect, DrawingOptions)

Saves the current drawing state and begins an isolated compositing layer whose content is transformed by an effect confined to a path region when the layer is restored, using the supplied drawing options for commands recorded into the layer.

```
public abstract int SaveLayer(GraphicsOptions layerOptions, IPath region, LayerEffect
effect, DrawingOptions options)
```

## Parameters

`layerOptions` GraphicsOptions

Graphics options controlling how the closed layer is composited against the parent canvas.

`region` IPath

The path region the effect processes, in local coordinates.

`effect` LayerEffect

The effect applied to the layer content on restore.

`options` DrawingOptions

Drawing options for the layer contents.

## Returns

int

The save count after the layer state has been pushed.

# SaveLayer(GraphicsOptions, Rectangle)

Saves the current drawing state and begins an isolated compositing layer bounded to a subregion. Subsequent draw commands target that layer until Restore() closes it.

```
public abstract int SaveLayer(GraphicsOptions layerOptions, Rectangle bounds)
```

## Parameters

`layerOptions` GraphicsOptions

Graphics options controlling how the closed layer is composited against the parent canvas.

`bounds` Rectangle

The local bounds of the layer. Only this region is allocated and composited.

## Returns

int

The save count after the layer state has been pushed.

## Remarks

The layer bounds are expressed in the current local coordinate system and are transformed with the active drawing transform when the layer is created. They limit allocation and compositing

only; they do not change the canvas coordinate system used by commands recorded inside the layer.

## SaveLayer(GraphicsOptions, Rectangle, DrawingOptions)

Saves the current drawing state and begins an isolated compositing layer using the supplied drawing options for commands recorded into the layer.

```
public abstract int SaveLayer(GraphicsOptions layerOptions, Rectangle bounds,
DrawingOptions options)
```

### Parameters

`layerOptions` GraphicsOptions

Graphics options controlling how the closed layer is composited against the parent canvas.

`bounds` Rectangle

The local bounds of the layer. Only this region is allocated and composited.

`options` DrawingOptions

Drawing options for the layer contents.

### Returns

int

The save count after the layer state has been pushed.

## SaveLayer(GraphicsOptions, Rectangle, LayerEffect)

Saves the current drawing state and begins an isolated compositing layer whose content is transformed by an effect when the layer is restored.

```
public abstract int SaveLayer(GraphicsOptions layerOptions, Rectangle bounds,
LayerEffect effect)
```

## Parameters

`layerOptions` GraphicsOptions

Graphics options controlling how the closed layer is composited against the parent canvas.

`bounds` Rectangle

The content bounds in local canvas coordinates.

`effect` LayerEffect

The effect applied to the layer content on restore.

## Returns

int

The save count after the layer state has been pushed.

## Remarks

The layer isolates the content, so the effect operates on exactly what is drawn between this call and the matching Restore(), against transparency; a DropShadowLayerEffect, for example, slots its shadow beneath that content before the layer composites onto the canvas. The bounds are expanded internally by the effect's reach so blurred or offset output is not cut off.

# SaveLayer(GraphicsOptions, Rectangle, LayerEffect, DrawingOptions)

Saves the current drawing state and begins an isolated compositing layer whose content is transformed by an effect when the layer is restored, using the supplied drawing options for commands recorded into the layer.

```
public abstract int SaveLayer(GraphicsOptions layerOptions, Rectangle bounds, LayerEffect
effect, DrawingOptions options)
```

## Parameters

`layerOptions` GraphicsOptions

Graphics options controlling how the closed layer is composited against the parent canvas.

`bounds` Rectangle

The content bounds in local canvas coordinates.

`effect` LayerEffect

The effect applied to the layer content on restore.

`options` DrawingOptions

Drawing options for the layer contents.

## Returns

int

The save count after the layer state has been pushed.

# Class DrawingCanvasFactoryExtensions

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Extension methods for creating drawing canvas instances over ImageSharp image frames.

```
public static class DrawingCanvasFactoryExtensions
```

**Inheritance**

object ← DrawingCanvasFactoryExtensions

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

# Methods

## CreateCanvas(ImageFrame, Configuration, DrawingOptions, params IPath[])

Creates a drawing canvas over an existing image frame.

```
public static DrawingCanvas CreateCanvas(this ImageFrame frame, Configuration configuration,
DrawingOptions options, params IPath[] clipPaths)
```

### Parameters

`frame` ImageFrame

The frame backing the canvas.

`configuration` Configuration

The configuration to use for this canvas instance.

`options` DrawingOptions

Initial drawing options for this canvas instance.

`clipPaths` IPath[]

Initial clip paths for this canvas instance.

## Returns

DrawingCanvas

A drawing canvas targeting `frame`.

## Remarks

The caller owns the returned canvas and must dispose it to replay recorded work into the frame.

# CreateCanvas(ImageFrame, Configuration, DrawingOptions, DrawingTextCache, params IPath[])

Creates a drawing canvas over an existing image frame.

```
public static DrawingCanvas CreateCanvas(this ImageFrame frame, Configuration configuration,
DrawingOptions options, DrawingTextCache textCache, params IPath[] clipPaths)
```

## Parameters

`frame` ImageFrame

The frame backing the canvas.

`configuration` Configuration

The configuration to use for this canvas instance.

`options` DrawingOptions

Initial drawing options for this canvas instance.

`textCache` DrawingTextCache

The text drawing cache used by this canvas instance.

`clipPaths` IPath[]

Initial clip paths for this canvas instance.

## Returns

DrawingCanvas

A drawing canvas targeting `frame`.

## Remarks

The caller owns the returned canvas and must dispose it to replay recorded work into the frame.

# CreateCanvas<TPixel>(ImageFrame<TPixel>, Configuration, DrawingOptions, params IPath[])

Creates a drawing canvas over an existing typed image frame.

```
public static DrawingCanvas CreateCanvas<TPixel>(this ImageFrame<TPixel> frame,
Configuration configuration, DrawingOptions options, params IPath[] clipPaths) where TPixel
: unmanaged, IPixel<TPixel>
```

## Parameters

`frame` ImageFrame<TPixel>

The frame backing the canvas.

`configuration` Configuration

The configuration to use for this canvas instance.

`options` DrawingOptions

Initial drawing options for this canvas instance.

`clipPaths` IPath[]

Initial clip paths for this canvas instance.

## Returns

DrawingCanvas

A drawing canvas targeting `frame`.

## Type Parameters

`TPixel`

The pixel format.

## Remarks

The caller owns the returned canvas and must dispose it to replay recorded work into the frame.

# CreateCanvas<TPixel>(ImageFrame<TPixel>, Configuration, DrawingOptions, DrawingTextCache, params IPath[])

Creates a drawing canvas over an existing typed image frame.

```
public static DrawingCanvas CreateCanvas<TPixel>(this ImageFrame<TPixel> frame,
Configuration configuration, DrawingOptions options, DrawingTextCache textCache, params
IPath[] clipPaths) where TPixel : unmanaged, IPixel<TPixel>
```

## Parameters

`frame` ImageFrame<TPixel>

The frame backing the canvas.

`configuration` Configuration

The configuration to use for this canvas instance.

`options` DrawingOptions

Initial drawing options for this canvas instance.

`textCache` DrawingTextCache

The text drawing cache used by this canvas instance.

`clipPaths` IPath[]

Initial clip paths for this canvas instance.

## Returns

DrawingCanvas

A drawing canvas targeting `frame`.

## Type Parameters

`TPixel`

The pixel format.

## Remarks

The caller owns the returned canvas and must dispose it to replay recorded work into the frame.

# Class DrawingCanvas<TPixel>

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

A drawing canvas over a frame target.

```
public sealed class DrawingCanvas<TPixel> : DrawingCanvas, IDisposable where TPixel :
unmanaged, IPixel<TPixel>
```

## Type Parameters

`TPixel`

The pixel format.

**Inheritance**

object ← DrawingCanvas ← DrawingCanvas<TPixel>

**Implements**

IDisposable

**Inherited Members**

DrawingCanvas.SaveLayer() , DrawingCanvas.SaveLayer(GraphicsOptions) , DrawingCanvas.Fill(Brush) , DrawingCanvas.Fill(Brush, Rectangle) , DrawingCanvas.Clear(Brush) , DrawingCanvas.Clear(Brush, Rectangle) , DrawingCanvas.Fill(Brush, IPathCollection) , DrawingCanvas.Fill(Brush, PathBuilder) , DrawingCanvas.FillEllipse(Brush, PointF, SizeF) , DrawingCanvas.FillArc(Brush, PointF, SizeF, float, float, float) , DrawingCanvas.FillPie(Brush, PointF, SizeF, float, float, float) , DrawingCanvas.FillPie(Brush, PointF, SizeF, float, float) , DrawingCanvas.DrawArc(Pen, PointF, SizeF, float, float, float) , DrawingCanvas.DrawBezier(Pen, params PointF[]) , DrawingCanvas.DrawEllipse(Pen, PointF, SizeF) , DrawingCanvas.DrawPie(Pen, PointF, SizeF, float, float, float) , DrawingCanvas.DrawPie(Pen, PointF, SizeF, float, float) , DrawingCanvas.Draw(Pen, Rectangle) , DrawingCanvas.Draw(Pen, IPathCollection) , DrawingCanvas.Draw(Pen, PathBuilder) , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## DrawingCanvas(Configuration, DrawingOptions, ICanvasFrame<TPixel>, params IPath[])

Initializes a new instance of the DrawingCanvas<TPixel> class.

```
public DrawingCanvas(Configuration configuration, DrawingOptions options,
ICanvasFrame<TPixel> targetFrame, params IPath[] clipPaths)
```

### Parameters

`configuration` Configuration

The active processing configuration.

`options` DrawingOptions

Initial drawing options for this canvas instance.

`targetFrame` ICanvasFrame<TPixel>

The destination frame.

`clipPaths` IPath[]

Initial clip paths for this canvas instance.

## DrawingCanvas(Configuration, DrawingOptions, IDrawingBackend, ICanvasFrame<TPixel>, params IPath[])

Initializes a new instance of the DrawingCanvas<TPixel> class with an explicit backend and initial state.

```
public DrawingCanvas(Configuration configuration, DrawingOptions options, IDrawingBackend
backend, ICanvasFrame<TPixel> targetFrame, params IPath[] clipPaths)
```

### Parameters

`configuration` Configuration

The active processing configuration.

`options` DrawingOptions

Initial drawing options for this canvas instance.

`backend` IDrawingBackend

The drawing backend implementation.

`targetFrame` ICanvasFrame<TPixel>

The destination frame.

`clipPaths` IPath[]

Initial clip paths for this canvas instance.

## DrawingCanvas(Configuration, DrawingOptions, DrawingTextCache, ICanvasFrame<TPixel>, params IPath[])

Initializes a new instance of the DrawingCanvas<TPixel> class.

```
public DrawingCanvas(Configuration configuration, DrawingOptions options, DrawingTextCache
textCache, ICanvasFrame<TPixel> targetFrame, params IPath[] clipPaths)
```

### Parameters

`configuration` Configuration

The active processing configuration.

`options` DrawingOptions

Initial drawing options for this canvas instance.

`textCache` DrawingTextCache

The text drawing cache used by this canvas instance.

`targetFrame` ICanvasFrame<TPixel>

The destination frame.

`clipPaths` IPath[]

Initial clip paths for this canvas instance.

## DrawingCanvas(Configuration, DrawingOptions, DrawingTextCache, IDrawingBackend, ICanvasFrame<TPixel>, params IPath[])

Initializes a new instance of the DrawingCanvas<TPixel> class with an explicit backend and initial state.

```
public DrawingCanvas(Configuration configuration, DrawingOptions options, DrawingTextCache
textCache, IDrawingBackend backend, ICanvasFrame<TPixel> targetFrame, params
IPath[] clipPaths)
```

### Parameters

`configuration` Configuration

The active processing configuration.

`options` DrawingOptions

Initial drawing options for this canvas instance.

`textCache` DrawingTextCache

The text drawing cache used by this canvas instance.

`backend` IDrawingBackend

The drawing backend implementation.

`targetFrame` ICanvasFrame<TPixel>

The destination frame.

`clipPaths` IPath[]

Initial clip paths for this canvas instance.

# DrawingCanvas(Configuration, DrawingOptions, DrawingTextCache, Buffer2DRegion<TPixel>, params IPath[])

Initializes a new instance of the DrawingCanvas<TPixel> class.

```
public DrawingCanvas(Configuration configuration, DrawingOptions options, DrawingTextCache
textCache, Buffer2DRegion<TPixel> targetRegion, params IPath[] clipPaths)
```

## Parameters

`configuration` Configuration

The active processing configuration.

`options` DrawingOptions

Initial drawing options for this canvas instance.

`textCache` DrawingTextCache

The text drawing cache used by this canvas instance.

`targetRegion` Buffer2DRegion<TPixel>

The destination target region.

`clipPaths` IPath[]

Initial clip paths for this canvas instance.

# DrawingCanvas(Configuration, DrawingOptions, Buffer2DRegion<TPixel>, params IPath[])

Initializes a new instance of the DrawingCanvas<TPixel> class.

```
public DrawingCanvas(Configuration configuration, DrawingOptions options,
Buffer2DRegion<TPixel> targetRegion, params IPath[] clipPaths)
```

## Parameters

`configuration` Configuration

The active processing configuration.

`options` DrawingOptions

Initial drawing options for this canvas instance.

`targetRegion` Buffer2DRegion<TPixel>

The destination target region.

`clipPaths` IPath[]

Initial clip paths for this canvas instance.

# Properties

## Bounds

Gets the local bounds of this canvas.

```
public override Rectangle Bounds { get; }
```

## Property Value

Rectangle

## SaveCount

Gets the number of saved states currently on the canvas stack.

```
public override int SaveCount { get; }
```

## Property Value

int

# Methods

## Apply(IPath, Action<IImageProcessingContext>)

Applies an image-processing operation to a path region.

```
public override void Apply(IPath path, Action<IImageProcessingContext> operation)
```

### Parameters

`path` IPath

The path region to process.

`operation` Action<IImageProcessingContext>

The image-processing operation to apply to the region.

### Remarks

The operation affects only pixels covered by the supplied path.

## Apply(IPath, Action<IImageProcessingContext>, GraphicsOptions, Point)

Applies an image-processing operation to a path region and composites the processed pixels back with explicit options, optionally offset from where they were read.

```
public override void Apply(IPath path, Action<IImageProcessingContext> operation,
GraphicsOptions writeBackOptions, Point writeBackOffset)
```

### Parameters

`path` IPath

The path region to process.

`operation` Action<IImageProcessingContext>

The image-processing operation to apply to the region.

`writeBackOptions` GraphicsOptions

The graphics options used to composite the processed pixels back.

`writeBackOffset` Point

The offset at which the processed pixels are written back.

### Remarks

The write-back affects only pixels covered by the supplied path translated by the offset.

## Apply(PathBuilder, Action<IImageProcessingContext>)

Applies an image-processing operation to a region described by a path builder.

```
public override void Apply(PathBuilder pathBuilder, Action<IImageProcessingContext>
operation)
```

### Parameters

`pathBuilder` PathBuilder

The path builder describing the region to process.

`operation` Action<IImageProcessingContext>

The image-processing operation to apply to the region.

## Apply(Rectangle, Action<IImageProcessingContext>)

Applies an image-processing operation to a local region.

```
public override void Apply(Rectangle region, Action<IImageProcessingContext> operation)
```

### Parameters

`region` Rectangle

The local region to process.

`operation` Action<IImageProcessingContext>

The image-processing operation to apply to the region.

## Apply(Rectangle, Action<IImageProcessingContext>, GraphicsOptions, Point)

Applies an image-processing operation to a local region and composites the processed pixels back with explicit options, optionally offset from where they were read. The target region is untouched while the operation runs, so the processed pixels can be blended against the original content; for example a drop shadow composites the tinted, blurred region back beneath the content with DestOver at the shadow offset.

```
public override void Apply(Rectangle region, Action<IImageProcessingContext> operation,
GraphicsOptions writeBackOptions, Point writeBackOffset)
```

### Parameters

`region` Rectangle

The local region to process.

`operation` Action<IImageProcessingContext>

The image-processing operation to apply to the region.

`writeBackOptions` GraphicsOptions

The graphics options used to composite the processed pixels back.

`writeBackOffset` Point

The offset at which the processed pixels are written back.

## Clear(Brush, IPath)

Fills a path region with the given brush using replacement composition: the brush output overwrites the covered pixels outright, including their alpha, rather than blending over them.

```
public override void Clear(Brush brush, IPath path)
```

### Parameters

`brush` Brush

Brush used to shade destination pixels during the clear.

`path` IPath

The path region to clear.

## Clip(ClipOperation, params IPath[])

Narrows the current clip region by applying the specified clipping operation with the supplied clip paths.

```
public override void Clip(ClipOperation operation, params IPath[] clipPaths)
```

### Parameters

`operation` ClipOperation

The operation to apply to the current clip.

`clipPaths` IPath[]

The clip paths to combine with the current clip, in local coordinates.

### Remarks

The clip paths are transformed by the active transform at the point this is called.

## Clip(params IPath[])

Narrows the current clip region by intersecting it with the supplied clip paths.

```
public override void Clip(params IPath[] clipPaths)
```

## Parameters

`clipPaths` IPath[]

The clip paths to intersect with the current clip, in local coordinates.

## Remarks

The clip paths are transformed by the active transform at the point this is called, then intersected with the existing clip; clipping only ever narrows. The resulting clip is part of the current saved state and is restored by Restore(). Multiple paths combine as a union before intersecting (e.g. a region built from several rectangles).

# CopyPixelsFrom(DrawingCanvas, Rectangle, Point)

Copies pixels from another canvas into this canvas.

```
public override void CopyPixelsFrom(DrawingCanvas source, Rectangle sourceRectangle,
Point targetPoint)
```

## Parameters

`source` DrawingCanvas

The source canvas.

`sourceRectangle` Rectangle

The source rectangle in source-local coordinates.

`targetPoint` Point

The target point in this canvas' local coordinates.

## Remarks

Any queued commands on both canvases are applied before the copy.

# CreateRegion(Rectangle)

Creates a child canvas over a subregion in local coordinates.

```
public override DrawingCanvas<TPixel> CreateRegion(Rectangle region)
```

### Parameters

`region` Rectangle

The child region in local coordinates.

### Returns

DrawingCanvas<TPixel>

A child canvas with local origin at (0,0).

## CreateScene()

Seals the commands recorded so far into a retained backend scene and clears them from the canvas queue.

```
public override DrawingBackendScene CreateScene()
```

### Returns

DrawingBackendScene

The created backend scene.

### Remarks

The active clip range is closed before the scene is built and reopened afterwards, so this acts as an ordering boundary in the recorded timeline. Image resources retained for the sealed commands are transferred to the returned scene, which owns and disposes them.

## Dispose()

Releases the canvas. Disposing the root canvas closes any open layers and clips and replays the recorded command timeline into the target frame; child region canvases share the root's command stream and do not flush on disposal.

```
public override void Dispose()
```

### Remarks

The root canvas also releases image resources retained for deferred draws and, when it owns the text drawing cache, clears that cache. Disposal is idempotent.

## Draw(Pen, IPath)

Draws a path outline in local coordinates using the given pen.

```
public override void Draw(Pen pen, IPath path)
```

### Parameters

`pen` Pen

Pen used to generate the outline fill path.

`path` IPath

The path to stroke.

## DrawGlyphs(Brush, Pen, IEnumerable<GlyphPathCollection>)

Draws layered glyph geometry.

```
public override void DrawGlyphs(Brush brush, Pen pen, IEnumerable<GlyphPathCollection>
glyphs)
```

### Parameters

`brush` Brush

Brush used to fill glyph layers.

`pen` Pen

Pen used to outline dominant painted layers.

`glyphs` IEnumerable<GlyphPathCollection>

Layered glyph geometry to draw.

## DrawImage(Image, Rectangle, RectangleF, WrapMode, WrapMode, IResampler?)

Draws an image source region into a destination rectangle, tiling the painted area by repeating the destination rectangle outwards per the supplied WrapModes.

```
public override void DrawImage(Image image, Rectangle sourceRect, RectangleF
destinationRect, WrapMode wrapX, WrapMode wrapY, IResampler? sampler)
```

## Parameters

`image` Image

The source image.

`sourceRect` Rectangle

The source rectangle within `image`.

`destinationRect` RectangleF

The destination rectangle in local canvas coordinates (defines a single tile cell).

`wrapX` WrapMode

The horizontal wrap mode applied when sampling beyond `destinationRect`.

`wrapY` WrapMode

The vertical wrap mode applied when sampling beyond `destinationRect`.

`sampler` IResampler

Optional resampler used when scaling or transforming the image. Defaults to Bicubic.

## DrawImage(Image, Rectangle, RectangleF, IResampler?)

Draws an image source region into a destination rectangle.

```
public override void DrawImage(Image image, Rectangle sourceRect, RectangleF
destinationRect, IResampler? sampler)
```

### Parameters

`image` Image

The source image.

`sourceRect` Rectangle

The source rectangle within `image`.

`destinationRect` RectangleF

The destination rectangle in local canvas coordinates.

`sampler` IResampler

Optional resampler used when scaling or transforming the image. Defaults to Bicubic.

## DrawImage(Image<TPixel>, Rectangle, RectangleF, WrapMode, WrapMode, IResampler?)

Draws an image source region into a destination rectangle, tiling the painted area by repeating the destination rectangle outwards per the supplied WrapModes.

```
public void DrawImage(Image<TPixel> image, Rectangle sourceRect, RectangleF destinationRect,
WrapMode wrapX, WrapMode wrapY, IResampler? sampler = null)
```

### Parameters

`image` Image<TPixel>

The source image.

`sourceRect` Rectangle

The source rectangle within `image`.

`destinationRect` RectangleF

The destination rectangle in local canvas coordinates (defines a single tile cell).

`wrapX` WrapMode

The horizontal wrap mode applied when sampling beyond `destinationRect`.

`wrapY` WrapMode

The vertical wrap mode applied when sampling beyond `destinationRect`.

`sampler` IResampler

Optional resampler used when scaling or transforming the image. Defaults to Bicubic.

## DrawImage(Image<TPixel>, Rectangle, RectangleF, IResampler?)

Draws an image source region into a destination rectangle.

```
public void DrawImage(Image<TPixel> image, Rectangle sourceRect, RectangleF destinationRect,
IResampler? sampler = null)
```

## Parameters

`image` Image<TPixel>

The source image.

`sourceRect` Rectangle

The source rectangle within `image`.

`destinationRect` RectangleF

The destination rectangle in local canvas coordinates.

`sampler` IResampler

Optional resampler used when scaling or transforming the image. Defaults to Bicubic.

## DrawLine(Pen, PointF, PointF)

Draws a two-point line segment using the provided pen and drawing options.

```
public void DrawLine(Pen pen, PointF start, PointF end)
```

### Parameters

`pen` Pen

Pen used to generate the line outline.

`start` PointF

Line start point.

`end` PointF

Line end point.

## DrawLine(Pen, params PointF[])

Draws a polyline outline using the provided pen and drawing options.

```
public override void DrawLine(Pen pen, params PointF[] points)
```

### Parameters

`pen` Pen

Pen used to generate the line outline.

`points` PointF[]

Polyline points.

## DrawText(LineLayout, IPath, Brush?, Pen?)

Draws one prepared line layout along a path baseline onto this canvas.

```
public override void DrawText(LineLayout lineLayout, IPath path, Brush? brush, Pen? pen)
```

### Parameters

`lineLayout` LineLayout

The prepared line layout to draw.

`path` IPath

The path used as the text baseline in local canvas coordinates.

`brush` Brush

Optional brush used to fill glyphs.

`pen` Pen

Optional pen used to outline glyphs.

## DrawText(LineLayout, PointF, Brush?, Pen?)

Draws one prepared line layout onto this canvas.

```
public override void DrawText(LineLayout lineLayout, PointF location, Brush? brush,
Pen? pen)
```

### Parameters

`lineLayout` LineLayout

The prepared line layout to draw.

`location` PointF

The drawing location in local canvas coordinates.

`brush` Brush

Optional brush used to fill glyphs.

`pen` Pen

Optional pen used to outline glyphs.

## DrawText(TextBlock, IPath, float, Brush?, Pen?)

Draws a prepared text block along a path baseline onto this canvas.

```
public override void DrawText(TextBlock textBlock, IPath path, float wrappingLength, Brush?
brush, Pen? pen)
```

### Parameters

`textBlock` TextBlock

The prepared text block to draw.

`path` IPath

The path used as the text baseline in local canvas coordinates.

`wrappingLength` float

The wrapping length in pixels. Use `-1` to disable wrapping.

`brush` Brush

Optional brush used to fill glyphs.

`pen` Pen

Optional pen used to outline glyphs.

## DrawText(TextBlock, PointF, float, Brush?, Pen?)

Draws a prepared text block onto this canvas.

```
public override void DrawText(TextBlock textBlock, PointF location, float wrappingLength,
Brush? brush, Pen? pen)
```

## Parameters

`textBlock` TextBlock

The prepared text block to draw.

`location` PointF

The drawing location in local canvas coordinates.

`wrappingLength` float

The wrapping length in pixels. Use `-1` to disable wrapping.

`brush` Brush

Optional brush used to fill glyphs.

`pen` Pen

Optional pen used to outline glyphs.

# DrawText(RichTextOptions, ReadOnlySpan<char>, IPath, Brush?, Pen?)

Draws text along a path baseline onto this canvas.

```
public override void DrawText(RichTextOptions textOptions, ReadOnlySpan<char> text, IPath
path, Brush? brush, Pen? pen)
```

## Parameters

`textOptions` RichTextOptions

The text rendering options.

`text` ReadOnlySpan<char>

The text to draw.

`path` IPath

The path used as the text baseline in local canvas coordinates.

`brush` Brush

Optional brush used to fill glyphs.

`pen` Pen

Optional pen used to outline glyphs.

## DrawText(RichTextOptions, ReadOnlySpan<char>, Brush?, Pen?)

Draws text onto this canvas.

```
public override void DrawText(RichTextOptions textOptions, ReadOnlySpan<char> text, Brush?
brush, Pen? pen)
```

### Parameters

`textOptions` RichTextOptions

The text rendering options.

`text` ReadOnlySpan<char>

The text to draw.

`brush` Brush

Optional brush used to fill glyphs.

`pen` Pen

Optional pen used to outline glyphs.

# DrawText(ReadOnlySpan<ushort>, ReadOnlySpan<Vector2>, RichGlyphOptions, Brush?, Pen?)

Draws positioned glyphs onto this canvas.

```
public override void DrawText(ReadOnlySpan<ushort> glyphIds, ReadOnlySpan<Vector2> points,
RichGlyphOptions options, Brush? brush, Pen? pen)
```

## Parameters

`glyphIds` ReadOnlySpan<ushort>

The glyph identifiers.

`points` ReadOnlySpan<Vector2>

The absolute glyph origins in pixel units.

`options` RichGlyphOptions

The glyph rendering options, including the font and optional glyph paint.

`brush` Brush

Default brush used to fill glyphs when Brush is not set.

`pen` Pen

Default pen used to outline glyphs when Pen is not set.

# DrawText(ushort, RichGlyphOptions, Brush?, Pen?)

Draws a single glyph, identified by its glyph id, onto this canvas.

```
public override void DrawText(ushort glyphId, RichGlyphOptions options, Brush? brush,
Pen? pen)
```

## Parameters

`glyphId` ushort

The id of the glyph within the font face referenced by `options`.

`options` RichGlyphOptions

The glyph rendering options, including the font, origin, grapheme index and optional per-glyph paint.

`brush` Brush

Default brush used to fill the glyph when Brush is not set.

`pen` Pen

Default pen used to outline the glyph when Pen is not set.

## Fill(Brush, IPath)

Fills a path in local coordinates using the given brush.

```
public override void Fill(Brush brush, IPath path)
```

### Parameters

`brush` Brush

Brush used to shade covered pixels.

`path` IPath

The path to fill.

## Flush()

Seals the drawing commands recorded so far into an ordered range so that subsequent commands begin a new range.

```
public override void Flush()
```

### Remarks

This does not rasterize or composite anything into the target frame. Recorded commands are replayed into the target only when the root canvas is disposed, or when they are lowered explicitly through CreateScene(), RenderScene(DrawingBackendScene) or CopyPixels From(DrawingCanvas, Rectangle, Point). Use this to establish an ordering boundary in the recorded timeline without ending the canvas.

## MeasureText(RichTextOptions, ReadOnlySpan<char>)

Measures the full set of layout metrics for the supplied text.

```
public override TextMetrics MeasureText(RichTextOptions textOptions,
ReadOnlySpan<char> text)
```

### Parameters

`textOptions` RichTextOptions

The text shaping and layout options.

`text` ReadOnlySpan<char>

The text to measure.

### Returns

TextMetrics

A TextMetrics value containing the metrics for the laid-out text.

## RenderScene(DrawingBackendScene)

Records a previously created backend scene into this canvas' command timeline.

```
public override void RenderScene(DrawingBackendScene scene)
```

### Parameters

`scene` DrawingBackendScene

The backend scene to render.

### Remarks

The active command range is sealed before the scene is inserted so the scene renders in submission order relative to the commands recorded around it. Like other recorded work, the scene is replayed into the target frame when the root canvas is disposed or otherwise flushed, not at the point of this call.

## Restore()

Restores the most recently saved state.

```
public override void Restore()
```

### Remarks

If the most recently saved state was created by a `SaveLayer` overload, the layer is closed and becomes part of subsequent rendering.

## RestoreTo(int)

Restores to a specific save count.

```
public override void RestoreTo(int saveCount)
```

### Parameters

`saveCount` int

The save count to restore to.

### Remarks

State frames above `saveCount` are discarded, and the last discarded frame becomes the current state. If any discarded state was created by a `SaveLayer` overload, those layers are closed and become part of subsequent rendering.

## Save()

Saves the current drawing state on the state stack.

```
public override int Save()
```

### Returns

int

The save count after the state has been pushed.

### Remarks

This operation stores the current canvas state by reference. If the same DrawingOptions instance is mutated after Save(), those mutations are visible when restoring.

## Save(DrawingOptions)

Saves the current drawing state and replaces the active state with the provided options.

```
public override int Save(DrawingOptions options)
```

### Parameters

`options` DrawingOptions

Drawing options for the new active state.

### Returns

int

The save count after the previous state has been pushed.

### Remarks

The provided `options` instance is stored by reference. Mutating it after this call mutates the active/restored state behavior.

# SaveLayer(GraphicsOptions, IPath, LayerEffect)

Saves the current drawing state and begins an isolated compositing layer whose content is transformed by an effect confined to a path region when the layer is restored.

```
public override int SaveLayer(GraphicsOptions layerOptions, IPath region,
LayerEffect effect)
```

## Parameters

`layerOptions` GraphicsOptions

Graphics options controlling how the closed layer is composited against the parent canvas.

`region` IPath

The path region the effect processes, in local coordinates.

`effect` LayerEffect

The effect applied to the layer content on restore.

## Returns

int

The save count after the layer state has been pushed.

## Remarks

The effect processes only pixels covered by the supplied path; its output lands on the path translated by the effect's offset. The layer bounds are derived from the path and expanded by the effect's reach.

# SaveLayer(GraphicsOptions, IPath, LayerEffect, DrawingOptions)

Saves the current drawing state and begins an isolated compositing layer whose content is transformed by an effect confined to a path region when the layer is restored, using the supplied drawing options for commands recorded into the layer.

```
public override int SaveLayer(GraphicsOptions layerOptions, IPath region, LayerEffect
effect, DrawingOptions options)
```

## Parameters

`layerOptions` GraphicsOptions

Graphics options controlling how the closed layer is composited against the parent canvas.

`region` IPath

The path region the effect processes, in local coordinates.

`effect` LayerEffect

The effect applied to the layer content on restore.

`options` DrawingOptions

Drawing options for the layer contents.

## Returns

int

The save count after the layer state has been pushed.

# SaveLayer(GraphicsOptions, Rectangle)

Saves the current drawing state and begins an isolated compositing layer bounded to a subregion. Subsequent draw commands target that layer until Restore() closes it.

```
public override int SaveLayer(GraphicsOptions layerOptions, Rectangle bounds)
```

## Parameters

`layerOptions` GraphicsOptions

Graphics options controlling how the closed layer is composited against the parent canvas.

`bounds` Rectangle

The local bounds of the layer. Only this region is allocated and composited.

## Returns

int

The save count after the layer state has been pushed.

## Remarks

The layer bounds are expressed in the current local coordinate system and are transformed with the active drawing transform when the layer is created. They limit allocation and compositing only; they do not change the canvas coordinate system used by commands recorded inside the layer.

# SaveLayer(GraphicsOptions, Rectangle, DrawingOptions)

Saves the current drawing state and begins an isolated compositing layer using the supplied drawing options for commands recorded into the layer.

```
public override int SaveLayer(GraphicsOptions layerOptions, Rectangle bounds,
DrawingOptions options)
```

## Parameters

`layerOptions` GraphicsOptions

Graphics options controlling how the closed layer is composited against the parent canvas.

`bounds` Rectangle

The local bounds of the layer. Only this region is allocated and composited.

`options` DrawingOptions

Drawing options for the layer contents.

## Returns

int

The save count after the layer state has been pushed.

# SaveLayer(GraphicsOptions, Rectangle, LayerEffect)

Saves the current drawing state and begins an isolated compositing layer whose content is transformed by an effect when the layer is restored.

```
public override int SaveLayer(GraphicsOptions layerOptions, Rectangle bounds,
LayerEffect effect)
```

## Parameters

`layerOptions` GraphicsOptions

Graphics options controlling how the closed layer is composited against the parent canvas.

`bounds` Rectangle

The content bounds in local canvas coordinates.

`effect` LayerEffect

The effect applied to the layer content on restore.

## Returns

int

The save count after the layer state has been pushed.

## Remarks

The layer isolates the content, so the effect operates on exactly what is drawn between this call and the matching Restore(), against transparency; a DropShadowLayerEffect, for example, slots its shadow beneath that content before the layer composites onto the canvas. The bounds are expanded internally by the effect's reach so blurred or offset output is not cut off.

# SaveLayer(GraphicsOptions, Rectangle, LayerEffect, DrawingOptions)

Saves the current drawing state and begins an isolated compositing layer whose content is transformed by an effect when the layer is restored, using the supplied drawing options for commands recorded into the layer.

```
public override int SaveLayer(GraphicsOptions layerOptions, Rectangle bounds, LayerEffect
effect, DrawingOptions options)
```

## Parameters

`layerOptions` GraphicsOptions

Graphics options controlling how the closed layer is composited against the parent canvas.

`bounds` Rectangle

The content bounds in local canvas coordinates.

`effect` LayerEffect

The effect applied to the layer content on restore.

`options` DrawingOptions

Drawing options for the layer contents.

## Returns

int

The save count after the layer state has been pushed.

# Class DrawingClipDescriptor

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Represents one clip primitive carried by a drawing command.

```
public sealed class DrawingClipDescriptor
```

**Inheritance**

object ← DrawingClipDescriptor

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Properties

## EdgeMode

Gets the edge mode used by rectangle and region clips.

```
public DrawingClipEdgeMode EdgeMode { get; }
```

## Property Value

DrawingClipEdgeMode

## IntegerRectangles

Gets the integer rectangles for an integer region clip.

```
public IReadOnlyList<Rectangle> IntegerRectangles { get; }
```

## Property Value

IReadOnlyList<Rectangle>

# IntersectionRule

Gets the fill rule used when this clip is represented as path geometry.

```
public IntersectionRule IntersectionRule { get; }
```

## Property Value

IntersectionRule

# Kind

Gets the clip primitive kind.

```
public DrawingClipKind Kind { get; }
```

## Property Value

DrawingClipKind

# Operation

Gets the operation used to combine this descriptor with the existing clip.

```
public ClipOperation Operation { get; }
```

## Property Value

ClipOperation

## PathScale

Gets the path-space scale represented by this descriptor.

```
public Vector2 PathScale { get; }
```

### Property Value

Vector2

## PathTransform

Gets the transform that maps scaled path geometry into descriptor coordinates.

```
public Matrix4x4 PathTransform { get; }
```

### Property Value

Matrix4×4

## Paths

Gets the path operands for a path clip.

```
public IReadOnlyList<IPath> Paths { get; }
```

### Property Value

IReadOnlyList<IPath>

## Rectangle

Gets the rectangle for a rectangle clip.

```
public RectangleF Rectangle { get; }
```

### Property Value

RectangleF

## Rectangles

Gets the floating-point rectangles for a region clip.

```
public IReadOnlyList<RectangleF> Rectangles { get; }
```

### Property Value

IReadOnlyList<RectangleF>

# Methods

## CreateIntegerRegion(IReadOnlyList<Rectangle>, ClipOperation)

Creates an integer region clip descriptor.

```
public static DrawingClipDescriptor CreateIntegerRegion(IReadOnlyList<Rectangle> rectangles,
ClipOperation operation)
```

### Parameters

`rectangles` IReadOnlyList<Rectangle>

The region rectangles in canvas-local coordinates.

`operation` ClipOperation

The operation used to combine the region with the existing clip.

## Returns

DrawingClipDescriptor

The descriptor.

# CreatePath(IReadOnlyList<IPath>, ClipOperation, DrawingClipEdgeMode)

Creates a path clip descriptor.

```
public static DrawingClipDescriptor CreatePath(IReadOnlyList<IPath> paths, ClipOperation
operation, DrawingClipEdgeMode edgeMode)
```

## Parameters

`paths` IReadOnlyList<IPath>

The path operands interpreted as one clip region.

`operation` ClipOperation

The operation used to combine the paths with the existing clip.

`edgeMode` DrawingClipEdgeMode

The clip edge mode.

## Returns

DrawingClipDescriptor

The descriptor.

# CreateRectangle(RectangleF, ClipOperation, DrawingClipEdgeMode)

Creates a rectangle clip descriptor.

```
public static DrawingClipDescriptor CreateRectangle(RectangleF rectangle, ClipOperation
operation, DrawingClipEdgeMode edgeMode)
```

## Parameters

`rectangle` RectangleF

The rectangle in canvas-local coordinates.

`operation` ClipOperation

The operation used to combine the rectangle with the existing clip.

`edgeMode` DrawingClipEdgeMode

The clip edge mode.

## Returns

DrawingClipDescriptor

The descriptor.

# CreateRegion(IReadOnlyList<RectangleF>, ClipOperation, DrawingClipEdgeMode)

Creates a floating-point region clip descriptor.

```
public static DrawingClipDescriptor CreateRegion(IReadOnlyList<RectangleF> rectangles,
ClipOperation operation, DrawingClipEdgeMode edgeMode)
```

## Parameters

`rectangles` IReadOnlyList<RectangleF>

The region rectangles in canvas-local coordinates.

`operation` ClipOperation

The operation used to combine the region with the existing clip.

`edgeMode` DrawingClipEdgeMode

The clip edge mode.

### Returns

DrawingClipDescriptor

The descriptor.

## FromPath(IPath, ClipOperation, DrawingClipEdgeMode)

Converts one incoming path to a clip descriptor.

```
public static DrawingClipDescriptor FromPath(IPath path, ClipOperation operation,
DrawingClipEdgeMode edgeMode)
```

### Parameters

`path` IPath

The incoming path.

`operation` ClipOperation

The operation used to combine the path with the existing clip.

`edgeMode` DrawingClipEdgeMode

The clip edge mode.

### Returns

DrawingClipDescriptor

The descriptor.

## GetConservativeBounds(Point)

Gets conservative integer bounds for this descriptor in absolute target coordinates.

```
public Rectangle GetConservativeBounds(Point destinationOffset)
```

### Parameters

`destinationOffset` Point

The absolute destination offset for the owning command.

### Returns

Rectangle

The conservative bounds.

## ToLinearGeometry(out Matrix4×4)

Converts a path clip descriptor to linear geometry.

```
public LinearGeometry ToLinearGeometry(out Matrix4x4 transform)
```

### Parameters

`transform` Matrix4×4

The transform that maps the returned geometry into descriptor coordinates.

### Returns

LinearGeometry

The scaled linear geometry for the path clip.

### Exceptions

InvalidOperationException

Thrown when this descriptor is not a path clip.

## ToPath()

Converts this descriptor to path geometry.

```
public IPath ToPath()
```

### Returns

IPath

A path describing the descriptor area.

## Transform(Matrix4×4)

Transforms this descriptor.

```
public DrawingClipDescriptor Transform(Matrix4x4 matrix)
```

### Parameters

`matrix` Matrix4×4

The transform matrix.

### Returns

DrawingClipDescriptor

The transformed descriptor.

## Translate(Vector2)

Translates this descriptor.

```
public DrawingClipDescriptor Translate(Vector2 offset)
```

### Parameters

`offset` Vector2

The translation offset.

## Returns

DrawingClipDescriptor

The translated descriptor.

# Enum DrawingClipEdgeMode

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Identifies how rectangle clip edges contribute coverage.

```
public enum DrawingClipEdgeMode : byte
```

# Fields

`Antialiased = 1`

The clip participates in antialias coverage.

`Hard = 0`

The clip has hard device-pixel edges.

# Enum DrawingClipKind

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Identifies the clip primitive represented by a drawing clip descriptor.

```
public enum DrawingClipKind : byte
```

# Fields

`IntegerRegion = 1`

The clip is a set of integer rectangles.

`Path = 3`

The clip is path geometry.

`Rectangle = 0`

The clip is an axis-aligned rectangle.

`Region = 2`

The clip is a set of floating-point rectangles.

# Struct DrawingClipState

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Represents an ordered set of exact clip primitives carried by a drawing command.

```
public readonly struct DrawingClipState
```

**Inherited Members**
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

# Properties

## Count

Gets the number of clip descriptors in stack order.

```
public int Count { get; }
```

## Property Value

int

## Empty

Gets an empty clip state.

```
public static DrawingClipState Empty { get; }
```

## Property Value

DrawingClipState

## HasClips

Gets a value indicating whether this state carries any clips.

```
public bool HasClips { get; }
```

### Property Value

bool

# Methods

## Append(DrawingClipDescriptor)

Appends one clip descriptor.

```
public DrawingClipState Append(DrawingClipDescriptor descriptor)
```

### Parameters

`descriptor` DrawingClipDescriptor

The descriptor to append.

### Returns

DrawingClipState

The updated clip state.

## Append(DrawingClipState)

Appends all descriptors from another state.

```
public DrawingClipState Append(DrawingClipState state)
```

## Parameters

`state` DrawingClipState

The clip state to append.

## Returns

DrawingClipState

The updated clip state.

# FromPaths(IReadOnlyList<IPath>, ClipOperation, DrawingClipEdgeMode)

Creates a clip state from path operands.

```
public static DrawingClipState FromPaths(IReadOnlyList<IPath> paths, ClipOperation
operation, DrawingClipEdgeMode edgeMode)
```

## Parameters

`paths` IReadOnlyList<IPath>

The clip paths.

`operation` ClipOperation

The operation used to combine the paths with the existing clip.

`edgeMode` DrawingClipEdgeMode

The clip edge mode.

## Returns

DrawingClipState

The normalized clip state.

## GetDescriptor(int)

Gets one descriptor by stack order.

```
public DrawingClipDescriptor GetDescriptor(int index)
```

### Parameters

`index` int

The descriptor index.

### Returns

DrawingClipDescriptor

The descriptor at the requested index.

## Transform(Matrix4×4)

Transforms this clip state.

```
public DrawingClipState Transform(Matrix4x4 matrix)
```

### Parameters

`matrix` Matrix4×4

The transform matrix.

### Returns

DrawingClipState

The transformed clip state.

## Translate(Vector2)

Translates this clip state.

```
public DrawingClipState Translate(Vector2 offset)
```

## Parameters

`offset` Vector2

The translation offset.

## Returns

DrawingClipState

The translated clip state.

# TryGetConservativeBounds(Point, out Rectangle)

Gets conservative integer bounds for intersection descriptors in absolute target coordinates.

```
public bool TryGetConservativeBounds(Point destinationOffset, out Rectangle bounds)
```

## Parameters

`destinationOffset` Point

The absolute destination offset for the owning command.

`bounds` Rectangle

The conservative bounds.

## Returns

bool

true when at least one intersection descriptor has clip bounds.

# TryGetTargetBoundsClip(Point, out Rectangle)

Gets bounds for a single pixel-aligned clip that can be represented entirely by target bounds.

```
public bool TryGetTargetBoundsClip(Point destinationOffset, out Rectangle bounds)
```

## Parameters

`destinationOffset` Point

The absolute destination offset for the owning canvas state.

`bounds` Rectangle

The exact integer clip bounds.

## Returns

bool

true when this state contains only one pixel-aligned rectangle clip.

# Class DrawingOptions

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Provides options for influencing drawing operations, combining graphics rendering settings, the fill-rule intersection mode, and an optional coordinate transform.

```
public class DrawingOptions
```

**Inheritance**

object ← DrawingOptions

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## DrawingOptions()

Initializes a new instance of the DrawingOptions class.

```
public DrawingOptions()
```

# Fields

## DefaultTextContrast

The default perceptual contrast boost applied to antialiased text rendering.

```
public const float DefaultTextContrast = 0.5
```

### Field Value

float

# Properties

## GraphicsOptions

Gets or sets the graphics rendering options that control antialiasing, blending, and alpha composition for the drawing operation.

```
public GraphicsOptions GraphicsOptions { get; set; }
```

### Property Value

GraphicsOptions

## IntersectionRule

Gets or sets the fill rule used to determine which regions of a self-intersecting or multi-contour path are inside the filled area. Defaults to NonZero.

```
public IntersectionRule IntersectionRule { get; set; }
```

### Property Value

IntersectionRule

## TextContrast

Gets or sets the perceptual contrast boost applied to antialiased text rendering. Coverage is remapped through an S-curve that darkens mostly covered pixels and lightens mostly empty ones, so glyph stems solidify while counters and gaps stay bright; empty, half-covered, and fully covered pixels are unchanged. `0` disables the boost and renders text identically to plain vector fills; `1` applies the full smoothstep remap. Because the curve lightens sub-half coverage, high

values gradually thin hairline stems in very light faces at very small sizes; lower the value if hairline preservation matters more than contrast. The boost applies only to text drawn through the text APIs; general vector fills and strokes are never affected. Defaults to DefaultTextContrast.

```
public float TextContrast { get; set; }
```

## Property Value

float

# Transform

Gets or sets the transform matrix applied to vector output before rasterization. For strokes, the pen is expanded in local geometry space and the resulting outline is transformed before rasterization. Defaults to Identity.

```
public Matrix4x4 Transform { get; set; }
```

## Property Value

Matrix4×4

# Class DrawingOptionsDefaultsExtensions

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Adds extensions that help working with DrawingOptions.

```
public static class DrawingOptionsDefaultsExtensions
```

**Inheritance**

object ← DrawingOptionsDefaultsExtensions

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

# Methods

## GetDrawingOptions(IImageProcessingContext)

Gets the default drawing options against the source image processing context.

```
public static DrawingOptions GetDrawingOptions(this IImageProcessingContext context)
```

## Parameters

`context` IImageProcessingContext

The image processing context to retrieve defaults from.

## Returns

DrawingOptions

The globally configured default options.

# Class DrawingTextCache

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Stores reusable text drawing data shared by one or more drawing canvases.

```
public sealed class DrawingTextCache
```

**Inheritance**

object ← DrawingTextCache

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Remarks

This class is thread-safe. Concurrent drawing operations must use separate canvas instances.

# Constructors

## DrawingTextCache()

Initializes a new instance of the DrawingTextCache class.

```
public DrawingTextCache()
```

## DrawingTextCache(int)

Initializes a new instance of the DrawingTextCache class.

```
public DrawingTextCache(int capacity)
```

### Parameters

`capacity` int

The maximum number of text cache entries.

# Fields

## DefaultCapacity

The default maximum number of text cache entries.

```
public const int DefaultCapacity = 16384
```

### Field Value

int

# Properties

## Capacity

Gets the maximum number of glyph cache entries. The smaller run-path cache capacity is derived from this value.

```
public int Capacity { get; }
```

### Property Value

int

## Count

Gets the number of glyph cache entries. Run-path entries are not included.

```
public int Count { get; }
```

### Property Value

int

# Methods

## Clear()

Removes all cached text drawing data.

```
public void Clear()
```

### Remarks

Draws already in progress can continue using previously cached data and can populate the cache again after this method returns.

# Class DropShadowLayerEffect

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Composites a drop shadow beneath the contents of a compositing layer when it is restored. The shadow is the content's own silhouette tinted with the shadow colour, blurred, and offset.

```
public sealed class DropShadowLayerEffect : LayerEffect
```

**Inheritance**

object ← LayerEffect ← DropShadowLayerEffect

**Inherited Members**

LayerEffect.IsPassThrough , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## DropShadowLayerEffect(Point, float, Color)

Initializes a new instance of the DropShadowLayerEffect class.

```
public DropShadowLayerEffect(Point offset, float sigma, Color color)
```

### Parameters

`offset` Point

The shadow offset, in pixels.

`sigma` float

The Gaussian blur sigma, in pixels; zero draws a hard shadow.

`color` Color

The shadow colour. Its alpha scales the content's alpha, so a semi-transparent colour draws a semi-transparent shadow.

# Properties

## Color

Gets the shadow colour.

```
public Color Color { get; }
```

### Property Value

Color

## Offset

Gets the shadow offset, in pixels.

```
public Point Offset { get; }
```

### Property Value

Point

## Reach

Gets the distance, in pixels, the effect can push content beyond its source region. Layer and processing bounds are expanded by this reach so the effect output is not cut off. Effects constructed over a fallback inherit the fallback's reach.

```
public override int Reach { get; }
```

### Property Value

int

## Sigma

Gets the Gaussian blur sigma, in pixels.

```
public float Sigma { get; }
```

### Property Value

float

## WriteBackOffset

Gets the offset, in pixels, at which the processed pixels are written back relative to the region they were read from.

```
public override Point WriteBackOffset { get; }
```

### Property Value

Point

## WriteBackOptions

Gets the graphics options used to composite the processed pixels back onto the layer, or null to replace the processed region outright.

```
public override GraphicsOptions? WriteBackOptions { get; }
```

### Property Value

GraphicsOptions

# Class EllipticGradientBrush

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Provides an implementation of a brush for painting elliptical gradients. The ellipse is defined by a center point, a point on the longest axis, and the ratio between the longest and shortest axes.

```
public sealed class EllipticGradientBrush : GradientBrush, IEquatable<Brush>
```

**Inheritance**

object ← Brush ← GradientBrush ← EllipticGradientBrush

**Implements**

IEquatable<Brush>

**Inherited Members**

GradientBrush.RepetitionMode , GradientBrush.GradientTransform , GradientBrush.ColorStops , GradientBrush.Equals(Brush) , GradientBrush.GetHashCode() , Brush.Equals(object) , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## EllipticGradientBrush(PointF, PointF, float, GradientRepetitionMode, params ColorStop[])

Initializes a new instance of the EllipticGradientBrush class.

```
public EllipticGradientBrush(PointF center, PointF referenceAxisEnd, float axisRatio,
GradientRepetitionMode repetitionMode, params ColorStop[] colorStops)
```

### Parameters

`center` PointF

The center of the elliptical gradient and 0 for the color stops.

`referenceAxisEnd` PointF

The end point of the reference axis of the ellipse.

`axisRatio` float

The ratio of the axis widths. The second axis is perpendicular to the reference axis and its length is the reference axis length multiplied by this factor.

`repetitionMode` GradientRepetitionMode

Defines how the colors of the gradients are repeated.

`colorStops` ColorStop[]

The color stops.

## EllipticGradientBrush(PointF, PointF, float, GradientRepetitionMode, Matrix4×4, params ColorStop[])

Initializes a new instance of the EllipticGradientBrush class with the ellipse defined in the gradient's own coordinate space.

```
public EllipticGradientBrush(PointF center, PointF referenceAxisEnd, float
axisRatio, GradientRepetitionMode repetitionMode, Matrix4x4 gradientTransform, params
ColorStop[] colorStops)
```

### Parameters

`center` PointF

The center of the elliptical gradient and 0 for the color stops.

`referenceAxisEnd` PointF

The end point of the reference axis of the ellipse.

`axisRatio` float

The ratio of the axis widths. The second axis is perpendicular to the reference axis and its length is the reference axis length multiplied by this factor.

`repetitionMode` GradientRepetitionMode

Defines how the colors of the gradients are repeated.

`gradientTransform` Matrix4×4

The transform from the gradient's coordinate space to the drawing.

`colorStops` ColorStop[]

The color stops.

# Properties

## AxisRatio

Gets the ratio of the secondary axis to the primary axis.

```
public float AxisRatio { get; }
```

### Property Value

float

## Center

Gets the center of the ellipse.

```
public PointF Center { get; }
```

### Property Value

PointF

## ReferenceAxisEnd

Gets the end point of the reference axis.

```
public PointF ReferenceAxisEnd { get; }
```

### Property Value

PointF

# Methods

## CreateRenderer<TPixel>(Configuration, GraphicsOptions, int, RectangleF)

Creates the prepared execution object for this brush.

```
public override BrushRenderer<TPixel> CreateRenderer<TPixel>(Configuration configuration,
GraphicsOptions options, int canvasWidth, RectangleF region) where TPixel :
unmanaged, IPixel<TPixel>
```

### Parameters

`configuration` Configuration

The configuration instance to use when performing operations.

`options` GraphicsOptions

The graphic options.

`canvasWidth` int

The canvas width for the current render pass.

`region` RectangleF

The region the brush will be applied to.

### Returns

BrushRenderer<TPixel>

The BrushRenderer<TPixel> for this brush.

## Type Parameters

`TPixel`

The pixel type.

## Remarks

The `region` when being applied to things like shapes would usually be the bounding box of the shape not necessarily the bounds of the whole image.

# Transform(Matrix4×4, Rectangle, Rectangle)

Returns a new brush with its defining geometry transformed by the given matrix.

```
public override Brush Transform(Matrix4x4 matrix, Rectangle sourceInterest,
Rectangle preparedInterest)
```

## Parameters

`matrix` Matrix4×4

The transformation matrix to apply.

`sourceInterest` Rectangle

The command interest before preparation.

`preparedInterest` Rectangle

The command interest after preparation.

## Returns

Brush

A transformed brush, or `this` if the brush has no spatial parameters.

# Class GlowLayerEffect

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Composites a glow beneath the contents of a compositing layer when it is restored: the content's own silhouette tinted with the glow colour and blurred, spreading evenly in all directions.

```
public sealed class GlowLayerEffect : LayerEffect
```

**Inheritance**

object ← LayerEffect ← GlowLayerEffect

**Inherited Members**

LayerEffect.IsPassThrough , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## GlowLayerEffect(float, Color)

Initializes a new instance of the GlowLayerEffect class.

```
public GlowLayerEffect(float sigma, Color color)
```

## Parameters

`sigma` float

The Gaussian blur sigma, in pixels, controlling the glow's spread.

`color` Color

The glow colour. Its alpha scales the content's alpha, so a semi-transparent colour draws a fainter glow.

# Properties

## Color

Gets the glow colour.

```
public Color Color { get; }
```

### Property Value

Color

## Reach

Gets the distance, in pixels, the effect can push content beyond its source region. Layer and processing bounds are expanded by this reach so the effect output is not cut off. Effects constructed over a fallback inherit the fallback's reach.

```
public override int Reach { get; }
```

### Property Value

int

## Sigma

Gets the Gaussian blur sigma, in pixels, controlling the glow's spread.

```
public float Sigma { get; }
```

### Property Value

float

## WriteBackOffset

Gets the offset, in pixels, at which the processed pixels are written back relative to the region they were read from.

```
public override Point WriteBackOffset { get; }
```

### Property Value

Point

## WriteBackOptions

Gets the graphics options used to composite the processed pixels back onto the layer, or null to replace the processed region outright.

```
public override GraphicsOptions? WriteBackOptions { get; }
```

### Property Value

GraphicsOptions

# Class GradientBrush

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Base class for gradient brushes. Derived brushes define a parameterization that maps each point to a scalar gradient position; this base class maps that position to a color via the sorted color stops and the GradientRepetitionMode.

```
public abstract class GradientBrush : Brush, IEquatable<Brush>
```

**Inheritance**

object ← Brush ← GradientBrush

**Implements**

IEquatable<Brush>

**Derived**

EllipticGradientBrush, LinearGradientBrush, RadialGradientBrush, SweepGradientBrush

**Inherited Members**

Brush.CreateRenderer<TPixel>(Configuration, GraphicsOptions, int, RectangleF) , Brush.Transform(Matrix4×4, Rectangle, Rectangle) , Brush.Equals(object) , object.Equals(object, object) , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## GradientBrush(GradientRepetitionMode, params ColorStop[])

Initializes a new instance of the GradientBrush class.

```
protected GradientBrush(GradientRepetitionMode repetitionMode, params
ColorStop[] colorStops)
```

## Parameters

`repetitionMode` GradientRepetitionMode

Defines how the colors are repeated beyond the interval [0..1].

`colorStops` ColorStop[]

The gradient colors.

## GradientBrush(GradientRepetitionMode, Matrix4×4, params ColorStop[])

Initializes a new instance of the GradientBrush class.

```
protected GradientBrush(GradientRepetitionMode repetitionMode, Matrix4x4 gradientTransform,
params ColorStop[] colorStops)
```

### Parameters

`repetitionMode` GradientRepetitionMode

Defines how the colors are repeated beyond the interval [0..1].

`gradientTransform` Matrix4×4

The transform from the gradient's coordinate space to the drawing.

`colorStops` ColorStop[]

The gradient colors.

# Properties

## ColorStops

Gets the color stops for this gradient.

```
public ReadOnlySpan<ColorStop> ColorStops { get; }
```

## Property Value

ReadOnlySpan<ColorStop>

# ColorStopsArray

Gets the color stops array for use by derived applicators.

```
protected ColorStop[] ColorStopsArray { get; }
```

## Property Value

ColorStop[]

# GradientTransform

Gets the transform from the gradient's coordinate space to the drawing. The geometry of the gradient is defined in its own space and every sample is mapped through the inverse of this transform, so a skew or a non-uniform scale changes the shape of the gradient. The transform is affine: a projective transform applied through Transform(Matrix4×4, Rectangle, Rectangle) projects the geometry point by point instead, because an affine gradient cannot express perspective.

```
public Matrix4x4 GradientTransform { get; }
```

## Property Value

Matrix4×4

# RepetitionMode

Gets how the colors are repeated beyond the interval [0..1].

```
public GradientRepetitionMode RepetitionMode { get; }
```

### Property Value

GradientRepetitionMode

# Methods

## Equals(Brush?)

Indicates whether the current object is equal to another object of the same type.

```
public override bool Equals(Brush? other)
```

### Parameters

`other` Brush

An object to compare with this object.

### Returns

bool

true if the current object is equal to the `other` parameter; otherwise, false.

## GetHashCode()

Serves as the default hash function.

```
public override int GetHashCode()
```

### Returns

int

A hash code for the current object.

# Enum GradientRepetitionMode

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Modes to repeat a gradient.

```
public enum GradientRepetitionMode
```

# Fields

`DontFill = 3`

With DontFill a gradient does not touch any pixel beyond its borders. For the LinearGradient Brush this is beyond the orthogonal through start and end, For RadialGradientBrush and Elliptic GradientBrush it's beyond 1.0.

`None = 0`

Don't repeat, keep the color of start and end beyond those points stable.

`Reflect = 2`

Reflect the gradient. Similar to Repeat, but each other repetition uses inverse order of Color Stops. Used on a Black-White gradient, Reflect leads to Black→{gray}→White→{gray}→Black...

`Repeat = 1`

Repeat the gradient. If it's a black-white gradient, with Repeat it will be Black→{gray}->White|Black→{gray}→White|...

# Class ImageBrush

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

The untyped base class for image brushes, used to support non-generic brush references in drawing contexts.

```
public abstract class ImageBrush : Brush, IEquatable<Brush>
```

**Inheritance**

object ← Brush ← ImageBrush

**Implements**

IEquatable<Brush>

**Derived**

ImageBrush<TPixel>

**Inherited Members**

Brush.Transform(Matrix4×4, Rectangle, Rectangle) , Brush.Equals(object) , object.Equals(object, object) , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## ImageBrush(Image)

Initializes a new instance of the ImageBrush class.

```
protected ImageBrush(Image image)
```

### Parameters

`image` Image

The source image to draw.

# ImageBrush(Image, WrapMode, WrapMode)

Initializes a new instance of the ImageBrush class.

```
protected ImageBrush(Image image, WrapMode wrapX, WrapMode wrapY)
```

## Parameters

`image` Image

The image.

`wrapX` WrapMode

The wrap mode used when sampling horizontally beyond the source region.

`wrapY` WrapMode

The wrap mode used when sampling vertically beyond the source region.

# ImageBrush(Image, Point)

Initializes a new instance of the ImageBrush class.

```
protected ImageBrush(Image image, Point offset)
```

## Parameters

`image` Image

The image.

`offset` Point

An offset to apply to the image while drawing the texture.

# ImageBrush(Image, Point, WrapMode, WrapMode)

Initializes a new instance of the ImageBrush class.

```
protected ImageBrush(Image image, Point offset, WrapMode wrapX, WrapMode wrapY)
```

### Parameters

`image` Image

The image.

`offset` Point

An offset to apply to the image while drawing the texture.

`wrapX` WrapMode

The wrap mode used when sampling horizontally beyond the source region.

`wrapY` WrapMode

The wrap mode used when sampling vertically beyond the source region.

## ImageBrush(Image, RectangleF)

Initializes a new instance of the ImageBrush class.

```
protected ImageBrush(Image image, RectangleF region)
```

### Parameters

`image` Image

The image.

`region` RectangleF

The region of interest. This overrides any region used to initialize the brush applicator.

## ImageBrush(Image, RectangleF, WrapMode, WrapMode)

Initializes a new instance of the ImageBrush class.

```
protected ImageBrush(Image image, RectangleF region, WrapMode wrapX, WrapMode wrapY)
```

## Parameters

`image` Image

The image.

`region` RectangleF

The region of interest. This overrides any region used to initialize the brush applicator.

`wrapX` WrapMode

The wrap mode used when sampling horizontally beyond the source region.

`wrapY` WrapMode

The wrap mode used when sampling vertically beyond the source region.

# ImageBrush(Image, RectangleF, Point)

Initializes a new instance of the ImageBrush class.

```
protected ImageBrush(Image image, RectangleF region, Point offset)
```

## Parameters

`image` Image

The image.

`region` RectangleF

The region of interest. This overrides any region used to initialize the brush applicator.

`offset` Point

An offset to apply to the image while drawing the texture.

## ImageBrush(Image, RectangleF, Point, WrapMode, WrapMode)

Initializes a new instance of the ImageBrush class.

```
protected ImageBrush(Image image, RectangleF region, Point offset, WrapMode wrapX,
WrapMode wrapY)
```

### Parameters

`image` Image

The image.

`region` RectangleF

The region of interest. This overrides any region used to initialize the brush applicator.

`offset` Point

An offset to apply to the image while drawing the texture.

`wrapX` WrapMode

The wrap mode used when sampling horizontally beyond the source region.

`wrapY` WrapMode

The wrap mode used when sampling vertically beyond the source region.

# Properties

## Offset

Gets the offset applied to the brush origin.

```
public Point Offset { get; }
```

### Property Value

Point

## SourceRegion

Gets the source region within the image.

```
public RectangleF SourceRegion { get; }
```

### Property Value

RectangleF

## UntypedImage

Gets the source image used by this brush.

```
public Image UntypedImage { get; }
```

### Property Value

Image

## WrapX

Gets the wrap mode used when sampling horizontally beyond the SourceRegion.

```
public WrapMode WrapX { get; }
```

### Property Value

WrapMode

## WrapY

Gets the wrap mode used when sampling vertically beyond the SourceRegion.

```
public WrapMode WrapY { get; }
```

### Property Value

WrapMode

## Methods

### CreateRenderer<TPixel>(Configuration, GraphicsOptions, int, RectangleF)

Creates the prepared execution object for this brush.

```
public override BrushRenderer<TPixel> CreateRenderer<TPixel>(Configuration configuration,
GraphicsOptions options, int canvasWidth, RectangleF region) where TPixel :
unmanaged, IPixel<TPixel>
```

### Parameters

`configuration` Configuration

The configuration instance to use when performing operations.

`options` GraphicsOptions

The graphic options.

`canvasWidth` int

The canvas width for the current render pass.

`region` RectangleF

The region the brush will be applied to.

### Returns

BrushRenderer<TPixel>

The BrushRenderer<TPixel> for this brush.

### Type Parameters

`TPixel`

The pixel type.

### Remarks

The `region` when being applied to things like shapes would usually be the bounding box of the shape not necessarily the bounds of the whole image.

## Equals(Brush?)

Indicates whether the current object is equal to another object of the same type.

```
public override bool Equals(Brush? other)
```

### Parameters

`other` Brush

An object to compare with this object.

### Returns

bool

true if the current object is equal to the `other` parameter; otherwise, false.

## GetHashCode()

Serves as the default hash function.

```
public override int GetHashCode()
```

### Returns

int

A hash code for the current object.

# Class ImageBrush<TPixel>

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Provides an implementation of an image brush for painting images within areas.

```
public sealed class ImageBrush<TPixel> : ImageBrush, IEquatable<Brush> where TPixel :
unmanaged, IPixel<TPixel>
```

## Type Parameters

TPixel

The pixel format of the source image.

**Inheritance**

object ← Brush ← ImageBrush ← ImageBrush<TPixel>

**Implements**

IEquatable<Brush>

**Inherited Members**

ImageBrush.UntypedImage , ImageBrush.SourceRegion , ImageBrush.Offset , ImageBrush.WrapX , ImageBrush.WrapY , ImageBrush.Equals(Brush) , ImageBrush.GetHashCode() , ImageBrush.CreateRenderer<TPixel>(Configuration, GraphicsOptions, int, RectangleF) , Brush.Equals(object) , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## ImageBrush(Image<TPixel>)

Initializes a new instance of the ImageBrush<TPixel> class.

```
public ImageBrush(Image<TPixel> image)
```

## Parameters

`image` Image<TPixel>

The source image to draw.

# ImageBrush(Image<TPixel>, WrapMode, WrapMode)

Initializes a new instance of the ImageBrush<TPixel> class.

```
public ImageBrush(Image<TPixel> image, WrapMode wrapX, WrapMode wrapY)
```

## Parameters

`image` Image<TPixel>

The source image to draw.

`wrapX` WrapMode

The wrap mode used when sampling horizontally beyond the source region.

`wrapY` WrapMode

The wrap mode used when sampling vertically beyond the source region.

# ImageBrush(Image<TPixel>, Point)

Initializes a new instance of the ImageBrush<TPixel> class.

```
public ImageBrush(Image<TPixel> image, Point offset)
```

## Parameters

`image` Image<TPixel>

The source image to draw.

`offset` Point

An offset to apply to the image while drawing the texture.

## ImageBrush(Image<TPixel>, Point, WrapMode, WrapMode)

Initializes a new instance of the ImageBrush<TPixel> class.

```
public ImageBrush(Image<TPixel> image, Point offset, WrapMode wrapX, WrapMode wrapY)
```

### Parameters

`image` Image<TPixel>

The source image to draw.

`offset` Point

An offset to apply to the image while drawing the texture.

`wrapX` WrapMode

The wrap mode used when sampling horizontally beyond the source region.

`wrapY` WrapMode

The wrap mode used when sampling vertically beyond the source region.

## ImageBrush(Image<TPixel>, RectangleF)

Initializes a new instance of the ImageBrush<TPixel> class.

```
public ImageBrush(Image<TPixel> image, RectangleF region)
```

### Parameters

`image` Image<TPixel>

The source image to draw.

`region` RectangleF

The region of interest within the source image.

## ImageBrush(Image<TPixel>, RectangleF, WrapMode, WrapMode)

Initializes a new instance of the ImageBrush<TPixel> class.

```
public ImageBrush(Image<TPixel> image, RectangleF region, WrapMode wrapX, WrapMode wrapY)
```

### Parameters

`image` Image<TPixel>

The source image to draw.

`region` RectangleF

The region of interest within the source image.

`wrapX` WrapMode

The wrap mode used when sampling horizontally beyond the source region.

`wrapY` WrapMode

The wrap mode used when sampling vertically beyond the source region.

## ImageBrush(Image<TPixel>, RectangleF, Point)

Initializes a new instance of the ImageBrush<TPixel> class.

```
public ImageBrush(Image<TPixel> image, RectangleF region, Point offset)
```

### Parameters

`image` Image<TPixel>

The source image to draw.

`region` RectangleF

The region of interest within the source image.

`offset` Point

An offset to apply to the image while drawing the texture.

## ImageBrush(Image<TPixel>, RectangleF, Point, WrapMode, WrapMode)

Initializes a new instance of the ImageBrush<TPixel> class.

```
public ImageBrush(Image<TPixel> image, RectangleF region, Point offset, WrapMode wrapX,
WrapMode wrapY)
```

### Parameters

`image` Image<TPixel>

The source image to draw.

`region` RectangleF

The region of interest within the source image.

`offset` Point

An offset to apply to the image while drawing the texture.

`wrapX` WrapMode

The wrap mode used when sampling horizontally beyond the source region.

`wrapY` WrapMode

The wrap mode used when sampling vertically beyond the source region.

# Properties

## SourceImage

Gets the typed source image used by this brush.

```
public Image<TPixel> SourceImage { get; }
```

### Property Value

Image<TPixel>

# Methods

## Transform(Matrix4×4, Rectangle, Rectangle)

Returns a new brush with its defining geometry transformed by the given matrix.

```
public override Brush Transform(Matrix4x4 matrix, Rectangle sourceInterest,
Rectangle preparedInterest)
```

### Parameters

`matrix` Matrix4×4

The transformation matrix to apply.

`sourceInterest` Rectangle

The command interest before preparation.

`preparedInterest` Rectangle

The command interest after preparation.

### Returns

Brush

A transformed brush, or `this` if the brush has no spatial parameters.

# Class InnerShadowLayerEffect

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Composites a shadow inside the contents of a compositing layer when it is restored: the darkness outside the content's silhouette is tinted with the shadow colour, blurred, offset, and clipped onto the content itself, so the shadow hugs the content's inside edges.

```
public sealed class InnerShadowLayerEffect : LayerEffect
```

**Inheritance**

object ← LayerEffect ← InnerShadowLayerEffect

**Inherited Members**

LayerEffect.IsPassThrough , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## InnerShadowLayerEffect(Point, float, Color)

Initializes a new instance of the InnerShadowLayerEffect class.

```
public InnerShadowLayerEffect(Point offset, float sigma, Color color)
```

### Parameters

`offset` Point

The shadow offset, in pixels. Positive values shade the content's top and left inside edges, matching a CSS inset shadow.

`sigma` float

The Gaussian blur sigma, in pixels; zero draws a hard shadow.

`color` Color

The shadow colour. Its alpha scales the shadow's alpha, so a semi-transparent colour draws a semi-transparent shadow.

# Properties

## Color

Gets the shadow colour.

```
public Color Color { get; }
```

### Property Value

Color

## Offset

Gets the shadow offset, in pixels.

```
public Point Offset { get; }
```

### Property Value

Point

## Reach

Gets the distance, in pixels, the effect can push content beyond its source region. Layer and processing bounds are expanded by this reach so the effect output is not cut off. Effects constructed over a fallback inherit the fallback's reach.

```
public override int Reach { get; }
```

## Property Value

int

# Sigma

Gets the Gaussian blur sigma, in pixels.

```
public float Sigma { get; }
```

## Property Value

float

# WriteBackOffset

Gets the offset, in pixels, at which the processed pixels are written back relative to the region they were read from.

```
public override Point WriteBackOffset { get; }
```

## Property Value

Point

# WriteBackOptions

Gets the graphics options used to composite the processed pixels back onto the layer, or null to replace the processed region outright.

```
public override GraphicsOptions? WriteBackOptions { get; }
```

## Property Value

GraphicsOptions

# Class LayerEffect

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Describes an image effect applied to the contents of a compositing layer when it is restored. The layer isolates the content the effect operates on, so the effect sees only what was drawn into the layer, against transparency, before the result is composited onto the canvas.

```
public abstract class LayerEffect
```

**Inheritance**

object ← LayerEffect

**Derived**

BackdropLayerEffect, WebGPUShaderLayerEffect, BlurLayerEffect, ColorMatrixLayerEffect, DropShadowLayerEffect, GlowLayerEffect, InnerShadowLayerEffect

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## LayerEffect(LayerEffect)

Initializes a new instance of the LayerEffect class with the observable behavior of another effect when this effect is not executed natively.

```
protected LayerEffect(LayerEffect fallbackEffect)
```

### Parameters

`fallbackEffect` LayerEffect

The effect whose behavior is used as the fallback.

## LayerEffect(Action<IImageProcessingContext>)

Initializes a new instance of the LayerEffect class.

```
protected LayerEffect(Action<IImageProcessingContext> operation)
```

### Parameters

operation Action<IImageProcessingContext>

The operation that implements the effect.

# Properties

## IsPassThrough

Gets a value indicating whether the effect leaves the layer content unchanged, in which case its application is skipped entirely.

```
public virtual bool IsPassThrough { get; }
```

### Property Value

bool

## Reach

Gets the distance, in pixels, the effect can push content beyond its source region. Layer and processing bounds are expanded by this reach so the effect output is not cut off. Effects constructed over a fallback inherit the fallback's reach.

```
public virtual int Reach { get; }
```

### Property Value

int

## WriteBackOffset

Gets the offset, in pixels, at which the processed pixels are written back relative to the region they were read from.

```
public virtual Point WriteBackOffset { get; }
```

### Property Value

Point

## WriteBackOptions

Gets the graphics options used to composite the processed pixels back onto the layer, or null to replace the processed region outright.

```
public virtual GraphicsOptions? WriteBackOptions { get; }
```

### Property Value

GraphicsOptions

# Class LinearGradientBrush

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Provides a brush that paints linear gradients within an area. Supports both classic two-point gradients and three-point (rotated) gradients.

```
public sealed class LinearGradientBrush : GradientBrush, IEquatable<Brush>
```

**Inheritance**

object ← Brush ← GradientBrush ← LinearGradientBrush

**Implements**

IEquatable<Brush>

**Inherited Members**

GradientBrush.RepetitionMode , GradientBrush.GradientTransform , GradientBrush.ColorStops , Brush.Equals(object) , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## LinearGradientBrush(PointF, PointF, GradientRepetitionMode, params ColorStop[])

Initializes a new instance of the LinearGradientBrush class using a start and end point.

```
public LinearGradientBrush(PointF p0, PointF p1, GradientRepetitionMode repetitionMode,
params ColorStop[] colorStops)
```

### Parameters

`p0` PointF

The start point of the gradient.

`p1` PointF

The end point of the gradient.

`repetitionMode` GradientRepetitionMode

Defines how the colors are repeated.

`colorStops` ColorStop[]

The ordered color stops of the gradient.

## LinearGradientBrush(PointF, PointF, GradientRepetitionMode, Matrix4×4, params ColorStop[])

Initializes a new instance of the LinearGradientBrush class using a start and end point defined in the gradient's own coordinate space.

```
public LinearGradientBrush(PointF p0, PointF p1, GradientRepetitionMode repetitionMode,
Matrix4x4 gradientTransform, params ColorStop[] colorStops)
```

### Parameters

`p0` PointF

The start point of the gradient.

`p1` PointF

The end point of the gradient.

`repetitionMode` GradientRepetitionMode

Defines how the colors are repeated.

`gradientTransform` Matrix4×4

The transform from the gradient's coordinate space to the drawing.

`colorStops` ColorStop[]

The ordered color stops of the gradient.

# LinearGradientBrush(PointF, PointF, PointF, GradientRepetitionMode, params ColorStop[])

Initializes a new instance of the LinearGradientBrush class using three points to define a rotated gradient axis.

```
public LinearGradientBrush(PointF p0, PointF p1, PointF rotationPoint,
GradientRepetitionMode repetitionMode, params ColorStop[] colorStops)
```

## Parameters

`p0` PointF

The first point (start of the gradient).

`p1` PointF

The second point (gradient vector endpoint).

`rotationPoint` PointF

The rotation reference point. This defines the rotation of the gradient axis.

`repetitionMode` GradientRepetitionMode

Defines how the colors are repeated.

`colorStops` ColorStop[]

The ordered color stops of the gradient.

# LinearGradientBrush(PointF, PointF, PointF, GradientRepetitionMode, Matrix4×4, params ColorStop[])

Initializes a new instance of the LinearGradientBrush class using three points, defined in the gradient's own coordinate space, to define a rotated gradient axis.

```
public LinearGradientBrush(PointF p0, PointF p1, PointF rotationPoint,
GradientRepetitionMode repetitionMode, Matrix4x4 gradientTransform, params
ColorStop[] colorStops)
```

### Parameters

`p0` PointF

The first point (start of the gradient).

`p1` PointF

The second point (gradient vector endpoint).

`rotationPoint` PointF

The rotation reference point. This defines the rotation of the gradient axis.

`repetitionMode` GradientRepetitionMode

Defines how the colors are repeated.

`gradientTransform` Matrix4×4

The transform from the gradient's coordinate space to the drawing.

`colorStops` ColorStop[]

The ordered color stops of the gradient.

# Properties

## EndPoint

Gets the end point of the gradient axis.

```
public PointF EndPoint { get; }
```

### Property Value

PointF

## StartPoint

Gets the start point of the gradient axis.

```
public PointF StartPoint { get; }
```

### Property Value

PointF

# Methods

## CreateRenderer<TPixel>(Configuration, GraphicsOptions, int, RectangleF)

Creates the prepared execution object for this brush.

```
public override BrushRenderer<TPixel> CreateRenderer<TPixel>(Configuration configuration,
GraphicsOptions options, int canvasWidth, RectangleF region) where TPixel :
unmanaged, IPixel<TPixel>
```

### Parameters

`configuration` Configuration

The configuration instance to use when performing operations.

`options` GraphicsOptions

The graphic options.

`canvasWidth` int

The canvas width for the current render pass.

`region` RectangleF

The region the brush will be applied to.

### Returns

BrushRenderer<TPixel>

The BrushRenderer<TPixel> for this brush.

### Type Parameters

`TPixel`

The pixel type.

### Remarks

The `region` when being applied to things like shapes would usually be the bounding box of the shape not necessarily the bounds of the whole image.

## Equals(Brush?)

Indicates whether the current object is equal to another object of the same type.

```
public override bool Equals(Brush? other)
```

### Parameters

`other` Brush

An object to compare with this object.

### Returns

bool

true if the current object is equal to the `other` parameter; otherwise, false.

## GetHashCode()

Serves as the default hash function.

```
public override int GetHashCode()
```

### Returns

int

A hash code for the current object.

## Transform(Matrix4×4, Rectangle, Rectangle)

Returns a new brush with its defining geometry transformed by the given matrix.

```
public override Brush Transform(Matrix4x4 matrix, Rectangle sourceInterest,
Rectangle preparedInterest)
```

### Parameters

`matrix` Matrix4×4

The transformation matrix to apply.

`sourceInterest` Rectangle

The command interest before preparation.

`preparedInterest` Rectangle

The command interest after preparation.

### Returns

Brush

A transformed brush, or `this` if the brush has no spatial parameters.

# Class PaintExtensions

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Adds image-processing extensions that paint each frame through DrawingCanvas.

```
public static class PaintExtensions
```

**Inheritance**

object ← PaintExtensions

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

# Methods

## Paint(IImageProcessingContext, CanvasAction)

Paints each image frame using drawing options from the current context.

```
public static IImageProcessingContext Paint(this IImageProcessingContext source,
CanvasAction action)
```

## Parameters

`source` IImageProcessingContext

The image processing context to paint.

`action` CanvasAction

The per-frame painting callback.

## Returns

IImageProcessingContext

The IImageProcessingContext so additional processing operations can be chained.

## Paint(IImageProcessingContext, DrawingOptions, CanvasAction)

Paints each image frame using the supplied drawing options.

```
public static IImageProcessingContext Paint(this IImageProcessingContext source,
DrawingOptions options, CanvasAction action)
```

### Parameters

`source` IImageProcessingContext

The image processing context to paint.

`options` DrawingOptions

The drawing options applied when creating each frame canvas.

`action` CanvasAction

The per-frame painting callback.

### Returns

IImageProcessingContext

The IImageProcessingContext so additional processing operations can be chained.

## Paint(IImageProcessingContext, DrawingOptions, DrawingTextCache, CanvasAction)

Paints each image frame using the supplied drawing options and text drawing cache.

```
public static IImageProcessingContext Paint(this IImageProcessingContext source,
DrawingOptions options, DrawingTextCache textCache, CanvasAction action)
```

## Parameters

`source` IImageProcessingContext

The image processing context to paint.

`options` DrawingOptions

The drawing options applied when creating each frame canvas.

`textCache` DrawingTextCache

The text drawing cache used by each frame canvas.

`action` CanvasAction

The per-frame painting callback.

## Returns

IImageProcessingContext

The IImageProcessingContext so additional processing operations can be chained.

# Class PaintProcessor

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Defines the image processor used by Paint(IImageProcessingContext, DrawingOptions, CanvasAction) to execute a canvas callback for each image frame.

```
public sealed class PaintProcessor : IImageProcessor
```

**Inheritance**

object ← PaintProcessor

**Implements**

IImageProcessor

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## PaintProcessor(DrawingOptions, CanvasAction)

Initializes a new instance of the PaintProcessor class.

```
public PaintProcessor(DrawingOptions options, CanvasAction action)
```

## Parameters

`options` DrawingOptions

The drawing options used when creating each frame canvas.

`action` CanvasAction

The per-frame painting callback.

## PaintProcessor(DrawingOptions, DrawingTextCache, CanvasAction)

Initializes a new instance of the PaintProcessor class.

```
public PaintProcessor(DrawingOptions options, DrawingTextCache textCache,
CanvasAction action)
```

### Parameters

`options` DrawingOptions

The drawing options used when creating each frame canvas.

`textCache` DrawingTextCache

The text drawing cache used by each frame canvas.

`action` CanvasAction

The per-frame painting callback.

# Properties

## Options

Gets the drawing options used when creating each frame canvas.

```
public DrawingOptions Options { get; }
```

### Property Value

DrawingOptions

## TextCache

Gets the text drawing cache used by each frame canvas, or null when each frame canvas owns a private cache.

```
public DrawingTextCache? TextCache { get; }
```

### Property Value

DrawingTextCache

# Methods

## CreatePixelSpecificProcessor<TPixel>(Configuration, Image<TPixel>, Rectangle)

Creates a pixel specific IImageProcessor<TPixel> that is capable of executing the processing algorithm on an Image<TPixel>.

```
public IImageProcessor<TPixel> CreatePixelSpecificProcessor<TPixel>(Configuration
configuration, Image<TPixel> source, Rectangle sourceRectangle) where TPixel :
unmanaged, IPixel<TPixel>
```

### Parameters

`configuration` Configuration

The configuration which allows altering default behaviour or extending the library.

`source` Image<TPixel>

The source image. Cannot be null.

`sourceRectangle` Rectangle

The Rectangle structure that specifies the portion of the image object to draw.

### Returns

IImageProcessor<TPixel>

The IImageProcessor<TPixel>

### Type Parameters

`TPixel`

The pixel type.

# Class PathGradientBrush

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Provides an implementation of a brush for painting gradients between multiple color positions in 2D coordinates.

```
public sealed class PathGradientBrush : Brush, IEquatable<Brush>
```

**Inheritance**

object ← Brush ← PathGradientBrush

**Implements**

IEquatable<Brush>

**Inherited Members**

Brush.Equals(object) , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Remarks

Colors are assigned to the polygon points in order; if fewer colors than points are supplied the colors are reused cyclically.

# Constructors

## PathGradientBrush(PointF[], Color[])

Initializes a new instance of the PathGradientBrush class.

```
public PathGradientBrush(PointF[] points, Color[] colors)
```

### Parameters

`points` PointF[]

Points that constitute a polygon that represents the gradient area.

`colors` Color[]

Array of colors that correspond to each point in the polygon.

## PathGradientBrush(PointF[], Color[], Color)

Initializes a new instance of the PathGradientBrush class.

```
public PathGradientBrush(PointF[] points, Color[] colors, Color centerColor)
```

### Parameters

`points` PointF[]

Points that constitute a polygon that represents the gradient area.

`colors` Color[]

Array of colors that correspond to each point in the polygon.

`centerColor` Color

Color at the center of the gradient area to which the other colors converge.

# Properties

## CenterColor

Gets the color at the center of the gradient area.

```
public Color CenterColor { get; }
```

### Property Value

Color

## Colors

Gets the colors that are mapped to the polygon points.

```
public ReadOnlySpan<Color> Colors { get; }
```

### Property Value

ReadOnlySpan<Color>

## HasExplicitCenterColor

Gets a value indicating whether the center color was explicitly supplied.

```
public bool HasExplicitCenterColor { get; }
```

### Property Value

bool

## Points

Gets the polygon points that define the gradient area.

```
public ReadOnlySpan<PointF> Points { get; }
```

### Property Value

ReadOnlySpan<PointF>

# Methods

## CreateRenderer<TPixel>(Configuration, GraphicsOptions, int, RectangleF)

Creates the prepared execution object for this brush.

```
public override BrushRenderer<TPixel> CreateRenderer<TPixel>(Configuration configuration,
GraphicsOptions options, int canvasWidth, RectangleF region) where TPixel :
unmanaged, IPixel<TPixel>
```

## Parameters

`configuration` Configuration

The configuration instance to use when performing operations.

`options` GraphicsOptions

The graphic options.

`canvasWidth` int

The canvas width for the current render pass.

`region` RectangleF

The region the brush will be applied to.

## Returns

BrushRenderer<TPixel>

The BrushRenderer<TPixel> for this brush.

## Type Parameters

`TPixel`

The pixel type.

## Remarks

The `region` when being applied to things like shapes would usually be the bounding box of the shape not necessarily the bounds of the whole image.

# Equals(Brush?)

Indicates whether the current object is equal to another object of the same type.

```
public override bool Equals(Brush? other)
```

## Parameters

`other` Brush

An object to compare with this object.

## Returns

bool

true if the current object is equal to the `other` parameter; otherwise, false.

# GetHashCode()

Serves as the default hash function.

```
public override int GetHashCode()
```

## Returns

int

A hash code for the current object.

# Transform(Matrix4×4, Rectangle, Rectangle)

Returns a new brush with its defining geometry transformed by the given matrix.

```
public override Brush Transform(Matrix4x4 matrix, Rectangle sourceInterest,
Rectangle preparedInterest)
```

## Parameters

`matrix` Matrix4×4

The transformation matrix to apply.

`sourceInterest` Rectangle

The command interest before preparation.

`preparedInterest` Rectangle

The command interest after preparation.

## Returns

Brush

A transformed brush, or `this` if the brush has no spatial parameters.

# Class PatternBrush

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Provides an implementation of a pattern brush for painting patterns.

```
public sealed class PatternBrush : Brush, IEquatable<Brush>
```

**Inheritance**

object ← Brush ← PatternBrush

**Implements**

IEquatable<Brush>

**Inherited Members**

Brush.Transform(Matrix4×4, Rectangle, Rectangle) , Brush.Equals(object) , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Remarks

The patterns that are used to create a custom pattern brush are made up of a repeating matrix of flags, where each flag denotes whether to draw the foreground color or the background color. so to create a new bool[,] with your flags

For example if you wanted to create a diagonal line that repeat every 4 pixels you would use a pattern like so 1000 0100 0010 0001

or you want a horizontal stripe which is 3 pixels apart you would use a pattern like 1 0 0

# Constructors

## PatternBrush(Color, Color, bool[,])

Initializes a new instance of the PatternBrush class.

```
public PatternBrush(Color foreColor, Color backColor, bool[,] pattern)
```

### Parameters

`foreColor` Color

The color used where the pattern flag is true.

`backColor` Color

The color used where the pattern flag is false.

`pattern` bool[,]

The pattern flags, indexed [row, column].

## Properties

### Pattern

Gets the pattern color matrix.

```
public DenseMatrix<Color> Pattern { get; }
```

### Property Value

DenseMatrix<Color>

## Methods

### CreateRenderer<TPixel>(Configuration, GraphicsOptions, int, RectangleF)

Creates the prepared execution object for this brush.

```
public override BrushRenderer<TPixel> CreateRenderer<TPixel>(Configuration configuration,
GraphicsOptions options, int canvasWidth, RectangleF region) where TPixel :
```

```
unmanaged, IPixel<TPixel>
```

## Parameters

`configuration` Configuration

The configuration instance to use when performing operations.

`options` GraphicsOptions

The graphic options.

`canvasWidth` int

The canvas width for the current render pass.

`region` RectangleF

The region the brush will be applied to.

## Returns

BrushRenderer<TPixel>

The BrushRenderer<TPixel> for this brush.

## Type Parameters

`TPixel`

The pixel type.

## Remarks

The `region` when being applied to things like shapes would usually be the bounding box of the shape not necessarily the bounds of the whole image.

# Equals(Brush?)

Indicates whether the current object is equal to another object of the same type.

```
public override bool Equals(Brush? other)
```

## Parameters

`other` Brush

An object to compare with this object.

## Returns

bool

true if the current object is equal to the `other` parameter; otherwise, false.

# GetHashCode()

Serves as the default hash function.

```
public override int GetHashCode()
```

## Returns

int

A hash code for the current object.

# Class PatternPen

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Defines a pen that can apply a pattern to a line with a set brush and thickness.

```
public class PatternPen : Pen, IEquatable<Pen>
```

**Inheritance**

object ← Pen ← PatternPen

**Implements**

IEquatable<Pen>

**Inherited Members**

Pen.StrokeFill , Pen.StrokeWidth , Pen.StrokePattern , Pen.StrokePatternOffset , Pen.StrokeOptions , Pen.GeneratePath(IPath) , Pen.Equals(object) , Pen.GetHashCode() , object.Equals(object, object) , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

# Remarks

The pattern is expressed in the form

```
[1f, 2f, 0.5f]
```

this will be converted into a pattern that is 3.5 times longer than the width with 3 sections.

- Section 1 will be width long (making a square) and will be filled by the brush.
- Section 2 will be width * 2 long and will be empty.
- Section 3 will be width/2 long and will be filled.

The pattern will immediately repeat without gap.

# Constructors

# PatternPen(Color, float, float[])

Initializes a new instance of the PatternPen class.

```
public PatternPen(Color color, float strokeWidth, float[] strokePattern)
```

## Parameters

`color` Color

The color.

`strokeWidth` float

The stroke width in the path's local coordinate space before any drawing transform is applied.

`strokePattern` float[]

The stroke pattern.

# PatternPen(Color, float, float[], float)

Initializes a new instance of the PatternPen class.

```
public PatternPen(Color color, float strokeWidth, float[] strokePattern,
float strokePatternOffset)
```

## Parameters

`color` Color

The color.

`strokeWidth` float

The stroke width in the path's local coordinate space before any drawing transform is applied.

`strokePattern` float[]

The stroke pattern.

`strokePatternOffset` float

The distance into the stroke pattern, expressed as a multiple of `strokeWidth`.

## PatternPen(Color, float[])

Initializes a new instance of the PatternPen class.

```
public PatternPen(Color color, float[] strokePattern)
```

### Parameters

`color` Color

The color.

`strokePattern` float[]

The stroke pattern.

## PatternPen(Color, float[], float)

Initializes a new instance of the PatternPen class.

```
public PatternPen(Color color, float[] strokePattern, float strokePatternOffset)
```

### Parameters

`color` Color

The color.

`strokePattern` float[]

The stroke pattern.

`strokePatternOffset` float

The distance into the stroke pattern, expressed as a multiple of the stroke width.

# PatternPen(Brush, float, float[])

Initializes a new instance of the PatternPen class.

```
public PatternPen(Brush strokeFill, float strokeWidth, float[] strokePattern)
```

## Parameters

`strokeFill` Brush

The brush used to fill the stroke outline.

`strokeWidth` float

The stroke width in the path's local coordinate space before any drawing transform is applied.

`strokePattern` float[]

The stroke pattern.

# PatternPen(Brush, float, float[], float)

Initializes a new instance of the PatternPen class.

```
public PatternPen(Brush strokeFill, float strokeWidth, float[] strokePattern,
float strokePatternOffset)
```

## Parameters

`strokeFill` Brush

The brush used to fill the stroke outline.

`strokeWidth` float

The stroke width in the path's local coordinate space before any drawing transform is applied.

`strokePattern` float[]

The stroke pattern.

`strokePatternOffset` float

The distance into the stroke pattern, expressed as a multiple of `strokeWidth`.

## PatternPen(PenOptions)

Initializes a new instance of the PatternPen class.

```
public PatternPen(PenOptions options)
```

### Parameters

`options` PenOptions

The pen options.

# Methods

## Equals(Pen?)

Indicates whether the current object is equal to another object of the same type.

```
public override bool Equals(Pen? other)
```

### Parameters

`other` Pen

An object to compare with this object.

### Returns

bool

true if the current object is equal to the `other` parameter; otherwise, false.

# GeneratePath(IPath, float)

Applies the styling from the pen to a path and generates a new path with the final vector.

```
public override IPath GeneratePath(IPath path, float strokeWidth)
```

## Parameters

`path` IPath

The source path.

`strokeWidth` float

The stroke width in the path's local coordinate space before any drawing transform is applied.

## Returns

IPath

The IPath with the pen styling applied.

# Class Pen

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

The base class for pens that can apply a pattern to a line with a set brush and thickness

```
public abstract class Pen : IEquatable<Pen>
```

**Inheritance**
object ← Pen

**Implements**
IEquatable<Pen>

**Derived**
PatternPen, SolidPen

**Inherited Members**
object.Equals(object, object) , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

# Remarks

The pattern is expressed in the form

```
[1f, 2f, 0.5f]
```

this will be converted into a pattern that is 3.5 times longer than the width with 3 sections.

- Section 1 will be width long (making a square) and will be filled by the brush.
- Section 2 will be width * 2 long and will be empty.
- Section 3 will be width/2 long and will be filled.

The pattern will immediately repeat without gap.

# Constructors

## Pen(Brush)

Initializes a new instance of the Pen class.

```
protected Pen(Brush strokeFill)
```

### Parameters

`strokeFill` Brush

The brush used to fill the stroke outline.

## Pen(Brush, float)

Initializes a new instance of the Pen class.

```
protected Pen(Brush strokeFill, float strokeWidth)
```

### Parameters

`strokeFill` Brush

The brush used to fill the stroke outline.

`strokeWidth` float

The stroke width in the path's local coordinate space before any drawing transform is applied.

## Pen(Brush, float, float[])

Initializes a new instance of the Pen class.

```
protected Pen(Brush strokeFill, float strokeWidth, float[] strokePattern)
```

### Parameters

`strokeFill` Brush

The brush used to fill the stroke outline.

`strokeWidth` float

The stroke width in the path's local coordinate space before any drawing transform is applied.

`strokePattern` float[]

The stroke pattern.

## Pen(PenOptions)

Initializes a new instance of the Pen class.

```
protected Pen(PenOptions options)
```

### Parameters

`options` PenOptions

The pen options.

# Properties

## StrokeFill

Gets the brush used to fill the stroke outline. Defaults to SolidBrush.

```
public Brush StrokeFill { get; }
```

### Property Value

Brush

## StrokeOptions

Gets or sets the stroke geometry options used to stroke paths drawn with this pen.

```
public StrokeOptions StrokeOptions { get; }
```

### Property Value

StrokeOptions

## StrokePattern

Gets the stroke pattern: alternating filled and empty segment lengths expressed as multiples of StrokeWidth, starting with a filled segment. An empty pattern produces a continuous stroke.

```
public ReadOnlyMemory<float> StrokePattern { get; }
```

### Property Value

ReadOnlyMemory<float>

## StrokePatternOffset

Gets or sets the distance into the stroke pattern, expressed as a multiple of StrokeWidth.

```
public float StrokePatternOffset { get; }
```

### Property Value

float

## StrokeWidth

Gets the stroke width in the path's local coordinate space before any drawing transform is applied. Defaults to 1.

```
public float StrokeWidth { get; }
```

### Property Value

float

# Methods

## Equals(Pen?)

Indicates whether the current object is equal to another object of the same type.

```
public virtual bool Equals(Pen? other)
```

### Parameters

`other` Pen

An object to compare with this object.

### Returns

bool

true if the current object is equal to the `other` parameter; otherwise, false.

## Equals(object?)

Determines whether the specified object is equal to the current object.

```
public override bool Equals(object? obj)
```

### Parameters

`obj` object

The object to compare with the current object.

### Returns

bool

true if the specified object is equal to the current object; otherwise, false.

## GeneratePath(IPath)

Applies the styling from the pen to a path and generates a new path with the final vector.

```
public IPath GeneratePath(IPath path)
```

### Parameters

`path` IPath

The source path.

### Returns

IPath

The IPath with the pen styling applied.

## GeneratePath(IPath, float)

Applies the styling from the pen to a path and generates a new path with the final vector.

```
public abstract IPath GeneratePath(IPath path, float strokeWidth)
```

### Parameters

`path` IPath

The source path.

`strokeWidth` float

The stroke width in the path's local coordinate space before any drawing transform is applied.

### Returns

IPath

The IPath with the pen styling applied.

## GetHashCode()

Serves as the default hash function.

```
public override int GetHashCode()
```

## Returns

int

A hash code for the current object.

# Struct PenOptions

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Provides a set of configurations options for pens.

```
public struct PenOptions
```

**Inherited Members**
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

# Constructors

## PenOptions(Color, float)

Initializes a new instance of the PenOptions struct.

```
public PenOptions(Color color, float strokeWidth)
```

### Parameters

`color` Color

The color.

`strokeWidth` float

The stroke width in the path's local coordinate space before any drawing transform is applied.

## PenOptions(Color, float, float[]?)

Initializes a new instance of the PenOptions struct.

```
public PenOptions(Color color, float strokeWidth, float[]? strokePattern)
```

## Parameters

`color` Color

The color.

`strokeWidth` float

The stroke width in the path's local coordinate space before any drawing transform is applied.

`strokePattern` float[]

The stroke pattern.

# PenOptions(Color, float, float[], float)

Initializes a new instance of the PenOptions struct.

```
public PenOptions(Color color, float strokeWidth, float[] strokePattern,
float strokePatternOffset)
```

## Parameters

`color` Color

The color.

`strokeWidth` float

The stroke width in the path's local coordinate space before any drawing transform is applied.

`strokePattern` float[]

The stroke pattern.

`strokePatternOffset` float

The distance into the stroke pattern, expressed as a multiple of `strokeWidth`.

# PenOptions(Brush, float, float[]?)

Initializes a new instance of the PenOptions struct.

```
public PenOptions(Brush strokeFill, float strokeWidth, float[]? strokePattern)
```

## Parameters

`strokeFill` Brush

The brush used to fill the stroke outline.

`strokeWidth` float

The stroke width in the path's local coordinate space before any drawing transform is applied.

`strokePattern` float[]

The stroke pattern.

# PenOptions(Brush, float, float[], float)

Initializes a new instance of the PenOptions struct.

```
public PenOptions(Brush strokeFill, float strokeWidth, float[] strokePattern,
float strokePatternOffset)
```

## Parameters

`strokeFill` Brush

The brush used to fill the stroke outline.

`strokeWidth` float

The stroke width in the path's local coordinate space before any drawing transform is applied.

`strokePattern` float[]

The stroke pattern.

`strokePatternOffset` float

The distance into the stroke pattern, expressed as a multiple of `strokeWidth`.

## PenOptions(float)

Initializes a new instance of the PenOptions struct.

```
public PenOptions(float strokeWidth)
```

### Parameters

`strokeWidth` float

The stroke width in the path's local coordinate space before any drawing transform is applied.

# Properties

## StrokeFill

Gets the brush used to fill the stroke outline. Defaults to SolidBrush.

```
public readonly Brush StrokeFill { get; }
```

### Property Value

Brush

## StrokeOptions

Gets or sets the stroke geometry options used to stroke paths drawn with this pen.

```
public StrokeOptions? StrokeOptions { readonly get; set; }
```

### Property Value

StrokeOptions

## StrokePattern

Gets the stroke pattern: alternating filled and empty segment lengths expressed as multiples of StrokeWidth, starting with a filled segment. An empty pattern produces a continuous stroke.

```
public readonly float[] StrokePattern { get; }
```

### Property Value

float[]

## StrokePatternOffset

Gets or sets the distance into the stroke pattern, expressed as a multiple of StrokeWidth.

```
public float StrokePatternOffset { readonly get; set; }
```

### Property Value

float

## StrokeWidth

Gets the stroke width in the path's local coordinate space before any drawing transform is applied. Defaults to 1.

```
public readonly float StrokeWidth { get; }
```

### Property Value

float

# Class Pens

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Contains a collection of common pen styles.

```
public static class Pens
```

**Inheritance**

object ← Pens

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

# Methods

## Dash(Color, float)

Creates a pen with a 'Dash' drawing pattern.

```
public static PatternPen Dash(Color color, float width)
```

## Parameters

`color` Color

The color.

`width` float

The width.

## Returns

PatternPen

The Pen.

## Dash(Brush, float)

Creates a pen with a 'Dash' drawing pattern.

```
public static PatternPen Dash(Brush brush, float width)
```

### Parameters

`brush` Brush

The brush.

`width` float

The width.

### Returns

PatternPen

The Pen.

## DashDot(Color, float)

Creates a pen with a 'Dash Dot' drawing pattern.

```
public static PatternPen DashDot(Color color, float width)
```

### Parameters

`color` Color

The color.

`width` float

The width.

## Returns

PatternPen

The Pen.

# DashDot(Brush, float)

Creates a pen with a 'Dash Dot' drawing pattern.

```
public static PatternPen DashDot(Brush brush, float width)
```

## Parameters

`brush` Brush

The brush.

`width` float

The width.

## Returns

PatternPen

The Pen.

# DashDotDot(Color, float)

Creates a pen with a 'Dash Dot Dot' drawing pattern.

```
public static PatternPen DashDotDot(Color color, float width)
```

## Parameters

`color` Color

The color.

`width` float

The width.

## Returns

PatternPen

The Pen.

# DashDotDot(Brush, float)

Creates a pen with a 'Dash Dot Dot' drawing pattern.

```
public static PatternPen DashDotDot(Brush brush, float width)
```

## Parameters

`brush` Brush

The brush.

`width` float

The width.

## Returns

PatternPen

The Pen.

# Dot(Color, float)

Creates a pen with a 'Dot' drawing pattern.

```
public static PatternPen Dot(Color color, float width)
```

## Parameters

`color` Color

The color.

`width` float

The width.

## Returns

PatternPen

The Pen.

# Dot(Brush, float)

Creates a pen with a 'Dot' drawing pattern.

```
public static PatternPen Dot(Brush brush, float width)
```

## Parameters

`brush` Brush

The brush.

`width` float

The width.

## Returns

PatternPen

The Pen.

# Solid(Color)

Creates a solid pen without any drawing pattern.

```
public static SolidPen Solid(Color color)
```

## Parameters

`color` Color

The color.

## Returns

SolidPen

The Pen.

# Solid(Color, float)

Creates a solid pen without any drawing pattern.

```
public static SolidPen Solid(Color color, float width)
```

## Parameters

`color` Color

The color.

`width` float

The width.

## Returns

SolidPen

The Pen.

# Solid(Brush)

Creates a solid pen without any drawing pattern.

```
public static SolidPen Solid(Brush brush)
```

## Parameters

`brush` Brush

The brush.

## Returns

SolidPen

The Pen.

# Solid(Brush, float)

Creates a solid pen without any drawing pattern.

```
public static SolidPen Solid(Brush brush, float width)
```

## Parameters

`brush` Brush

The brush.

`width` float

The width.

## Returns

SolidPen

The Pen.

# Class RadialGradientBrush

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

A radial gradient brush defined by either one circle or two circles. When one circle is provided, the gradient parameter is the distance from the center divided by the radius. When two circles are provided, the gradient parameter is computed along the family of circles interpolating between the start and end circles.

```
public sealed class RadialGradientBrush : GradientBrush, IEquatable<Brush>
```

### Inheritance

object ← Brush ← GradientBrush ← RadialGradientBrush

### Implements

IEquatable<Brush>

### Inherited Members

GradientBrush.RepetitionMode , GradientBrush.GradientTransform , GradientBrush.ColorStops , Brush.Equals(object) , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## RadialGradientBrush(PointF, float, GradientRepetitionMode, params ColorStop[])

Initializes a new instance of the RadialGradientBrush class using a single circle.

```
public RadialGradientBrush(PointF center, float radius, GradientRepetitionMode
repetitionMode, params ColorStop[] colorStops)
```

### Parameters

`center` PointF

The center of the circular gradient.

`radius` float

The radius of the circular gradient.

`repetitionMode` GradientRepetitionMode

Defines how the colors in the gradient are repeated.

`colorStops` ColorStop[]

The ordered gradient stops.

## RadialGradientBrush(PointF, float, GradientRepetitionMode, Matrix4×4, params ColorStop[])

Initializes a new instance of the RadialGradientBrush class using a single circle defined in the gradient's own coordinate space.

```
public RadialGradientBrush(PointF center, float radius, GradientRepetitionMode
repetitionMode, Matrix4x4 gradientTransform, params ColorStop[] colorStops)
```

### Parameters

`center` PointF

The center of the circular gradient.

`radius` float

The radius of the circular gradient.

`repetitionMode` GradientRepetitionMode

Defines how the colors in the gradient are repeated.

`gradientTransform` Matrix4×4

The transform from the gradient's coordinate space to the drawing.

`colorStops` ColorStop[]

The ordered gradient stops.

## RadialGradientBrush(PointF, float, PointF, float, GradientRepetitionMode, params ColorStop[])

Initializes a new instance of the RadialGradientBrush class using two circles.

```
public RadialGradientBrush(PointF startCenter, float startRadius, PointF endCenter, float
endRadius, GradientRepetitionMode repetitionMode, params ColorStop[] colorStops)
```

### Parameters

`startCenter` PointF

The center of the starting circle.

`startRadius` float

The radius of the starting circle.

`endCenter` PointF

The center of the ending circle.

`endRadius` float

The radius of the ending circle.

`repetitionMode` GradientRepetitionMode

Defines how the colors in the gradient are repeated.

`colorStops` ColorStop[]

The ordered gradient stops.

## RadialGradientBrush(PointF, float, PointF, float, GradientRepetitionMode, Matrix4×4, params ColorStop[])

Initializes a new instance of the RadialGradientBrush class using two circles defined in the gradient's own coordinate space.

```
public RadialGradientBrush(PointF startCenter, float startRadius, PointF endCenter, float
endRadius, GradientRepetitionMode repetitionMode, Matrix4x4 gradientTransform, params
ColorStop[] colorStops)
```

## Parameters

`startCenter` PointF

The center of the starting circle.

`startRadius` float

The radius of the starting circle.

`endCenter` PointF

The center of the ending circle.

`endRadius` float

The radius of the ending circle.

`repetitionMode` GradientRepetitionMode

Defines how the colors in the gradient are repeated.

`gradientTransform` Matrix4×4

The transform from the gradient's coordinate space to the drawing.

`colorStops` ColorStop[]

The ordered gradient stops.

# Properties

## Center0

Gets the center of the starting circle.

```
public PointF Center0 { get; }
```

## Property Value

PointF

# Center1

Gets the center of the ending circle, or null for single-circle form.

```
public PointF? Center1 { get; }
```

## Property Value

PointF?

# IsTwoCircle

Gets a value indicating whether this is a two-circle radial gradient.

```
public bool IsTwoCircle { get; }
```

## Property Value

bool

# Radius0

Gets the radius of the starting circle.

```
public float Radius0 { get; }
```

## Property Value

float

## Radius1

Gets the radius of the ending circle, or null for single-circle form.

```
public float? Radius1 { get; }
```

### Property Value

float?

# Methods

## CreateRenderer<TPixel>(Configuration, GraphicsOptions, int, RectangleF)

Creates the prepared execution object for this brush.

```
public override BrushRenderer<TPixel> CreateRenderer<TPixel>(Configuration configuration,
GraphicsOptions options, int canvasWidth, RectangleF region) where TPixel :
unmanaged, IPixel<TPixel>
```

### Parameters

`configuration` Configuration

The configuration instance to use when performing operations.

`options` GraphicsOptions

The graphic options.

`canvasWidth` int

The canvas width for the current render pass.

`region` RectangleF

The region the brush will be applied to.

### Returns

BrushRenderer<TPixel>

The BrushRenderer<TPixel> for this brush.

### Type Parameters

`TPixel`

The pixel type.

### Remarks

The `region` when being applied to things like shapes would usually be the bounding box of the shape not necessarily the bounds of the whole image.

## Equals(Brush?)

Indicates whether the current object is equal to another object of the same type.

```
public override bool Equals(Brush? other)
```

### Parameters

`other` Brush

An object to compare with this object.

### Returns

bool

true if the current object is equal to the `other` parameter; otherwise, false.

## GetHashCode()

Serves as the default hash function.

```
public override int GetHashCode()
```

## Returns

int

A hash code for the current object.

# Transform(Matrix4×4, Rectangle, Rectangle)

Returns a new brush with its defining geometry transformed by the given matrix.

```
public override Brush Transform(Matrix4x4 matrix, Rectangle sourceInterest,
Rectangle preparedInterest)
```

## Parameters

`matrix` Matrix4×4

The transformation matrix to apply.

`sourceInterest` Rectangle

The command interest before preparation.

`preparedInterest` Rectangle

The command interest after preparation.

## Returns

Brush

A transformed brush, or `this` if the brush has no spatial parameters.

# Class RasterizerDefaultsExtensions

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Adds extensions that allow configuring the drawing backend implementation.

```
public static class RasterizerDefaultsExtensions
```

**Inheritance**

object ← RasterizerDefaultsExtensions

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

# Methods

## SetDrawingBackend(Configuration, IDrawingBackend)

Sets the default drawing backend against the configuration.

```
public static void SetDrawingBackend(this Configuration configuration,
IDrawingBackend backend)
```

## Parameters

`configuration` Configuration

The configuration to store the backend against.

`backend` IDrawingBackend

The backend to use.

# Class RecolorBrush

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Provides an implementation of a brush that can recolor an image. Pixels within Threshold of SourceColor are blended towards TargetColor, with the blend strength falling off as the color distance approaches the threshold.

```
public sealed class RecolorBrush : Brush, IEquatable<Brush>
```

### Inheritance

object ← Brush ← RecolorBrush

### Implements

IEquatable<Brush>

### Inherited Members

Brush.Transform(Matrix4×4, Rectangle, Rectangle) , Brush.Equals(object) , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## RecolorBrush(Color, Color, float)

Initializes a new instance of the RecolorBrush class.

```
public RecolorBrush(Color sourceColor, Color targetColor, float threshold)
```

## Parameters

`sourceColor` Color

The color to match against existing pixels.

`targetColor` Color

The color to recolor matched pixels with.

`threshold` float

The color-matching threshold as a value between 0 and 1.

# Properties

## SourceColor

Gets the color to match against existing pixels.

```
public Color SourceColor { get; }
```

### Property Value

Color

## TargetColor

Gets the color to recolor matched pixels with.

```
public Color TargetColor { get; }
```

### Property Value

Color

## Threshold

Gets the color-matching threshold as a value between 0 and 1.

```
public float Threshold { get; }
```

### Property Value

float

# Methods

## CreateRenderer<TPixel>(Configuration, GraphicsOptions, int, RectangleF)

Creates the prepared execution object for this brush.

```
public override BrushRenderer<TPixel> CreateRenderer<TPixel>(Configuration configuration,
GraphicsOptions options, int canvasWidth, RectangleF region) where TPixel :
unmanaged, IPixel<TPixel>
```

### Parameters

`configuration` Configuration

The configuration instance to use when performing operations.

`options` GraphicsOptions

The graphic options.

`canvasWidth` int

The canvas width for the current render pass.

`region` RectangleF

The region the brush will be applied to.

### Returns

BrushRenderer<TPixel>

The BrushRenderer<TPixel> for this brush.

### Type Parameters

`TPixel`

The pixel type.

### Remarks

The `region` when being applied to things like shapes would usually be the bounding box of the shape not necessarily the bounds of the whole image.

## Equals(Brush?)

Indicates whether the current object is equal to another object of the same type.

```
public override bool Equals(Brush? other)
```

### Parameters

`other` Brush

An object to compare with this object.

### Returns

bool

true if the current object is equal to the `other` parameter; otherwise, false.

## GetHashCode()

Serves as the default hash function.

```
public override int GetHashCode()
```

### Returns

int

A hash code for the current object.

# Class RichGlyphOptions

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Provides GlyphOptions for rendering a glyph by id with per-glyph Brush and Pen support. The paint is carried to the renderer via a RichTextRun, so it can vary per glyph across a run.

```
public class RichGlyphOptions : GlyphOptions
```

**Inheritance**

object ← GlyphOptions ← RichGlyphOptions

**Inherited Members**

GlyphOptions.Font , GlyphOptions.Dpi , GlyphOptions.HintingMode , GlyphOptions.LayoutMode , GlyphOptions.ColorFontSupport , GlyphOptions.FontPalette , GlyphOptions.Origin , GlyphOptions.VisibleBounds , GlyphOptions.TextBaseline , GlyphOptions.BaselineOffset , GlyphOptions.GraphemeIndex , GlyphOptions.TextAttributes , GlyphOptions.TextDecorations , GlyphOptions.DecorationPositioningMode , GlyphOptions.TextDecorationSkipInk , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

# Properties

## Brush

Gets or sets the brush used for filling this run.

```
public Brush? Brush { get; set; }
```

## Property Value

Brush

## OverlinePen

Gets or sets the pen used for drawing overline features for this run.

```
public Pen? OverlinePen { get; set; }
```

### Property Value

Pen

## Pen

Gets or sets the pen used for outlining this run.

```
public Pen? Pen { get; set; }
```

### Property Value

Pen

## StrikeoutPen

Gets or sets the pen used for drawing strikeout features for this run.

```
public Pen? StrikeoutPen { get; set; }
```

### Property Value

Pen

## UnderlinePen

Gets or sets the pen used for drawing underline features for this run.

```
public Pen? UnderlinePen { get; set; }
```

### Property Value

Pen

# Methods

## CreateTextRun()

Creates the text run associated with the glyph.

```
protected override TextRun CreateTextRun()
```

### Returns

TextRun

The text run associated with the glyph.

# Class RichTextOptions

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Provides configuration options for rendering and shaping of rich text.

```
public class RichTextOptions : TextOptions
```

### Inheritance

object ← TextOptions ← RichTextOptions

### Inherited Members

TextOptions.Font , TextOptions.FontWeight , TextOptions.FallbackFontFamilies , TextOptions.FontFallbackResolver , TextOptions.Dpi , TextOptions.TabWidth , TextOptions.HintingMode , TextOptions.LineSpacing , TextOptions.Origin , TextOptions.WrappingLength , TextOptions.VisibleBounds , TextOptions.TextBaseline , TextOptions.BaselineOffset , TextOptions.MaxLines , TextOptions.WordBreaking , TextOptions.TextEllipsis , TextOptions.CustomEllipsis , TextOptions.TextHyphenation , TextOptions.CustomHyphen , TextOptions.TextDirection , TextOptions.TextBidiMode , TextOptions.Script , TextOptions.TextInteractionMode , TextOptions.TextAlignment , TextOptions.TextJustification , TextOptions.HorizontalAlignment , TextOptions.VerticalAlignment , TextOptions.LayoutMode , TextOptions.KerningMode , TextOptions.Tracking , TextOptions.DecorationPositioningMode , TextOptions.TextDecorationSkipInk , TextOptions.ColorFontSupport , TextOptions.FontPalette , TextOptions.FeatureTags , TextOptions.Culture , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## RichTextOptions(Font)

Initializes a new instance of the RichTextOptions class.

```
public RichTextOptions(Font font)
```

### Parameters

`font` Font

The font.

## RichTextOptions(RichTextOptions)

Initializes a new instance of the RichTextOptions class from properties copied from the given instance.

```
public RichTextOptions(RichTextOptions options)
```

### Parameters

`options` RichTextOptions

The options whose properties are copied into this instance.

# Properties

## TextRuns

Gets or sets an optional collection of text runs to apply to the body of text.

```
public IReadOnlyList<RichTextRun> TextRuns { get; set; }
```

### Property Value

IReadOnlyList<RichTextRun>

# Class RichTextRun

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Represents a run of drawable text spanning a series of graphemes within a string.

```
public class RichTextRun : TextRun
```

### Inheritance

object ← TextRun ← RichTextRun

### Inherited Members

TextRun.Slice(ReadOnlySpan<char>) , TextRun.ToString() , TextRun.Start , TextRun.End , TextRun.Font , TextRun.FontWeight , TextRun.Script , TextRun.Culture , TextRun.FeatureTags , TextRun.ColorFontSupport , TextRun.FontPalette , TextRun.TextAttributes , TextRun.TextDecorations , TextRun.Placeholder , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object)

# Properties

## Brush

Gets or sets the brush used for filling this run.

```
public Brush? Brush { get; set; }
```

### Property Value

Brush

## OverlinePen

Gets or sets the pen used for drawing overline features for this run.

```
public Pen? OverlinePen { get; set; }
```

## Property Value

Pen

# Pen

Gets or sets the pen used for outlining this run.

```
public Pen? Pen { get; set; }
```

## Property Value

Pen

# StrikeoutPen

Gets or sets the pen used for drawing strikeout features for this run.

```
public Pen? StrikeoutPen { get; set; }
```

## Property Value

Pen

# UnderlinePen

Gets or sets the pen used for drawing underline features for this run.

```
public Pen? UnderlinePen { get; set; }
```

## Property Value

Pen

# Methods

## GetDecorationOptions(TextDecorations)

Gets the geometry overrides to apply to the specified decoration, or null to use the values derived from the font's metrics.

```
public override TextDecorationOptions? GetDecorationOptions(TextDecorations decoration)
```

### Parameters

`decoration` TextDecorations

The decoration to retrieve overrides for.

### Returns

TextDecorationOptions

The TextDecorationOptions for the decoration, or null.

### Remarks

The base implementation returns null for every decoration. Derived runs can override this to supply values sourced from styling that the font metrics do not describe, such as an explicit stroke width.

# Class SolidBrush

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Provides an implementation of a solid brush for painting solid color areas.

```
public sealed class SolidBrush : Brush, IEquatable<Brush>
```

### Inheritance

object ← Brush ← SolidBrush

### Implements

IEquatable<Brush>

### Inherited Members

Brush.Transform(Matrix4×4, Rectangle, Rectangle) , Brush.Equals(object) , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## SolidBrush(Color)

Initializes a new instance of the SolidBrush class.

```
public SolidBrush(Color color)
```

### Parameters

`color` Color

The color.

# Properties

## Color

Gets the color.

```
public Color Color { get; }
```

### Property Value

Color

# Methods

## CreateRenderer<TPixel>(Configuration, GraphicsOptions, int, RectangleF)

Creates the prepared execution object for this brush.

```
public override BrushRenderer<TPixel> CreateRenderer<TPixel>(Configuration configuration,
GraphicsOptions options, int canvasWidth, RectangleF region) where TPixel :
unmanaged, IPixel<TPixel>
```

### Parameters

`configuration` Configuration

The configuration instance to use when performing operations.

`options` GraphicsOptions

The graphic options.

`canvasWidth` int

The canvas width for the current render pass.

`region` RectangleF

The region the brush will be applied to.

### Returns

BrushRenderer<TPixel>

The BrushRenderer<TPixel> for this brush.

### Type Parameters

`TPixel`

The pixel type.

### Remarks

The `region` when being applied to things like shapes would usually be the bounding box of the shape not necessarily the bounds of the whole image.

## Equals(Brush?)

Indicates whether the current object is equal to another object of the same type.

```
public override bool Equals(Brush? other)
```

### Parameters

`other` Brush

An object to compare with this object.

### Returns

bool

true if the current object is equal to the `other` parameter; otherwise, false.

## GetHashCode()

Serves as the default hash function.

```
public override int GetHashCode()
```

## Returns

int

A hash code for the current object.

# Class SolidPen

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Defines a pen that draws a continuous, unpatterned stroke with a set brush and thickness.

```
public class SolidPen : Pen, IEquatable<Pen>
```

**Inheritance**

object ← Pen ← SolidPen

**Implements**

IEquatable<Pen>

**Inherited Members**

Pen.StrokeFill , Pen.StrokeWidth , Pen.StrokePattern , Pen.StrokePatternOffset , Pen.StrokeOptions , Pen.GeneratePath(IPath) , Pen.Equals(object) , Pen.GetHashCode() , object.Equals(object, object) , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## SolidPen(Color)

Initializes a new instance of the SolidPen class.

```
public SolidPen(Color color)
```

### Parameters

`color` Color

The color.

## SolidPen(Color, float)

Initializes a new instance of the SolidPen class.

```
public SolidPen(Color color, float width)
```

## Parameters

`color` Color

The color.

`width` float

The width.

# SolidPen(Brush)

Initializes a new instance of the SolidPen class.

```
public SolidPen(Brush strokeFill)
```

## Parameters

`strokeFill` Brush

The brush used to fill the stroke outline.

# SolidPen(Brush, float)

Initializes a new instance of the SolidPen class.

```
public SolidPen(Brush strokeFill, float strokeWidth)
```

## Parameters

`strokeFill` Brush

The brush used to fill the stroke outline.

`strokeWidth` float

The stroke width in the path's local coordinate space before any drawing transform is applied.

## SolidPen(PenOptions)

Initializes a new instance of the SolidPen class.

```
public SolidPen(PenOptions options)
```

### Parameters

`options` PenOptions

The pen options.

# Methods

## Equals(Pen?)

Indicates whether the current object is equal to another object of the same type.

```
public override bool Equals(Pen? other)
```

### Parameters

`other` Pen

An object to compare with this object.

### Returns

bool

true if the current object is equal to the `other` parameter; otherwise, false.

# GeneratePath(IPath, float)

Applies the styling from the pen to a path and generates a new path with the final vector.

```
public override IPath GeneratePath(IPath path, float strokeWidth)
```

## Parameters

`path` IPath

The source path.

`strokeWidth` float

The stroke width in the path's local coordinate space before any drawing transform is applied.

## Returns

IPath

The IPath with the pen styling applied.

# Class StrokeOptions

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Provides configuration options for geometric stroke generation.

```
public sealed class StrokeOptions : IEquatable<StrokeOptions?>
```

**Inheritance**

object ← StrokeOptions

**Implements**

IEquatable<StrokeOptions>

**Inherited Members**

object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Properties

## ArcDetailScale

Gets or sets the tessellation detail scale for round joins and round caps. Higher values produce more vertices (smoother curves, more work). Lower values produce fewer vertices.

```
public double ArcDetailScale { get; set; }
```

## Property Value

double

## Exceptions

ArgumentOutOfRangeException

The value is not greater than zero.

## LineCap

Gets or sets the line cap style used for open path ends.

```
public LineCap LineCap { get; set; }
```

### Property Value

LineCap

## LineJoin

Gets or sets the outer line join style used for stroking corners.

```
public LineJoin LineJoin { get; set; }
```

### Property Value

LineJoin

## MiterLimit

Gets or sets the miter limit used to clamp outer miter joins.

```
public double MiterLimit { get; set; }
```

### Property Value

double

# Methods

## Equals(StrokeOptions?)

Indicates whether the current object is equal to another object of the same type.

```
public bool Equals(StrokeOptions? other)
```

## Parameters

`other` StrokeOptions

An object to compare with this object.

## Returns

bool

true if the current object is equal to the `other` parameter; otherwise, false.

# Equals(object?)

Determines whether the specified object is equal to the current object.

```
public override bool Equals(object? obj)
```

## Parameters

`obj` object

The object to compare with the current object.

## Returns

bool

true if the specified object is equal to the current object; otherwise, false.

# GetHashCode()

Serves as the default hash function.

```
public override int GetHashCode()
```

## Returns

int

A hash code for the current object.

# Class SweepGradientBrush

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Provides an implementation of a brush for painting sweep (conic) gradients within areas. Angles increase counter-clockwise from +X on the design grid.

```
public sealed class SweepGradientBrush : GradientBrush, IEquatable<Brush>
```

### Inheritance

object ← Brush ← GradientBrush ← SweepGradientBrush

### Implements

IEquatable<Brush>

### Inherited Members

GradientBrush.RepetitionMode , GradientBrush.GradientTransform , GradientBrush.ColorStops , Brush.Equals(object) , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## SweepGradientBrush(PointF, float, float, GradientRepetitionMode, params ColorStop[])

Initializes a new instance of the SweepGradientBrush class.

```
public SweepGradientBrush(PointF center, float startAngleDegrees, float endAngleDegrees,
GradientRepetitionMode repetitionMode, params ColorStop[] colorStops)
```

### Parameters

`center` PointF

The center point of the sweep gradient in device space.

`startAngleDegrees` float

The starting angle, in degrees, measured counter-clockwise from +X on the design grid. This value is stored as provided so the sign and magnitude of the sweep remain intact.

`endAngleDegrees` float

The ending angle, in degrees, measured counter-clockwise from +X on the design grid. If equal to `startAngleDegrees`, the gradient is treated as a full 360 degree sweep. Otherwise, the signed difference between start and end determines the sweep direction.

`repetitionMode` GradientRepetitionMode

Defines how the gradient colors are repeated beyond the interval [0..1].

`colorStops` ColorStop[]

The gradient color stops. Ratios must be in [0..1] and are interpreted along the angular sweep.

## SweepGradientBrush(PointF, float, float, GradientRepetitionMode, Matrix4×4, params ColorStop[])

Initializes a new instance of the SweepGradientBrush class with the center and angles defined in the gradient's own coordinate space.

```
public SweepGradientBrush(PointF center, float startAngleDegrees, float endAngleDegrees,
GradientRepetitionMode repetitionMode, Matrix4x4 gradientTransform, params
ColorStop[] colorStops)
```

## Parameters

`center` PointF

The center point of the sweep gradient.

`startAngleDegrees` float

The starting angle, in degrees, measured counter-clockwise from +X on the design grid.

`endAngleDegrees` float

The ending angle, in degrees, measured counter-clockwise from +X on the design grid.

`repetitionMode` GradientRepetitionMode

Defines how the gradient colors are repeated beyond the interval [0..1].

`gradientTransform` Matrix4×4

The transform from the gradient's coordinate space to the drawing.

`colorStops` ColorStop[]

The gradient color stops. Ratios must be in [0..1] and are interpreted along the angular sweep.

# Properties

## Center

Gets the center point of the sweep gradient.

```
public PointF Center { get; }
```

### Property Value

PointF

## EndAngleDegrees

Gets the ending angle in degrees.

```
public float EndAngleDegrees { get; }
```

### Property Value

float

## StartAngleDegrees

Gets the starting angle in degrees.

```
public float StartAngleDegrees { get; }
```

### Property Value

float

# Methods

## CreateRenderer<TPixel>(Configuration, GraphicsOptions, int, RectangleF)

Creates the prepared execution object for this brush.

```
public override BrushRenderer<TPixel> CreateRenderer<TPixel>(Configuration configuration,
GraphicsOptions options, int canvasWidth, RectangleF region) where TPixel :
unmanaged, IPixel<TPixel>
```

### Parameters

`configuration` Configuration

The configuration instance to use when performing operations.

`options` GraphicsOptions

The graphic options.

`canvasWidth` int

The canvas width for the current render pass.

`region` RectangleF

The region the brush will be applied to.

### Returns

BrushRenderer<TPixel>

The BrushRenderer<TPixel> for this brush.

### Type Parameters

`TPixel`

The pixel type.

### Remarks

The `region` when being applied to things like shapes would usually be the bounding box of the shape not necessarily the bounds of the whole image.

## Equals(Brush?)

Indicates whether the current object is equal to another object of the same type.

```
public override bool Equals(Brush? other)
```

### Parameters

`other` Brush

An object to compare with this object.

### Returns

bool

true if the current object is equal to the `other` parameter; otherwise, false.

## GetHashCode()

Serves as the default hash function.

```
public override int GetHashCode()
```

### Returns

int

A hash code for the current object.

## Transform(Matrix4×4, Rectangle, Rectangle)

Returns a new brush with its defining geometry transformed by the given matrix.

```
public override Brush Transform(Matrix4x4 matrix, Rectangle sourceInterest,
Rectangle preparedInterest)
```

### Parameters

`matrix` Matrix4×4

The transformation matrix to apply.

`sourceInterest` Rectangle

The command interest before preparation.

`preparedInterest` Rectangle

The command interest after preparation.

### Returns

Brush

A transformed brush, or `this` if the brush has no spatial parameters.

# Enum WrapMode

Namespace: SixLabors.ImageSharp.Drawing.Processing

Assembly: SixLabors.ImageSharp.Drawing.dll

Defines how an ImageBrush samples beyond its source region along a single axis.

```
public enum WrapMode
```

# Fields

`Clamp = 3`

The source region is not repeated; the nearest edge pixel is sampled for coordinates outside it.

`Mirror = 2`

The source region is tiled, mirroring every other repetition.

`None = 0`

The source region is not repeated; pixels outside it are left transparent.

`Repeat = 1`

The source region is tiled, repeating edge-to-edge.

# Namespace SixLabors.ImageSharp.Drawing.Processing.Backends

## Classes

CompositionSceneCommand
: Base type for one draw-order command in a flush-scoped scene stream.

DefaultDrawingBackend
: CPU backend that executes path coverage rasterization and brush composition directly against a CPU region.

DefaultDrawingBackendScene
: Retained scene created by the CPU drawing backend.

DrawingBackendScene
: Base type for a retained backend scene: the lowered, reusable form of a batch of drawing commands produced by CreateScene(Configuration, Rectangle, DrawingCommandBatch, IReadOnlyList<IDisposable>?) and later rendered into a target through RenderScene<TPixel>(Configuration, ICanvasFrame<TPixel>, DrawingBackendScene).

LineSegmentCompositionSceneCommand
: Scene command wrapper for explicit stroked line-segment commands.

MemoryCanvasFrame<TPixel>
: Canvas frame backed by a Buffer2DRegion<T>.

NativeCanvasFrame<TPixel>
: Canvas frame backed by a NativeSurface.

NativeSurface
: Opaque base type for a backend-specific native (non-CPU) drawing target, such as a GPU surface.

PathCompositionSceneCommand
: Scene command wrapper for fill-path and layer-based composition commands.

PolylineCompositionSceneCommand
: Scene command wrapper for explicit stroked polyline commands.

StrokePathCompositionSceneCommand
: Scene command wrapper for stroked path commands.

WebGPUBackdropAcrylicLayerEffect
Provides a shader-accelerated counterpart to BackdropAcrylicLayerEffect.

WebGPUBackdropColorMatrixLayerEffect
Provides a shader-accelerated counterpart to BackdropColorMatrixLayerEffect.

WebGPUBackdropDropShadowLayerEffect
Provides a shader-accelerated counterpart to BackdropDropShadowLayerEffect.

WebGPUBackdropGaussianBlurLayerEffect
Provides a shader-accelerated counterpart to BackdropBlurLayerEffect.

WebGPUBackdropShaderLayerEffect
Defines a backdrop effect with complete WGSL shader passes and an equivalent ImageSharp CPU fallback.

WebGPUColorMatrixLayerEffect
Provides a shader-accelerated counterpart to ColorMatrixLayerEffect.

WebGPUDeviceContext
Owns the WebGPU device-scoped drawing state shared by render targets, surfaces, shader programs, and pipelines.

WebGPUDrawingBackend
Pixel-format registration for composite session I/O.

WebGPUDrawingBackendScene
Retained scene created by the WebGPU drawing backend.

WebGPUDropShadowLayerEffect
Provides a shader-accelerated counterpart to DropShadowLayerEffect.

WebGPUEnvironment
Provides explicit support probes for the library-managed WebGPU environment. Use this type when you want to check availability or compute pipeline support before constructing WebGPU objects.

WebGPUEnvironmentOptions
Options for the library-managed WebGPU environment.

WebGPUExternalSurface
A WebGPU rendering surface bound to an externally-owned native host. Unlike WebGPUWindow, this type does not own a platform window; the host application owns the UI object, its lifecycle, and the drawable size notifications forwarded through Resize(Size).

WebGPUExternalSurfaceOptions
: Options for creating a WebGPUExternalSurface.

WebGPUGaussianBlurLayerEffect
: Provides a shader-accelerated counterpart to BlurLayerEffect.

WebGPUGlowLayerEffect
: Provides a shader-accelerated counterpart to GlowLayerEffect.

WebGPUInnerShadowLayerEffect
: Provides a shader-accelerated counterpart to InnerShadowLayerEffect.

WebGPURenderTarget
: An offscreen WebGPU render target.

WebGPUShaderCompilationException
: The exception thrown when a WebGPU layer-effect shader cannot be compiled.

WebGPUShaderLayerEffect
: Defines a layer effect with complete WGSL shader passes and an equivalent ImageSharp CPU fallback.

WebGPUShaderUniformBuilder
: Builds a set of named values for a WebGPUShaderUniformLayout.

WebGPUShaderUniformLayout
: Describes the names, types, and fixed array lengths of values supplied to a WebGPU layer-effect shader.

WebGPUShaderUniforms
: Contains an immutable snapshot of values supplied to a WebGPU layer-effect shader.

WebGPUSurfaceFrame
: A single acquired drawable frame returned by a WebGPU surface. Use the Canvas to draw the frame contents, then dispose the frame to show it on screen.

WebGPUSurfaceSession
: Provides shared WebGPU resources for a group of external presentation surfaces.

WebGPUWindow
: A self-contained WebGPU-backed window that owns the platform window, the WebGPU device and queue, the surface and swap chain, and the per-frame texture acquire/present cycle, exposing a DrawingCanvas for each frame. Use Run(Action<DrawingCanvas>) to let the

window drive rendering, or TryAcquireFrame(out WebGPUSurfaceFrame?) to drive the frame loop yourself.

WebGPUWindowOptions
: Options for creating a WebGPUWindow.

## Structs

CompositionCommand
: One normalized fill-path or layer-based composition command queued for backend execution.

DrawingCommandBatch
: One prepared draw-order command batch consumed by a drawing backend.

RasterizerOptions
: Immutable options used by rasterizers when scan-converting vector geometry.

StrokeLineSegmentCommand
: One explicit stroked two-point line-segment command queued by the canvas batcher.

StrokePathCommand
: One stroked path command queued by the canvas batcher.

StrokePolylineCommand
: One explicit stroked open polyline command queued by the canvas batcher.

WebGPUShaderDiagnostic
: Describes one message produced while compiling a WebGPU layer-effect shader.

WebGPUShaderUniform
: Describes one named value or fixed-size array supplied to a WebGPU layer-effect shader.

WebGPUSurfaceHost
: Describes the externally-owned native drawable that a WebGPUExternalSurface should attach to. Use the factory method that matches the host toolkit or platform that owns the drawable surface.

## Interfaces

ICanvasFrame<TPixel>
: A render destination for DrawingCanvas<TPixel>. Implementations back the canvas with either a CPU-accessible pixel region or an opaque native surface; a usable frame must expose at least one of the two.

ICompositionSceneCommandVisitor
: Visitor contract for one flush-scoped composition scene command.

IDrawingBackend
: Defines the contract a drawing backend implements to turn recorded canvas commands into pixels: creating retained scenes from command batches, rendering those scenes into a canvas frame, and transferring pixels between frames (copy and readback).

IWebGPUShaderEffect
: Identifies a layer effect that supplies a native WebGPU implementation and a CPU fallback.

## Enums

CompositionCommandKind
: Identifies the flush-time role carried by a CompositionCommand.

RasterizationMode
: Describes whether rasterizers should emit continuous coverage or binary aliased coverage.

WebGPUCompositeAlphaMode
: Specifies how the native compositor interprets alpha in a WebGPU surface.

WebGPUEnvironmentError
: Error codes returned by WebGPUEnvironment probe methods.

WebGPUErrorType
: Error categories reported by the native WebGPU runtime.

WebGPUPowerPreference
: Describes the adapter power preference requested for the library-managed WebGPU device.

WebGPUPresentMode
: Presentation mode used by WebGPUWindow.

WebGPUShaderDiagnosticSeverity
: Identifies the severity of a WebGPU shader compiler message.

WebGPUShaderUniformType
: Identifies a value type that can be supplied to a WebGPU layer-effect shader.

WebGPUTextureFormat
: Supported texture format identifiers for native WebGPU targets.

WebGPUWindowBorder
: Describes the window border around a WebGPUWindow.

## WebGPUWindowState

Describes the startup or current state of a WebGPUWindow.

# Struct CompositionCommand

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.dll

One normalized fill-path or layer-based composition command queued for backend execution.

```
public readonly struct CompositionCommand
```

**Inherited Members**
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

# Remarks

This type carries fill-path commands plus inline layer boundaries.

# Properties

## ApplyCanvasBounds

Gets the canvas bounds available to an Apply command.

```
public Rectangle ApplyCanvasBounds { get; }
```

## Property Value

Rectangle

## ApplyEffect

Gets the layer effect carried by an Apply command, or null when the command represents a direct Apply operation.

```
public LayerEffect? ApplyEffect { get; }
```

### Property Value

LayerEffect

## ApplyOperation

Gets the image processor carried by an Apply command.

```
public Action<IImageProcessingContext> ApplyOperation { get; }
```

### Property Value

Action<IImageProcessingContext>

## ApplyOutputBounds

Gets the local bounds within which an Apply command writes its processed output.

```
public RectangleF ApplyOutputBounds { get; }
```

### Property Value

RectangleF

## ApplyWriteBackOffset

Gets the offset subtracted from an Apply command's write rectangle when reading the source pixels, so a write-back recorded at an offset still reads the pre-offset region.

```
public Point ApplyWriteBackOffset { get; }
```

### Property Value

Point

## Brush

Gets the brush used during composition.

```
public Brush Brush { get; }
```

### Property Value

Brush

## ClipDescriptor

Gets the clip descriptor opened by a BeginClip command.

```
public DrawingClipDescriptor ClipDescriptor { get; }
```

### Property Value

DrawingClipDescriptor

### Remarks

The ordered begin/end-clip command stream is the single source of truth for clipping. Draw commands do not carry clip state; backends resolve the active clip stack from the stream commands surrounding each draw.

## DestinationOffset

Gets the absolute destination offset where the local coverage should be composited.

```
public Point DestinationOffset { get; }
```

### Property Value

Point

## DrawingOptions

Gets the drawing options carried by the command.

```
public DrawingOptions DrawingOptions { get; }
```

### Property Value

DrawingOptions

## IsInsideLayer

Gets a value indicating whether the command was recorded inside a layer.

```
public bool IsInsideLayer { get; }
```

### Property Value

bool

## Kind

Gets the command kind.

```
public CompositionCommandKind Kind { get; }
```

### Property Value

CompositionCommandKind

## LayerBounds

Gets the absolute bounds of the layer opened by this command.

```
public Rectangle LayerBounds { get; }
```

## Property Value

Rectangle

## Remarks

Only meaningful for BeginLayer and EndLayer.

# LayerOptions

Gets graphics options used by layer compositing commands.

```
public GraphicsOptions LayerOptions { get; }
```

## Property Value

GraphicsOptions

## Remarks

Only valid for commands that carry layer state; accessing it on other commands throws.

# RasterizerOptions

Gets rasterizer options used to generate coverage.

```
public RasterizerOptions RasterizerOptions { get; }
```

## Property Value

RasterizerOptions

## RequiresScopedApply

Gets a value indicating whether this layer command must be rendered as an isolated target for Apply.

```
public bool RequiresScopedApply { get; }
```

### Property Value

bool

## SourcePath

Gets the source path carried by the command.

```
public IPath SourcePath { get; }
```

### Property Value

IPath

## SubPixelOffset

Gets the fractional translation applied after the drawing options transform. Glyph geometry rides an integer destination offset; the sub-pixel remainder travels here so the queued command does not need per-operation drawing options to carry it.

```
public Vector2 SubPixelOffset { get; }
```

### Property Value

Vector2

## TargetBounds

Gets the absolute bounds of the logical target for this command.

```
public Rectangle TargetBounds { get; }
```

## Property Value

Rectangle

# Transform

Gets the command transform: the drawing options transform followed by the sub-pixel translation.

```
public Matrix4x4 Transform { get; }
```

## Property Value

Matrix4×4

# Enum CompositionCommandKind

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.dll

Identifies the flush-time role carried by a CompositionCommand.

```
public enum CompositionCommandKind : byte
```

# Fields

`Apply = 3`

Applies an image processor to the current target before later commands are rendered.

`BeginClip = 4`

Starts a clip scope.

`BeginLayer = 1`

Starts an isolated compositing layer.

`EndClip = 5`

Ends the most recently opened clip scope.

`EndLayer = 2`

Ends the most recently opened layer.

`FillLayer = 0`

A fill-path command.

# Class CompositionSceneCommand

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.dll

Base type for one draw-order command in a flush-scoped scene stream.

```
public abstract class CompositionSceneCommand
```

**Inheritance**

object ← CompositionSceneCommand

**Derived**

LineSegmentCompositionSceneCommand, PathCompositionSceneCommand, PolylineCompositionSceneCommand, StrokePathCompositionSceneCommand

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

# Methods

## Accept(ICompositionSceneCommandVisitor)

Dispatches the command to a visitor without a per-item kind switch at the call site.

```
public abstract void Accept(ICompositionSceneCommandVisitor visitor)
```

## Parameters

`visitor` ICompositionSceneCommandVisitor

The visitor receiving the command.

# Class DefaultDrawingBackend

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.dll

CPU backend that executes path coverage rasterization and brush composition directly against a CPU region.

```
public sealed class DefaultDrawingBackend : IDrawingBackend
```

### Inheritance

object ← DefaultDrawingBackend

### Implements

IDrawingBackend

### Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Properties

## Instance

Gets the default backend instance.

```
public static DefaultDrawingBackend Instance { get; }
```

## Property Value

DefaultDrawingBackend

# Methods

# ComposeLayer<TPixel>(Configuration, ICanvasFrame<TPixel>, ICanvasFrame<TPixel>, Point, GraphicsOptions)

Composites one CPU-backed frame onto another using the supplied graphics options.

```
public static void ComposeLayer<TPixel>(Configuration configuration, ICanvasFrame<TPixel>
source, ICanvasFrame<TPixel> destination, Point destinationOffset, GraphicsOptions options)
where TPixel : unmanaged, IPixel<TPixel>
```

## Parameters

`configuration` Configuration

The active processing configuration.

`source` ICanvasFrame<TPixel>

The source frame.

`destination` ICanvasFrame<TPixel>

The destination frame.

`destinationOffset` Point

The destination offset relative to `destination`.

`options` GraphicsOptions

The graphics options controlling composition.

## Type Parameters

`TPixel`

The pixel format.

# CopyPixels<TPixel>(Configuration, ICanvasFrame<TPixel>, ICanvasFrame<TPixel>, Rectangle, Point)

Copies pixels from a source frame into a target frame.

```
public void CopyPixels<TPixel>(Configuration configuration, ICanvasFrame<TPixel> source,
ICanvasFrame<TPixel> target, Rectangle sourceRectangle, Point targetPoint) where TPixel :
unmanaged, IPixel<TPixel>
```

## Parameters

`configuration` Configuration

The active processing configuration.

`source` ICanvasFrame<TPixel>

The source frame.

`target` ICanvasFrame<TPixel>

The target frame.

`sourceRectangle` Rectangle

The source rectangle in source-local coordinates.

`targetPoint` Point

The target point in target-local coordinates.

## Type Parameters

`TPixel`

The pixel format.

# CreateScene(Configuration, Rectangle, DrawingCommandBatch, IReadOnlyList<IDisposable>?)

Creates a backend scene from a prepared command batch.

```
public DrawingBackendScene CreateScene(Configuration configuration, Rectangle targetBounds,
DrawingCommandBatch commandBatch, IReadOnlyList<IDisposable>? ownedResources = null)
```

## Parameters

`configuration` Configuration

The active processing configuration.

`targetBounds` Rectangle

The target bounds used for target-dependent scene data.

`commandBatch` DrawingCommandBatch

The scene commands in submission order.

`ownedResources` IReadOnlyList<IDisposable>

The resources that must stay alive for the returned scene.

## Returns

DrawingBackendScene

The created backend scene.

# ReadRegion<TPixel>(Configuration, ICanvasFrame<TPixel>, Rectangle, Buffer2DRegion<TPixel>)

Reads source pixels from the target into the destination region.

```
public void ReadRegion<TPixel>(Configuration configuration, ICanvasFrame<TPixel>
target, Rectangle sourceRectangle, Buffer2DRegion<TPixel> destination) where TPixel :
unmanaged, IPixel<TPixel>
```

## Parameters

`configuration` Configuration

The active processing configuration.

`target` ICanvasFrame<TPixel>

The target frame.

`sourceRectangle` Rectangle

The source rectangle in target-local coordinates.

`destination` Buffer2DRegion<TPixel>

The destination region that receives the copied pixels.

## Type Parameters

`TPixel`

The pixel format.

# RenderScene<TPixel>(Configuration, ICanvasFrame<TPixel>, DrawingBackendScene)

Renders a backend scene into the target.

```
public void RenderScene<TPixel>(Configuration configuration, ICanvasFrame<TPixel> target,
DrawingBackendScene scene) where TPixel : unmanaged, IPixel<TPixel>
```

## Parameters

`configuration` Configuration

The active processing configuration.

`target` ICanvasFrame<TPixel>

The target frame.

`scene` DrawingBackendScene

The backend scene to render.

## Type Parameters

`TPixel`

The pixel format.

# Class DefaultDrawingBackendScene

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.dll

Retained scene created by the CPU drawing backend.

```
public sealed class DefaultDrawingBackendScene : DrawingBackendScene, IDisposable
```

**Inheritance**

object ← DrawingBackendScene ← DefaultDrawingBackendScene

**Implements**

IDisposable

**Inherited Members**

DrawingBackendScene.Bounds , DrawingBackendScene.Dispose() , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Methods

## DisposeCore()

Disposes backend-specific resources retained by this scene.

```
protected override void DisposeCore()
```

# Class DrawingBackendScene

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.dll

Base type for a retained backend scene: the lowered, reusable form of a batch of drawing commands produced by CreateScene(Configuration, Rectangle, DrawingCommandBatch, IReadOnlyList<IDisposable>?) and later rendered into a target through RenderScene<TPixel>(Configuration, ICanvasFrame<TPixel>, DrawingBackendScene).

```
public abstract class DrawingBackendScene : IDisposable
```

**Inheritance**

object ← DrawingBackendScene

**Implements**

IDisposable

**Derived**

DefaultDrawingBackendScene, WebGPUDrawingBackendScene

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

# Remarks

A scene captures everything a backend needs to replay the commands, including any resources (such as image-brush sources) that must remain alive until the scene is disposed. Disposing the scene releases backend-specific state through DisposeCore() and then the retained owned resources.

# Constructors

## DrawingBackendScene(Rectangle, IReadOnlyList<IDisposable>?)

Initializes a new instance of the DrawingBackendScene class.

```
protected DrawingBackendScene(Rectangle bounds, IReadOnlyList<IDisposable>? ownedResources)
```

### Parameters

`bounds` Rectangle

The target bounds used to create the scene.

`ownedResources` IReadOnlyList<IDisposable>

Resources that must stay alive for the retained scene.

## Properties

### Bounds

Gets the target bounds used to create the scene.

```
public Rectangle Bounds { get; }
```

### Property Value

Rectangle

## Methods

### Dispose()

Performs application-defined tasks associated with freeing, releasing, or resetting unmanaged resources.

```
public void Dispose()
```

# DisposeCore()

Disposes backend-specific resources retained by this scene.

```
protected virtual void DisposeCore()
```

# Struct DrawingCommandBatch

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.dll

One prepared draw-order command batch consumed by a drawing backend.

```
public readonly struct DrawingCommandBatch
```

**Inherited Members**
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

# Constructors

## DrawingCommandBatch(IReadOnlyList<CompositionSceneCommand>, bool)

Initializes a new instance of the DrawingCommandBatch struct.

```
public DrawingCommandBatch(IReadOnlyList<CompositionSceneCommand> commands, bool hasLayers)
```

### Parameters

`commands` IReadOnlyList<CompositionSceneCommand>

The draw-order scene commands.

`hasLayers` bool

Indicates whether the command stream contains layer boundaries.

### Remarks

Batches created through this constructor never contain apply barriers or clip controls; those commands are only produced by the internal canvas batcher.

# Properties

## CommandCount

Gets the total number of draw-order commands in the scene.

```
public int CommandCount { get; }
```

### Property Value

int

## Commands

Gets the draw-order scene commands.

```
public IReadOnlyList<CompositionSceneCommand> Commands { get; }
```

### Property Value

IReadOnlyList<CompositionSceneCommand>

## HasApply

Gets a value indicating whether this scene contains apply barriers.

```
public bool HasApply { get; }
```

### Property Value

bool

## HasClipControls

Gets a value indicating whether this scene contains clip control commands.

```
public bool HasClipControls { get; }
```

## Property Value

bool

## HasLayers

Gets a value indicating whether this scene contains inline layer commands.

```
public bool HasLayers { get; }
```

## Property Value

bool

# Interface ICanvasFrame<TPixel>

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.dll

A render destination for DrawingCanvas<TPixel>. Implementations back the canvas with either a CPU-accessible pixel region or an opaque native surface; a usable frame must expose at least one of the two.

```
public interface ICanvasFrame<TPixel> where TPixel : unmanaged, IPixel<TPixel>
```

## Type Parameters

`TPixel`

The pixel format.

# Remarks

A backend inspects the frame through TryGetCpuRegion(out Buffer2DRegion<TPixel>) and TryGetNativeSurface(out NativeSurface?) to choose its render path. Exactly one of the two succeeds for a given frame: a CPU frame yields a Buffer2DRegion<T>, a native frame yields a NativeSurface.

# Properties

## Bounds

Gets the frame bounds in root target coordinates.

```
Rectangle Bounds { get; }
```

## Property Value

Rectangle

# Methods

## TryGetCpuRegion(out Buffer2DRegion<TPixel>)

Attempts to get a CPU-accessible destination region.

```
bool TryGetCpuRegion(out Buffer2DRegion<TPixel> region)
```

### Parameters

`region` Buffer2DRegion<TPixel>

The CPU region when available.

### Returns

bool

true when a CPU region is available.

## TryGetNativeSurface(out NativeSurface?)

Attempts to get an opaque native destination surface.

```
bool TryGetNativeSurface(out NativeSurface? surface)
```

### Parameters

`surface` NativeSurface

The native surface when available.

### Returns

bool

true when a native surface is available.

# Interface ICompositionSceneCommandVisitor

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.dll

Visitor contract for one flush-scoped composition scene command.

```
public interface ICompositionSceneCommandVisitor
```

# Methods

## Visit(LineSegmentCompositionSceneCommand)

Visits one explicit stroked line-segment command.

```
void Visit(LineSegmentCompositionSceneCommand command)
```

### Parameters

`command` LineSegmentCompositionSceneCommand

The command being visited.

## Visit(PathCompositionSceneCommand)

Visits one fill-path or layer-based composition command.

```
void Visit(PathCompositionSceneCommand command)
```

### Parameters

`command` PathCompositionSceneCommand

The command being visited.

## Visit(PolylineCompositionSceneCommand)

Visits one explicit stroked polyline command.

```
void Visit(PolylineCompositionSceneCommand command)
```

### Parameters

`command` PolylineCompositionSceneCommand

The command being visited.

## Visit(StrokePathCompositionSceneCommand)

Visits one stroked path command.

```
void Visit(StrokePathCompositionSceneCommand command)
```

### Parameters

`command` StrokePathCompositionSceneCommand

The command being visited.

# Interface IDrawingBackend

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.dll

Defines the contract a drawing backend implements to turn recorded canvas commands into pixels: creating retained scenes from command batches, rendering those scenes into a canvas frame, and transferring pixels between frames (copy and readback).

```
public interface IDrawingBackend
```

# Methods

## CopyPixels<TPixel>(Configuration, ICanvasFrame<TPixel>, ICanvasFrame<TPixel>, Rectangle, Point)

Copies pixels from a source frame into a target frame.

```
void CopyPixels<TPixel>(Configuration configuration, ICanvasFrame<TPixel> source,
ICanvasFrame<TPixel> target, Rectangle sourceRectangle, Point targetPoint) where TPixel :
unmanaged, IPixel<TPixel>
```

### Parameters

`configuration` Configuration

The active processing configuration.

`source` ICanvasFrame<TPixel>

The source frame.

`target` ICanvasFrame<TPixel>

The target frame.

`sourceRectangle` Rectangle

The source rectangle in source-local coordinates.

`targetPoint` Point

The target point in target-local coordinates.

## Type Parameters

`TPixel`

The pixel format.

# CreateScene(Configuration, Rectangle, DrawingCommandBatch, IReadOnlyList<IDisposable>?)

Creates a backend scene from a prepared command batch.

```
DrawingBackendScene CreateScene(Configuration configuration, Rectangle targetBounds,
DrawingCommandBatch commandBatch, IReadOnlyList<IDisposable>? ownedResources = null)
```

## Parameters

`configuration` Configuration

The active processing configuration.

`targetBounds` Rectangle

The target bounds used for target-dependent scene data.

`commandBatch` DrawingCommandBatch

The scene commands in submission order.

`ownedResources` IReadOnlyList<IDisposable>

The resources that must stay alive for the returned scene.

## Returns

DrawingBackendScene

The created backend scene.

## ReadRegion<TPixel>(Configuration, ICanvasFrame<TPixel>, Rectangle, Buffer2DRegion<TPixel>)

Reads source pixels from the target into the destination region.

```
void ReadRegion<TPixel>(Configuration configuration, ICanvasFrame<TPixel> target,
Rectangle sourceRectangle, Buffer2DRegion<TPixel> destination) where TPixel :
unmanaged, IPixel<TPixel>
```

### Parameters

`configuration` Configuration

The active processing configuration.

`target` ICanvasFrame<TPixel>

The target frame.

`sourceRectangle` Rectangle

The source rectangle in target-local coordinates.

`destination` Buffer2DRegion<TPixel>

The destination region that receives the copied pixels.

### Type Parameters

`TPixel`

The pixel format.

## RenderScene<TPixel>(Configuration, ICanvasFrame<TPixel>, DrawingBackendScene)

Renders a backend scene into the target.

```
void RenderScene<TPixel>(Configuration configuration, ICanvasFrame<TPixel> target,
DrawingBackendScene scene) where TPixel : unmanaged, IPixel<TPixel>
```

## Parameters

`configuration` Configuration

The active processing configuration.

`target` ICanvasFrame<TPixel>

The target frame.

`scene` DrawingBackendScene

The backend scene to render.

## Type Parameters

`TPixel`

The pixel format.

# Interface IWebGPUShaderEffect

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.WebGPU.dll

Identifies a layer effect that supplies a native WebGPU implementation and a CPU fallback.

```
public interface IWebGPUShaderEffect
```

# Remarks

Derive from WebGPUShaderLayerEffect or WebGPUBackdropShaderLayerEffect to implement this contract. Pass an instance to Precompile(IWebGPUShaderEffect) to compile its pipelines before first use.

# Class LineSegmentCompositionSceneCommand

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.dll

Scene command wrapper for explicit stroked line-segment commands.

```
public sealed class LineSegmentCompositionSceneCommand : CompositionSceneCommand
```

**Inheritance**

object ← CompositionSceneCommand ← LineSegmentCompositionSceneCommand

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## LineSegmentCompositionSceneCommand(in StrokeLineSegmentCommand)

Initializes a new instance of the LineSegmentCompositionSceneCommand class.

```
public LineSegmentCompositionSceneCommand(in StrokeLineSegmentCommand command)
```

### Parameters

`command` StrokeLineSegmentCommand

The wrapped stroke line-segment command.

# Properties

## Command

Gets the wrapped stroke line-segment command.

```
public StrokeLineSegmentCommand Command { get; }
```

### Property Value

StrokeLineSegmentCommand

# Methods

## Accept(ICompositionSceneCommandVisitor)

Dispatches the command to a visitor without a per-item kind switch at the call site.

```
public override void Accept(ICompositionSceneCommandVisitor visitor)
```

### Parameters

`visitor` ICompositionSceneCommandVisitor

The visitor receiving the command.

# Class MemoryCanvasFrame<TPixel>

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.dll

Canvas frame backed by a Buffer2DRegion<T>.

```
public sealed class MemoryCanvasFrame<TPixel> : ICanvasFrame<TPixel> where TPixel :
unmanaged, IPixel<TPixel>
```

## Type Parameters

`TPixel`

The pixel format.

**Inheritance**

object ← MemoryCanvasFrame<TPixel>

**Implements**

ICanvasFrame<TPixel>

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## MemoryCanvasFrame(Buffer2DRegion<TPixel>)

Initializes a new instance of the MemoryCanvasFrame<TPixel> class.

```
public MemoryCanvasFrame(Buffer2DRegion<TPixel> region)
```

## Parameters

`region` Buffer2DRegion<TPixel>

The pixel buffer region backing this frame.

# Properties

## Bounds

Gets the frame bounds in root target coordinates.

```
public Rectangle Bounds { get; }
```

### Property Value

Rectangle

# Methods

## TryGetCpuRegion(out Buffer2DRegion<TPixel>)

Attempts to get a CPU-accessible destination region.

```
public bool TryGetCpuRegion(out Buffer2DRegion<TPixel> region)
```

### Parameters

`region` Buffer2DRegion<TPixel>

The CPU region when available.

### Returns

bool

true when a CPU region is available.

## TryGetNativeSurface(out NativeSurface?)

Attempts to get an opaque native destination surface.

```
public bool TryGetNativeSurface(out NativeSurface? surface)
```

## Parameters

`surface` NativeSurface

The native surface when available.

## Returns

bool

true when a native surface is available.

# Class NativeCanvasFrame<TPixel>

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.dll

Canvas frame backed by a NativeSurface.

```
public sealed class NativeCanvasFrame<TPixel> : ICanvasFrame<TPixel> where TPixel :
unmanaged, IPixel<TPixel>
```

## Type Parameters

`TPixel`

The pixel format.

**Inheritance**

object ← NativeCanvasFrame<TPixel>

**Implements**

ICanvasFrame<TPixel>

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## NativeCanvasFrame(Rectangle, NativeSurface)

Initializes a new instance of the NativeCanvasFrame<TPixel> class.

```
public NativeCanvasFrame(Rectangle bounds, NativeSurface surface)
```

## Parameters

`bounds` Rectangle

The frame bounds.

`surface` NativeSurface

The native surface backing this frame.

# Properties

## Bounds

Gets the frame bounds in root target coordinates.

```
public Rectangle Bounds { get; }
```

## Property Value

Rectangle

# Methods

## TryGetCpuRegion(out Buffer2DRegion<TPixel>)

Attempts to get a CPU-accessible destination region.

```
public bool TryGetCpuRegion(out Buffer2DRegion<TPixel> region)
```

## Parameters

`region` Buffer2DRegion<TPixel>

The CPU region when available.

## Returns

bool

true when a CPU region is available.

# TryGetNativeSurface(out NativeSurface?)

Attempts to get an opaque native destination surface.

```
public bool TryGetNativeSurface(out NativeSurface? surface)
```

## Parameters

`surface` NativeSurface

The native surface when available.

## Returns

bool

true when a native surface is available.

# Class NativeSurface

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.dll

Opaque base type for a backend-specific native (non-CPU) drawing target, such as a GPU surface.

```
public abstract class NativeSurface
```

**Inheritance**

object ← NativeSurface

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

# Remarks

A canvas frame exposes an instance through TryGetNativeSurface(out NativeSurface?). The concrete backend that owns the frame downcasts this to its own surface type; the base type carries no members because its contents are entirely backend-defined.

# Constructors

## NativeSurface()

Initializes a new instance of the NativeSurface class.

```
protected NativeSurface()
```

# Class PathCompositionSceneCommand

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.dll

Scene command wrapper for fill-path and layer-based composition commands.

```
public sealed class PathCompositionSceneCommand : CompositionSceneCommand
```

**Inheritance**

object ← CompositionSceneCommand ← PathCompositionSceneCommand

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## PathCompositionSceneCommand(in CompositionCommand)

Initializes a new instance of the PathCompositionSceneCommand class.

```
public PathCompositionSceneCommand(in CompositionCommand command)
```

### Parameters

`command` CompositionCommand

The wrapped composition command.

# Properties

## Command

Gets the wrapped composition command.

```
public CompositionCommand Command { get; }
```

### Property Value

CompositionCommand

### Remarks

The internal setter lets the batcher rewrite the command in place during flush preparation (for example brush normalization) without reallocating the scene node.

# Methods

## Accept(ICompositionSceneCommandVisitor)

Dispatches the command to a visitor without a per-item kind switch at the call site.

```
public override void Accept(ICompositionSceneCommandVisitor visitor)
```

### Parameters

`visitor` ICompositionSceneCommandVisitor

The visitor receiving the command.

# Class PolylineCompositionSceneCommand

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.dll

Scene command wrapper for explicit stroked polyline commands.

```
public sealed class PolylineCompositionSceneCommand : CompositionSceneCommand
```

**Inheritance**

object ← CompositionSceneCommand ← PolylineCompositionSceneCommand

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## PolylineCompositionSceneCommand(in StrokePolylineCommand)

Initializes a new instance of the PolylineCompositionSceneCommand class.

```
public PolylineCompositionSceneCommand(in StrokePolylineCommand command)
```

### Parameters

`command` StrokePolylineCommand

The wrapped stroke polyline command.

# Properties

## Command

Gets the wrapped stroke polyline command.

```
public StrokePolylineCommand Command { get; }
```

### Property Value

StrokePolylineCommand

# Methods

## Accept(ICompositionSceneCommandVisitor)

Dispatches the command to a visitor without a per-item kind switch at the call site.

```
public override void Accept(ICompositionSceneCommandVisitor visitor)
```

### Parameters

`visitor` ICompositionSceneCommandVisitor

The visitor receiving the command.

# Enum RasterizationMode

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.dll

Describes whether rasterizers should emit continuous coverage or binary aliased coverage.

```
public enum RasterizationMode
```

# Fields

`Aliased = 1`

Emit binary coverage values (0 or 1).

`Antialiased = 0`

Emit continuous coverage in the range [0, 1].

# Struct RasterizerOptions

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.dll

Immutable options used by rasterizers when scan-converting vector geometry.

```
public readonly struct RasterizerOptions
```

**Inherited Members**
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

# Constructors

## RasterizerOptions(Rectangle, IntersectionRule, RasterizationMode, float)

Initializes a new instance of the RasterizerOptions struct.

```
public RasterizerOptions(Rectangle interest, IntersectionRule intersectionRule,
RasterizationMode rasterizationMode, float coverageBoost = 0)
```

### Parameters

`interest` Rectangle

Destination bounds to rasterize into.

`intersectionRule` IntersectionRule

Polygon intersection rule.

`rasterizationMode` RasterizationMode

Rasterization coverage mode.

`coverageBoost` float

Perceptual coverage boost for antialiased mode (0 disables).

# Properties

## CoverageBoost

Gets the perceptual coverage boost used when RasterizationMode is Antialiased. Partial coverage values are remapped by the S-curve `a + boost * a * (1 - a) * (2a - 1)`, darkening mostly covered pixels and lightening mostly empty ones while 0, 0.5, and 1 stay fixed; at `1` the remap is exactly smoothstep. `0` disables the boost. Text rendering uses this to counter the soft, washed-out appearance of small blended glyphs.

```
public float CoverageBoost { get; }
```

### Property Value

float

## Interest

Gets destination bounds to rasterize into.

```
public Rectangle Interest { get; }
```

### Property Value

Rectangle

## IntersectionRule

Gets the polygon intersection rule.

```
public IntersectionRule IntersectionRule { get; }
```

### Property Value

IntersectionRule

## RasterizationMode

Gets the rasterization coverage mode.

```
public RasterizationMode RasterizationMode { get; }
```

### Property Value

RasterizationMode

# Methods

## WithInterest(Rectangle)

Creates a copy of the current options with a different interest rectangle.

```
public RasterizerOptions WithInterest(Rectangle interest)
```

### Parameters

`interest` Rectangle

The replacement interest rectangle.

### Returns

RasterizerOptions

A new RasterizerOptions value.

# Struct StrokeLineSegmentCommand

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.dll

One explicit stroked two-point line-segment command queued by the canvas batcher.

```
public readonly struct StrokeLineSegmentCommand
```

**Inherited Members**
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

# Remarks

Stroke commands carry pen geometry only; they do not carry clip state. Backends resolve the active clip stack from the begin/end-clip commands surrounding each draw in the stream.

# Constructors

## StrokeLineSegmentCommand(PointF, PointF, Brush, DrawingOptions, in RasterizerOptions, Rectangle, Point, Pen, bool)

Initializes a new instance of the StrokeLineSegmentCommand struct.

```
public StrokeLineSegmentCommand(PointF sourceStart, PointF sourceEnd, Brush brush,
DrawingOptions drawingOptions, in RasterizerOptions rasterizerOptions, Rectangle
targetBounds, Point destinationOffset, Pen pen, bool isInsideLayer)
```

### Parameters

`sourceStart` PointF

The source line start point.

`sourceEnd` PointF

The source line end point.

`brush` Brush

The brush used to shade the stroke.

`drawingOptions` DrawingOptions

The drawing options (graphics, shape, transform) used during composition.

`rasterizerOptions` RasterizerOptions

The rasterizer options used to generate coverage.

`targetBounds` Rectangle

The absolute bounds of the logical target.

`destinationOffset` Point

The absolute destination offset of the command.

`pen` Pen

The stroke metadata.

`isInsideLayer` bool

True if the command was recorded inside a layer.

# Properties

## Brush

Gets the brush used during composition.

```
public Brush Brush { get; }
```

## Property Value

Brush

## DestinationOffset

Gets the absolute destination offset where the local coverage should be composited.

```
public Point DestinationOffset { get; }
```

### Property Value

Point

## DrawingOptions

Gets the drawing options carried by the command.

```
public DrawingOptions DrawingOptions { get; }
```

### Property Value

DrawingOptions

## GraphicsOptions

Gets the graphics options used during composition.

```
public GraphicsOptions GraphicsOptions { get; }
```

### Property Value

GraphicsOptions

## IsInsideLayer

Gets a value indicating whether the command was recorded inside a layer.

```
public bool IsInsideLayer { get; }
```

## Property Value

bool

# Pen

Gets the stroke metadata for this command.

```
public Pen Pen { get; }
```

## Property Value

Pen

# RasterizerOptions

Gets the rasterizer options used to generate coverage.

```
public RasterizerOptions RasterizerOptions { get; }
```

## Property Value

RasterizerOptions

# SourceEnd

Gets the source line end point.

```
public PointF SourceEnd { get; }
```

## Property Value

PointF

## SourceStart

Gets the source line start point.

```
public PointF SourceStart { get; }
```

### Property Value

PointF

## TargetBounds

Gets the absolute bounds of the logical target for this command.

```
public Rectangle TargetBounds { get; }
```

### Property Value

Rectangle

## Transform

Gets the command transform.

```
public Matrix4x4 Transform { get; }
```

### Property Value

Matrix4×4

# Methods

# GetConservativeBounds(PointF, PointF, Pen)

Computes the conservative stroked bounds of one two-point line segment.

```
public static RectangleF GetConservativeBounds(PointF start, PointF end, Pen pen)
```

## Parameters

`start` PointF

The line start point.

`end` PointF

The line end point.

`pen` Pen

The stroke metadata.

## Returns

RectangleF

The conservative stroked bounds.

# Struct StrokePathCommand

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.dll

One stroked path command queued by the canvas batcher.

```
public readonly struct StrokePathCommand
```

**Inherited Members**
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

# Remarks

Stroke commands carry pen geometry only; they do not carry clip state. Backends resolve the active clip stack from the begin/end-clip commands surrounding each draw in the stream.

# Properties

## Brush

Gets the brush used during composition.

```
public Brush Brush { get; }
```

### Property Value

Brush

## DestinationOffset

Gets the absolute destination offset where the local coverage should be composited.

```
public Point DestinationOffset { get; }
```

### Property Value

Point

## DrawingOptions

Gets the drawing options carried by the command.

```
public DrawingOptions DrawingOptions { get; }
```

### Property Value

DrawingOptions

## GraphicsOptions

Gets the graphics options used during composition.

```
public GraphicsOptions GraphicsOptions { get; }
```

### Property Value

GraphicsOptions

## IsInsideLayer

Gets a value indicating whether the command was recorded inside a layer.

```
public bool IsInsideLayer { get; }
```

### Property Value

bool

## Pen

Gets the stroke metadata for this command.

```
public Pen Pen { get; }
```

### Property Value

Pen

## RasterizerOptions

Gets the rasterizer options used to generate coverage.

```
public RasterizerOptions RasterizerOptions { get; }
```

### Property Value

RasterizerOptions

## SourcePath

Gets the source stroke path.

```
public IPath SourcePath { get; }
```

### Property Value

IPath

## SubPixelOffset

Gets the fractional translation applied after the drawing options transform. Glyph geometry rides an integer destination offset; the sub-pixel remainder travels here so the queued command does not need per-operation drawing options to carry it.

```
public Vector2 SubPixelOffset { get; }
```

## Property Value

Vector2

## TargetBounds

Gets the absolute bounds of the logical target for this command.

```
public Rectangle TargetBounds { get; }
```

## Property Value

Rectangle

## Transform

Gets the drawing transform: the drawing options transform followed by the sub-pixel translation.

```
public Matrix4x4 Transform { get; }
```

## Property Value

Matrix4×4

# Class StrokePathCompositionSceneCommand

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.dll

Scene command wrapper for stroked path commands.

```
public sealed class StrokePathCompositionSceneCommand : CompositionSceneCommand
```

### Inheritance

object ← CompositionSceneCommand ← StrokePathCompositionSceneCommand

### Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## StrokePathCompositionSceneCommand(in StrokePathCommand)

Initializes a new instance of the StrokePathCompositionSceneCommand class.

```
public StrokePathCompositionSceneCommand(in StrokePathCommand command)
```

### Parameters

`command` StrokePathCommand

The wrapped stroke path command.

# Properties

## Command

Gets the wrapped stroke path command.

```
public StrokePathCommand Command { get; }
```

### Property Value

StrokePathCommand

### Remarks

The internal setter lets the batcher rewrite the command in place during flush preparation (for example dash or transform expansion) without reallocating the scene node.

# Methods

## Accept(ICompositionSceneCommandVisitor)

Dispatches the command to a visitor without a per-item kind switch at the call site.

```
public override void Accept(ICompositionSceneCommandVisitor visitor)
```

### Parameters

`visitor` ICompositionSceneCommandVisitor

The visitor receiving the command.

# Struct StrokePolylineCommand

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.dll

One explicit stroked open polyline command queued by the canvas batcher.

```
public readonly struct StrokePolylineCommand
```

**Inherited Members**
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

# Remarks

Stroke commands carry pen geometry only; they do not carry clip state. Backends resolve the active clip stack from the begin/end-clip commands surrounding each draw in the stream.

# Constructors

## StrokePolylineCommand(PointF[], Brush, DrawingOptions, in RasterizerOptions, Rectangle, Point, Pen, bool)

Initializes a new instance of the StrokePolylineCommand struct.

```
public StrokePolylineCommand(PointF[] sourcePoints, Brush brush, DrawingOptions
drawingOptions, in RasterizerOptions rasterizerOptions, Rectangle targetBounds, Point
destinationOffset, Pen pen, bool isInsideLayer)
```

### Parameters

`sourcePoints` PointF[]

The source polyline points.

`brush` Brush

The brush used to shade the stroke.

`drawingOptions` DrawingOptions

The drawing options (graphics, shape, transform) used during composition.

`rasterizerOptions` RasterizerOptions

The rasterizer options used to generate coverage.

`targetBounds` Rectangle

The absolute bounds of the logical target.

`destinationOffset` Point

The absolute destination offset of the command.

`pen` Pen

The stroke metadata.

`isInsideLayer` bool

True if the command was recorded inside a layer.

# Properties

## Brush

Gets the brush used during composition.

```
public Brush Brush { get; }
```

### Property Value

Brush

## DestinationOffset

Gets the absolute destination offset where the local coverage should be composited.

```
public Point DestinationOffset { get; }
```

### Property Value

Point

## DrawingOptions

Gets the drawing options carried by the command.

```
public DrawingOptions DrawingOptions { get; }
```

### Property Value

DrawingOptions

## GraphicsOptions

Gets the graphics options used during composition.

```
public GraphicsOptions GraphicsOptions { get; }
```

### Property Value

GraphicsOptions

## IsInsideLayer

Gets a value indicating whether the command was recorded inside a layer.

```
public bool IsInsideLayer { get; }
```

### Property Value

bool

## Pen

Gets the stroke metadata for this command.

```
public Pen Pen { get; }
```

### Property Value

Pen

## RasterizerOptions

Gets the rasterizer options used to generate coverage.

```
public RasterizerOptions RasterizerOptions { get; }
```

### Property Value

RasterizerOptions

## SourcePoints

Gets the source polyline points.

```
public PointF[] SourcePoints { get; }
```

### Property Value

PointF[]

## TargetBounds

Gets the absolute bounds of the logical target for this command.

```
public Rectangle TargetBounds { get; }
```

### Property Value

Rectangle

## Transform

Gets the command transform.

```
public Matrix4x4 Transform { get; }
```

### Property Value

Matrix4×4

# Methods

## GetConservativeBounds(PointF[], Pen)

Computes the conservative stroked bounds of one open polyline.

```
public static RectangleF GetConservativeBounds(PointF[] points, Pen pen)
```

### Parameters

`points` PointF[]

The polyline points.

`pen` Pen

The stroke metadata.

### Returns

RectangleF

The conservative stroked bounds.

# Class WebGPUBackdropAcrylicLayerEffect

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.WebGPU.dll

Provides a shader-accelerated counterpart to BackdropAcrylicLayerEffect.

```
public sealed class WebGPUBackdropAcrylicLayerEffect : WebGPUBackdropShaderLayerEffect,
IWebGPUShaderEffect
```

**Inheritance**

object ← LayerEffect ← BackdropLayerEffect ← WebGPUBackdropShaderLayerEffect ← WebGPUBackdropAcrylicLayerEffect

**Implements**

IWebGPUShaderEffect

**Inherited Members**

BackdropLayerEffect.Reach , LayerEffect.IsPassThrough , LayerEffect.WriteBackOptions , LayerEffect.WriteBackOffset , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Remarks

This effect can be used with both WebGPU and CPU drawing backends. WebGPU blurs and tints the backdrop with shader passes; a CPU backend applies the equivalent BackdropAcrylicLayerEffect.

# Constructors

## WebGPUBackdropAcrylicLayerEffect(float, Color)

Initializes a new instance of the WebGPUBackdropAcrylicLayerEffect class.

```
public WebGPUBackdropAcrylicLayerEffect(float sigma, Color tint)
```

### Parameters

`sigma` float

The Gaussian blur sigma, in pixels.

`tint` Color

The tint blended over the blurred backdrop.

# Properties

## Sigma

Gets the Gaussian blur sigma, in pixels.

```
public float Sigma { get; }
```

### Property Value

float

## Tint

Gets the tint blended over the blurred backdrop.

```
public Color Tint { get; }
```

### Property Value

Color

# Class WebGPUBackdropColorMatrixLayerEffect

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.WebGPU.dll

Provides a shader-accelerated counterpart to BackdropColorMatrixLayerEffect.

```
public sealed class WebGPUBackdropColorMatrixLayerEffect : WebGPUBackdropShaderLayerEffect,
IWebGPUShaderEffect
```

**Inheritance**

object ← LayerEffect ← BackdropLayerEffect ← WebGPUBackdropShaderLayerEffect ← WebGPUBackdropColorMatrixLayerEffect

**Implements**

IWebGPUShaderEffect

**Inherited Members**

BackdropLayerEffect.Reach , LayerEffect.IsPassThrough , LayerEffect.WriteBackOptions , LayerEffect.WriteBackOffset , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Remarks

This effect can be used with both WebGPU and CPU drawing backends. WebGPU transforms the backdrop with a shader; a CPU backend applies the equivalent BackdropColorMatrixLayerEffect.

# Constructors

## WebGPUBackdropColorMatrixLayerEffect(ColorMatrix)

Initializes a new instance of the WebGPUBackdropColorMatrixLayerEffect class.

```
public WebGPUBackdropColorMatrixLayerEffect(ColorMatrix matrix)
```

### Parameters

`matrix` ColorMatrix

The colour matrix applied to the backdrop.

# Properties

## Matrix

Gets the colour matrix applied to the backdrop.

```
public ColorMatrix Matrix { get; }
```

## Property Value

ColorMatrix

# Class WebGPUBackdropDropShadowLayerEffect

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.WebGPU.dll

Provides a shader-accelerated counterpart to BackdropDropShadowLayerEffect.

```
public sealed class WebGPUBackdropDropShadowLayerEffect : WebGPUBackdropShaderLayerEffect,
IWebGPUShaderEffect
```

**Inheritance**

object ← LayerEffect ← BackdropLayerEffect ← WebGPUBackdropShaderLayerEffect ← WebGPUBackdropDropShadowLayerEffect

**Implements**

IWebGPUShaderEffect

**Inherited Members**

BackdropLayerEffect.Reach , LayerEffect.IsPassThrough , LayerEffect.WriteBackOptions , LayerEffect.WriteBackOffset , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Remarks

This effect can be used with both WebGPU and CPU drawing backends. WebGPU creates the backdrop shadow with shader passes; a CPU backend applies the equivalent BackdropDropShadowLayerEffect.

# Constructors

## WebGPUBackdropDropShadowLayerEffect(Point, float, Color)

Initializes a new instance of the WebGPUBackdropDropShadowLayerEffect class.

```
public WebGPUBackdropDropShadowLayerEffect(Point offset, float sigma, Color color)
```

### Parameters

`offset` Point

The shadow offset, in pixels.

`sigma` float

The Gaussian blur sigma, in pixels; zero draws a hard shadow.

`color` Color

The shadow colour.

## Properties

### Color

Gets the shadow colour.

```
public Color Color { get; }
```

### Property Value

Color

### Offset

Gets the shadow offset, in pixels.

```
public Point Offset { get; }
```

### Property Value

Point

# Sigma

Gets the Gaussian blur sigma, in pixels.

```
public float Sigma { get; }
```

## Property Value

float

# Class WebGPUBackdropGaussianBlurLayerEffect

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.WebGPU.dll

Provides a shader-accelerated counterpart to BackdropBlurLayerEffect.

```
public sealed class WebGPUBackdropGaussianBlurLayerEffect : WebGPUBackdropShaderLayerEffect,
IWebGPUShaderEffect
```

**Inheritance**

object ← LayerEffect ← BackdropLayerEffect ← WebGPUBackdropShaderLayerEffect ← WebGPUBackdropGaussianBlurLayerEffect

**Implements**

IWebGPUShaderEffect

**Inherited Members**

BackdropLayerEffect.Reach , LayerEffect.IsPassThrough , LayerEffect.WriteBackOptions , LayerEffect.WriteBackOffset , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Remarks

This effect can be used with both WebGPU and CPU drawing backends. WebGPU blurs the backdrop with separable shader passes; a CPU backend applies the equivalent BackdropBlurLayerEffect.

# Constructors

## WebGPUBackdropGaussianBlurLayerEffect(float)

Initializes a new instance of the WebGPUBackdropGaussianBlurLayerEffect class.

```
public WebGPUBackdropGaussianBlurLayerEffect(float sigma)
```

### Parameters

`sigma` float

The Gaussian blur sigma, in pixels; zero leaves the backdrop unchanged.

## Properties

### Sigma

Gets the Gaussian blur sigma, in pixels.

```
public float Sigma { get; }
```

### Property Value

float

# Class WebGPUBackdropShaderLayerEffect

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.WebGPU.dll

Defines a backdrop effect with complete WGSL shader passes and an equivalent ImageSharp CPU fallback.

```
public abstract class WebGPUBackdropShaderLayerEffect : BackdropLayerEffect,
IWebGPUShaderEffect
```

**Inheritance**

object ← LayerEffect ← BackdropLayerEffect ← WebGPUBackdropShaderLayerEffect

**Implements**

IWebGPUShaderEffect

**Derived**

WebGPUBackdropAcrylicLayerEffect, WebGPUBackdropColorMatrixLayerEffect, WebGPUBackdropDropShadowLayerEffect, WebGPUBackdropGaussianBlurLayerEffect

**Inherited Members**

BackdropLayerEffect.Reach , LayerEffect.IsPassThrough , LayerEffect.WriteBackOptions , LayerEffect.WriteBackOffset , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

# Remarks

Instances work with both drawing backends. WebGPU executes the configured WGSL passes, while CPU rendering executes the required fallback effect. Source follows the same complete-shader contract as WebGPUShaderLayerEffect.

# Constructors

## WebGPUBackdropShaderLayerEffect(BackdropLayerEffect, string, WebGPUShaderUniformLayout)

Initializes a new instance of the WebGPUBackdropShaderLayerEffect class.

```
protected WebGPUBackdropShaderLayerEffect(BackdropLayerEffect fallbackEffect, string
shaderSource, WebGPUShaderUniformLayout uniformLayout)
```

## Parameters

`fallbackEffect` BackdropLayerEffect

The equivalent backdrop effect used by CPU rendering.

`shaderSource` string

The complete WGSL source for each pass added by this effect.

`uniformLayout` WebGPUShaderUniformLayout

The named uniform values available to the shader.

# WebGPUBackdropShaderLayerEffect(string, WebGPUShaderUniformLayout, Action<IImageProcessingContext>)

Initializes a new instance of the WebGPUBackdropShaderLayerEffect class.

```
protected WebGPUBackdropShaderLayerEffect(string shaderSource, WebGPUShaderUniformLayout
uniformLayout, Action<IImageProcessingContext> fallback)
```

## Parameters

`shaderSource` string

The complete WGSL source for each pass added by this effect.

`uniformLayout` WebGPUShaderUniformLayout

The named uniform values available to the shader.

`fallback` Action<IImageProcessingContext>

The equivalent operation used by CPU rendering.

# Methods

## AddShaderPass(BorderWrappingMode, BorderWrappingMode, Action<WebGPUShaderUniformBuilder>)

Configures and adds an invocation whose filtered samples use the supplied border modes.

```
protected void AddShaderPass(BorderWrappingMode xBorderMode, BorderWrappingMode yBorderMode,
Action<WebGPUShaderUniformBuilder> configureUniforms)
```

### Parameters

`xBorderMode` BorderWrappingMode

The wrapping mode applied beyond the horizontal input borders.

`yBorderMode` BorderWrappingMode

The wrapping mode applied beyond the vertical input borders.

`configureUniforms` Action<WebGPUShaderUniformBuilder>

The action that assigns this pass's named uniform values.

## AddShaderPass(Action<WebGPUShaderUniformBuilder>)

Configures and adds an invocation of the complete shader source to this effect's ordered pass sequence.

```
protected void AddShaderPass(Action<WebGPUShaderUniformBuilder> configureUniforms)
```

### Parameters

`configureUniforms` Action<WebGPUShaderUniformBuilder>

The action that assigns this pass's named uniform values.

# Class WebGPUColorMatrixLayerEffect

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.WebGPU.dll

Provides a shader-accelerated counterpart to ColorMatrixLayerEffect.

```
public sealed class WebGPUColorMatrixLayerEffect : WebGPUShaderLayerEffect,
IWebGPUShaderEffect
```

**Inheritance**

object ← LayerEffect ← WebGPUShaderLayerEffect ← WebGPUColorMatrixLayerEffect

**Implements**

IWebGPUShaderEffect

**Inherited Members**

LayerEffect.Reach , LayerEffect.IsPassThrough , LayerEffect.WriteBackOptions , LayerEffect.WriteBackOffset , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Remarks

This effect can be used with both WebGPU and CPU drawing backends. WebGPU transforms the layer with a shader; a CPU backend applies the equivalent ColorMatrixLayerEffect.

# Constructors

## WebGPUColorMatrixLayerEffect(ColorMatrix)

Initializes a new instance of the WebGPUColorMatrixLayerEffect class.

```
public WebGPUColorMatrixLayerEffect(ColorMatrix matrix)
```

### Parameters

`matrix` ColorMatrix

The colour matrix applied to the layer content.

# Properties

## Matrix

Gets the colour matrix applied to the layer content.

```
public ColorMatrix Matrix { get; }
```

## Property Value

ColorMatrix

# Enum WebGPUCompositeAlphaMode

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.WebGPU.dll

Specifies how the native compositor interprets alpha in a WebGPU surface.

```
public enum WebGPUCompositeAlphaMode
```

# Fields

`Auto = 0`

Allows WebGPU to select the surface composition mode.

`Opaque = 1`

Ignores the surface alpha channel and presents every pixel as opaque.

`Premultiplied = 2`

Presents color components that have already been multiplied by alpha.

# Class WebGPUDeviceContext

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.WebGPU.dll

Owns the WebGPU device-scoped drawing state shared by render targets, surfaces, shader programs, and pipelines.

```
public sealed class WebGPUDeviceContext
```

**Inheritance**

object ← WebGPUDeviceContext

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Properties

## Backend

Gets the WebGPU drawing backend owned by this context. Use this to inspect per-flush diagnostics for chunked rendering.

```
public WebGPUDrawingBackend Backend { get; }
```

## Property Value

WebGPUDrawingBackend

## Configuration

Gets the configuration provided when the context was created.

```
public Configuration Configuration { get; }
```

## Property Value

Configuration

# Methods

## Precompile(IWebGPUShaderEffect)

Compiles and caches every pipeline used by a shader effect on this WebGPU device.

```
public void Precompile(IWebGPUShaderEffect effect)
```

## Parameters

`effect` IWebGPUShaderEffect

The shader effect to compile.

## Remarks

Call this before the effect's first rendered frame when shader compilation latency must not occur during presentation.

# Class WebGPUDrawingBackend

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.WebGPU.dll

Pixel-format registration for composite session I/O.

```
public sealed class WebGPUDrawingBackend : IDrawingBackend, IDisposable
```

**Inheritance**

object ← WebGPUDrawingBackend

**Implements**

IDrawingBackend, IDisposable

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Remarks

SixLabors.ImageSharp.Drawing.Processing.Backends.WebGPUDrawingBackend.CompositePixelRegistrations is intentionally explicit and only includes one-to-one format mappings where the GPU texture format can round-trip the pixel payload without channel swizzle or custom conversion logic. Only formats that support `storage` texture binding (required by the compute compositor) are included. Formats that lack storage support are omitted and fall back to the CPU backend.

# Constructors

## WebGPUDrawingBackend()

Initializes a new instance of the WebGPUDrawingBackend class.

```
public WebGPUDrawingBackend()
```

# Methods

## CopyPixels<TPixel>(Configuration, ICanvasFrame<TPixel>, ICanvasFrame<TPixel>, Rectangle, Point)

Copies pixels from a source frame into a target frame.

```
public void CopyPixels<TPixel>(Configuration configuration, ICanvasFrame<TPixel> source,
ICanvasFrame<TPixel> target, Rectangle sourceRectangle, Point targetPoint) where TPixel :
unmanaged, IPixel<TPixel>
```

### Parameters

`configuration` Configuration

The active processing configuration.

`source` ICanvasFrame<TPixel>

The source frame.

`target` ICanvasFrame<TPixel>

The target frame.

`sourceRectangle` Rectangle

The source rectangle in source-local coordinates.

`targetPoint` Point

The target point in target-local coordinates.

### Type Parameters

`TPixel`

The pixel format.

## CreateScene(Configuration, Rectangle, DrawingCommandBatch, IReadOnlyList<IDisposable>?)

Creates a backend scene from a prepared command batch.

```
public DrawingBackendScene CreateScene(Configuration configuration, Rectangle targetBounds,
DrawingCommandBatch commandBatch, IReadOnlyList<IDisposable>? ownedResources = null)
```

## Parameters

`configuration` Configuration

The active processing configuration.

`targetBounds` Rectangle

The target bounds used for target-dependent scene data.

`commandBatch` DrawingCommandBatch

The scene commands in submission order.

`ownedResources` IReadOnlyList<IDisposable>

The resources that must stay alive for the returned scene.

## Returns

DrawingBackendScene

The created backend scene.

# Dispose()

Releases all cached shared WebGPU resources.

```
public void Dispose()
```

# ReadRegion<TPixel>(Configuration, ICanvasFrame<TPixel>, Rectangle, Buffer2DRegion<TPixel>)

Reads source pixels from the target into the destination region.

```
public void ReadRegion<TPixel>(Configuration configuration, ICanvasFrame<TPixel>
target, Rectangle sourceRectangle, Buffer2DRegion<TPixel> destination) where TPixel :
unmanaged, IPixel<TPixel>
```

## Parameters

`configuration` Configuration

The active processing configuration.

`target` ICanvasFrame<TPixel>

The target frame.

`sourceRectangle` Rectangle

The source rectangle in target-local coordinates.

`destination` Buffer2DRegion<TPixel>

The destination region that receives the copied pixels.

## Type Parameters

`TPixel`

The pixel format.

# RenderScene<TPixel>(Configuration, ICanvasFrame<TPixel>, DrawingBackendScene)

Renders a backend scene into the target.

```
public void RenderScene<TPixel>(Configuration configuration, ICanvasFrame<TPixel> target,
DrawingBackendScene scene) where TPixel : unmanaged, IPixel<TPixel>
```

## Parameters

`configuration` Configuration

The active processing configuration.

`target` ICanvasFrame<TPixel>

The target frame.

`scene` DrawingBackendScene

The backend scene to render.

## Type Parameters

`TPixel`

The pixel format.

# Class WebGPUDrawingBackendScene

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.WebGPU.dll

Retained scene created by the WebGPU drawing backend.

```
public sealed class WebGPUDrawingBackendScene : DrawingBackendScene, IDisposable
```

**Inheritance**

object ← DrawingBackendScene ← WebGPUDrawingBackendScene

**Implements**

IDisposable

**Inherited Members**

DrawingBackendScene.Bounds , DrawingBackendScene.Dispose() , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Remarks

A retained scene can be rendered multiple times, including concurrently. The scene caches reusable GPU arenas and monotonically growing scratch capacities between renders so repeat renders skip the dynamic-growth discovery passes.

# Methods

## DisposeCore()

Disposes backend-specific resources retained by this scene.

```
protected override void DisposeCore()
```

# Class WebGPUDropShadowLayerEffect

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.WebGPU.dll

Provides a shader-accelerated counterpart to DropShadowLayerEffect.

```
public sealed class WebGPUDropShadowLayerEffect : WebGPUShaderLayerEffect,
IWebGPUShaderEffect
```

**Inheritance**

object ← LayerEffect ← WebGPUShaderLayerEffect ← WebGPUDropShadowLayerEffect

**Implements**

IWebGPUShaderEffect

**Inherited Members**

LayerEffect.Reach , LayerEffect.IsPassThrough , LayerEffect.WriteBackOptions , LayerEffect.WriteBackOffset , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Remarks

This effect can be used with both WebGPU and CPU drawing backends. WebGPU creates the shadow with shader passes; a CPU backend applies the equivalent DropShadowLayerEffect.

# Constructors

## WebGPUDropShadowLayerEffect(Point, float, Color)

Initializes a new instance of the WebGPUDropShadowLayerEffect class.

```
public WebGPUDropShadowLayerEffect(Point offset, float sigma, Color color)
```

### Parameters

`offset` Point

The shadow offset, in pixels.

`sigma` float

The Gaussian blur sigma, in pixels; zero draws a hard shadow.

`color` Color

The shadow colour.

# Properties

## Color

Gets the shadow colour.

```
public Color Color { get; }
```

### Property Value

Color

## Offset

Gets the shadow offset, in pixels.

```
public Point Offset { get; }
```

### Property Value

Point

## Sigma

Gets the Gaussian blur sigma, in pixels.

```
public float Sigma { get; }
```

## Property Value

float

# Class WebGPUEnvironment

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.WebGPU.dll

Provides explicit support probes for the library-managed WebGPU environment. Use this type when you want to check availability or compute pipeline support before constructing WebGPU objects.

```
public static class WebGPUEnvironment
```

**Inheritance**

object ← WebGPUEnvironment

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

# Properties

## Options

Gets or sets the options used when the library-managed WebGPU environment is initialized.

```
public static WebGPUEnvironmentOptions Options { get; set; }
```

## Property Value

WebGPUEnvironmentOptions

## Remarks

Assign this before constructing WebGPU objects or calling support probes. The shared WebGPU runtime reads the current options during first initialization; changing them later does not reconfigure an existing device.

## UncapturedError

Gets or sets the callback invoked when the native WebGPU runtime reports an uncaptured device error.

```
public static Action<WebGPUErrorType, string>? UncapturedError { get; set; }
```

### Property Value

Action<WebGPUErrorType, string>

### Remarks

The callback can be raised from a native WebGPU callback thread. Keep handlers short and non-blocking. Exceptions thrown by the handler are not propagated back through the native callback.

# Methods

## EnableNativeLogging(Action<string>?)

Routes the native wgpu-native log stream (including the validation errors that precede a lost device) to a managed sink. Unlike UncapturedError, this surfaces errors that abort the process through the binding's submit panic before the uncaptured callback runs.

```
public static void EnableNativeLogging(Action<string>? sink)
```

### Parameters

`sink` Action<string>

The managed sink invoked for each native log line, or null to disable.

### Remarks

The sink can be invoked from native wgpu threads; keep it short and non-blocking. Passing null only mutes the managed sink; a previously registered native callback stays installed and simply drops its messages. The first call that installs a sink also fixes the native log level at the warning level; that level is not lowered again by later calls.

# ProbeAvailability()

Probes whether the required wgpu-native entry points and the library-managed WebGPU device and queue are available.

```
public static WebGPUEnvironmentError ProbeAvailability()
```

## Returns

WebGPUEnvironmentError

Success when the required wgpu-native entry points and the library-managed WebGPU device and queue are available; otherwise, the stable failure code describing why the probe failed.

## Remarks

Returns DxcUnavailable on Windows when the packaged DirectX Shader Compiler runtime is unavailable. Returns WgpuExtensionUnavailable when the core WebGPU API loads but a wgpu-native extension entry point required by the backend is unavailable.

# ProbeComputePipelineSupport()

Probes whether the library-managed WebGPU device can create a trivial compute pipeline.

```
public static WebGPUEnvironmentError ProbeComputePipelineSupport()
```

## Returns

WebGPUEnvironmentError

Success when compute pipeline creation succeeds; otherwise, the stable failure code describing why the probe failed.

# Enum WebGPUEnvironmentError

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.WebGPU.dll

Error codes returned by WebGPUEnvironment probe methods.

```
public enum WebGPUEnvironmentError
```

# Fields

`AdapterRequestFailed = 4`

The runtime failed to acquire a WebGPU adapter.

`AdapterRequestTimedOut = 3`

The adapter request callback did not complete before the timeout expired.

`ApiInitializationFailed = 1`

The shared WebGPU API loader could not be initialized.

`ComputePipelineCreationFailed = 10`

The isolated compute-pipeline probe ran to completion and reported that compute pipeline creation failed.

`ComputePipelineProbeProcessFailed = 11`

The isolated compute-pipeline probe process terminated before it could report a normal result.

`DeviceAcquisitionFailed = 9`

The runtime could not provision the shared device/queue pair for an unspecified reason.

`DeviceRequestFailed = 7`

The runtime failed to acquire a WebGPU device.

`DeviceRequestTimedOut = 6`

The device request callback did not complete before the timeout expired.

`DxcUnavailable = 13`

The DirectX Shader Compiler required by the Windows WebGPU backend is unavailable.

`InstanceCreationFailed = 2`

The runtime could not create a WebGPU instance.

`QueueAcquisitionFailed = 8`

The runtime acquired a device but could not retrieve its default queue.

`SoftwareAdapterUnsupported = 5`

The only available adapter is a software rasterizer. Software adapters are not supported by this backend; the CPU drawing backend serves GPU-less environments instead.

`Success = 0`

The probe succeeded.

`WgpuExtensionUnavailable = 12`

The required WGPU extension is unavailable.

# Remarks

Success is the only successful result. All other values describe one stable failure case that callers can branch on without parsing diagnostic strings.

# Class WebGPUEnvironmentOptions

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.WebGPU.dll

Options for the library-managed WebGPU environment.

```
public sealed class WebGPUEnvironmentOptions
```

**Inheritance**

object ← WebGPUEnvironmentOptions

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Remarks

Assign these options before constructing WebGPU objects or calling support probes. The shared WebGPU runtime reads the current options during first initialization; changing them later does not reconfigure an existing device.

# Properties

## Default

Gets the default WebGPU environment options.

```
public static WebGPUEnvironmentOptions Default { get; }
```

## Property Value

WebGPUEnvironmentOptions

## PowerPreference

Gets or sets the adapter power preference requested for the library-managed device.

```
public WebGPUPowerPreference PowerPreference { get; set; }
```

## Property Value

WebGPUPowerPreference

# Enum WebGPUErrorType

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.WebGPU.dll

Error categories reported by the native WebGPU runtime.

```
public enum WebGPUErrorType
```

# Fields

`DeviceLost = 5`

The WebGPU device was lost.

`Internal = 3`

The native runtime reported an internal implementation failure.

`NoError = 0`

No error was reported.

`OutOfMemory = 2`

The native runtime could not allocate the requested GPU resource.

`Unknown = 4`

The native runtime reported an uncategorized failure.

`Validation = 1`

The native runtime rejected a command or resource because it violated WebGPU validation rules.

# Class WebGPUExternalSurface

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.WebGPU.dll

A WebGPU rendering surface bound to an externally-owned native host. Unlike WebGPUWindow, this type does not own a platform window; the host application owns the UI object, its lifecycle, and the drawable size notifications forwarded through Resize(Size).

```
public sealed class WebGPUExternalSurface : IDisposable
```

### Inheritance

object ← WebGPUExternalSurface

### Implements

IDisposable

### Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## WebGPUExternalSurface(Configuration, WebGPUSurfaceHost, Size, WebGPUExternalSurfaceOptions)

Initializes a new instance of the WebGPUExternalSurface class.

```
public WebGPUExternalSurface(Configuration configuration, WebGPUSurfaceHost host, Size
framebufferSize, WebGPUExternalSurfaceOptions options)
```

### Parameters

`configuration` Configuration

The configuration instance to bind to the created backend.

`host` WebGPUSurfaceHost

The native surface host that the WebGPU surface will attach to.

`framebufferSize` Size

The initial framebuffer size in pixels.

`options` WebGPUExternalSurfaceOptions

The external surface options.

## WebGPUExternalSurface(WebGPUSurfaceHost, Size)

Initializes a new instance of the WebGPUExternalSurface class.

```
public WebGPUExternalSurface(WebGPUSurfaceHost host, Size framebufferSize)
```

### Parameters

`host` WebGPUSurfaceHost

The native surface host that the WebGPU surface will attach to.

`framebufferSize` Size

The initial framebuffer size in pixels.

## WebGPUExternalSurface(WebGPUSurfaceHost, Size, WebGPUExternalSurfaceOptions)

Initializes a new instance of the WebGPUExternalSurface class.

```
public WebGPUExternalSurface(WebGPUSurfaceHost host, Size framebufferSize,
WebGPUExternalSurfaceOptions options)
```

### Parameters

`host` WebGPUSurfaceHost

The native surface host that the WebGPU surface will attach to.

`framebufferSize` Size

The initial framebuffer size in pixels.

`options` WebGPUExternalSurfaceOptions

The external surface options.

# Properties

## DeviceContext

Gets the WebGPU device context used to render this surface.

```
public WebGPUDeviceContext DeviceContext { get; }
```

### Property Value

WebGPUDeviceContext

### Remarks

The owning surface session controls the returned context's lifetime.

# Methods

## Dispose()

Releases the WebGPU surface stack. The native host handles supplied through WebGPUSurfaceHost remain owned by the caller and are not released.

```
public void Dispose()
```

## Resize(Size)

Notifies the external surface that the drawable framebuffer has resized and reconfigures the swapchain when the size changes. Zero-area sizes (minimize, mid-layout) pause frame acquisition while the last valid native configuration is retained.

```
public void Resize(Size framebufferSize)
```

### Parameters

`framebufferSize` Size

The new framebuffer size in pixels.

## TryAcquireFrame(out WebGPUSurfaceFrame?)

Tries to acquire the next drawable frame using default drawing options.

```
public bool TryAcquireFrame(out WebGPUSurfaceFrame? frame)
```

### Parameters

`frame` WebGPUSurfaceFrame

Receives the acquired frame on success.

### Returns

bool

true when a frame is available; otherwise false.

### Remarks

A false result means no drawable frame is available right now, for example because the surface was lost, outdated, timed out, or has a zero-sized framebuffer. Dispose the returned frame when you are done with it to present it and release its per-frame resources.

## TryAcquireFrame(DrawingOptions, out WebGPUSurfaceFrame?)

Tries to acquire the next drawable frame.

```
public bool TryAcquireFrame(DrawingOptions options, out WebGPUSurfaceFrame? frame)
```

## Parameters

`options` DrawingOptions

The drawing options for the acquired frame.

`frame` WebGPUSurfaceFrame

Receives the acquired frame on success.

## Returns

bool

true when a frame is available; otherwise false.

## Remarks

A false result means no drawable frame is available right now, for example because the surface was lost, outdated, timed out, or has a zero-sized framebuffer. Dispose the returned frame when you are done with it to present it and release its per-frame resources.

# Class WebGPUExternalSurfaceOptions

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.WebGPU.dll

Options for creating a WebGPUExternalSurface.

```
public sealed class WebGPUExternalSurfaceOptions
```

**Inheritance**

object ← WebGPUExternalSurfaceOptions

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Remarks

Only values that are meaningful for an externally-owned surface are exposed here. Lifecycle, title, position, border, and state belong to the host application and its UI framework.

# Properties

## AlphaMode

Gets or sets how the native compositor interprets the surface alpha channel.

```
public WebGPUCompositeAlphaMode AlphaMode { get; set; }
```

### Property Value

WebGPUCompositeAlphaMode

### Remarks

The requested mode is used when available. Otherwise, a compatible mode is selected automatically.

## Format

Gets or sets the swapchain texture format used by acquired frames.

```
public WebGPUTextureFormat Format { get; set; }
```

### Property Value

WebGPUTextureFormat

### Remarks

The requested format is used when available. Otherwise, a compatible format is selected automatically.

## PresentMode

Gets or sets how completed frames are queued for presentation to the display.

```
public WebGPUPresentMode PresentMode { get; set; }
```

### Property Value

WebGPUPresentMode

### Remarks

When the requested mode is unavailable, Fifo is used.

# Class WebGPUGaussianBlurLayerEffect

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.WebGPU.dll

Provides a shader-accelerated counterpart to BlurLayerEffect.

```
public sealed class WebGPUGaussianBlurLayerEffect : WebGPUShaderLayerEffect,
IWebGPUShaderEffect
```

**Inheritance**

object ← LayerEffect ← WebGPUShaderLayerEffect ← WebGPUGaussianBlurLayerEffect

**Implements**

IWebGPUShaderEffect

**Inherited Members**

LayerEffect.Reach , LayerEffect.IsPassThrough , LayerEffect.WriteBackOptions , LayerEffect.WriteBackOffset , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Remarks

This effect can be used with both WebGPU and CPU drawing backends. WebGPU performs the Gaussian blur with separable shader passes; a CPU backend applies the equivalent BlurLayer Effect.

# Constructors

## WebGPUGaussianBlurLayerEffect(float)

Initializes a new instance of the WebGPUGaussianBlurLayerEffect class.

```
public WebGPUGaussianBlurLayerEffect(float sigma)
```

### Parameters

`sigma` float

The Gaussian blur sigma, in pixels; zero leaves the content unchanged.

### Exceptions

ArgumentOutOfRangeException

Thrown when `sigma` is negative.

## Properties

### Sigma

Gets the Gaussian blur sigma, in pixels.

```
public float Sigma { get; }
```

### Property Value

float

# Class WebGPUGlowLayerEffect

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.WebGPU.dll

Provides a shader-accelerated counterpart to GlowLayerEffect.

```
public sealed class WebGPUGlowLayerEffect : WebGPUShaderLayerEffect, IWebGPUShaderEffect
```

**Inheritance**

object ← LayerEffect ← WebGPUShaderLayerEffect ← WebGPUGlowLayerEffect

**Implements**

IWebGPUShaderEffect

**Inherited Members**

LayerEffect.Reach , LayerEffect.IsPassThrough , LayerEffect.WriteBackOptions , LayerEffect.WriteBackOffset , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Remarks

This effect can be used with both WebGPU and CPU drawing backends. WebGPU creates the glow with shader passes; a CPU backend applies the equivalent GlowLayerEffect.

# Constructors

## WebGPUGlowLayerEffect(float, Color)

Initializes a new instance of the WebGPUGlowLayerEffect class.

```
public WebGPUGlowLayerEffect(float sigma, Color color)
```

## Parameters

`sigma` float

The Gaussian blur sigma, in pixels.

`color` Color

The glow colour.

# Properties

## Color

Gets the glow colour.

```
public Color Color { get; }
```

## Property Value

Color

## Sigma

Gets the Gaussian blur sigma, in pixels.

```
public float Sigma { get; }
```

## Property Value

float

# Class WebGPUInnerShadowLayerEffect

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.WebGPU.dll

Provides a shader-accelerated counterpart to InnerShadowLayerEffect.

```
public sealed class WebGPUInnerShadowLayerEffect : WebGPUShaderLayerEffect,
IWebGPUShaderEffect
```

**Inheritance**

object ← LayerEffect ← WebGPUShaderLayerEffect ← WebGPUInnerShadowLayerEffect

**Implements**

IWebGPUShaderEffect

**Inherited Members**

LayerEffect.Reach , LayerEffect.IsPassThrough , LayerEffect.WriteBackOptions , LayerEffect.WriteBackOffset , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Remarks

This effect can be used with both WebGPU and CPU drawing backends. WebGPU creates the inner shadow with shader passes; a CPU backend applies the equivalent InnerShadowLayerEffect.

# Constructors

## WebGPUInnerShadowLayerEffect(Point, float, Color)

Initializes a new instance of the WebGPUInnerShadowLayerEffect class.

```
public WebGPUInnerShadowLayerEffect(Point offset, float sigma, Color color)
```

### Parameters

`offset` Point

The shadow offset, in pixels.

`sigma` float

The Gaussian blur sigma, in pixels; zero draws a hard shadow.

`color` Color

The shadow colour.

# Properties

## Color

Gets the shadow colour.

```
public Color Color { get; }
```

### Property Value

Color

## Offset

Gets the shadow offset, in pixels.

```
public Point Offset { get; }
```

### Property Value

Point

## Sigma

Gets the Gaussian blur sigma, in pixels.

```
public float Sigma { get; }
```

## Property Value

float

# Enum WebGPUPowerPreference

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.WebGPU.dll

Describes the adapter power preference requested for the library-managed WebGPU device.

```
public enum WebGPUPowerPreference
```

# Fields

`Default = 0`

Uses the native WebGPU runtime default.

`HighPerformance = 2`

Prefers a high-performance adapter when the platform provides one.

`LowPower = 1`

Prefers a lower-power adapter when the platform provides one.

# Enum WebGPUPresentMode

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.WebGPU.dll

Presentation mode used by WebGPUWindow.

```
public enum WebGPUPresentMode
```

# Fields

`Fifo = 0`

Presents frames in first-in, first-out order and waits for the display's vertical refresh.

This is the usual v-synced mode. It avoids tearing and is the safest default for general apps, but frames can spend longer waiting in the presentation queue than in lower-latency modes.

`Immediate = 1`

Presents the newest completed frame as soon as possible without waiting for the next vertical refresh.

This minimizes presentation queueing latency, but it can show partial frame updates on screen, which most people experience as tearing.

`Mailbox = 2`

Queues frames like v-synced presentation, but keeps only the most recent queued frame when the display is not ready yet.

Use this when you want lower latency than FIFO without the visible tearing of immediate presentation. Older queued frames may be dropped in favor of newer ones, and support depends on the backend and platform.

# Remarks

Presentation mode controls how completed frames wait for the display. The choice affects tearing, latency, and whether older queued frames can be skipped in favor of newer ones.

# Class WebGPURenderTarget

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.WebGPU.dll

An offscreen WebGPU render target.

```
public sealed class WebGPURenderTarget : IDisposable
```

**Inheritance**

object ← WebGPURenderTarget

**Implements**

IDisposable

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Remarks

The constructors on this type allocate a target on the shared process WebGPU device.

# Constructors

## WebGPURenderTarget(Configuration, WebGPUTextureFormat, PixelAlphaRepresentation, int, int)

Initializes a new instance of the WebGPURenderTarget class using the shared process-level device.

```
public WebGPURenderTarget(Configuration configuration, WebGPUTextureFormat format,
PixelAlphaRepresentation alphaRepresentation, int width, int height)
```

### Parameters

`configuration` Configuration

The configuration instance to bind to the created backend.

`format` WebGPUTextureFormat

The target texture format.

`alphaRepresentation` PixelAlphaRepresentation

The alpha representation stored by the target.

`width` int

The target width in pixels.

`height` int

The target height in pixels.

## WebGPURenderTarget(Configuration, WebGPUTextureFormat, int, int)

Initializes a new instance of the WebGPURenderTarget class using the shared process-level device.

```
public WebGPURenderTarget(Configuration configuration, WebGPUTextureFormat format, int
width, int height)
```

## Parameters

`configuration` Configuration

The configuration instance to bind to the created backend.

`format` WebGPUTextureFormat

The target texture format.

`width` int

The target width in pixels.

`height` int

The target height in pixels.

## WebGPURenderTarget(Configuration, int, int)

Initializes a new instance of the WebGPURenderTarget class using the shared process-level device and default RGBA8 format.

```
public WebGPURenderTarget(Configuration configuration, int width, int height)
```

### Parameters

`configuration` Configuration

The configuration instance to bind to the created backend.

`width` int

The target width in pixels.

`height` int

The target height in pixels.

## WebGPURenderTarget(WebGPUTextureFormat, PixelAlphaRepresentation, int, int)

Initializes a new instance of the WebGPURenderTarget class using the shared process-level device.

```
public WebGPURenderTarget(WebGPUTextureFormat format, PixelAlphaRepresentation
alphaRepresentation, int width, int height)
```

### Parameters

`format` WebGPUTextureFormat

The target texture format.

`alphaRepresentation` PixelAlphaRepresentation

The alpha representation stored by the target.

`width` int

The target width in pixels.

`height` int

The target height in pixels.

## WebGPURenderTarget(WebGPUTextureFormat, int, int)

Initializes a new instance of the WebGPURenderTarget class using the shared process-level device.

```
public WebGPURenderTarget(WebGPUTextureFormat format, int width, int height)
```

### Parameters

`format` WebGPUTextureFormat

The target texture format.

`width` int

The target width in pixels.

`height` int

The target height in pixels.

## WebGPURenderTarget(int, int)

Initializes a new instance of the WebGPURenderTarget class using the shared process-level device and default RGBA8 format.

```
public WebGPURenderTarget(int width, int height)
```

### Parameters

`width` int

The target width in pixels.

`height` int

The target height in pixels.

# Properties

## AlphaRepresentation

Gets the alpha representation stored by the target.

```
public PixelAlphaRepresentation AlphaRepresentation { get; }
```

### Property Value

PixelAlphaRepresentation

## Bounds

Gets the target bounds in pixels.

```
public Rectangle Bounds { get; }
```

### Property Value

Rectangle

## DeviceContext

Gets the device context used by this render target.

```
public WebGPUDeviceContext DeviceContext { get; }
```

## Property Value

WebGPUDeviceContext

# Format

Gets the allocated texture format.

```
public WebGPUTextureFormat Format { get; }
```

## Property Value

WebGPUTextureFormat

# Height

Gets the target height in pixels.

```
public int Height { get; }
```

## Property Value

int

# Width

Gets the target width in pixels.

```
public int Width { get; }
```

## Property Value

int

# Methods

## CreateCanvas()

Creates a drawing canvas over this render target.

```
public DrawingCanvas CreateCanvas()
```

### Returns

DrawingCanvas

A drawing canvas targeting this render target.

## CreateCanvas(DrawingOptions)

Creates a drawing canvas over this render target.

```
public DrawingCanvas CreateCanvas(DrawingOptions options)
```

### Parameters

`options` DrawingOptions

The initial drawing options.

### Returns

DrawingCanvas

A drawing canvas targeting this render target.

## CreateCanvas(DrawingOptions, DrawingTextCache)

Creates a drawing canvas over this render target.

```
public DrawingCanvas CreateCanvas(DrawingOptions options, DrawingTextCache textCache)
```

## Parameters

`options` DrawingOptions

The initial drawing options.

`textCache` DrawingTextCache

The text drawing cache used by this canvas instance.

## Returns

DrawingCanvas

A drawing canvas targeting this render target.

# CreateRenderTarget(int, int)

Creates an empty render target with the same texture format as this target.

```
public WebGPURenderTarget CreateRenderTarget(int width, int height)
```

## Parameters

`width` int

The target width in pixels.

`height` int

The target height in pixels.

## Returns

WebGPURenderTarget

The created render target.

## Remarks

The created target does not contain a copy of this target's pixels. This target must remain undisposed while the created target is in use.

# Dispose()

Releases the owned texture view and texture, and the device context when this target created it. Targets created from a shared context leave that context untouched.

```
public void Dispose()
```

# ReadbackImage()

Reads the current GPU texture contents back into a new CPU image whose pixel type matches Format and AlphaRepresentation.

```
public Image ReadbackImage()
```

## Returns

Image

The readback image whose pixel type has the target's channel layout, numeric encoding, and alpha representation.

# ReadbackImage<TPixel>()

Reads the current GPU texture contents back into a new CPU image.

```
public Image<TPixel> ReadbackImage<TPixel>() where TPixel : unmanaged, IPixel<TPixel>
```

## Returns

Image<TPixel>

The readback image.

## Type Parameters

`TPixel`

The destination image pixel format. Must match Format and AlphaRepresentation; the backend read throws NotSupportedException on a mismatch. Use ReadbackImage() to dispatch by format.

# ReadbackInto<TPixel>(Buffer2DRegion<TPixel>)

Reads the current GPU texture contents back into an existing CPU buffer region.

```
public void ReadbackInto<TPixel>(Buffer2DRegion<TPixel> destination) where TPixel :
unmanaged, IPixel<TPixel>
```

## Parameters

`destination` Buffer2DRegion<TPixel>

The destination buffer region that receives the readback pixels.

## Type Parameters

`TPixel`

The destination image pixel format. Must match Format and AlphaRepresentation; the backend read throws NotSupportedException on a mismatch.

# Class WebGPUShaderCompilationException

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.WebGPU.dll

The exception thrown when a WebGPU layer-effect shader cannot be compiled.

```
public sealed class WebGPUShaderCompilationException : Exception, ISerializable
```

### Inheritance

object ← Exception ← WebGPUShaderCompilationException

### Implements

ISerializable

### Inherited Members

Exception.GetBaseException() , Exception.GetType() , Exception.ToString() , Exception.Data , Exception.HelpLink , Exception.HResult , Exception.InnerException , Exception.Message , Exception.Source , Exception.StackTrace , Exception.TargetSite , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.ReferenceEquals(object, object)

# Properties

## Diagnostics

Gets the compiler diagnostics associated with the failure.

```
public IReadOnlyList<WebGPUShaderDiagnostic> Diagnostics { get; }
```

## Property Value

IReadOnlyList<WebGPUShaderDiagnostic>

# Struct WebGPUShaderDiagnostic

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.WebGPU.dll

Describes one message produced while compiling a WebGPU layer-effect shader.

```
public readonly struct WebGPUShaderDiagnostic
```

**Inherited Members**
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

# Constructors

## WebGPUShaderDiagnostic(WebGPUShaderDiagnosticSeverity, string, int, int)

Initializes a new instance of the WebGPUShaderDiagnostic struct.

```
public WebGPUShaderDiagnostic(WebGPUShaderDiagnosticSeverity severity, string message, int
line, int column)
```

### Parameters

`severity` WebGPUShaderDiagnosticSeverity

The diagnostic severity.

`message` string

The compiler message.

`line` int

The one-based user-source line, or zero when the message belongs to generated framework code.

`column` int

The one-based source column, or zero when the compiler does not provide one.

# Properties

## Column

Gets the one-based source column, or zero when the compiler does not provide one.

```
public int Column { get; }
```

### Property Value

int

## Line

Gets the one-based user-source line, or zero when the message belongs to generated framework code.

```
public int Line { get; }
```

### Property Value

int

## Message

Gets the compiler message.

```
public string Message { get; }
```

### Property Value

string

## Severity

Gets the diagnostic severity.

```
public WebGPUShaderDiagnosticSeverity Severity { get; }
```

## Property Value

WebGPUShaderDiagnosticSeverity

# Enum WebGPUShaderDiagnosticSeverity

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.WebGPU.dll

Identifies the severity of a WebGPU shader compiler message.

```
public enum WebGPUShaderDiagnosticSeverity
```

# Fields

`Error = 2`

A compiler error that prevents execution.

`Information = 0`

An informational compiler message.

`Warning = 1`

A compiler warning.

# Class WebGPUShaderLayerEffect

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.WebGPU.dll

Defines a layer effect with complete WGSL shader passes and an equivalent ImageSharp CPU fallback.

```
public abstract class WebGPUShaderLayerEffect : LayerEffect, IWebGPUShaderEffect
```

**Inheritance**

object ← LayerEffect ← WebGPUShaderLayerEffect

**Implements**

IWebGPUShaderEffect

**Derived**

WebGPUColorMatrixLayerEffect, WebGPUDropShadowLayerEffect, WebGPUGaussianBlurLayerEffect, WebGPUGlowLayerEffect, WebGPUInnerShadowLayerEffect

**Inherited Members**

LayerEffect.Reach , LayerEffect.IsPassThrough , LayerEffect.WriteBackOptions , LayerEffect.WriteBackOffset , object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

# Remarks

Instances work with both drawing backends. WebGPU executes the configured WGSL passes, while CPU rendering executes the required fallback effect. Each pass receives the preceding image as its input.

Shader source must define `fn layer_effect(position: vec2<f32>) -> vec4<f32>`. Position uses effect-local pixel-center coordinates. The return value is associated-alpha RGBA in `[0, 1]`. ImageSharp supplies `layer_load`, `layer_load_unassociated`, `layer_sample`, and the declared `imagesharp_uniforms` fields. Each source defines one complete shader pass.

# Constructors

# WebGPUShaderLayerEffect(LayerEffect, string, WebGPUShaderUniformLayout)

Initializes a new instance of the WebGPUShaderLayerEffect class.

```
protected WebGPUShaderLayerEffect(LayerEffect fallbackEffect, string shaderSource,
WebGPUShaderUniformLayout uniformLayout)
```

## Parameters

`fallbackEffect` LayerEffect

The equivalent effect used by CPU rendering.

`shaderSource` string

The complete WGSL source for each pass added by this effect.

`uniformLayout` WebGPUShaderUniformLayout

The named uniform values available to the shader.

# WebGPUShaderLayerEffect(string, WebGPUShaderUniformLayout, Action<IImageProcessingContext>)

Initializes a new instance of the WebGPUShaderLayerEffect class.

```
protected WebGPUShaderLayerEffect(string shaderSource, WebGPUShaderUniformLayout
uniformLayout, Action<IImageProcessingContext> fallback)
```

## Parameters

`shaderSource` string

The complete WGSL source for each pass added by this effect.

`uniformLayout` WebGPUShaderUniformLayout

The named uniform values available to the shader.

`fallback` Action<IImageProcessingContext>

The equivalent operation used by CPU rendering.

# Methods

## AddShaderPass(BorderWrappingMode, BorderWrappingMode, Action<WebGPUShaderUniformBuilder>)

Configures and adds an invocation whose filtered samples use the supplied border modes.

```
protected void AddShaderPass(BorderWrappingMode xBorderMode, BorderWrappingMode yBorderMode,
Action<WebGPUShaderUniformBuilder> configureUniforms)
```

### Parameters

`xBorderMode` BorderWrappingMode

The wrapping mode applied beyond the horizontal input borders.

`yBorderMode` BorderWrappingMode

The wrapping mode applied beyond the vertical input borders.

`configureUniforms` Action<WebGPUShaderUniformBuilder>

The action that assigns this pass's named uniform values.

## AddShaderPass(Action<WebGPUShaderUniformBuilder>)

Configures and adds an invocation of the complete shader source to this effect's ordered pass sequence.

```
protected void AddShaderPass(Action<WebGPUShaderUniformBuilder> configureUniforms)
```

### Parameters

`configureUniforms` Action<WebGPUShaderUniformBuilder>

The action that assigns this pass's named uniform values.

# Struct WebGPUShaderUniform

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.WebGPU.dll

Describes one named value or fixed-size array supplied to a WebGPU layer-effect shader.

```
public readonly struct WebGPUShaderUniform
```

**Inherited Members**
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

# Constructors

## WebGPUShaderUniform(string, WebGPUShaderUniformType, int)

Initializes a new instance of the WebGPUShaderUniform struct.

```
public WebGPUShaderUniform(string name, WebGPUShaderUniformType type, int elementCount)
```

### Parameters

`name` string

The WGSL member name used to access the value.

`type` WebGPUShaderUniformType

The value type.

`elementCount` int

The fixed number of elements. Use one for a scalar, vector, or matrix value.

# Properties

## ElementCount

Gets the fixed number of elements.

```
public int ElementCount { get; }
```

### Property Value

int

## Name

Gets the WGSL member name used to access the value.

```
public string Name { get; }
```

### Property Value

string

## Type

Gets the value type.

```
public WebGPUShaderUniformType Type { get; }
```

### Property Value

WebGPUShaderUniformType

# Class WebGPUShaderUniformBuilder

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.WebGPU.dll

Builds a set of named values for a WebGPUShaderUniformLayout.

```
public sealed class WebGPUShaderUniformBuilder
```

**Inheritance**

object ← WebGPUShaderUniformBuilder

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Methods

## SetFloat32(string, float)

Sets a named 32-bit floating-point value.

```
public void SetFloat32(string name, float value)
```

### Parameters

`name` string

The declared member name.

`value` float

The value to set.

## SetFloat32Array(string, ReadOnlySpan<float>)

Sets a named fixed-size array of 32-bit floating-point values.

```
public void SetFloat32Array(string name, ReadOnlySpan<float> values)
```

## Parameters

`name` string

The declared member name.

`values` ReadOnlySpan<float>

The values to set.

# SetInt32(string, int)

Sets a named signed 32-bit integer value.

```
public void SetInt32(string name, int value)
```

## Parameters

`name` string

The declared member name.

`value` int

The value to set.

# SetInt32Array(string, ReadOnlySpan<int>)

Sets a named fixed-size array of signed 32-bit integer values.

```
public void SetInt32Array(string name, ReadOnlySpan<int> values)
```

## Parameters

`name` string

The declared member name.

`values` ReadOnlySpan<int>

The values to set.

## SetMatrix4×4(string, Matrix4×4)

Sets a named four-by-four floating-point matrix.

```
public void SetMatrix4x4(string name, Matrix4x4 value)
```

### Parameters

`name` string

The declared member name.

`value` Matrix4×4

The value to set.

## SetMatrix4×4Array(string, ReadOnlySpan<Matrix4×4>)

Sets a named fixed-size array of four-by-four floating-point matrices.

```
public void SetMatrix4x4Array(string name, ReadOnlySpan<Matrix4x4> values)
```

### Parameters

`name` string

The declared member name.

`values` ReadOnlySpan<Matrix4×4>

The values to set.

## SetUInt32(string, uint)

Sets a named unsigned 32-bit integer value.

```
public void SetUInt32(string name, uint value)
```

### Parameters

`name` string

The declared member name.

`value` uint

The value to set.

## SetUInt32Array(string, ReadOnlySpan<uint>)

Sets a named fixed-size array of unsigned 32-bit integer values.

```
public void SetUInt32Array(string name, ReadOnlySpan<uint> values)
```

### Parameters

`name` string

The declared member name.

`values` ReadOnlySpan<uint>

The values to set.

## SetVector2(string, Vector2)

Sets a named two-component floating-point vector.

```
public void SetVector2(string name, Vector2 value)
```

## Parameters

`name` string

The declared member name.

`value` Vector2

The value to set.

# SetVector2Array(string, ReadOnlySpan<Vector2>)

Sets a named fixed-size array of two-component floating-point vectors.

```
public void SetVector2Array(string name, ReadOnlySpan<Vector2> values)
```

## Parameters

`name` string

The declared member name.

`values` ReadOnlySpan<Vector2>

The values to set.

# SetVector3(string, Vector3)

Sets a named three-component floating-point vector.

```
public void SetVector3(string name, Vector3 value)
```

## Parameters

`name` string

The declared member name.

`value` Vector3

The value to set.

## SetVector3Array(string, ReadOnlySpan<Vector3>)

Sets a named fixed-size array of three-component floating-point vectors.

```
public void SetVector3Array(string name, ReadOnlySpan<Vector3> values)
```

### Parameters

`name` string

The declared member name.

`values` ReadOnlySpan<Vector3>

The values to set.

## SetVector4(string, Vector4)

Sets a named four-component floating-point vector.

```
public void SetVector4(string name, Vector4 value)
```

### Parameters

`name` string

The declared member name.

`value` Vector4

The value to set.

## SetVector4Array(string, ReadOnlySpan<Vector4>)

Sets a named fixed-size array of four-component floating-point vectors.

```
public void SetVector4Array(string name, ReadOnlySpan<Vector4> values)
```

## Parameters

`name` string

The declared member name.

`values` ReadOnlySpan<Vector4>

The values to set.

# Class WebGPUShaderUniformLayout

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.WebGPU.dll

Describes the names, types, and fixed array lengths of values supplied to a WebGPU layer-effect shader.

```
public sealed class WebGPUShaderUniformLayout
```

**Inheritance**

object ← WebGPUShaderUniformLayout

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Constructors

## WebGPUShaderUniformLayout(ReadOnlySpan<WebGPUShaderUniform>)

Initializes a new instance of the WebGPUShaderUniformLayout class.

```
public WebGPUShaderUniformLayout(ReadOnlySpan<WebGPUShaderUniform> uniforms)
```

### Parameters

`uniforms` ReadOnlySpan<WebGPUShaderUniform>

The ordered uniform declarations.

# Fields

## MaximumByteLength

The maximum packed byte length accepted for a shader uniform layout.

```
public const int MaximumByteLength = 65536
```

### Field Value

int

# Properties

## Uniforms

Gets the ordered uniform declarations.

```
public ReadOnlySpan<WebGPUShaderUniform> Uniforms { get; }
```

### Property Value

ReadOnlySpan<WebGPUShaderUniform>

# Enum WebGPUShaderUniformType

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.WebGPU.dll

Identifies a value type that can be supplied to a WebGPU layer-effect shader.

```
public enum WebGPUShaderUniformType
```

# Fields

`Float32 = 0`

A 32-bit floating-point value.

`Int32 = 1`

A signed 32-bit integer value.

`Matrix4x4 = 6`

A four-by-four 32-bit floating-point matrix.

`UInt32 = 2`

An unsigned 32-bit integer value.

`Vector2 = 3`

A two-component 32-bit floating-point vector.

`Vector3 = 4`

A three-component 32-bit floating-point vector.

`Vector4 = 5`

A four-component 32-bit floating-point vector.

# Class WebGPUShaderUniforms

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.WebGPU.dll

Contains an immutable snapshot of values supplied to a WebGPU layer-effect shader.

```
public sealed class WebGPUShaderUniforms
```

### Inheritance

object ← WebGPUShaderUniforms

### Inherited Members

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Properties

## Layout

Gets the layout that describes the values.

```
public WebGPUShaderUniformLayout Layout { get; }
```

## Property Value

WebGPUShaderUniformLayout

# Class WebGPUSurfaceFrame

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.WebGPU.dll

A single acquired drawable frame returned by a WebGPU surface. Use the Canvas to draw the frame contents, then dispose the frame to show it on screen.

```
public sealed class WebGPUSurfaceFrame : IDisposable
```

**Inheritance**

object ← WebGPUSurfaceFrame

**Implements**

IDisposable

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Remarks

Disposal submits the recorded canvas work, presents the surface, releases the per-frame texture and view, and signals the owning surface so the next `TryAcquireFrame` call can succeed. Only one frame can be outstanding per surface at a time.

# Properties

## Canvas

Gets the drawing canvas for the acquired frame.

```
public DrawingCanvas Canvas { get; }
```

## Property Value

DrawingCanvas

# Methods

## CreateRenderTarget(int, int)

Creates an empty render target with the same texture format as this frame.

```
public WebGPURenderTarget CreateRenderTarget(int width, int height)
```

### Parameters

`width` int

The target width in pixels.

`height` int

The target height in pixels.

### Returns

WebGPURenderTarget

The created render target.

### Remarks

The created target does not contain a copy of this frame's pixels.

## Dispose()

Disposes the frame, rendering and presenting it, then releasing the per-frame WebGPU resources.

```
public void Dispose()
```

# Struct WebGPUSurfaceHost

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.WebGPU.dll

Describes the externally-owned native drawable that a WebGPUExternalSurface should attach to. Use the factory method that matches the host toolkit or platform that owns the drawable surface.

```
public readonly struct WebGPUSurfaceHost
```

**Inherited Members**
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

# Remarks

Construct via the platform-specific factory methods. The caller retains ownership of the underlying handles; the external surface never releases them. GLFW and SDL handles are translated to their underlying platform source during surface creation. The remaining factories correspond directly to surface source descriptors accepted by the bundled wgpu-native API.

# Methods

## Android(nint)

Creates a host descriptor for an Android native window.

```
public static WebGPUSurfaceHost Android(nint aNativeWindow)
```

### Parameters

`aNativeWindow` nint

The Android native window pointer (`ANativeWindow*`).

### Returns

WebGPUSurfaceHost

An Android host descriptor.

## Cocoa(nint)

Creates a host descriptor for a Cocoa window.

```
public static WebGPUSurfaceHost Cocoa(nint nsWindow)
```

### Parameters

`nsWindow` nint

The Cocoa window pointer (`NSWindow*`).

### Returns

WebGPUSurfaceHost

A Cocoa host descriptor.

## Glfw(nint)

Creates a host descriptor for a GLFW-owned window.

```
public static WebGPUSurfaceHost Glfw(nint glfwWindow)
```

### Parameters

`glfwWindow` nint

The GLFW window pointer (`GLFWwindow*`).

### Returns

WebGPUSurfaceHost

A GLFW host descriptor.

## MetalLayer(nint)

Creates a host descriptor for a Core Animation metal layer.

```
public static WebGPUSurfaceHost MetalLayer(nint metalLayer)
```

### Parameters

`metalLayer` nint

The Core Animation metal layer pointer (`CAMetalLayer*`).

### Returns

WebGPUSurfaceHost

A metal-layer host descriptor.

### Remarks

The caller owns the layer and must keep it valid for the lifetime of the external surface.

## Sdl(nint)

Creates a host descriptor for an SDL-owned window.

```
public static WebGPUSurfaceHost Sdl(nint sdlWindow)
```

### Parameters

`sdlWindow` nint

The SDL window pointer (`SDL_Window*`).

### Returns

WebGPUSurfaceHost

An SDL host descriptor.

## SwapChainPanel(nint)

Creates a host descriptor for a WinUI swap-chain panel.

```
public static WebGPUSurfaceHost SwapChainPanel(nint panelNative)
```

### Parameters

`panelNative` nint

The swap-chain panel's `ISwapChainPanelNative*` interface pointer.

### Returns

WebGPUSurfaceHost

A swap-chain-panel host descriptor.

### Remarks

The caller owns the interface pointer and must keep it valid for the lifetime of the external surface.

## Wayland(nint, nint)

Creates a host descriptor for a Wayland surface.

```
public static WebGPUSurfaceHost Wayland(nint display, nint surface)
```

### Parameters

`display` nint

The Wayland display pointer (`wl_display*`).

`surface` nint

The Wayland surface pointer (`wl_surface*`).

### Returns

WebGPUSurfaceHost

A Wayland host descriptor.

## Win32(nint, nint)

Creates a host descriptor for a Win32 window.

```
public static WebGPUSurfaceHost Win32(nint hwnd, nint hinstance)
```

### Parameters

`hwnd` nint

The Win32 window handle (`HWND`).

`hinstance` nint

The module instance handle (`HINSTANCE`) associated with the window.

### Returns

WebGPUSurfaceHost

A Win32 host descriptor.

## X11(nint, nuint)

Creates a host descriptor for an X11 window.

```
public static WebGPUSurfaceHost X11(nint display, nuint window)
```

### Parameters

`display` nint

The X11 display pointer (`Display*`).

`window` nuint

The X11 window identifier.

## Returns

WebGPUSurfaceHost

An X11 host descriptor.

# Class WebGPUSurfaceSession

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.WebGPU.dll

Provides shared WebGPU resources for a group of external presentation surfaces.

```
public sealed class WebGPUSurfaceSession : IDisposable
```

**Inheritance**

object ← WebGPUSurfaceSession

**Implements**

IDisposable

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Remarks

Create one session for related windows and popups so they use the same GPU device and drawing context. Dispose every surface and render target created by the session before disposing the session.

# Constructors

## WebGPUSurfaceSession()

Initializes a new instance of the WebGPUSurfaceSession class.

```
public WebGPUSurfaceSession()
```

## WebGPUSurfaceSession(Configuration)

Initializes a new instance of the WebGPUSurfaceSession class.

```
public WebGPUSurfaceSession(Configuration configuration)
```

### Parameters

`configuration` Configuration

The configuration used by surfaces and render targets created by this session.

## Properties

### DeviceContext

Gets the device context shared by every surface and render target in this session.

```
public WebGPUDeviceContext DeviceContext { get; }
```

### Property Value

WebGPUDeviceContext

### Remarks

The session owns the returned context; callers must not dispose it separately.

### Exceptions

InvalidOperationException

Thrown when no presentation surface has initialized the session.

### IsLost

Gets a value indicating whether the shared WebGPU device has been lost.

```
public bool IsLost { get; }
```

## Property Value

bool

# Methods

## CreateRenderTarget(WebGPUTextureFormat, PixelAlphaRepresentation, int, int)

Creates an offscreen render target on the shared WebGPU device.

```
public WebGPURenderTarget CreateRenderTarget(WebGPUTextureFormat format,
PixelAlphaRepresentation alphaRepresentation, int width, int height)
```

## Parameters

`format` WebGPUTextureFormat

The texture format.

`alphaRepresentation` PixelAlphaRepresentation

The alpha representation stored by the target.

`width` int

The target width in pixels.

`height` int

The target height in pixels.

## Returns

WebGPURenderTarget

The created render target.

## Exceptions

InvalidOperationException

Thrown when no presentation surface has initialized the session.

## CreateSurface(WebGPUSurfaceHost, Size)

Creates a presentation surface attached to an externally owned native host.

```
public WebGPUExternalSurface CreateSurface(WebGPUSurfaceHost host, Size framebufferSize)
```

### Parameters

`host` WebGPUSurfaceHost

The native surface host.

`framebufferSize` Size

The initial framebuffer size in pixels.

### Returns

WebGPUExternalSurface

The created presentation surface.

## CreateSurface(WebGPUSurfaceHost, Size, WebGPUExternalSurfaceOptions)

Creates a presentation surface attached to an externally owned native host.

```
public WebGPUExternalSurface CreateSurface(WebGPUSurfaceHost host, Size framebufferSize,
WebGPUExternalSurfaceOptions options)
```

### Parameters

`host` WebGPUSurfaceHost

The native surface host.

`framebufferSize` Size

The initial framebuffer size in pixels.

`options` WebGPUExternalSurfaceOptions

The presentation options.

## Returns

WebGPUExternalSurface

The created presentation surface.

# Dispose()

Releases the shared WebGPU resources.

```
public void Dispose()
```

# Enum WebGPUTextureFormat

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.WebGPU.dll

Supported texture format identifiers for native WebGPU targets.

```
public enum WebGPUTextureFormat
```

# Fields

`Bgra8Unorm = 2`

Four-channel 8-bit normalized unsigned BGRA format, mapped to Bgra32. Storage binding for this format requires the optional `bgra8unorm-storage` device feature.

`Rgba16Float = 3`

Four-channel 16-bit floating-point RGBA format, mapped to RgbaHalf.

`Rgba8Snorm = 1`

Four-channel 8-bit normalized signed RGBA format, mapped to NormalizedByte4.

`Rgba8Unorm = 0`

Four-channel 8-bit normalized unsigned RGBA format, mapped to Rgba32.

# Remarks

The compute pipeline writes its result through a storage texture binding, so a target format must be storage-bindable. Most formats here are storage-capable in core WebGPU; Bgra8Unorm is the exception and requires the optional `bgra8unorm-storage` device feature, which the backend enables when the adapter reports it.

# Class WebGPUWindow

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.WebGPU.dll

A self-contained WebGPU-backed window that owns the platform window, the WebGPU device and queue, the surface and swap chain, and the per-frame texture acquire/present cycle, exposing a DrawingCanvas for each frame. Use Run(Action<DrawingCanvas>) to let the window drive rendering, or TryAcquireFrame(out WebGPUSurfaceFrame?) to drive the frame loop yourself.

```
public sealed class WebGPUWindow : IDisposable
```

**Inheritance**

object ← WebGPUWindow

**Implements**

IDisposable

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Remarks

Use this type when ImageSharp.Drawing owns the application's window. To render into a window owned by a host application or UI framework, use WebGPUExternalSurface.

# Constructors

## WebGPUWindow()

Initializes a new instance of the WebGPUWindow class.

```
public WebGPUWindow()
```

## WebGPUWindow(Configuration, WebGPUWindowOptions)

Initializes a new instance of the WebGPUWindow class.

```
public WebGPUWindow(Configuration configuration, WebGPUWindowOptions options)
```

### Parameters

`configuration` Configuration

The configuration instance to bind to the created backend.

`options` WebGPUWindowOptions

The window creation options.

## WebGPUWindow(WebGPUWindowOptions)

Initializes a new instance of the WebGPUWindow class.

```
public WebGPUWindow(WebGPUWindowOptions options)
```

### Parameters

`options` WebGPUWindowOptions

The window creation options.

# Properties

## AlphaMode

Gets how the native compositor interprets the window surface alpha channel.

```
public WebGPUCompositeAlphaMode AlphaMode { get; }
```

### Property Value

WebGPUCompositeAlphaMode

## ClientSize

Gets or sets the client-area size in window coordinates.

```
public Size ClientSize { get; set; }
```

### Property Value

Size

## Configuration

Gets the configuration provided when the window was created.

```
public Configuration Configuration { get; }
```

### Property Value

Configuration

## DeviceContext

Gets the WebGPU device context used to render this window.

```
public WebGPUDeviceContext DeviceContext { get; }
```

### Property Value

WebGPUDeviceContext

### Remarks

The window owns the returned context; callers must not dispose it separately.

## Format

Gets the swapchain texture format.

```
public WebGPUTextureFormat Format { get; }
```

### Property Value

WebGPUTextureFormat

## FramebufferSize

Gets the framebuffer size in pixels.

```
public Size FramebufferSize { get; }
```

### Property Value

Size

## FramesPerSecond

Gets or sets the maximum number of render callbacks per second.

```
public double FramesPerSecond { get; set; }
```

### Property Value

double

## IsClosing

Gets or sets a value indicating whether the window has been requested to close.

```
public bool IsClosing { get; set; }
```

### Property Value

bool

## IsEventDriven

Gets or sets a value indicating whether the window is event-driven.

```
public bool IsEventDriven { get; set; }
```

### Property Value

bool

## IsTopMost

Gets or sets a value indicating whether the window stays above normal windows.

```
public bool IsTopMost { get; set; }
```

### Property Value

bool

## IsVisible

Gets or sets a value indicating whether the window is visible.

```
public bool IsVisible { get; set; }
```

### Property Value

bool

## Position

Gets or sets the window position in screen coordinates.

```
public Point Position { get; set; }
```

### Property Value

Point

## PresentMode

Gets or sets the swapchain present mode.

```
public WebGPUPresentMode PresentMode { get; set; }
```

### Property Value

WebGPUPresentMode

## RenderScale

Gets the ratio between framebuffer pixels and client coordinate units.

```
public float RenderScale { get; }
```

### Property Value

float

## Title

Gets or sets the window title.

```
public string Title { get; set; }
```

## Property Value

string

## UpdatesPerSecond

Gets or sets the maximum number of update callbacks per second.

```
public double UpdatesPerSecond { get; set; }
```

## Property Value

double

## WindowBorder

Gets or sets the window border mode.

```
public WebGPUWindowBorder WindowBorder { get; set; }
```

## Property Value

WebGPUWindowBorder

## WindowState

Gets or sets the window state.

```
public WebGPUWindowState WindowState { get; set; }
```

### Property Value

WebGPUWindowState

# Methods

## Close()

Closes the window immediately.

```
public void Close()
```

## ContinueEvents()

Continues the event loop when running in event-driven mode.

```
public void ContinueEvents()
```

## Dispose()

Disposes the window and its WebGPU resources.

```
public void Dispose()
```

## DoEvents()

Polls the underlying window for events.

```
public void DoEvents()
```

## Focus()

Sets keyboard focus to the window.

```
public void Focus()
```

## Hide()

Hides the window.

```
public void Hide()
```

## PointToClient(Point)

Converts a screen-space point to client coordinates.

```
public Point PointToClient(Point point)
```

### Parameters

`point` Point

The point to convert.

### Returns

Point

The converted point.

## PointToFramebuffer(Point)

Converts a client-space point to framebuffer coordinates.

```
public Point PointToFramebuffer(Point point)
```

### Parameters

`point` Point

The point to convert.

## Returns

Point

The converted point.

# PointToScreen(Point)

Converts a client-space point to screen coordinates.

```
public Point PointToScreen(Point point)
```

## Parameters

`point` Point

The point to convert.

## Returns

Point

The converted point.

# RequestClose()

Requests that the window close.

```
public void RequestClose()
```

# Run(DrawingOptions, DrawingTextCache, Action<WebGPUSurfaceFrame>)

Runs the window's event loop using a caller-owned text cache shared by every frame canvas.

```
public void Run(DrawingOptions options, DrawingTextCache textCache,
Action<WebGPUSurfaceFrame> render)
```

## Parameters

`options` DrawingOptions

The drawing options applied to each acquired frame.

`textCache` DrawingTextCache

The text cache shared by acquired frame canvases.

`render` Action<WebGPUSurfaceFrame>

The per-frame render callback.

# Run(DrawingOptions, Action<WebGPUSurfaceFrame, TimeSpan>)

Runs the window's event loop and renders one WebGPU frame per render callback.

```
public void Run(DrawingOptions options, Action<WebGPUSurfaceFrame, TimeSpan> render)
```

## Parameters

`options` DrawingOptions

The drawing options applied to each acquired frame.

`render` Action<WebGPUSurfaceFrame, TimeSpan>

The per-frame render callback. The second argument is elapsed time since the previous render callback.

# Run(DrawingOptions, Action<WebGPUSurfaceFrame>)

Runs the window's event loop and renders one WebGPU frame per render callback.

```
public void Run(DrawingOptions options, Action<WebGPUSurfaceFrame> render)
```

### Parameters

`options` DrawingOptions

The drawing options applied to each acquired frame.

`render` Action<WebGPUSurfaceFrame>

The per-frame render callback.

## Run(DrawingOptions, Action<DrawingCanvas>)

Runs the window's event loop and renders one WebGPU canvas per render callback.

```
public void Run(DrawingOptions options, Action<DrawingCanvas> render)
```

### Parameters

`options` DrawingOptions

The drawing options applied to each acquired frame.

`render` Action<DrawingCanvas>

The per-frame render callback.

## Run(Action<WebGPUSurfaceFrame, TimeSpan>)

Runs the window's event loop and renders one WebGPU frame per render callback.

```
public void Run(Action<WebGPUSurfaceFrame, TimeSpan> render)
```

### Parameters

`render` Action<WebGPUSurfaceFrame, TimeSpan>

The per-frame render callback. The second argument is elapsed time since the previous render callback.

## Run(Action<WebGPUSurfaceFrame>)

Runs the window's event loop and renders one WebGPU frame per render callback.

```
public void Run(Action<WebGPUSurfaceFrame> render)
```

### Parameters

`render` Action<WebGPUSurfaceFrame>

The per-frame render callback.

## Run(Action<DrawingCanvas>)

Runs the window's event loop and renders one WebGPU canvas per render callback.

```
public void Run(Action<DrawingCanvas> render)
```

### Parameters

`render` Action<DrawingCanvas>

The per-frame render callback.

## Show()

Shows the window.

```
public void Show()
```

# TryAcquireFrame(out WebGPUSurfaceFrame?)

Tries to acquire the next drawable frame using default drawing options.

```
public bool TryAcquireFrame(out WebGPUSurfaceFrame? frame)
```

## Parameters

`frame` WebGPUSurfaceFrame

Receives the acquired frame on success.

## Returns

bool

true when a frame is available; otherwise false when the frame should be retried later.

## Remarks

Use this overload when the default drawing options are sufficient.

# TryAcquireFrame(DrawingOptions, out WebGPUSurfaceFrame?)

Tries to acquire the next drawable frame.

```
public bool TryAcquireFrame(DrawingOptions options, out WebGPUSurfaceFrame? frame)
```

## Parameters

`options` DrawingOptions

The drawing options for the acquired frame.

`frame` WebGPUSurfaceFrame

Receives the acquired frame on success.

## Returns

bool

true when a frame is available; otherwise false when the frame should be retried later.

### Remarks

Use this overload when you are driving the render loop yourself and need explicit drawing options. A false result means no drawable frame is available right now, for example because the surface was lost, outdated, timed out, or has a zero-sized framebuffer. Dispose the returned frame when you are done with it to present it and release its per-frame resources.

## TryAcquireFrame(DrawingOptions, DrawingTextCache, out WebGPUSurfaceFrame?)

Tries to acquire the next drawable frame using a caller-owned text cache.

```
public bool TryAcquireFrame(DrawingOptions options, DrawingTextCache textCache, out
WebGPUSurfaceFrame? frame)
```

### Parameters

`options` DrawingOptions

The drawing options for the acquired frame.

`textCache` DrawingTextCache

The text cache shared by acquired frame canvases.

`frame` WebGPUSurfaceFrame

Receives the acquired frame on success.

### Returns

bool

true when a frame is available; otherwise false.

# Events

## Closing

Raised when the window is closing.

```
public event Action? Closing
```

### Event Type

Action

## FilesDropped

Raised when files are dropped onto the window.

```
public event Action<string[]>? FilesDropped
```

### Event Type

Action<string[]>

## FocusChanged

Raised when the window focus changes.

```
public event Action<bool>? FocusChanged
```

### Event Type

Action<bool>

## FramebufferResized

Raised when the framebuffer size changes.

```
public event Action<Size>? FramebufferResized
```

## Event Type

Action<Size>

# Moved

Raised when the window moves.

```
public event Action<Point>? Moved
```

## Event Type

Action<Point>

# Resized

Raised when the client-area size in window coordinates changes.

```
public event Action<Size>? Resized
```

## Event Type

Action<Size>

# StateChanged

Raised when the window state changes.

```
public event Action<WebGPUWindowState>? StateChanged
```

## Event Type

Action<WebGPUWindowState>

## Update

Raised when the window update loop runs.

```
public event Action<TimeSpan>? Update
```

## Event Type

Action<TimeSpan>

# Enum WebGPUWindowBorder

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.WebGPU.dll

Describes the window border around a WebGPUWindow.

```
public enum WebGPUWindowBorder
```

# Fields

`Fixed = 1`

Uses the normal decorated window frame but does not allow user resizing.

`Hidden = 2`

Hides the standard window border and title bar.

`Resizable = 0`

Uses the normal decorated window frame and allows the user to resize the window.

# Class WebGPUWindowOptions

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.WebGPU.dll

Options for creating a WebGPUWindow.

```
public sealed class WebGPUWindowOptions
```

**Inheritance**

object ← WebGPUWindowOptions

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Remarks

These values describe the initial window and scheduling configuration used during construction. Most callers only need to set Title, Size, and PresentMode.

# Properties

## AlphaMode

Gets or sets how the native compositor interprets the window surface alpha channel.

```
public WebGPUCompositeAlphaMode AlphaMode { get; set; }
```

### Property Value

WebGPUCompositeAlphaMode

### Remarks

The requested mode is used when available. Otherwise, a compatible mode is selected automatically.

## Format

Gets or sets the swapchain texture format used by acquired frames.

```
public WebGPUTextureFormat Format { get; set; }
```

### Property Value

WebGPUTextureFormat

### Remarks

The requested format is used when available. Otherwise, a compatible format is selected automatically.

## FramesPerSecond

Gets or sets the requested upper bound for render callbacks per second.

```
public double FramesPerSecond { get; set; }
```

### Property Value

double

### Remarks

This is a scheduling hint for the underlying window loop rather than a guarantee of presented frame rate. The chosen PresentMode and the platform's display timing still influence what the user sees. Negative or non-finite values use the default unrestricted rate.

## IsEventDriven

Gets or sets a value indicating whether the window should use event-driven scheduling.

```
public bool IsEventDriven { get; set; }
```

## Property Value

bool

## Remarks

Event-driven mode is useful for UI-style apps that should sleep while idle instead of continuously driving a tight loop. Continuous rendering scenarios often leave this disabled.

# IsTopMost

Gets or sets a value indicating whether the window should stay above normal windows.

```
public bool IsTopMost { get; set; }
```

## Property Value

bool

# IsVisible

Gets or sets a value indicating whether the window should be visible immediately after creation.

```
public bool IsVisible { get; set; }
```

## Property Value

bool

# Position

Gets or sets the initial requested window position in screen coordinates.

```
public Point Position { get; set; }
```

## Property Value

Point

# PresentMode

Gets or sets how completed frames are queued for presentation to the display.

```
public WebGPUPresentMode PresentMode { get; set; }
```

## Property Value

WebGPUPresentMode

## Remarks

Choose Fifo for the usual v-synced behavior, Immediate for the lowest latency with possible tearing, or Mailbox when you want newer-frame-wins behavior and the backend supports it. When the requested mode is unavailable, Fifo is used.

# Size

Gets or sets the initial client-area size in window coordinates.

```
public Size Size { get; set; }
```

## Property Value

Size

## Remarks

A size with a non-positive dimension uses the default initial size.

# Title

Gets or sets the initial window title shown in the title bar.

```
public string Title { get; set; }
```

### Property Value

string

## UpdatesPerSecond

Gets or sets the requested upper bound for update callbacks per second.

```
public double UpdatesPerSecond { get; set; }
```

### Property Value

double

### Remarks

Use this when you want simulation or input updates to run at a different cadence from rendering. Negative or non-finite values use the default unrestricted rate.

## WindowBorder

Gets or sets the initial window border mode.

```
public WebGPUWindowBorder WindowBorder { get; set; }
```

### Property Value

WebGPUWindowBorder

### Remarks

Unrecognized values use Resizable.

# WindowState

Gets or sets the initial window state such as normal, maximized, or fullscreen.

```
public WebGPUWindowState WindowState { get; set; }
```

## Property Value

WebGPUWindowState

## Remarks

Unrecognized values use Normal.

# Enum WebGPUWindowState

Namespace: SixLabors.ImageSharp.Drawing.Processing.Backends

Assembly: SixLabors.ImageSharp.Drawing.WebGPU.dll

Describes the startup or current state of a WebGPUWindow.

```
public enum WebGPUWindowState
```

# Fields

`Fullscreen = 3`

Opens in fullscreen mode and occupies the full display.

`Maximized = 2`

Opens maximized to fill the normal desktop work area.

`Minimized = 1`

Opens minimized to the taskbar or dock.

`Normal = 0`

Opens as a normal restored window using its configured size.

# Namespace SixLabors.ImageSharp.Drawing.Text

## Classes

GlyphPathCollection
: A geometry + paint container for a single glyph, preserving painted layer boundaries.

TextBuilder
: Builds vector shapes from text using the provided layout and rendering options.

## Structs

GlyphLayerInfo
: Describes a single painted layer as a span within the glyph's path list.

## Enums

GlyphLayerKind
: Optional semantic classification for layers to aid monochrome projection or decoration handling.

# Struct GlyphLayerInfo

Namespace: SixLabors.ImageSharp.Drawing.Text

Assembly: SixLabors.ImageSharp.Drawing.dll

Describes a single painted layer as a span within the glyph's path list.

```
public readonly struct GlyphLayerInfo
```

**Inherited Members**
ValueType.Equals(object) , ValueType.GetHashCode() , ValueType.ToString() , object.Equals(object, object) , object.GetType() , object.ReferenceEquals(object, object)

# Properties

## Bounds

Gets the bounds of the layer geometry (device space).

```
public RectangleF Bounds { get; }
```

## Property Value

RectangleF

## Count

Gets the number of paths in this layer.

```
public int Count { get; }
```

## Property Value

int

## IntersectionRule

Gets the fill rule for rasterization of this layer.

```
public IntersectionRule IntersectionRule { get; }
```

### Property Value

IntersectionRule

## Kind

Gets the semantic kind of the layer (for policy decisions).

```
public GlyphLayerKind Kind { get; }
```

### Property Value

GlyphLayerKind

## Paint

Gets the paint definition to use for this layer; may be null.

```
public Paint? Paint { get; }
```

### Property Value

Paint

## PixelAlphaCompositionMode

Gets the pixel alpha composition mode to use for this layer.

```
public PixelAlphaCompositionMode PixelAlphaCompositionMode { get; }
```

## Property Value

PixelAlphaCompositionMode

# PixelColorBlendingMode

Gets the pixel color blending mode to use for this layer.

```
public PixelColorBlendingMode PixelColorBlendingMode { get; }
```

## Property Value

PixelColorBlendingMode

# StartIndex

Gets the start index (inclusive) of the layer span within the glyph's path list.

```
public int StartIndex { get; }
```

## Property Value

int

# Enum GlyphLayerKind

Namespace: SixLabors.ImageSharp.Drawing.Text

Assembly: SixLabors.ImageSharp.Drawing.dll

Optional semantic classification for layers to aid monochrome projection or decoration handling.

```
public enum GlyphLayerKind
```

# Fields

`Decoration = 1`

Text decoration geometry (underline/overline/strikethrough).

`Glyph = 0`

Regular glyph geometry layer.

`Painted = 2`

Painted layer (e.g. color emoji glyph).

# Class GlyphPathCollection

Namespace: SixLabors.ImageSharp.Drawing.Text

Assembly: SixLabors.ImageSharp.Drawing.dll

A geometry + paint container for a single glyph, preserving painted layer boundaries.

```
public sealed class GlyphPathCollection
```

**Inheritance**

object ← GlyphPathCollection

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.ReferenceEquals(object, object) , object.ToString()

# Properties

## Bounds

Gets an axis-aligned bounding box of the entire glyph in device space.

```
public RectangleF Bounds { get; }
```

## Property Value

RectangleF

## LayerCount

Gets the number of layers.

```
public int LayerCount { get; }
```

## Property Value

int

## Layers

Gets a read-only list of layer descriptors preserving paint, fill rule and path spans.

```
public IReadOnlyList<GlyphLayerInfo> Layers { get; }
```

## Property Value

IReadOnlyList<GlyphLayerInfo>

## PathList

Gets a read-only view of all individual paths in z-order.

```
public IReadOnlyList<IPath> PathList { get; }
```

## Property Value

IReadOnlyList<IPath>

## Paths

Gets the flattened geometry for the glyph (all paths in z-order). This is equivalent to concatenating all layer spans.

```
public IPathCollection Paths { get; }
```

## Property Value

IPathCollection

# Methods

## GetLayerPaths(int)

Gets a PathCollection view of a single layer's geometry.

```
public PathCollection GetLayerPaths(int layerIndex)
```

### Parameters

`layerIndex` int

The zero-based layer index.

### Returns

PathCollection

A path collection comprising only that layer's span.

## ToPathCollection(Func<GlyphLayerInfo, bool>?)

Creates a PathCollection containing only the paths from layers that satisfy `predicate`. Useful to project to monochrome.

```
public PathCollection ToPathCollection(Func<GlyphLayerInfo, bool>? predicate = null)
```

### Parameters

`predicate` Func<GlyphLayerInfo, bool>

A filter deciding whether to keep a layer.

### Returns

PathCollection

A new PathCollection with the selected paths.

# Transform(Matrix4×4)

Transforms the glyph using the specified matrix.

```
public GlyphPathCollection Transform(Matrix4x4 matrix)
```

## Parameters

`matrix` Matrix4×4

The transform matrix.

## Returns

GlyphPathCollection

A new GlyphPathCollection with the matrix applied to it.

# Class TextBuilder

Namespace: SixLabors.ImageSharp.Drawing.Text

Assembly: SixLabors.ImageSharp.Drawing.dll

Builds vector shapes from text using the provided layout and rendering options.

```
public static class TextBuilder
```

**Inheritance**

object ← TextBuilder

**Inherited Members**

object.Equals(object) , object.Equals(object, object) , object.GetHashCode() , object.GetType() , object.MemberwiseClone() , object.ReferenceEquals(object, object) , object.ToString()

# Methods

## GenerateGlyphs(ReadOnlySpan<ushort>, ReadOnlySpan<Vector2>, GlyphOptions)

Generates per-glyph path data and metadata for positioned glyphs. Each entry contains the combined outline paths for a glyph and associated metadata that enables intelligent fill or stroke decisions at the glyph level.

```
public static IReadOnlyList<GlyphPathCollection> GenerateGlyphs(ReadOnlySpan<ushort>
glyphIds, ReadOnlySpan<Vector2> points, GlyphOptions glyphOptions)
```

### Parameters

`glyphIds` ReadOnlySpan<ushort>

The glyph identifiers.

`points` ReadOnlySpan<Vector2>

The absolute glyph origins in pixel units.

`glyphOptions` GlyphOptions

The glyph rendering options.

### Returns

IReadOnlyList<GlyphPathCollection>

A read-only list of GlyphPathCollection entries, one for each rendered glyph.

## GenerateGlyphs(string, TextOptions)

Generates per-glyph path data and metadata for the rendered `text`. Each entry contains the combined outline paths for a glyph and associated metadata that enables intelligent fill or stroke decisions at the glyph level.

```
public static IReadOnlyList<GlyphPathCollection> GenerateGlyphs(string text,
TextOptions textOptions)
```

### Parameters

`text` string

The text to shape and render.

`textOptions` TextOptions

The text rendering and layout options.

### Returns

IReadOnlyList<GlyphPathCollection>

A read-only list of GlyphPathCollection entries, one for each rendered glyph.

## GenerateGlyphs(string, IPath, TextOptions)

Generates per-glyph path data and metadata for the rendered `text`, laid out along the supplied `path` baseline. Each entry contains the combined outline paths for a glyph and associated metadata.

```
public static IReadOnlyList<GlyphPathCollection> GenerateGlyphs(string text, IPath path,
TextOptions textOptions)
```

## Parameters

`text` string

The text to shape and render.

`path` IPath

The path that defines the text baseline.

`textOptions` TextOptions

The text rendering and layout options.

## Returns

IReadOnlyList<GlyphPathCollection>

A read-only list of GlyphPathCollection entries, one for each rendered glyph.

# GeneratePaths(ReadOnlySpan<ushort>, ReadOnlySpan<Vector2>, GlyphOptions)

Generates the combined outline paths for positioned glyphs. The result merges per-glyph outlines into a single IPathCollection suitable for filling or stroking as one unit.

```
public static IPathCollection GeneratePaths(ReadOnlySpan<ushort> glyphIds,
ReadOnlySpan<Vector2> points, GlyphOptions glyphOptions)
```

## Parameters

`glyphIds` ReadOnlySpan<ushort>

The glyph identifiers.

`points` ReadOnlySpan<Vector2>

The absolute glyph origins in pixel units.

`glyphOptions` GlyphOptions

The glyph rendering options.

## Returns

IPathCollection

The combined IPathCollection for the rendered glyphs.

# GeneratePaths(string, TextOptions)

Generates the combined outline paths for all rendered glyphs in `text`. The result merges per-glyph outlines into a single IPathCollection suitable for filling or stroking as one unit.

```
public static IPathCollection GeneratePaths(string text, TextOptions textOptions)
```

## Parameters

`text` string

The text to shape and render.

`textOptions` TextOptions

The text rendering and layout options.

## Returns

IPathCollection

The combined IPathCollection for the rendered glyphs.

# GeneratePaths(string, IPath, TextOptions)

Generates the combined outline paths for all rendered glyphs in `text`, laid out along the supplied `path` baseline. The result merges per-glyph outlines into a single IPathCollection.

```
public static IPathCollection GeneratePaths(string text, IPath path,
TextOptions textOptions)
```

## Parameters

`text` string

The text to shape and render.

`path` IPath

The path that defines the text baseline.

`textOptions` TextOptions

The text rendering and layout options.

## Returns

IPathCollection

The combined IPathCollection for the rendered glyphs.